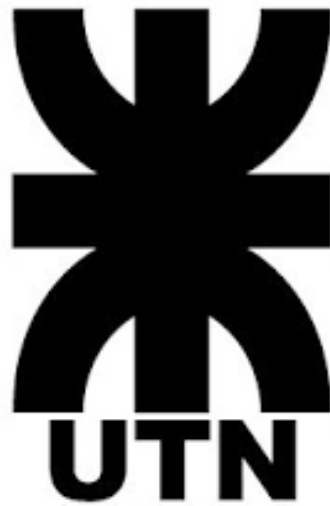




UNIVERSIDAD TECNOLÓGICA NACIONAL FACULTAD REGIONAL CÓRDOBA

Resumen Final

Diseño de Sistemas

Contenido

Introducción Paradigma orientado a objetos	6
UNIDAD 1: Análisis de Sistemas de Información	8
UML	8
Bloques básicos de construcción	9
Diagramas	9
Reglas de UML	13
Mecanismos comunes de UML	13
1. Especificaciones:	13
2. Adornos:	13
3. Divisiones comunes	14
4. Mecanismos de extensibilidad	14
Proceso unificado de desarrollo	16
MÁQUINAS DE ESTADOS	21
Workflows del PUD	24
WORKFLOW DE ANALISIS	25
Beneficio de porque hacer análisis:	25
¿Cuál es el rol del análisis en el ciclo de vida iterativo e incremental del PUD?	26
Como utilizar el análisis:	26
Integración del Análisis con el Diseño:	26
Artefactos (Cualquier pieza de código que es producida o modificada por un proceso)	27
Trabajadores	29
Flujo de trabajo	30
Actividades	30
RESUMEN DEL ANÁLISIS	33
Patrones GRASP	34
UNIDAD 2: WORKFLOW DE DISEÑO	41
Análisis vs Diseño	42
Para que diseñamos	43
Rol en el PUD	45
Propósitos del diseño	45
Workflow de diseño como proceso	45
Artefactos	46
Trabajadores del diseño	50
Flujo de trabajo	51
Actividades	52
RESUMEN DEL DISEÑO	62

¿Qué aspectos deben diseñarse?	63
Arquitectura	63
Base de datos	64
Procesos (P. de diseño)	64
Interfaces -> interacción humano-máquina	64
Formas de entrada/salida	65
Procedimientos manuales.	66
Guía para evaluar un buen diseño	66
Propósito de WF de diseño	66
Diseño de software orientado a objetos	66
Principios de diseño de software	69
Relaciones que tenemos en el SW:	75
ESTRATEGIA DE ENSAMBLADO DE COMPONENTES	76
Procesos ISBC	82
ISBC con reutilización	83
Diferencias con los procesos de software originales:	83
Características importantes de algunas etapas del proceso de CBSE:	84
Composición de componentes.	84
Beneficios	86
Ventajas	86
Patrones de Diseño (Solución concreta a un problema bien definido)	86
Los patrones de diseño:	87
Diferencias entre PATRONES E IDIOMAS	87
Elementos de un patrón de diseño	87
¿Por qué utilizar patrones de diseño?	88
¿Qué es un patrón de diseño?	88
Tipos de patrones de diseño	88
¿Cómo resuelven los patrones los problemas de diseño?	89
Patrones de diseño	95
Patrones de creación	95
Propósito específico	95
Patrones de creación	95
BUILDER	96
Herencia o Realización	97
FACTORY METHOD	98
-Realización o herencia	99

ADAPTER	101
Solo se aplica con herencia.	101
COMPOSITE.....	103
Se aplica con realización y herencia.	103
El composite no aplica el principio de diseño (Segregación de interfaces):	104
TEMPLATE METHOD.....	106
-Solo Herencia	106
-Realización o herencia.....	109
STATE.....	111
Solo herencia	111
STRATEGY.....	113
Se puede resolver con herencia y realización.	113
Conceptos del diseño	115
División de problemas	115
Modularidad	115
Ocultamiento de información	115
Independencia funcional	115
Refinamiento	116
Aspectos	116
Rediseño	116
Conceptos de diseño orientado a objetos	116
Estrategias de prototipado.....	118
Características:.....	118
Beneficios.....	118
Desventajas/Riesgos/Críticas:.....	119
Proceso de desarrollo de prototipos	119
¿Cuándo es conveniente usar prototipos?	119
¿Cómo se utilizan?	119
Uso de los Prototipos:	120
Clasificación de prototipos	120
Técnicas de prototipado.....	121
Estrategias para el Desarrollo de Prototipos	121
Creencias erróneas sobre el tema Prototipos.....	121
Prototipación de interfaz de usuario:.....	121
Prototipado en papel:.....	121
Evaluación de interfaz de usuario	122

Proceso de desarrollo de prototipos	122
Etapas del Modelo de Prototipos	122
Diseño de la interfaz de usuario	123
DATO VS INFORMACIÓN.....	123
Evolución de la IHM.....	123
Reglas doradas:	124
Factores que componen la UX:	125
COMPLEJIDAD DEL DISEÑO DE IUS.....	126
HEURÍSTICAS DE NIELSEN	126
8 REGLAS DE SHNEIDERMAN (CoARDEDeCoM).....	127
Diseño de la persistencia.....	128
Mapeo	141
Modelos de persistencia	141
Principios SOLID.....	141
UNIDAD 3.....	153
Diseño arquitectónico.....	153
VISTAS EN EL PROCESO UNIFICADO DE DESARROLLO (PUD)	185
Patrones arquitectónicos.....	187
UNIDAD 4.....	214
Workflow de implementación	214
Artefactos (6)	215
Trabajadores (3)	218
Flujo de trabajo	218
Actividades	219
Workflow de prueba	222
Artefactos (6)	223
Trabajadores (4)	225
Flujo de trabajo	226
Actividades	226
Workflow de despliegue.....	229
EL ROL DEL DESPLIEGUE EN EL CICLO DE VIDA ITERATIVO E INCREMENTAL DE RUP	229
Artefactos (1)	229
Trabajadores (7)	230
Actividades	230
PRODUCTOS ENLATADOS.....	232
RESUMEN DEL DESPLIEGUE.....	232
UNIDAD 5.....	232

Evolución del software	233
¿Porque se necesita tanto mantenimiento?	233
Procesos de evolución.....	234
Dinámica de evolución del software	236
Estrategias de evolución del software	237

Paradigma Orientado a Objetos

Definición: Es una forma de estructurar la realidad en el ámbito de la programación y el diseño de software, donde la interacción y colaboración entre objetos son clave para lograr funcionalidades que individualmente no podrían alcanzarse.

Objetos: Representan elementos o entidades, ya sean concretas o abstractas, con roles definidos en el dominio del problema. Cada objeto se caracteriza por:

Comportamiento: Las acciones o funciones que se espera que el objeto realice.

Estado: Los valores asignados a los atributos del objeto en un momento dado.

Identidad: Una característica extrínseca que no depende de su estado, sino de su posición en el tiempo o espacio, permitiendo que el objeto sea referenciado de manera única.

Clases: Describen un grupo de objetos que comparten una estructura y comportamiento comunes, manejados bajo una única definición. Las clases pueden representar roles, lugares, cosas, conceptos y transacciones.

Importancia: Permiten la abstracción y manejo de la complejidad en el diseño de software, agrupando objetos con características similares.

Principios Clave:

Abstracción: Utilizada para manejar la complejidad, enfocándose en las características esenciales de un objeto, ignorando los detalles irrelevantes.

Cohesión: Refiere a lo que sucede dentro de una clase. Cada clase debe tener una sola responsabilidad, asegurando que sus componentes estén estrechamente relacionados y enfocados en una única tarea.

Acoplamiento: Mide la interrelación entre clases. Un buen diseño busca minimizar el acoplamiento para reducir la dependencia entre distintas partes del sistema.

Encapsulamiento: Separa lo que un objeto hace (su interfaz) de cómo lo hace (su implementación), permitiendo el ocultamiento de información. Los atributos y la implementación se encapsulan, mientras que la parte pública son las firmas de los métodos, que están compuestas por nombre y visibilidad, parámetros y tipos, y retorno.

Relaciones entre Objetos:

Jerárquicas:

Generalización: Relación donde una clase "hija" hereda características de una clase "padre".

Composición/Agregación: Relaciones tipo Todo-Parte, donde un objeto se compone de otros.

No Jerárquicas:

Asociación: Relación donde un objeto colabora con otro, pero no existe una relación de herencia.

Herencia: Importante destacar que la herencia no puede darse entre objetos, ya que no se puede

heredar la identidad.

Paradigma orientado a objetos: Conjunto de objetos que colaboran entre sí para lograr algo que solos no podrían lograr

UNIDAD 1: Análisis de Sistemas de Información

Orientado a Objetos con UML

Revisión de UML 2.0 (falta un poco)

Revisión de Proceso Unificado de Desarrollo

Análisis en el Proceso Unificado de Desarrollo o Objetivo, actividades, trabajadores, artefactos

Análisis Orientado a Objetos o Modelado de comportamiento en el análisis

Modelado de estructura en el análisis

Patrones Generales de Asignación de Responsabilidades (GRASP)

UML

El lenguaje unificado de modelado es un lenguaje estándar para describir planos de software. Es independiente del proceso, sirve para visualizar, construir, especificar y documentar artefactos de un sistema. Nos ayuda a la comprensión del sistema, ya que indican cómo crear y leer modelos bien formados.

UML es un lenguaje: Proporciona un vocabulario y reglas posibilitando la comunicación en el equipo de trabajo, que se centran en la representación conceptual y física de un sistema: indican cómo crear y leer modelos bien formados, pero no dicen qué modelos crear ni cuándo (eso hace el PUD). Nos ayuda a la comprensión del sistema, a través de múltiples modelos conectados entre sí.

UML nos permite

✓ **Visualizar:** Detrás de cada símbolo hay un significado bien definido y sin ambigüedad. Así, un desarrollador puede escribir un modelo UML y otro desarrollador o herramienta, puede interpretarlo sin ambigüedad.

✓ **Especificar:** Construir modelos precisos, no ambiguos y completos.

✓ **Construir:** Te permite establecer correspondencias desde un modelo UML a un lenguaje de programación, permitiendo ingeniería directa (generación de código a partir de un modelo UML) e ingeniería inversa (reconstruir un modelo UML a partir de código).

✓ **Documentar:** Nos permite documentar la arquitectura de un sistema y sus detalles, como también

los requerimientos, pruebas y modelar las actividades de planificación de proyectos y gestión de versiones.

Bloques básicos de construcción

Elementos

- **Estructurales:** Partes estáticas de un modelo, representan conceptos o cosas.
- **De comportamiento:** Partes dinámicas de un modelo.
- **De agrupación:** Partes organizativas de un modelo.
- **De anotación:** Partes explicativas de un modelo.

Relaciones

- **Relación de dependencia:** Relación semántica entre dos elementos, en la cual un cambio a un elemento independiente puede afectar la semántica del dependiente.
- **Relación de asociación:** Relación estructural entre clases que describe un conjunto de conexiones entre objetos
 - **2.1 Agregación:** tipo especial de asociación, representa una relación entre un todo y sus partes
 - **2.2 Composición:** Composición Fuerte. Formado por un componente y un compuesto. Si se elimina el compuesto, se elimina el componente. Dependen uno del otro
- **Relación de generalización:** Relación de especialización/generalización en la cual el elemento especializado se basa en la especificación del elemento generalizado (hijo-padre)
- **Relación de realización:** Relación semántica entre clasificadores, donde uno especifica un contrato que otro cumplirá

Diagramas

Representación gráfica de un conjunto de elementos que sirven para visualizar un sistema desde diferentes perspectivas. Proyección de un sistema. Son herramientas para construir modelos, las vistas se construyen en cada workflow con diagramas. Se organizan en paquetes. Puede contener cualquier combinación de elementos y relaciones. En la práctica surgen un número de combinaciones habituales, que son consistentes con las cinco vistas que comprenden la arquitectura de un sistema.

Se dividen en dos grupos:

Diagramas de Estructura (Modelan Clases)

Diagrama de Clases: Muestra las clases de un sistema y las relaciones entre ellas. Incluye propiedades, métodos y dependencias entre clases, siendo fundamental para entender la **estructura estática** del

sistema.

Diagrama de Paquetes: Agrupa elementos relacionados del modelo, como clases o interfaces, en paquetes. Es útil para organizar el modelo y reducir su complejidad. Muestra la descomposición del propio modelo en unidades organizativas y sus dependencias.

Diagrama de Estructura Compuesta: Describe la estructura interna de una clase y sus interacciones con otras clases. Se enfoca en cómo se compone una clase a través de sus partes y cómo se relacionan estas partes entre sí.

Diagrama de Componentes: Representa los componentes de software de alto nivel y sus interdependencias. Esencial para comprender cómo se organiza el sistema en términos de componentes reutilizables. Representa la encapsulación de una clase, junto con sus interfaces, puertos y estructura interna, la cual está formada por otros componentes anidados y conectores. Cubren la vista de implementación estática del diseño de un sistema.

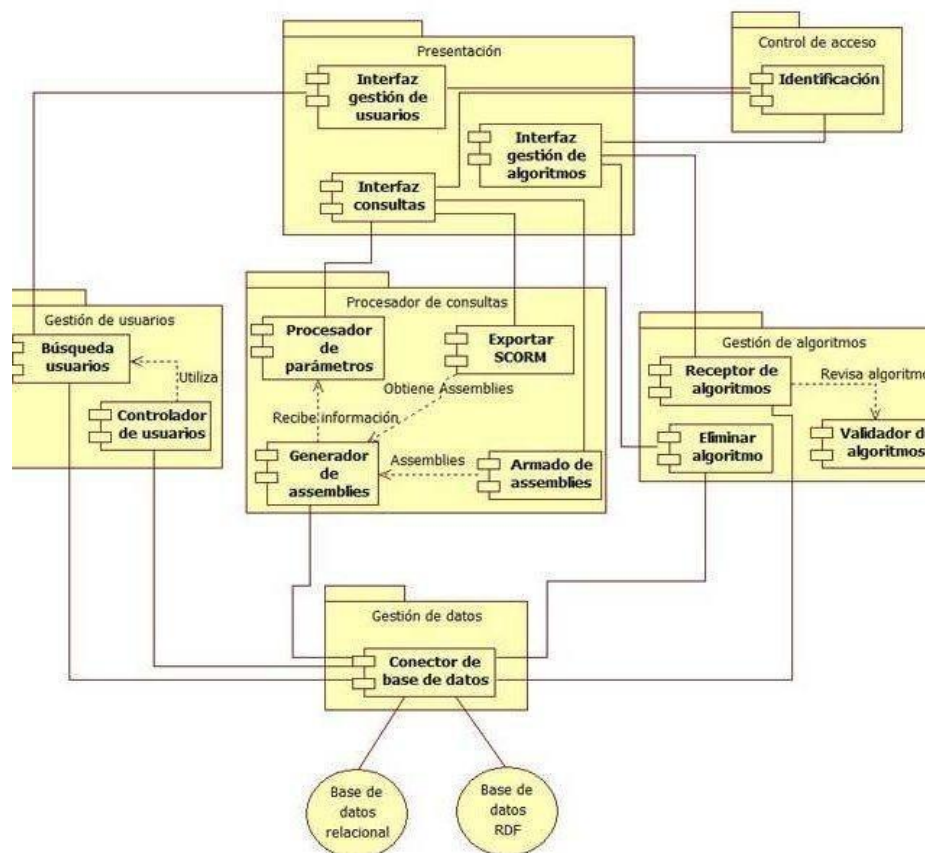
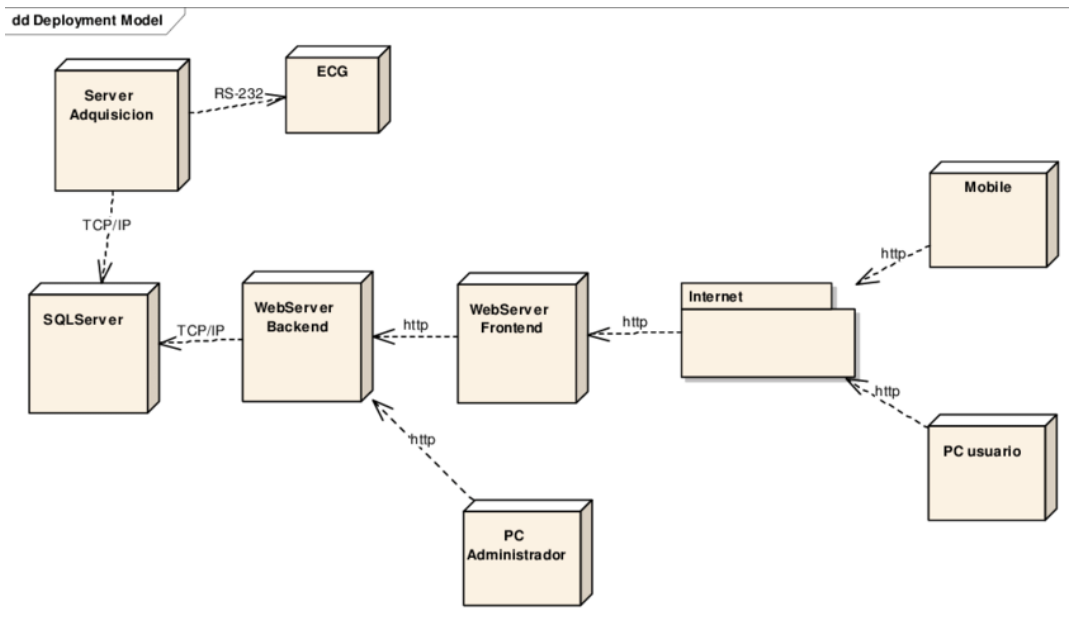


Diagrama de Despliegue: Muestra la configuración física del hardware y cómo se distribuyen en él los componentes del software. Es clave para entender cómo se implementará y ejecutará el sistema en un entorno real. Abordan la vista de despliegue estática de una arquitectura.



Diagramas de Estructura (Modelan Objetos)

Diagrama de Objetos: Similar al diagrama de clases, pero enfocado en instancias específicas de clases (objetos) y sus relaciones en un momento específico. Útil para ilustrar ejemplos concretos del modelo de clases. Representan instantáneas estáticas de instancias de los elementos de los diagramas de clases (clases). Cubren la vista de diseño estática.

Diagramas de Comportamiento (Modelan Clases)

Diagrama de Casos de Uso: Representa las funcionalidades del sistema desde la perspectiva del usuario. Muestra un conjunto de casos de uso y actores y sus relaciones. Organizan y modelan el comportamiento del sistema. Cubren la vista de casos de uso estática de un sistema.

Diagrama de Máquina de Estados: Describe los estados por los que pasa un objeto a lo largo de su vida en respuesta a eventos, junto con sus respuestas y acciones. consta de estados, transiciones, eventos y actividades. Muestra la vista dinámica de un objeto.

Diagrama de Actividad: Ilustra el flujo de trabajo o las actividades dentro del sistema y datos paso a paso. Cubren la vista dinámica del sistema. Es útil para modelar procesos de negocio o flujos de trabajo.

Diagramas de Interacción (Modelan Objetos)

Diagrama de Secuencia: Muestra cómo los objetos interactúan en función del tiempo. Representa la secuencia de mensajes entre objetos para un escenario particular. **Resalta la ordenación temporal de los mensajes. Permite modelar la línea de vida de un objeto.** Un mensaje se representa con una

flecha de una línea de vida a otra.

línea de vida: es discontinua y vertical, representa la existencia de un objeto en un período de **tiempo**

foco de control: rectángulo que representa el período de tiempo en el que un objeto ejecuta una acción.

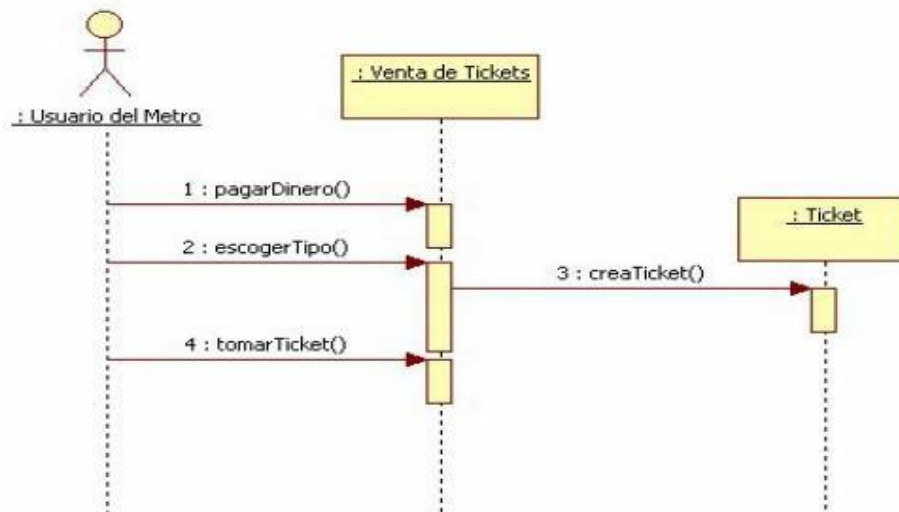


Diagrama de Timing: Similar al diagrama de secuencia, pero se enfoca en el tiempo y la duración de las interacciones. Útil para sistemas donde el tiempo es un factor crítico. Es un diagrama de interacción que muestra los tiempos reales entre diferentes objetos o roles, en oposición a la simple secuencia relativa de mensajes.

Diagrama de Comunicación: Representa las interacciones entre objetos en términos de mensajes intercambiados, enfatizando la organización y relaciones de los objetos. Resalta la **organización estructural de los objetos que envían y reciben mensajes**. Permite modelar los enlaces estructurales entre los objetos de la interacción.

- **camino:** línea que se dibuja haciéndola corresponder con una asociación.
- **número de secuencia:** indica el orden del mensaje.

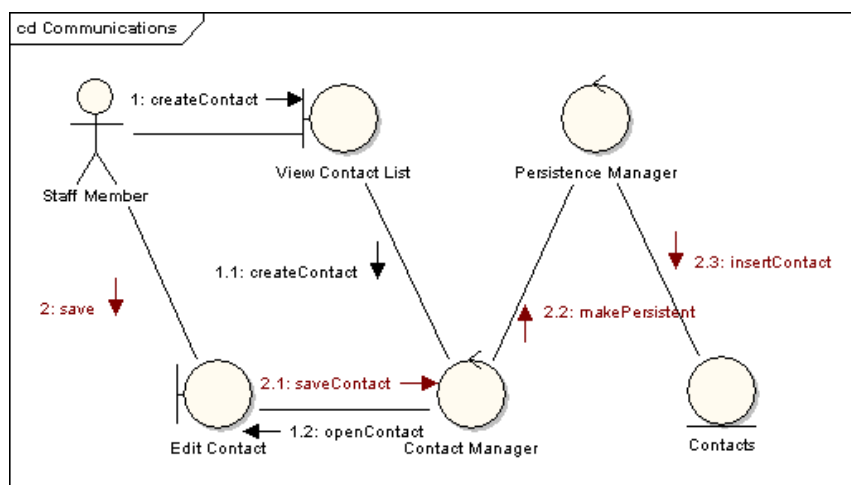


Diagrama de Descripción de Interacción: Es una categoría general que incluye diagramas de secuencia, comunicación, timing y otros tipos, centrados en cómo los objetos interactúan. muestra una interacción, que consta de un conjunto de objetos o roles, incluyendo los mensajes que pueden ser enviados entre ellos. Modelan los aspectos dinámicos de un sistema lo cual implica modelas instancias concretas de clases, interfaces, componentes y nodos, todo en el contexto de un escenario que ilustra un comportamiento. Se pueden usar para modelar un flujo de control particular de un caso de uso. Permiten construir sistemas ejecutables por medio de la ingeniería directa e inversa.

Reglas de UML

UML tiene reglas para lograr un modelo bien formado, que es aquel que es semánticamente auto consistente y está en armonía con sus modelos relacionados. UML tiene reglas para nombres, alcance, visibilidad, integridad y ejecución.

UML tiene reglas sintácticas y semánticas para:

- **Nombres:** como llamar a los elementos, relaciones y diagramas.
- **Alcance:** el contexto que da un significado específico a un nombre.
- **Visibilidad:** cómo se pueden ver y utilizar esos nombres por otros.
- **Integridad:** cómo se relacionan apropiada y consistentemente unos elementos con otros.
- **Ejecución:** que significa ejecutar o simular un modelo dinámico.

¿Qué es un modelo?

Representación de algo en el mismo u otro medio. Abstracción a un cierto nivel que captura aspectos importantes de lo que estamos modelando y simplifica el resto. Un modelo de sistema de software está construido en un lenguaje de modelado como el UML, y tiene semántica y notación.

Mecanismos comunes de UML

1. **Especificaciones:** detrás de cada elemento de su notación gráfica hay una especificación que proporciona una explicación textual de la sintaxis y semántica de ese bloque de construcción.
2. **Adornos:** todos los elementos en la notación UML parten de un símbolo básico, al cual pueden añadirse adornos específicos de ese símbolo. Para atributos y métodos: + (public), - (private) y # (protected). Ej: La clase fue pensada para dibujarse con un rectángulo porque es fácil y aparece con frecuencia, sus aspectos más importantes son: nombre, atributo y operaciones. Todos los elementos en la notación UML parten de un símbolo básico, al cual pueden añadirse una variedad de adornos específicos de ese símbolo.

3. Divisiones comunes

- a. **Entre clase y objeto:** una clase es una abstracción; un objeto es una manifestación concreta de esa abstracción.
- b. **Entre interfaz e implementación:** una interfaz declara un contrato, y una implementación representa una realización de ese contrato.
- c. **Entre tipo y rol:** el tipo declara la clase de una entidad (como un objeto, un atributo o un parámetro); un rol describe el significado de una entidad en un contexto, como una clase, un componente o una colaboración.

4. Mecanismos de extensibilidad:

permiten configurar y extender UML para las necesidades de un proyecto. Y adaptarlo a nuevas tecnologías de SW.

- a. **Estereotipo:** extiende el vocabulario de UML, permitiendo crear nuevos bloques de construcción que derivan de los existentes, pero son específicos a un problema.
- b. **Valor etiquetado:** extiende las propiedades de un estereotipo de UML, permitiendo añadir nueva información en especificación de ese estereotipo.
- c. **Restricción:** extiende la semántica de un bloque de construcción de UML, permitiendo añadir nuevas reglas o modificar las existentes.

¿Para qué se usan?

- Para pensar en su diseño antes de comenzar a construirlo, y permitir cambios en las decisiones a partir de los requisitos.
- Para generar productos aprovechables para el trabajo, es decir, las herramientas que necesitaremos para su desarrollo.
- Para encontrar, organizar, examinar, corregir información en sistemas grandes donde la misma no está a simple vista.
- Para explorar económicamente múltiples soluciones.
- Para domesticar los sistemas complejos, disminuir su complejidad sin riesgos.
- Niveles de los modelos: dependen de su propósito
- Guías al proceso de pensamiento
- Especificaciones abstractas de la estructura esencial de un sistema
- Especificaciones completas de un sistema final
- Ejemplos de sistemas típicos o posibles
- Descripciones completas o parciales de un sistema

En un modelo, encontramos **información semántica**, que capta el significado de una aplicación como una red de construcciones lógicas, y **presentación visual** que presenta la información de forma accesible al usuario.

También encontraremos **contexto**, que es lo que les da el significado completo a los modelos.

Siempre debemos considerar los siguientes aspectos en los modelos:

- Abstracción frente al detalle
- Especificación frente a implementación. Primero debemos determinar qué vamos a hacer antes de pasar al cómo hacerlo.
- Descripción frente a instancia.
- Variaciones en la interpretación.

Vistas

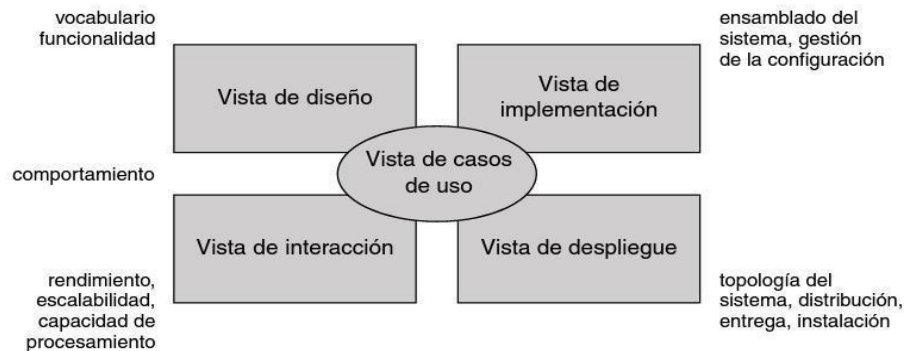
La arquitectura de un sistema puede describirse mejor a través de cinco vistas interrelacionadas. Cada vista es una proyección del sistema desde una perspectiva en particular, se hace foco o énfasis en un aspecto particular que se quiere mostrar. Es más, lo que oculta que lo que muestra una vista.

La vista se crea para un interesado permitiendo explicar lo que se quiere comunicar a ese interesado. Los modelos se hacen para comunicar algo a los interesados y, se modela a través de vistas porque es la forma que se tiene para manejar la complejidad haciendo foco en lo que se quiere comunicar y sea más fácil y simple. Las vistas en PUD se hacen con diagramas de UML porque es la recomendación que PUD propone ya que es el lenguaje de modelado más usado en todo el mundo. Cada una de estas vistas implica modelado estructural, así como modelado de comportamiento.

- **Vista de casos de uso:** comprende los casos de uso que describen el comportamiento del sistema tal y como es percibido por los usuarios finales, analistas y encargados de las pruebas. Con UML, los aspectos estáticos de esta vista se capturan en los diagramas de casos de uso; los aspectos dinámicos de esta vista se capturan en los diagramas de interacción, diagramas de estados y diagramas de actividades. Comportamiento.
- **Vista de diseño:** comprende las clases, interfaces y colaboraciones que forman el vocabulario del problema y su solución. Soporta principalmente los requerimientos funcionales del sistema. Con UML, los aspectos estáticos se capturan con diagramas de clases y de objetos; los dinámicos ídem a la vista de casos de uso. Vocabulario.
- **Vista de interacción (o de procesos):** muestra el flujo de control entre las diversas partes del sistema, incluyendo mecanismos de concurrencia y sincronización. Con UML, los aspectos estáticos y dinámicos se capturan con los mismos diagramas que en la vista de diseño, pero haciendo énfasis en clases activas que controlan el sistema y los mensajes que fluyen entre ellas. Flujo de control.
- **Vista de implementación:** comprende los artefactos que se utilizan ensamblar y poner en ejecución el sistema físico. Con UML, los aspectos estáticos de esta vista se capturan con el

diagrama de artefactos; los dinámicos ídem vista de casos de uso. Ensamblado.

- **Vista de despliegue:** contiene los nodos que forman la topología de hardware sobre la que se ejecuta el sistema. Con UML, los aspectos estáticos se capturan con el diagrama de despliegue; los dinámicos ídem a la vista de casos de uso. Topología de hardware.



Estereotipo

Es un recurso para potenciar la capacidad de modelado, es una especialización de un concepto base ya existente, con variaciones de acuerdo a las necesidades de quien modela

Vista

Es una representación del sistema desde determinada perspectiva, ilustran los artefactos más significativos para la arquitectura y se ilustran a través de diagramas. (otra def) Proyecciones de un modelo/sistema para un involucrado en particular y sus intereses

Modelo

Es una representación simplificada de la realidad, construimos modelos para comprender mejor el sistema en desarrollo, nos ayuda a visualizar como queremos que sea el sistema

Visibilidad (Necesaria para intercambiar mensajes)

- **Por atributos:** Se refiere a la forma en que los atributos (o variables miembro) de una clase son accedidos y modificados desde fuera de la clase. Pueden ser definidos como públicos, privados o protegidos. (Asociación, agregación y composición)
- **Por parámetros:** Se refiere a la forma en que los parámetros de un método o función son accedidos y modificados dentro de la función o método. Los parámetros a menudo son tratados como variables locales, y su visibilidad se limita solo al cuerpo de la función o método. (Dependencia)

Proceso unificado de desarrollo

Es un conjunto de actividades o procesos o tareas que logran transformar los requerimientos de un cliente en un sistema de SW. Es un marco de desarrollo de software. Se define como un entorno genérico de proceso que se puede especializar según las necesidades que tengamos en cada proyecto.

Es un proceso de desarrollo de software que utiliza UML para preparar todos los esquemas de un

sistema de software.

- Es un proceso definido
- Es un framework Puede especializarse
- Posee capacidad evolutiva
- Su ciclo de vida es iterativo e incremental.

Iterativo: involucra la gestión de un flujo de versiones ejecutables. Repite el proceso de trabajo.

Incremental: se integran las versiones, y cada nuevo ejecutable incorpora mejoras incrementales sobre los otros.

Evolución del producto

Centrado en la arquitectura (Intenta desarrollar el sistema de forma conjunta, abordando lo funcional y lo no funcional) Nos da una visión general de que puede y no puede hacer el sistema, que es utilizado como cimiento. Se avanza con vistas arquitectónicas desde el WF de Requerimientos. Vamos haciendo evolucionar el software teniendo en cuenta a la arquitectura desde el 1er momento.

Conducido por casos de uso (Desglosa el producto): Se utilizan para establecer el comportamiento deseado del sistema, para verificar y validar la arquitectura del sistema, para las pruebas y para la comunicación entre las personas involucradas en el proyecto. Los CU guían el proceso de desarrollo. Los CU se utilizan para capturar los requerimientos para desarrollarlos a través de los distintos WF en las distintas fases.

El CU que mapea un requerimiento, lo llevamos como hilo conductor a lo largo de todos los WF del proceso, manteniendo una trazabilidad (responde al cliente sobre cierto requerimiento en qué etapa se encuentra, en que se transformó, nivel de avance)

Iterativo e incremental ¿Que es? Es el ciclo de vida que el proceso unificado de desarrollo (PUD) recomienda. Tiene fases, las cuales identifican los momentos de evolución del proyecto, se trabaja con iteraciones, que entregan una versión del producto. Tenemos la libertad de elegir otro ciclo de vida, un proceso se puede trabajar con diferentes ciclos de vida.

Dirigido por casos de uso y centrado en la arquitectura van de la mano, porque la funcionalidad y la arquitectura se condicionan entre sí, a veces un RNF determina que la arquitectura se comporte de una determinada manera u otra. Ej: q el sistema sea web o Mobile, cliente liviano (no se debe instalar nada), es un RNF que impacta en la arquitectura.

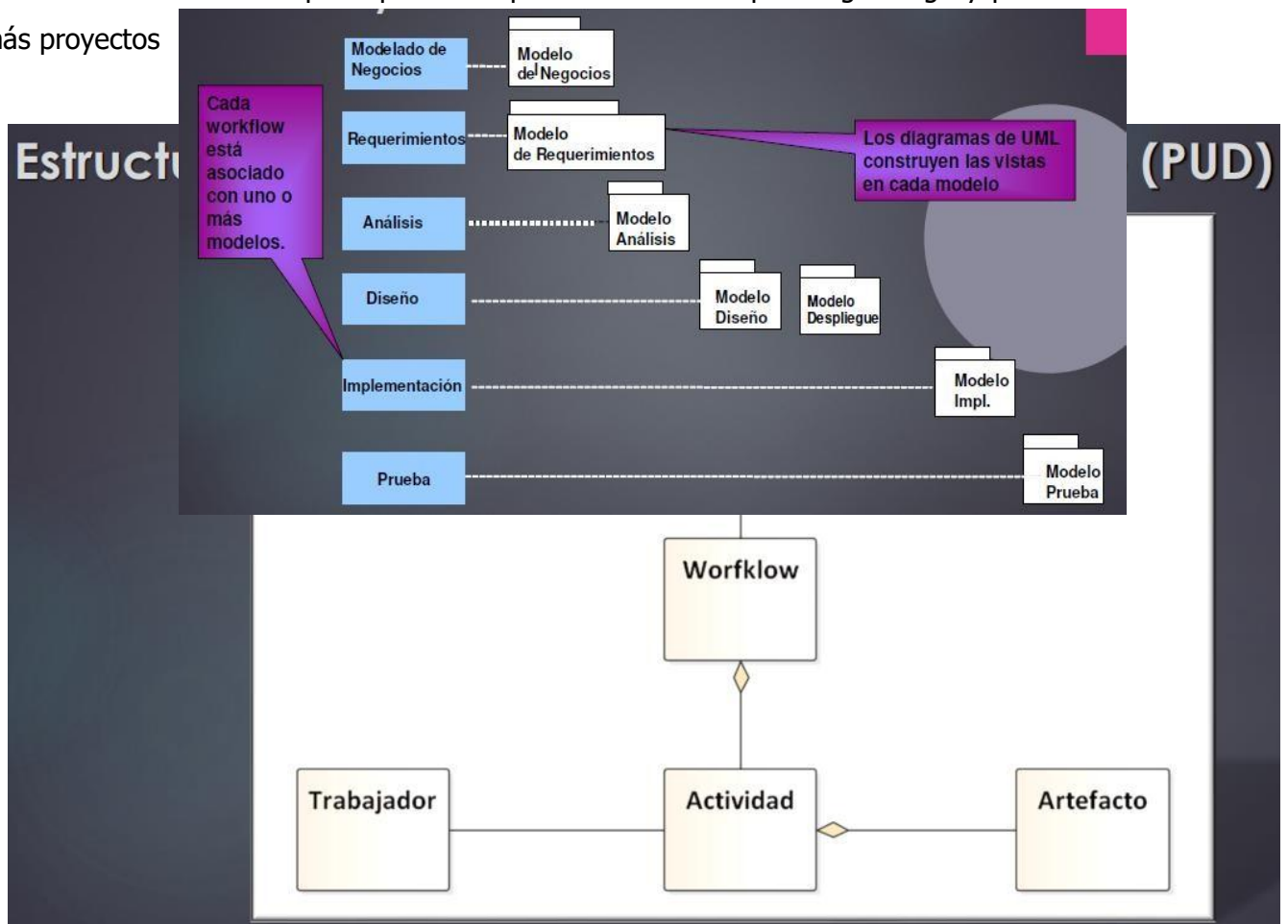
Otra característica del PUD, es que cada WF genera como salida un modelo. El WF de diseño nos entrega 2 modelos, uno que se centra en el software (modelo de diseño) y otro en el hardware (modelo de despliegue), que hace falta para darle soporte al software que diseñamos. Nosotros no diseñamos

hardware, se encarga de la distribución de componentes de software en que elementos de hardware lo vamos a alojar.



Proyecto: Adapta a los procesos a sus necesidades particulares, debe tener objetivos claros, medibles y alcanzables. Un resultado único, fechas de inicio y fin

Proceso: Describe conceptos que es lo que se debe hacer para lograr algo y puede usarse en uno o más proyectos



Estructura de un workflow en el PUD

- **Trabajador:** rol que ocupa una persona en un equipo de desarrollo.

Los procesos siempre se definen en términos de roles porque esto le permite al proceso la flexibilidad de adaptarse a una u otra organización en base a las necesidades (ej: organización

pequeña donde una sola persona desempeña varios roles o, al revés, en una organización grande varias personas desempeñan el mismo rol).

- **Artefacto:** es una pieza de información que es producida, modificada o usada por un proceso. Las características de los artefactos van cambiando de acuerdo al workflow, por ejemplo, los artefactos en el workflow de implementación son piezas de código mientras que en el de análisis son modelos lógicos/conceptuales.

- **Actividad:** es una unidad de trabajo para una persona que actúa como trabajador

Fases: intervalo de tiempo entre dos hitos importantes del proceso, muestran/determinan momentos de evolución del producto en el proyecto.

1. **Inicio:** contribuye a la planificación de los incrementos. El momento de evolución para salir del inicio y entrar a la fase de elaboración es cuando tiene claro el dominio del problema.

2. **Elaboración:** se especifican en detalle la mayoría de los CU (requerimientos) y se diseña la arquitectura. Contribuye a obtener una estructura robusta y estable, facilita una comprensión más profunda de los requerimientos

3. **Construcción:** se crea el producto.

4. **Transición:** el producto se convierte en versión beta. Un número reducido de usuarios con experiencia lo prueba e informa los defectos y deficiencias. Los desarrolladores los corrigen e incorporan mejoras.

Iteración: conjunto bien definido de actividades, con un plan y criterios de evaluación bien establecidos, que acaba en un sistema que puede ejecutarse, probarse y evaluarse.

Interacciones

Interacción: es un comportamiento que incluye un conjunto de mensajes que se intercambian un conjunto de objetos dentro de un contexto para lograr un propósito.

Se utilizan para modelar los aspectos dinámicos de las colaboraciones, que representan sociedades de objetos que desempeñan roles específicos, y colaboran entre sí para desarrollar un comportamiento mayor que la suma de sus elementos. Cada interacción puede modelarse de **dos formas:** destacando la ordenación temporal de los mensajes, o destacando la secuencia de mensajes en el contexto de una organización estructural de objetos.

En UML, los aspectos dinámicos de un sistema se modelan mediante interacciones. Una interacción establece el escenario para su comportamiento presentando todos los objetos que colaboran para realizar alguna acción. Las interacciones incluyen mensajes enviados entre objetos. Un mensaje implica la invocación de una operación o el envío de una señal; un mensaje también puede incluir la creación o destrucción de otros objetos.

Contexto

Una interacción puede aparecer siempre que haya unos objetos enlazados a otros. Las interacciones aparecerán en la colaboración de objetos existentes en el contexto de un sistema o subsistema, en el contexto de una operación y en el contexto de una clase.

Objetos y roles

Los objetos que participan en una interacción son o bien elementos concretos (algo del mundo real) o bien elementos prototípicos (instancias).

En el contexto de una interacción podemos encontrar instancias de clases, componentes, nodos y casos de uso. Aunque las clases abstractas y las interfaces, por definición, no pueden tener instancias directas, podemos representar instancias de estos elementos en una interacción (clases hijas concretas o clases que realicen la interfaz).

Enlaces y conectores

Enlace: conexión semántica entre objetos. Es una instancia de una asociación.

Un enlace especifica un camino a lo largo del cual un objeto puede enviar un mensaje a otro objeto (o a sí mismo).

Mensajes

Mensaje: es la especificación de una comunicación entre objetos que transmite información, con la expectativa de que se desencadenará una actividad.

Cuando se pasa un mensaje, su recepción produce normalmente una acción. Una acción puede producir un cambio en el estado del destinatario y en los objetos accesibles desde él.

- Se pueden modelar varios tipos de acciones:
- Llamada: invoca una operación sobre un objeto.
- Retorno: devuelve un valor al invocador.
- Envío: envía una señal a un objeto.
- Creación: crea un objeto.
- Destrucción: destruye un objeto.

El tipo más común de mensaje que se modelará será la llamada, en la cual un objeto invoca una operación de otro objeto (o de él mismo).

Los mensajes también pueden corresponderse con el envío de señales. Una señal es un valor objeto que se comunica de manera asíncrona a un objeto destinatario. Después de enviar la señal, el objeto que la envió continúa su propia ejecución. Cuando el objeto destinatario recibe la señal, él decide qué hacer con ella. Normalmente las señales disparan transiciones en la máquina de estados del receptor.

Secuenciación

Cuando un objeto envía un mensaje a otro, el objeto receptor puede, a su vez, enviar un mensaje a otro objeto, el cual puede enviar un mensaje a otro objeto diferente, y así sucesivamente. Este flujo de

mensajes forma una secuencia. El inicio de cada secuencia tiene su origen en algún proceso o hilo que la contiene.

Los mensajes se ordenan en secuencia conforme sucedan en el tiempo. Para visualizar mejor la secuencia de mensajes, se puede expresar explícitamente la posición de un mensaje con relación al inicio de la secuencia, precediéndolo de un número de secuencia, separado por dos puntos.

Creación, modificación y destrucción

La mayoría de las veces, los objetos que participan en una interacción existen durante todo el tiempo que dura la interacción. Sin embargo, algunas interacciones pueden conllevar la creación y destrucción de objetos. Esto también se aplica a los enlaces.

En un diagrama de secuencia se representan de manera explícita la vida, la creación y destrucción de objetos o roles a través de la longitud vertical de sus líneas de vida. Dentro de un diagrama de comunicación, la creación y la destrucción deben indicarse con notas.

Representación – Diagramas de Interacción

Una forma de modelar los aspectos dinámicos de un sistema es construir representaciones gráficas de escenarios que impliquen la interacción de ciertos objetos interesantes y los mensajes que se envían. En UML, estas representaciones gráficas se modelan con los diagramas de interacción. Estas representaciones se pueden construir de dos formas: destacando la ordenación temporal de sus mensajes, o destacando la organización estructural de los roles que envían y reciben mensajes. El primero se llama diagrama de secuencia, y el segundo, diagrama de comunicación.

Los diagramas de secuencia y de comunicación son similares, lo que significa que podemos partir de uno de ellos y transformarlo en el otro, aunque a veces muestren información diferente. Los diagramas de secuencia permiten modelar la línea de vida de un objeto, mientras que los de comunicación permiten modelar los enlaces estructurales que pueden existir entre los objetos de la interacción.

Un diagrama de interacción es un tipo especial de diagrama y comparte las propiedades comunes al resto de los diagramas (un nombre y un contenido gráfico que es una proyección de un modelo). Lo que distingue a un diagrama de interacción de los otros tipos de diagramas es su contenido. Normalmente, los diagramas de interacción contienen: roles u objetos, comunicaciones o enlaces y mensajes (definido anteriormente).

MÁQUINAS DE ESTADOS

Es un comportamiento que especifica la secuencia de estados por los que pasa un objeto durante su vida, en respuesta a eventos, junto con sus respuestas a esos eventos. Se utilizan para modelar los aspectos dinámicos de un sistema, lo que implica modelar la vida de las instancias de una clase, un CU o un sistema completo, las cuales pueden responder a eventos tales como señales, operaciones o el paso del tiempo. Mientras una interacción modela una sociedad de objetos que colaboran para llevar a cabo

una acción, una máquina de estados modela la vida de un único objeto, sea una instancia de una clase, un CU o un sistema completo. Durante su vida, un objeto está expuesto a varios tipos de eventos y como respuesta, reacciona con alguna acción, y cambia su estado a un nuevo valor.

Estado: es una condición o situación en la vida de un objeto durante el cual satisface alguna condición, realiza alguna actividad o espera algún evento.

Evento: es la especificación de un acontecimiento significativo situado en el tiempo y en el espacio. Es la aparición de un estímulo que puede disparar una transición de estados (o no). Pueden incluir señales, llamadas, el paso del tiempo o un cambio en el estado.

Transición: es una relación entre dos estados que indica que un objeto que esté en el primer estado realizará ciertas acciones y entrará en el segundo estado cuando ocurra un evento especificado y se satisfagan ciertas condiciones especificadas. Cuando se produce ese cambio de estado se dice que la transición se ha disparado. Hasta que se dispara el objeto está en el estado origen y después de dispararse está en el estado destino.

Actividad: es una ejecución no atómica en curso, dentro de una máquina de estados.

Acción: es una computación atómica ejecutable que produce un cambio en el estado del modelo o devuelve un valor.

Contexto

Cada objeto tiene una vida, se crea, se destruye y entretanto puede actuar sobre otros objetos y los demás pueden actuar sobre él. Las máquinas de estado se utilizan para especificar el comportamiento de los objetos que deben responder a estímulos asíncronos o cuyo comportamiento actual depende de su pasado es utilizar una máquina de estados. También se utilizan para modelar el comportamiento de sistemas completos, especialmente sistemas que deben responder a señales de actores externos.

Estados

Un objeto permanece en un estado durante una cantidad de tiempo finita. Cuando la máquina de estados de un objeto se encuentra en un estado dado, se dice que el objeto está en ese estado.

Un estado se representa con un rectángulo con las esquinas redondeadas. Un estado tiene varias partes:

- **Nombre:** cadena de texto que distingue al estado de otros. Un estado puede no tener un nombre (anónimo).
- **Efectos de entrada/salida:** acciones ejecutadas al entrar y salir del estado. Permite realizar alguna configuración cada vez que se entra en un estado o, al abandonar un estado se desea realizar una acción de limpieza. En UML esto se indica con una etiqueta entry y exit respectivamente junto a una acción apropiada.
- **Transiciones internas:** transiciones que se manejan sin causar un cambio de estado. Es distinto a una auto transición porque en esta, un evento dispara la transición, se abandona el

estado, se ejecuta una acción y se vuelve a entrar al mismo estado. En cambio, en una transición interna no se ejecutan acciones de entrada y salida. Se identifica incluyendo su texto dentro del símbolo del estado.

- **Eventos diferidos:** lista de eventos que no se manejan en este estado, sino que se posponen y se añaden a una cola para ser manejado por el objeto en otro estado. El evento diferido se mantiene hasta que se encuentre finalmente un estado donde el evento no esté diferido. Se especifica listando el evento con la acción especial defer.

- **Subestados – Estado compuesto:** estructura anidada de un estado, que engloba subestados disjuntos (activos secuencialmente – no ortogonales) o concurrentes (activos concurrentemente - ortogonales). Es un estado anidado dentro de otro. Se representa con un compartimiento gráfico opcional que muestra una máquina de estados anidada. Es un estado que tiene una máquina de estados dentro y que va a manejar los estados que son propios de él.

Otras características avanzadas:

- **Actividades:** cuando un objeto está en un estado, normalmente se encuentra ocioso esperando a que ocurra un evento. A veces puede que se quiera modelar una actividad en curso, mientras está en el estado, el objeto realiza alguna tarea que continuará hasta que sea interrumpido por un evento. En UML se utiliza la transición especial do para especificar el trabajo que se realizará en ese estado. También se puede especificar un comportamiento como una secuencia de operaciones.

- **Estados de historia:** cuando una transición entra en un estado compuesto la acción de la máquina de estados anidada comienza de nuevo en su estado inicial, a menos que se especifique lo contrario. A veces se desea modelar un objeto de forma que recuerde el último subestado que estaba activo antes de abandonar el estado compuesto. En UML una forma de modelar esto es utilizar los estados de historia que permite que un estado compuesto que contiene subestados secuenciales recuerde el último subestado activo antes de la transición que provocó la salida del estado compuesto. Se representa con un pequeño círculo que contiene el símbolo H.

En la máquina de estados de un objeto se pueden definir tres PSEUDOESTADOS:

- **Pseudoestado:** indicadores de cierta condición que cumple el estado al que están asociados. Pueden tener las partes habituales de un estado excepto el nombre.

- **Estado inicial:** indica el punto de comienzo por defecto para la máquina de estados o del subestado. Se representa mediante un círculo negro. Está indicando cuál es el estado en el que va a crear el objeto.

- **Estado final:** indica que la ejecución de la máquina de estados o del estado que lo contiene ha finalizado. Se representa con un círculo negro dentro de un círculo blanco. Indica que no se puede hacer más transiciones cuando un objeto llega al estado indicado como estado final.

- **Exit point:** Es un pseudoestado y estado final usado en el contexto de un estado compuesto y es un punto de salida anticipado. Indica una interrupción que hace que se

salte directamente al final y se pueda salir a la máquina de estados principal.

- **Historia:** descripto anteriormente. Las transiciones a este pseudoestado son las que indican qué estado necesita memoria para poder restaurar al objeto en el estado donde se encontraba.

Transiciones: se representa con una línea continua desde el estado origen hacia el destino. Una autotransición es una transición donde el estado origen y el estado destino son el mismo. Tiene cinco partes:

- Estado origen: el estado afectado por la transición.
- Evento de disparo: el evento cuya recepción por el objeto que está en el estado origen provoca el disparo de la transición si se satisface su condición de guarda. Se puede incluir una transición de terminación, es una transición sin evento de disparo. Ésta se dispara implícitamente cuando su estado origen ha completado su comportamiento.
- Condición de guarda: expresión booleana que se evalúa cuando la transición se activa por la recepción del evento de disparo, si es verdadero la transición se dispara, si es falso, no se dispara. Se coloca entre corchetes tras el evento de disparo. Sólo se evalúa después de ocurrir el evento de disparo. Se evalúa una vez por cada transición, puede volver a evaluarse si la transición se vuelve a disparar.
- Efecto: un comportamiento ejecutable, como una acción. Es un comportamiento que se evalúa cuando se dispara una transición, puede incluir una llamada a operaciones sobre el objeto de la máquina de estados o sobre otros objetos visibles, la creación o destrucción de otro objeto, el envío de una señal o una computación en línea. Las transiciones ocurren solo cuando la máquina de estados está inactiva, es decir cuando no se está ejecutando el efecto de una transición previa. No se permiten interrupciones de nuevos eventos que causen transiciones adicionales.
- Estado destino: el estado activo tras completarse la transición.

Una transición se representa con una línea continua dirigida desde el estado origen hacia el destino. Una autotransición es una transición donde el estado origen y el estado destino son el mismo.

NOTA: a los diagramas de interacción se le agregan los diagramas de timing y los diagramas de descripción de interacción.

El diagrama de una máquina de estados es a nivel de clase, mientras que el de timing es a nivel de instancia (o sea a nivel de objeto).

Workflows del PUD

- **WF de Modelado de negocios (Opcional)**
- **WF de Requerimientos → Modelan el dominio del problema**
- **WF de Análisis (Puede ser Opcional) → Modela el dominio de la solución**
- **WF de Diseño → Modela el dominio de la solución**
- **WF de Implementación**
- **WF de Prueba → 4 niveles de testing (Prueba unitaria, integración, de sistema, aceptación de usuario)**

• **WF de Despliegue (Opcional) → Transferir la versión del producto al usuario final**

WORKFLOW DE ANALISIS

El "Workflow de Análisis" en el Proceso Unificado de Desarrollo (PUD) es una fase clave donde se examinan, refinan y estructuran los requerimientos previamente capturados en el workflow de requerimientos. Esta etapa tiene como principal objetivo lograr una comprensión más precisa y detallada de los requerimientos, facilitando una descripción que sea fácil de mantener y útil para la estructuración global del sistema. Durante este proceso, se abordan y resuelven las ambigüedades o incertidumbres remanentes de la etapa de captura de requerimientos, donde a menudo se utiliza un lenguaje más intuitivo, pero menos preciso. En el workflow de análisis, se adopta un enfoque más técnico y detallado, utilizando el lenguaje y los términos propios del desarrollo, lo que permite un análisis más profundo y lógico de los requerimientos para una mejor implementación y diseño del sistema. Habla en términos conceptuales (lógicos).

Modelo de casos de uso	Modelo de análisis
Descrito con el lenguaje del cliente.	Descrito con el lenguaje del desarrollador.
Vista externa del sistema.	Vista interna del sistema.
Estructurado por los casos de uso; proporciona la estructura a la vista externa.	Estructurado por clases y paquetes estereotipados; proporciona la estructura a la vista interna.
Utilizado fundamentalmente como contrato entre el cliente y los desarrolladores sobre qué debería y qué no debería hacer el sistema.	Utilizado fundamentalmente por los desarrolladores para comprender cómo debería darse forma al sistema, es decir, cómo debería ser diseñado e implementado.
Puede contener redundancias, inconsistencias, etc., entre requisitos.	No debería contener redundancias, inconsistencias, etc., entre requisitos.
Captura la funcionalidad del sistema, incluida la funcionalidad significativa para la arquitectura.	Esboza cómo llevar a cabo la funcionalidad dentro del sistema, incluida la funcionalidad significativa para la arquitectura; sirve como una primera aproximación al diseño.
Define casos de uso que se analizarán con más profundidad en el modelo de análisis.	Define realizaciones de casos de uso, y cada una de ellas representa el análisis de un caso de uso del modelo de casos de uso.

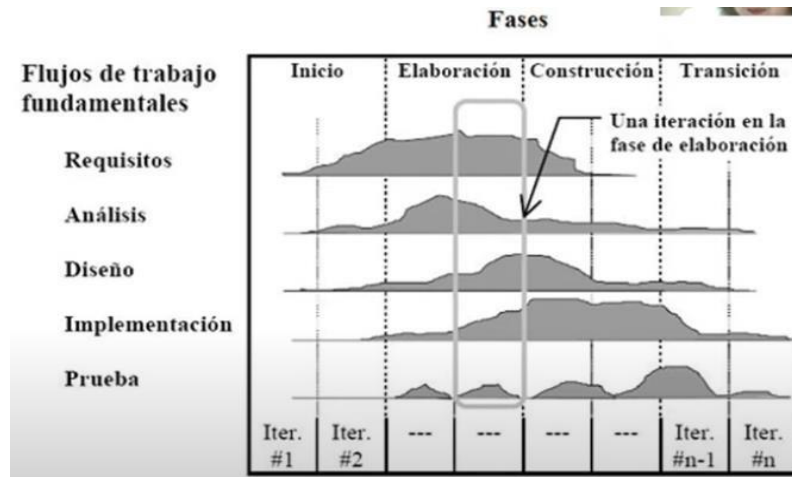
Beneficio de porque hacer análisis:

- ★ Podemos analizar gran parte del sistema sin grandes costes. Se puede utilizar el resultado para planificar el trabajo de diseño e implementación subsiguiente en varios incrementos sucesivos, donde cada uno diseña e implementa una pequeña parte del sistema.
- ★ El análisis proporciona una visión más general del sistema, ya que al no hacer análisis puede ser difícil de obtener mediante el estudio de los resultados del diseño y la implementación, ya que son muy detallados. Una visión general puede ser muy valiosa para recién llegados al sistema o para desarrolladores que en general mantienen el sistema.
- ★ El modelo de análisis puede proporcionar una vista conceptual, precisa y unificadora de las implementaciones alternativas.
- ★ El sistema se construye utilizando un sistema heredado complejo, cuya reingeniería puede hacerse en

términos del modelo de análisis donde los desarrolladores puedan comprender el sistema heredado sin profundizar en detalles de diseño e implementación y utilizarlo como un bloque de construcción reutilizable.

¿Cuál es el rol del análisis en el ciclo de vida iterativo e incremental del PUD?

El foco del análisis está puesto en las primeras iteraciones de la fase de elaboración (donde tiene el pico máximo de la curva de análisis), tiene incidencia en la fase de inicio, pero conforme va saliendo de la fase de elaboración ya no tiene tanta incidencia en el de construcción. Esta curva del análisis va cambiando de acuerdo a las 3 opciones que tenemos de uso del análisis en el contexto de Proceso Unificado



Como utilizar el análisis:

Mantener el Modelo de Análisis a lo Largo del Ciclo de Vida del Software: En esta aproximación, el proyecto utiliza el modelo de análisis para detallar los resultados del análisis, manteniendo y actualizando este modelo durante todo el ciclo de vida del software. Esta opción garantiza consistencia y una referencia continua a lo largo de las diversas etapas del desarrollo.

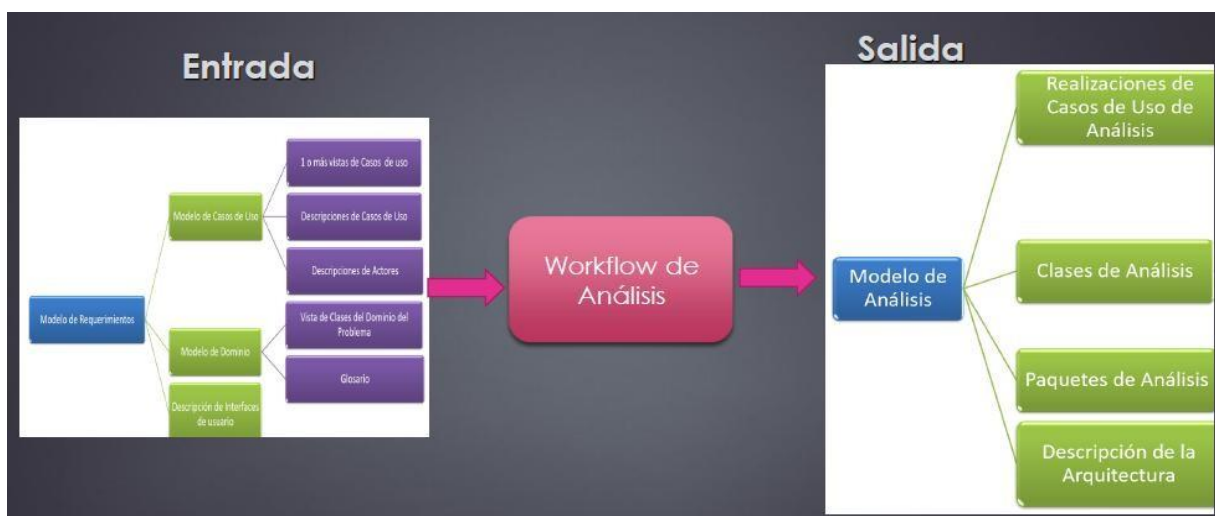
Uso Transitorio del Modelo de Análisis: Aquí, el modelo de análisis se utiliza para describir los resultados del análisis, pero se considera como una herramienta intermedia. Una vez que la fase de diseño y desarrollo comienza, el modelo de análisis deja de actualizarse. Esta opción es comúnmente preferida, especialmente cuando el análisis se enfoca principalmente en la fase de elaboración, pues permite una transición eficiente a la fase de diseño y construcción.

Integración del Análisis con el Diseño:

En esta modalidad, el proyecto no separa el análisis del diseño. Los requerimientos funcionales, no funcionales y las restricciones de negocio se resuelven en un modelo unificado. Aunque esta opción puede resultar más compleja debido a la falta de separación entre funcionalidades, puede ser adecuada para sistemas excepcionalmente simples o en casos donde el análisis está embebido en el diseño. Es la opción por la cual opta el RUP

	Modelo de Requerimientos	Modelo de Análisis
¿Qué hace? ¿Qué representa?	Captura la funcionalidad del sistema. Se obtienen los RF, RNF, reglas de negocio, etc, resultado del relevamiento con el Cliente	Esboza o diagrama como llevar a cabo la fucionalidad del sistema (RF) sirviendo como aproximacion al diseño pero abstrayéndose de los detalles de implementación.
Punto de vista del sistema	Vista externa del sistema.	Vista interna del sistema.
¿Qué define?	Define casos de uso.	Define realizaciones de casos de uso.
¿Cómo está estructurado?	Estructurado por casos de uso. Proporciona la estructura a la vista externa.	Estructurado por clases y paquetes estereotipados. Proporciona la estructura a la vista interna.
¿Cual es su utilidad o finalidad?	Es utilizado fundamentalmente como contrato entre el cliente y los desarrolladores sobre QUÉ debería y QUÉ NO debería hacer el sistema.	Es utilizado fundamentalmente por los desarrolladores para comprender CÓMO debería darse forma al sistema, es decir, CÓMO debería ser diseñado e implementado.
Lenguaje utilizado para comunicar y describir	Lenguaje del Cliente.	Lenguaje de los desarrolladores.
Precisión en la comunicación	Puede contener redudancia e inconsistencia.	No debería contener redudancia e inconsistencias.
¿En qué fase o fases del PUD está más presente?	Inicio y Elaboración.	Elaboración.
¿Es opcional?	No. Siempre se debe conocer los requerimientos del software que queremos construir.	Se recomienda que se realice cómo un workflow en sí y separado de los requerimientos o diseño. Pero para sistemas muy sencillos o de un equipo de desarrolladores expertos en el dominio, se lo puede fusionar con el anterior o siguiente.

WORKFLOW DE ANÁLISIS COMO PROCESO



Artefactos (Cualquier pieza de código que es producida o modificada por un proceso)

Modelo de análisis

Es un modelo conceptual o lógico de objetos que desempeña un papel fundamental en la refinación y

estructuración de los requerimientos funcionales (RF), mientras que pospone los requerimientos no funcionales (RNF) y restricciones para etapas posteriores. Este modelo actúa como una herramienta clave para razonar sobre los aspectos internos del sistema, proporcionando una estructura enfocada en el mantenimiento, la flexibilidad ante cambios y la reutilización.

Aunque el modelo de análisis se considera una primera aproximación al modelo de diseño, es un modelo independiente por sí mismo. Sirve como entrada importante para las actividades de diseño e implementación, intentando preservar su estructura a medida que se le da forma al sistema. Sin embargo, no siempre es posible mantener esta estructura intacta, ya que el diseño debe tener en cuenta factores como el lenguaje de programación, los sistemas operativos, frameworks, y sistemas heredados.

Clase de análisis (Fabricación pura)

Representa una abstracción de una o varias clases del diseño del sistema. Se centra en los requisitos funcionales, lo que provoca que tenga mayor granularidad. Su comportamiento se refleja en responsabilidades y suelen poseer atributos de alto nivel. Lo mismo ocurre con las relaciones.

Características clave de las Clases de Análisis incluyen:

Comportamiento y Responsabilidades: Se definen en un alto nivel y de manera menos formal, no mediante interfaces detalladas en términos de operaciones y sus firmas.

Atributos: Se definen también en un alto nivel, con tipos conceptuales reconocibles en el dominio del problema, a diferencia del diseño donde los tipos pertenecen al lenguaje de programación.

Relaciones: Participan en relaciones más conceptuales que las relaciones de diseño e implementación.

Pueden ser de 3 tipos:

Clases de Interfaz (Boundary): Modelan la interacción entre el sistema y sus actores, incluyendo la recepción y presentación de información y peticiones. Suelen representar abstracciones de interfaces de usuario y comunicaciones, asociándose directamente con los actores. Son fronteras entre el sistema y el exterior con el que interactúa

Clases de Control (Gestor): Encapsulan la coordinación, secuencia y control de otros objetos, enfocándose en casos de uso específicos, derivaciones y cálculos complejos. Son el nexo entre las clases de entidad y de interfaz, manejando los aspectos dinámicos del sistema.

Clases de Entidad: Se enfocan en modelar información con una vida larga y que suele ser persistente. Reflejan conceptos o fenómenos del mundo real, siendo candidatas a ser persistidas en bases de datos. Difieren de las clases de entidad de negocio en que consideran objetos manejados por el sistema.

Realización de caso de uso de análisis

Proporcionan una traza hacia un caso de uso, es una colaboración dentro del modelo de análisis en términos de las clases de análisis. Describe como se lleva a cabo y se ejecuta un caso de uso en términos de clases de análisis y sus objetos de análisis en interacción. Modela aspectos dinámicos del sistema y se hace a través del diagrama de comunicaciones o diagrama de secuencia. Son el artefacto principal de la salida del modelo de análisis. Las mismas tienen una parte estática, que se modela con el diagrama de clases y, una parte dinámica que se modela con un diagrama de comunicación o de secuencia.

Paquetes de análisis

Los paquetes del análisis proporcionan un medio para organizar los artefactos del modelo de análisis en piezas manejables. Puede estar compuesto de clases de análisis, de realizaciones de casos de uso de análisis y de otros paquetes (recursivamente). Deberían estar débilmente acoplados y tener alta cohesión. Deberían estar enfocados en los requerimientos funcionales y en el dominio del problema. Proporcionan un medio para organizar los artefactos del modelo de análisis en piezas manejables, pueden contener clases de análisis, realizaciones de caso de uso de análisis y otros paquetes.

- o Pueden representar una separación de intereses de análisis.
- o Deberían crearse basándose en los RF y en el dominio del problema.
- o Probablemente se convertirán en subsistemas o se distribuirán entre ellos

Descripción de la arquitectura

La descripción de la arquitectura contiene una vista de la arquitectura del modelo de análisis, que muestra sus artefactos significativos para la arquitectura.

Artefactos que se consideran significativos para la arquitectura:

- La descomposición del modelo de análisis en paquetes de análisis y sus dependencias.
- Las clases fundamentales del análisis (entidad, control e interfaz).
- Realizaciones de caso de uso que describen cierta funcionalidad importante y crítica.

Trabajadores

Arquitecto

Responsable de la integridad del modelo de análisis, garantizando que sea correcto, consistente y legible como un todo. Esto ocurre cuando el mismo realiza la funcionalidad descrita en el modelo de casos de uso, y sólo esa funcionalidad. También se ocupa de la arquitectura del modelo. El arquitecto también es responsable de la arquitectura, es decir, de la existencia de sus partes significativas para la arquitectura tal y como se muestran en la vista de la arquitectura del modelo.

Tiene responsabilidad por el planteo general del modelo de análisis, pero no es el responsable directo de la generación de los artefactos principales del análisis que son las realizaciones de caso de uso y las clases, es el responsable de delinearlas (identifica las clases de entidad), pero sí es responsable de la vista arquitectónica del análisis, ya que es un proceso centrado en la arquitectura (una de las 3 características).

Ingeniero de casos de uso

Responsable de la integridad de las realizaciones de casos de uso, garantizando que cumplen los requerimientos que recaen sobre ellos. También diseñará estas realizaciones. Modela los escenarios más importantes, no modela todos, identifica clases de control e interfaz. Toma como entrada el modelo de casos de uso, requisitos, modelo de dominio y la vista del modelo de análisis. Tiene las 2 vistas, estática (diagrama de clases) y dinámica (diagrama de comunicación y secuencia)

Ingeniero de componentes

Define y mantiene las responsabilidades, relaciones, s y requerimientos especiales de una o varias clases del análisis, asegurándose de que cada clase del análisis cumple con los requerimientos que se esperan de ella de acuerdo a las realizaciones de caso de uso en las que participa. Se encarga de mantener la integridad de los paquetes de análisis y clases de análisis.

Flujo de trabajo

Los arquitectos comienzan definiendo la arquitectura del modelo, identificando los paquetes y clases evidentes y los requisitos comunes. Luego, el ingeniero de casos de uso realiza los casos de uso y el Ing.



Actividades

Análisis de la arquitectura ► Arquitecto

Se busca esbozar el modelo de análisis y la arquitectura en paquetes de análisis, clases de análisis evidentes y requerimientos especiales comunes. Paquetes de análisis, de servicio... dependencias entre



Autor: Federico Lovera Baldi

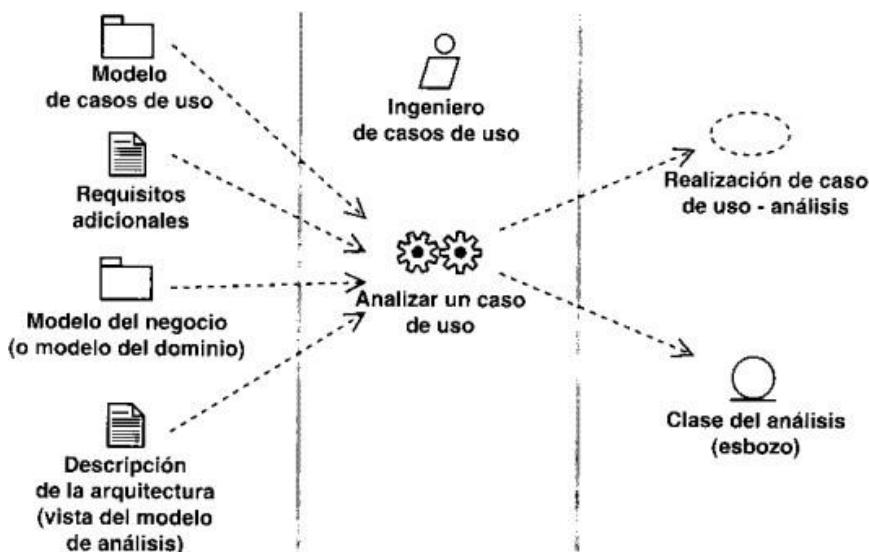
paquetes.

Identificación de paquetes del análisis Forma de dividir el trabajo. Una identificación inicial de los paquetes se basa en los RF y el dominio del problema: se asigna un número de CU a un paquete y luego se realiza la funcionalidad correspondiente al mismo. Pueden hacerse para:

- Dar soporte a un determinado proceso de negocio
- Dar soporte a un determinado actor del sistema
- Los casos de uso que están relacionados mediante relaciones de generalización y de extensión.

Identificación de clases de entidad evidentes Consiste en una propuesta preliminar de las clases de entidad más importantes basado en las clases del dominio o las entidades de negocio que se identificaron durante la captura de requerimientos.

Identificación de requerimientos especiales comunes (RNF) Un requerimiento especial es un requerimiento que aparece durante el análisis y que es importante anotar de forma que pueda ser tratado adecuadamente en las actividades de diseño e implementación (req. No funcionales).



Análisis de un caso de uso ► Ing. De Casos de Uso

Identificar clases de análisis, clases de entidad, control e interfaz, cuyos objetos son necesarios para realizar el CU. Se esbozan sus nombres, responsabilidades, atributos y relaciones.

Descripción de interacciones entre objetos del análisis: Se distribuye el comportamiento del CU entre los objetos del análisis que interactúan, mediante diagramas de colaboración que contienen instancias de actores participantes, objetos del análisis y sus enlaces.

Capturar requisitos funcionales del caso de uso: Se capturan los requerimientos no funcionales que aparecen, pero se los posterga para el diseño e implementación.

Análisis de una clase ► Ing. De Componentes

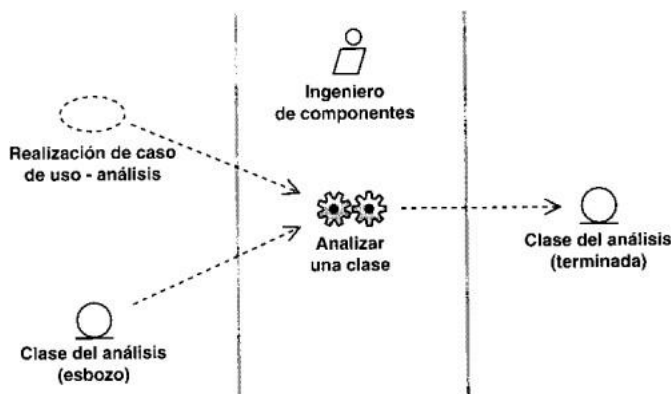
✓ **Identificar responsabilidades** de una clase de análisis y mantenerlas, combinando todos los roles que cumple en diferentes realizaciones de CU. Se obtiene de los diagramas de interacción y del diagrama de clases.

✓ **Identificar atributos** que especifican propiedades de una clase de análisis, y son necesarios para las responsabilidades de su clase. Los atributos de las clases de interfaz suelen ser elementos de información (etiquetas, combo box, list box). Los atributos de las clases de control son valores acumulados o calculados en la realización de caso de uso.

✓ **Identificar asociaciones, agregaciones y generalizaciones**, se estudian los diagramas de colaboración para determinar las asociaciones necesarias. El ingeniero de componentes debería estudiar los enlaces empleados en los diagramas de interacción para determinar qué asociaciones son necesarias reduciendo al mínimo las relaciones entre clases. También debe definir la multiplicidad. Las generalizaciones deberían utilizarse en el análisis para extraer comportamiento compartido y común entre varias clases de análisis diferentes. Deben mantenerse en un nivel alto y conceptual para facilitar la comprensión del modelo de análisis. Luego en el diseño las generalizaciones se ajustan para que encajen con el lenguaje de programación.

✓ **Captura de requerimientos especiales:** se postergan para el diseño e implementación.

✓ **Identificar responsabilidades** Las responsabilidades de una clase pueden recopilarse combinando todos los roles que cumple en diferentes realizaciones de caso de uso.

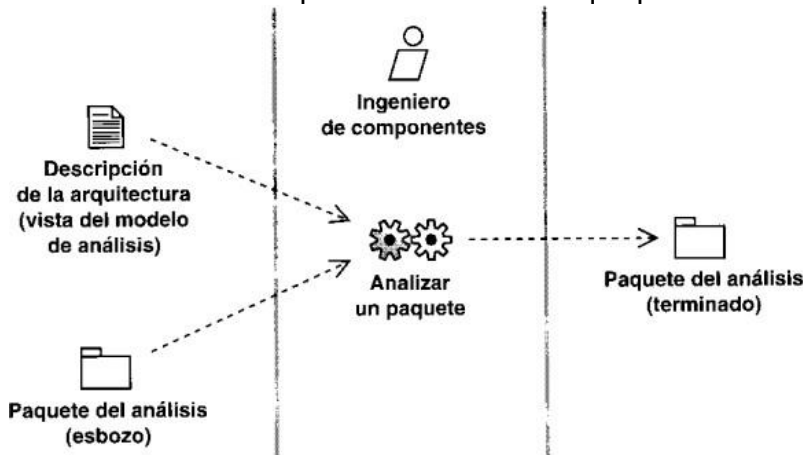


Análisis de un paquete ► Ing. De Componentes

Se encarga de mantener su integridad, describiendo las dependencias de forma que pueda estimarse el efecto de los cambios futuros y garantizando:

1. Que sea tan independiente de otros paquetes como sea posible

2. Que cumple su objetivo de realizar algunas clases de dominio o CU.
3. Describir las dependencias de forma que pueda estimarse el efecto de los cambios futuros

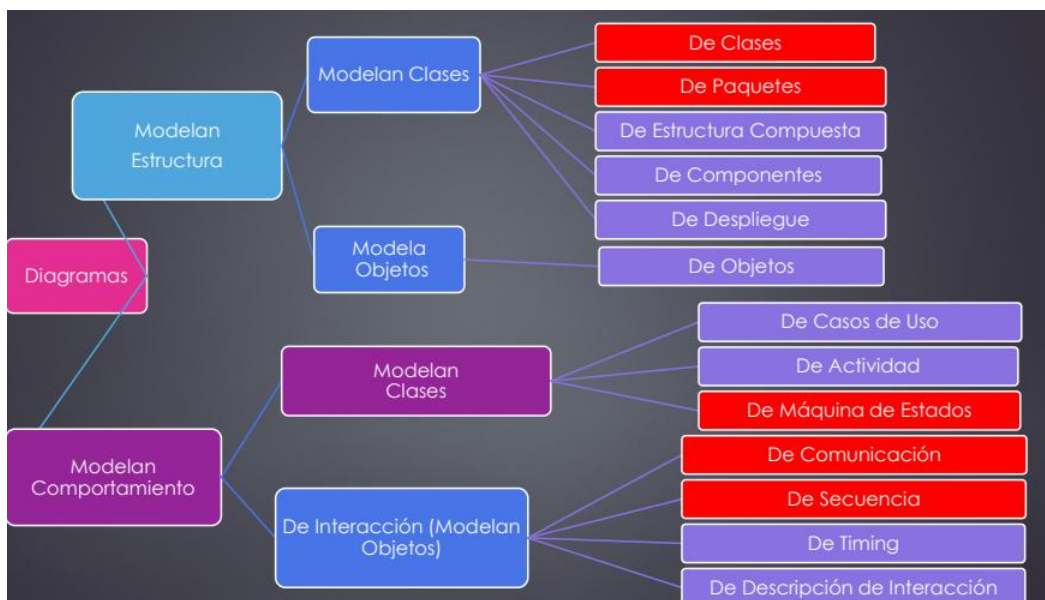


RESUMEN DEL ANÁLISIS

El resultado del flujo de trabajo del análisis es el modelo de análisis, que es un modelo de objetos conceptual que analiza los requisitos mediante su refinamiento y estructuración. El modelo de análisis incluye:

- Paquetes del análisis: con su contenido y dependencias.
- Clases del análisis con sus atributos, responsabilidades, relaciones y requerimientos no funcionales.
- Realizaciones de casos de uso de análisis.
- Vista de la arquitectura

El modelo de análisis se considera la entrada fundamental para las actividades de diseño subsiguientes.



Un patrón es una descripción de un problema y su solución a la que se le da un nombre y se puede

aplicar en nuevos contextos. Los patrones GRASP son una codificación de principios básicos.

Lo importante de los patrones no es expresar nuevas ideas de diseño, sino justamente lo contrario. Los patrones pretenden codificar conocimientos, estilos y principios existentes y que se han probado que son válidos. Todos los patrones, idealmente, tienen nombres sugerentes que apoyan la identificación e incorporación de ese concepto en nuestro conocimiento y memoria, y facilita la comunicación.

Patrones GRASP: Son patrones que nos ayudan a modelar las realizaciones de caso de uso de análisis, para ayudarnos a definir a donde vamos a determinar la responsabilidad, o sea donde vamos a asignar un método en que clase es más conveniente ubicar un determinado comportamiento.

Responsabilidades y métodos

UML define responsabilidad como “un contrato u obligación de un contratador”. Están relacionadas con las obligaciones de un objeto en cuanto a su comportamiento. No es lo mismo que un método: estos se implementan para llevar a cabo responsabilidades. Las responsabilidades se implementan utilizando métodos que actúan solos o colaboran.

Responsabilidades de hacer

- **Hacer algo él mismo:** Crear un objeto o hacer un cálculo, responder peticiones.
- **Iniciar una acción en otro objeto:** Delegar responsabilidades. Ej: Cuando un gestor crea un objeto
- **Controlar actividades en otro objeto:** Clases de control, Gestores

Responsabilidades de conocer

- **Conocer datos privados encapsulados:** Conocer datos propios como nombre, apellido, dni
- **Conocer objetos relacionados:** Objetos con los que está relacionado. Son los métodos que me permiten a mi tener visibilidad de los objetos con los que yo ya tengo una relación. Permite conocer los atributos de las clases relacionadas (Ej get())
- **Conocer información que puede calcular:** Los atributos que son derivables o los que se pueden calcular, no deben existir como atributos en una clase, tiene que estar el método que calcule ese atributo. Ej FechaNacimiento(), no usar edad como atributo.

GRASP: Patrones de principios generales del diseño de objetos y de asignación de responsabilidades. Estos patrones se deben utilizar en diagramas de interacción.

1. Experto en información.
2. Creador.
3. Alta cohesión.
4. Bajo acoplamiento.
5. Controlador.

Experto en información

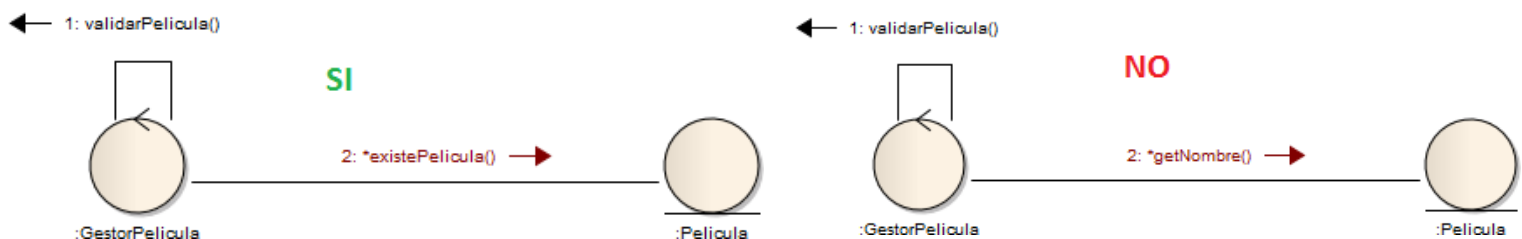
Durante el modelo de diseño, se definen muchas clases con cientos de responsabilidades. Si no sabemos a qué clase le correspondería una responsabilidad, aplicamos este patrón, que nos dice que debemos asignar la responsabilidad al experto, quien tiene la información para realizar lo que se necesita.

Asignar una responsabilidad a la clase que tiene la información necesaria para realizarla. Poner el comportamiento (métodos) lo más cerca posible de los datos, en la clase que los tiene.

“Poner el comportamiento lo más cerca posible de los datos”, o sea, poner los métodos en la clase que tiene los datos para cumplir con el comportamiento

Está relacionado a los principios bajo acoplamiento y alta cohesión.

Se mantiene el encapsulamiento y la legibilidad.



En el primer ejemplo, el gestor “junta” todos los nombres de las películas y luego resuelve el problema el mismo (verificar si existe).

En el segundo ejemplo, se delega la responsabilidad a la clase Película, donde directamente el gestor le pregunta si existe, y la misma Película, utilizando su atributo nombre se encargará de resolver el problema.

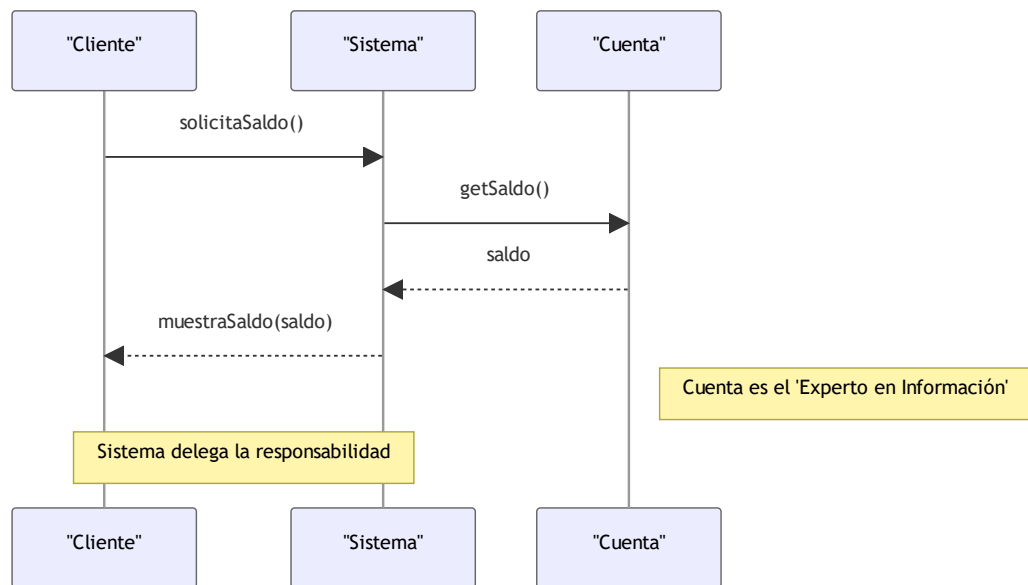
Beneficios:

- Se mantiene el encapsulamiento de la información, puesto que los objetos utilizan su propia información para llevar a cabo las tareas. Esto conlleva a un bajo acoplamiento, lo que da lugar a sistemas más robustos y fáciles de mantener.

- Se distribuye el comportamiento entre las clases que contienen la información requerida, por tanto, se estimula las definiciones de clases más cohesivas y ligeras que son más fáciles de entender y mantener.

Ejemplo:

- Un Cliente solicita el saldo a un Sistema.
- El Sistema, a su vez, delega esta solicitud a una Cuenta, que es el "Experto en Información" en este contexto, ya que posee la información necesaria (el saldo).
- La Cuenta devuelve el saldo al Sistema.
- Finalmente, el Sistema muestra el saldo al Cliente.



Creador

Es un tipo especial del patrón experto. Resuelve a quien le da la responsabilidad de invocar ese `new()`, que cada clase tiene su `new()`. Una clase instancia un objeto a través del `new()`. La decisión es quien invoca el constructor de esa clase para crear el objeto, esto nos ayuda a decidir el patrón creador.

Cuando no estamos seguros de quién debería tener la responsabilidad de crear una instancia de un objeto.

B debería crear la instancia de la clase A si:

1. B agrega objetos de A: B es el todo, está formado por objetos de A. En el caso de agregación, la responsabilidad de las partes de A, se las tengo q dar a B (al todo). Tiene responsabilidad de creación sobre sus partes.
2. B contiene objetos de A: Relación de composición, contención. El todo tiene responsabilidad de creación de sus partes
3. B registra instancias de objetos de A
4. B utiliza muchos objetos de A

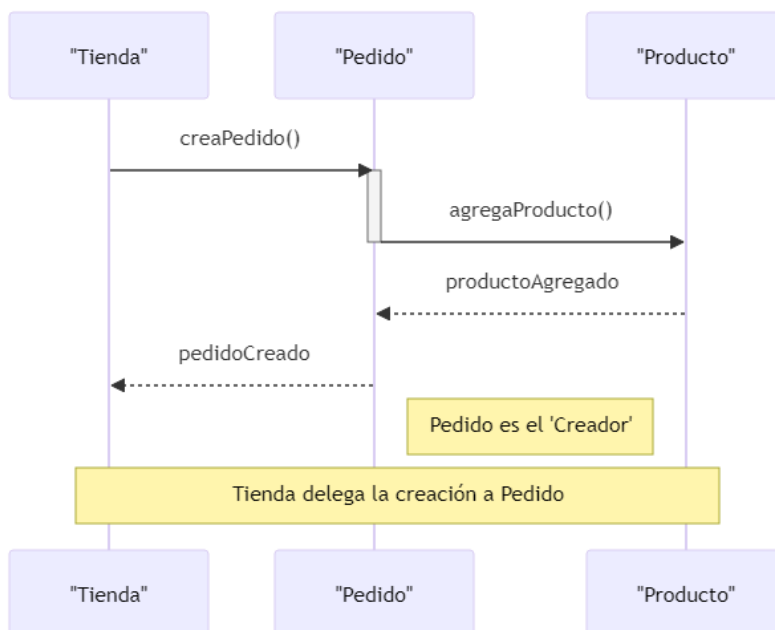
6. B contiene los datos de inicialización de A

Está relacionado a los patrones bajo acoplamiento, factoría, y todo-parte.

Beneficios: se eliminan dependencias innecesarias, soportando bajo acoplamiento y mayores oportunidades para reutilizar.

Ejemplo

1. Una Tienda inicia el proceso de creación de un Pedido.
2. El Pedido, que es el "Creador" en este contexto, procede a agregar un Producto.
3. El Producto confirma que ha sido agregado al Pedido.
4. Finalmente, el Pedido informa a la Tienda que ha sido creado exitosamente.



Bajo acoplamiento

Nos ayuda a soportar bajas dependencias, bajo impacto del cambio e incremento de reutilización, buscando asignar responsabilidades manteniendo bajo el acoplamiento. Se busca tener las clases lo menos ligadas entre sí como sea posible, para que en caso de producirse una modificación en alguna se tenga la mínima repercusión posible en el resto, potenciando la reutilización, y disminuyendo la dependencia entre las clases.

Acoplamiento: medida de la fuerza con la que un objeto está conectado a otro.

Esto nos permite reutilizar código y hacer que el mismo sea legible y entendible por sí solo. Además, cambios futuros no deberían afectar tanto el código.

- Los cambios en las clases relacionadas fuerzan cambios locales.
- Son difíciles de entender de manera aislada.

- Son difíciles de reutilizar puesto que su uso requiere la presencia adicional de las clases de las que depende.

Beneficios:

- No afectan los cambios en otros componentes.
- Fácil de entender de manera aislada.
- Conveniente para reutilizar.

Alta cohesión

Solución: Asignar una responsabilidad de manera que la cohesión permanezca alta. Cada atributo y método tiene que tener sentido en el concepto de la clase. Una alta cohesión funcional provoca mayor cantidad de clases. Un concepto, una clase. Cada clase debe modelar un concepto

Cohesión: Es una medida de la fuerza con la que se relacionan los objetos y de grado de focalización de las responsabilidades de un elemento. Un elemento con responsabilidades altamente relacionadas, y que no hace una gran cantidad de trabajo, tiene alta cohesión. Una clase con baja cohesión hace muchas cosas no relacionadas, o hace demasiado trabajo, no son convenientes y tienen problemas:

- Son difíciles de entender, reutilizar y mantener.
- Son delicadas, constantemente afectadas por los cambios

Explicación: Una clase de alta cohesión tiene un número relativamente pequeño de operaciones con funcionalidad relacionada y poco trabajo por hacer. Colabora con otros objetos para compartir esfuerzo si la tarea es grande.

Beneficios:

- Se incrementa la claridad y facilita la compresión del diseño.
- Se simplifican el mantenimiento y las mejoras.
- Se soporta a menudo bajo acoplamiento.
- Incrementa la reutilización porque una clase cohesiva se puede utilizar para un propósito muy específico.

Para mantener la complejidad manejable, se busca asignar responsabilidades manteniendo alta la cohesión. Cohesión: medida de la fuerza con la que se relacionan los objetos, y del grado de focalización de las responsabilidades de un objeto.

Controlador

Problema: ¿Quién debería encargarse de atender un evento del sistema? Un evento del sistema es un evento de alto nivel generado por el actor externo. Se asocian con operaciones del sistema tal como se relacionan los mensajes y los métodos.

Solución: asignar la responsabilidad de recibir o manejar un mensaje de evento del sistema a una clase controladora de esos eventos. En nuestro caso usamos la clase controladora para manejar un escenario de caso de uso en el que tiene lugar el evento del sistema. Se utiliza la misma clase controladora para todos los eventos del sistema en el mismo caso de uso.

Controlador: es un objeto que no pertenece a la interfaz de usuario, responsable de recibir o manejar un evento del sistema. Un controlador define el método para la operación del sistema.

Explicación: Un controlador de caso de uso es una buena alternativa cuando hay muchos eventos del sistema entre varios procesos: asigna su manejo a clases individuales controlables.

Es un patrón que sirve como intermediario entre una determinada interfaz y el algoritmo que la implementa, de tal forma que es la que recibe los datos del usuario y la que los envía a las distintas clases según el método llamado. Este patrón sugiere que la lógica de negocios debe estar separada de la capa de presentación, esto para aumentar la reutilización de código y a la vez tener un mayor control.

Normalmente, un controlador debería **delegar** en otros objetos el trabajo que necesita hacer, coordina o controla la actividad.

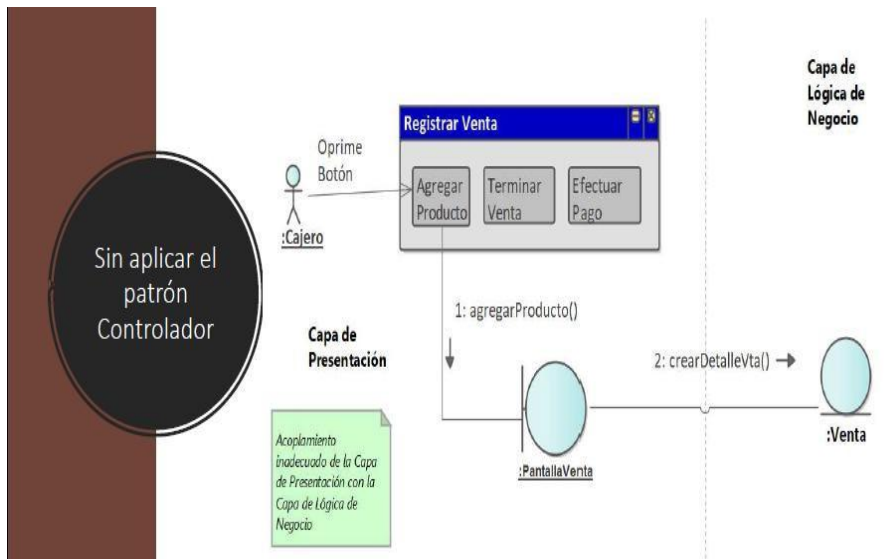
Hay dos categorías de controlador:

- Controlador de fachada: representa al sistema global, dispositivo o subsistema. Son adecuados cuando no existen “demasiados” eventos del sistema, o no es posible que la interfaz de usuario (UI) redirija mensajes de los eventos del sistema a controladores alternativos.
- Controlador de casos de uso: hay un controlador diferente para cada caso de uso. Aquí, el controlador no es un objeto de dominio; es una construcción artificial para dar soporte al sistema (clase de fabricación pura).

Un importante corolario del patrón Controlador es que los objetos interfaz y la capa de presentación no deberían ser responsables de llevar a cabo los eventos del sistema. Las operaciones del sistema se deberían manejar en la lógica de la aplicación o capa del dominio en lugar de en la capa de interfaz del sistema. El controlador recibe la solicitud del servicio desde la capa de UI y coordina su realización, normalmente delegando a otros objetos. Intermediario entre la interfaz y las entidades.

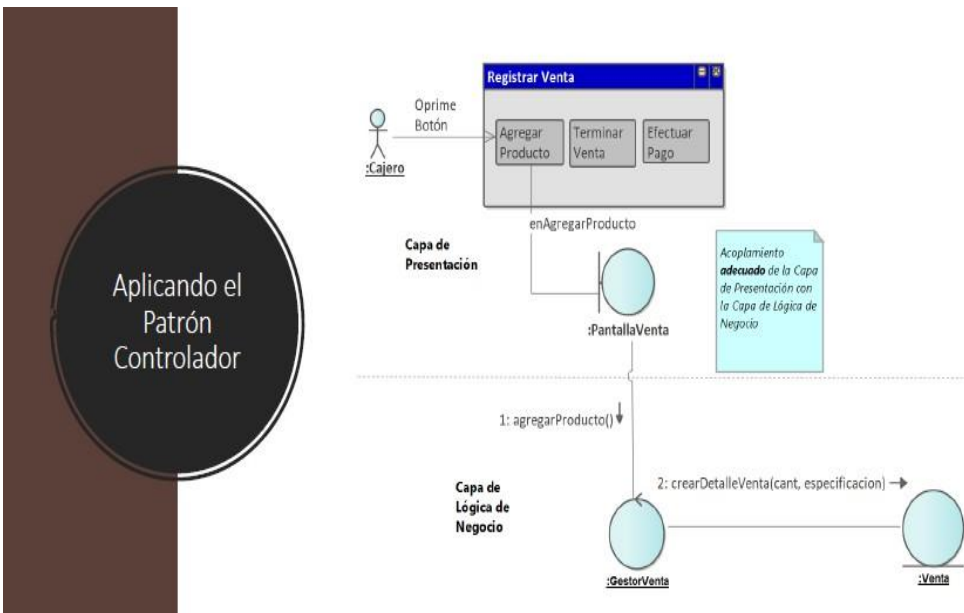
Beneficios:

- Aumenta el potencial para reutilizar y las interfaces conectables: asegura que la lógica de la aplicación no se maneja en la capa de interfaz. Delegando la responsabilidad de una operación del sistema a un controlador ayuda a la reutilización de la lógica en futuras aplicaciones. Y puesto que la lógica no está ligada a la capa de interfaz, puede sustituirse por una interfaz nueva.
- Razonamiento sobre el estado de los casos de uso: a veces es necesario asegurar que las operaciones del sistema tienen lugar en una secuencia válida, o ser capaces de razonar sobre el estado actual de la actividad y operaciones del caso de uso que está en marcha.



Sin aplicar el patrón Controlador, quien debería tener responsabilidades respecto de cómo accionar sobre las clases de entidad sería el objeto de la clase boundary (interfaz con el actor), entonces tendríamos que acoplar la capa de presentación con la siguiente capa. Se generaría un acoplamiento inadecuado por eso surge este patrón controlador que hace necesario el uso de un objeto de control.

El propósito de la capa de presentación es vincularse con el ambiente y NO tener la lógica de negocio metida, sin usar clases de control se acopla la capa de lógica de negocio con la capa de presentación. Luego, si hay que hacer algún cambio en las interfaces, hay que hacer un sistema nuevo porque muchas veces lo que se actualiza es la interfaz y no la lógica de negocios.



Fabricación pura: es una creación arbitraria del diseñador, no una clase software cuyo nombre se inspira en el modelo de dominio.

Un controlador de caso de uso es un tipo de fabricación pura.

Para saber quién debe ser responsable de gestionar un evento de entrada a un sistema, usamos este

patrón, que nos permite asignar a una clase la responsabilidad de manejar un evento del sistema, disparado por un actor externo. Estos patrones nos aseguran que no habrá ninguna lógica en clases de interfaz, se ocupan las clases controladoras y eso permite que se reutilice el código.

Relacionado con patrones command y facade.

Modelo de Dominio + Escenarios de Casos de uso = Realizaciones de Casos de uso

UNIDAD 2: WORKFLOW DE DISEÑO

- Definición de Diseño, principios de diseño de software orientado a objetos
- Aspectos que se diseñan en un sistema de información
- Estrategias de Prototipado y de Ensamblaje de Componentes
- Diseño en el Proceso Unificado de Desarrollo o Objetivo, actividades, trabajadores, artefactos
- Diseño de Software OO.
- Diseño del Comportamiento del Software.
- Diseño de la Estructura del Software
- Patrones de Diseño.
- Mapeo de estructuras de clases a bases de datos relacionales – Patrones de Persistencia.
- Diseño de Interacción Humano-Máquina (lo tengo en papel)
- Diseño de Procedimientos.
- Calidad en el diseño: Validación y verificación de los modelos de diseño

Es un proceso iterativo de transformar un modelo lógico en un modelo físico (componentes de código, lenguaje de programación, tecnologías, seguridad, tiempos de rta), modelamos el sistema y definimos su forma (incluida la arquitectura) para que soporte todos los requerimientos, incluyendo los RNF y otras restricciones, entrando en el dominio de la solución. Generando un modelo de diseño, más detallado que nos acerca a la implementación. Tiene suficiente detalle que nos va a permitir su realización física.

En el WF de Diseño, se toma el modelo de análisis como entrada obtenida en el WF de Análisis y se aplican los RNF obtenidos en el WF de Requerimientos, buscando generar un modelo que respete la estructura del análisis tanto como sea posible al momento de darle forma al sistema. Es el proceso de decidir cómo se va a construir el producto final.

El diseño es el WF que se encarga de avanzar en el refinamiento de la solución que empezó a

estructurarse en el WF de análisis.

Aplicamos Patrones de diseño con el Objetivo de rediseñar y mejorar la estructura que se había conseguido en el WF de análisis, hay muchos aspectos de la solución que no había sido tenido en cuenta. Nivel de cohesión y acoplamiento lo más óptimo posible

Diseño (Se modela en términos físicos, diag. de componentes y despliegue)

- No es genérico dado que se especifica para una implementación
- Muy centrado en la secuencia
- Mas formal

Este modelo busca:

- Adquirir una comprensión profunda de los aspectos relacionados a requerimientos no funcionales y restricciones de implementación física.
- Generar una entrada para las actividades de implementación.
- Descomponer las actividades de implementación en piezas más manejables, teniendo en cuenta la concurrencia.
- Capturar interfaces entre subsistemas.
- Visualizar el diseño usando una notación común.
- Crear una abstracción de la implementación del sistema, que es la actividad que luego refinará los resultados del diseño.

Modelo de análisis	Modelo de diseño
Modelo conceptual, porque es una abstracción del sistema y permite aspectos de la implementación.	Modelo físico, porque es un plano de la implementación.
Genérico respecto al diseño (aplicable a varios diseños).	No genérico, específico para una implementación.
Tres estereotipos conceptuales sobre las clases: Control, Entidad e Interfaz.	Cualquier número de estereotipos (físicos) sobre las clases, dependiendo del lenguaje de implementación.
Menos formal.	Más formal.
Menos caro de desarrollar (ratio al análisis 1:5).	Más caro de desarrollar (ratio al análisis 5:1).
Menos capas.	Más capas.
Dinámico (no muy centrado en la secuencia).	Dinámico (muy centrado en las secuencias).
Bosquejo del diseño del sistema, incluyendo su arquitectura.	Manifiesto del diseño del sistema, incluyendo su arquitectura (una de sus vistas).
Creado principalmente como "trabajo de a pie" en talleres o similares.	Creado principalmente como "programación visual" en ingeniería de ida y vuelta; el modelo de diseño es realizado según la ingeniería de ida y vuelta con el modelo de implementación (descrito en el Capítulo 10).
Puede no estar mantenido durante todo el ciclo de vida del software.	Debe ser mantenido durante todo el ciclo de vida del software.
Define una estructura que es una entrada esencial para modelar el sistema —incluyendo la creación del modelo de diseño.	Da forma al sistema mientras que intenta preservar la estructura definida por el modelo de análisis lo más posible.

Análisis vs Diseño

El análisis:

- Ofrece una especificación más precisa de los requerimientos.

- Se describe utilizando el lenguaje de los desarrolladores, introduciendo mayor formalismo.
- Estructura los requerimientos de modo que facilita su comprensión, su preparación, su modificación, y su mantenimiento.
- Es la primera aproximación al diseño.

El diseño:

- Es un proceso mediante el cual se aplican varias técnicas y principios con el objetivo de definir un dispositivo, un proceso o un sistema con suficiente nivel de detalle como para permitir su realización física.
- Es un proceso iterativo de transformar un modelo lógico en un modelo físico, teniendo en cuenta las restricciones del negocio (se debe incorporar toda la situación del contexto en donde el SW va a funcionar, es decir, el negocio) Se deja de asumir el concepto de "Tecnología Interna Perfecta" o "Tecnología Neutra" que se asumió en el análisis. Se necesita un refinamiento iterativo que se va dando a medida que contamos con más información. En él se traducen los requerimientos en un plano para construir el software.
- El diseño modela la solución en términos físicos, se ocupa de la solución de todos los aspectos que tienen que ver con la implementación.
- Es el proceso de decidir de cómo se va a construir el producto final.

Para que diseñamos

1. **Comprensión Profunda de Requerimientos No Funcionales (RNF) y Restricciones Tecnológicas:** Ambas descripciones enfatizan la importancia de entender a fondo los aspectos relacionados con los RNF y las limitaciones del entorno tecnológico, como el lenguaje de programación, componentes reutilizables, sistemas operativos, tecnologías de distribución y concurrencia.
2. **Refinamiento de Requerimientos para Subsistemas, Clases e Interfaces:** Se hace énfasis en la necesidad de detallar y perfeccionar los requerimientos para diferentes componentes del sistema, incluyendo subsistemas, clases e interfaces.
3. **Descomposición del Trabajo de Implementación:** Ambas descripciones sugieren dividir el trabajo de implementación en partes más pequeñas y manejables, lo que facilita la distribución de tareas entre diversos equipos de desarrollo.
4. **Identificación y Captura de Interfaces entre Subsistemas:** Se destaca la importancia de identificar y documentar las interfaces entre diferentes subsistemas, lo que incluye determinar qué pantalla corresponde a cada subsistema y la creación de diagramas de componentes.

5. **Creación de una Abstracción de la Implementación del Sistema:** Se propone la creación de una representación abstracta del sistema, como un diagrama de despliegue o plano, que sirve como una guía para la implementación efectiva del sistema.

DIFERENCIA FUNDAMENTAL entre Análisis y Diseño:

El Diseño tiene dos modelos de salida:

- Modelo de diseño
- Modelo de despliegue (incluye aspectos de hardware y comunicaciones).

Ha habido una evolución muy importante en los procesos, métodos y metodologías que se usan para construir software, antes del auge del paradigma O.O, la etapa de análisis era muy larga e importante (consumía el 50% de los recursos), no estaba la etapa de requerimientos separada y no se le daba importancia al diseño (muchas veces se lo salteaba) pasando directamente a la implementación, lo cual significaba cometer muchos errores y problemas en el resultado final del producto. Esto era así ya que el nivel de complejidad que se resolvía era muchísimo menor (no existía internet, ni la distribución/descentralización que hay hoy en el software y tampoco había tanta demanda de integración).

Las tendencias de la Ingeniería de Software hoy en día son:

- Darle un papel protagonista a la etapa de requerimientos
- Poner especial atención en la arquitectura de software
- Utilizar las mismas herramientas para modelar
- Utilizar un proceso.

Con esto, la etapa de análisis quedo muy pequeña e incluso hay opción de fusionarla con los requerimientos o el diseño. El RUP lo que hace es fusionarlo en un solo workflow llamado Análisis&Diseño.

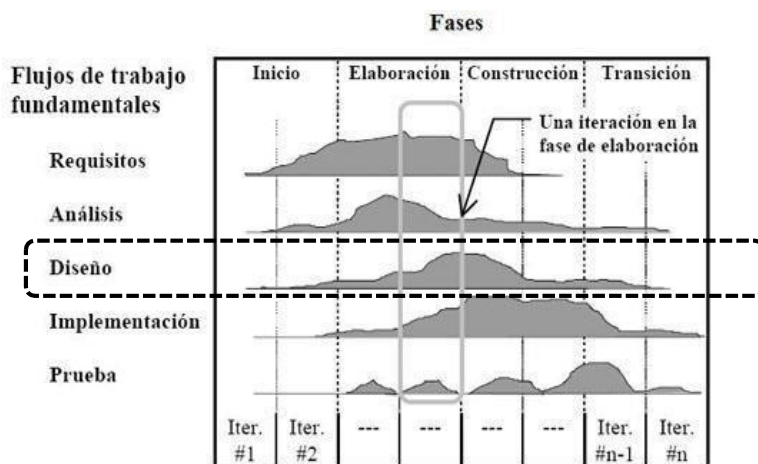
Con los nuevos métodos, la división entre el análisis y el diseño es cada vez más difusa. El modelo físico que se va a construir durante el diseño debe tener suficiente nivel de detalle para que de ahí se pueda obtener el código. Durante el diseño se refinan los modelos aplicando patrones de diseño, modificando la estructura del análisis para que sea más eficiente. Aparecen más clases de fabricación pura (que ya no tienen traza con el dominio del problema) porque queremos que nuestro diseño tenga calidad (bajo acoplamiento y alta cohesión, los principios básicos de calidad). Esto es lo que se hace cuando se diseña software.

Rol en el PUD

El diseño es el centro de atención en las últimas iteraciones de la fase de elaboración y el comienzo de las iteraciones de la fase de construcción.

- **Fase de elaboración (últimas iteraciones):** contribuye a una arquitectura estable y robusta.
- **Fase de construcción (primeras iteraciones):** crea un plano para la implementación.

El modelo de diseño se puede utilizar para visualizar la implementación y para soportar las técnicas de programación gráfica. A esto se lo denomina ingeniería de ida y vuelta.



Propósitos del diseño

- Adquirir una comprensión profunda de aspectos relacionados con requerimientos no funcionales y restricciones tecnológicas (lenguajes de programación, sistemas operativos, tecnologías de distribución y concurrencia, tecnologías de interfaz de usuario, etc.).
- Crear una entrada para el workflow de implementación.
- Descomponer el trabajo de implementación en piezas manejables que puedan ser llevadas a cabo por diferentes equipos de desarrollo, teniendo en cuenta la posible concurrencia.
- Capturar interfaces entre subsistemas.
- Crear una abstracción de la implementación del sistema, teniendo en cuenta que la implementación es un refinamiento del diseño que rellena lo existente sin cambiar la estructura. Esto permite no estar restringidos a una tecnología y la ingeniería de ida y vuelta entre el diseño y la implementación.

Workflow de diseño como proceso

Entradas

1. Realizaciones de CU Clases de análisis
2. Paquete de análisis
3. Desc de arquitectura
4. Req. No funcionales
5. Vista arquitectónica de la funcionalidad

Salidas

1. Modelo de diseño
2. Realizaciones de CU de diseño
3. Clases de diseño
4. Interfaces
5. Desc de la arquitectura Modelo de Despliegue
6. Desc. De la arquitectura

Artefactos

Artefactos de diseño

1. Modelo de diseño.
2. Clase de diseño.
3. Realización de caso de uso-diseño.
4. Subsistema de diseño.
5. Interfaz.
6. Descripción de la arquitectura (vista del modelo de diseño).
7. Modelo de despliegue.
8. Descripción de la arquitectura (vista del modelo de despliegue).

Modelo de diseño

Modelo de objetos, que describe la realización física de los casos de uso centrándose en requisitos (funcionales y no funcionales) y restricciones de implementación y su impacto en el sistema. Sirve de abstracción de su implementación, y como entrada a las actividades de implementación.

En este modelo, los casos de uso son realizados por clases de diseño y sus objetos, que, junto con los subsistemas de diseño, representan abstracciones del subsistema.

El modelo de diseño se representa por un sistema de diseño que denota el subsistema de nivel más alto del modelo. La utilización de otro subsistema es una forma de organización del modelo de diseño en porciones más manejables. Los casos de uso son realizados por clases de diseño y sus objetos. Esto se representa por colaboraciones en el modelo de diseño y denota realización de caso de uso-diseño, ésta describe cómo se realiza un caso de uso en términos de interacción entre objetos del diseño

Clase de diseño

Representa una abstracción sin costuras de una clase de la implementación del sistema. Sin costuras porque:

- Se especifican en lenguaje de programación
- Tienen visibilidad y operaciones especificadas
- Tienen relaciones con otras clases, las cuales tienen un significado directo
- Tienen métodos especificados que se corresponden con los métodos implementados en código (generalmente se los escribe en pseudocódigo en el diseño)
- Permiten posponer manejos de requisitos que sería inapropiado manejar en el diseño
- Pueden activarse y mantener hilos separados
- Pueden realizar interfaces

Las clases de diseño provienen de dos lados:

- El ámbito del problema vía una mejora de las clases de análisis: a las clases de análisis se le añaden detalles de implementación.
- El ámbito de la solución: es el ámbito de librerías de clases de utilidad y componentes reutilizables (Time, Date, String, colecciones, middleware, frameworks, etc.).

En las clases de diseño hay que especificar: atributos (nombre, tipo, visibilidad y opcionalmente un valor predeterminado), operaciones o métodos (nombre, parámetros con tipo y tipo de retorno). Esto además sirve para generar código que después les servirá a los programadores.

Cuatro características que debe tener una clase de diseño para que se considere bien diseñada:

- **Completa y suficiente**: proporciona a los clientes lo que se espera de ella y contiene el conjunto esperado de operaciones y nada más.
- **Sencilla**: ofrece un solo servicio sencillo. A veces se "relaja" un poco por cuestiones de rendimiento.
- **Alta cohesión**: tiene un pequeño número de responsabilidades que están íntimamente relacionadas. Son fáciles de entender y mantener, y son reutilizables.
- **Bajo acoplamiento**: debería asociarse con clases si solamente tienen vínculo semántico entre ellas.

Realización de caso de uso de diseño

Una realización de caso de uso-diseño es una colaboración en el modelo de diseño que describe como se realiza un caso de uso específico, y como se ejecuta, en términos de clases de diseño y sus objetos.

Proporciona una traza directa a una realización de caso de uso de análisis del modelo anterior. Para modelarlas, se usan diagramas de interacción, de clases y descripciones textuales.

Una realización de caso de uso-diseño tiene una descripción de flujo de eventos textual, diagramas de clases que muestra sus clases de diseño participantes; y diagramas de interacción que muestran la realización de un flujo o escenario concreto de un caso de uso en términos de interacción entre objetos del diseño.

Pueden posponer requerimientos hasta la siguiente actividad anotándolos como requerimientos de implementación en la realización.

- Diagramas de clases: contienen clases de diseño, y se utilizan conectados a una realización de caso de uso diseño para mostrar las clases de diseño participantes, subsistemas y sus relaciones.
- Diagramas de interacción: muestran la interacción entre objetos para llevar a cabo el caso de uso. En el diseño es preferible utilizar diagramas de secuencia ya que es importante encontrar secuencias de interacciones detalladas y ordenadas en el tiempo.
- Flujo de sucesos-diseño: es una descripción textual que explica y complementa a los diagramas y a sus etiquetas.
- Requerimientos de implementación: requerimientos capturados en el diseño, pero que es mejor tratarlos en la implementación.

Subsistema de diseño

Son una forma de organizar los artefactos mencionados en piezas más manejables. Un subsistema debería ser cohesivo, es decir, sus contenidos deberían estar fuertemente asociados. Además, los subsistemas deberían ser débilmente acoplados, esto es, sus dependencias entre unos y otros, o entre sus interfaces, deberían ser mínimas.

Deberían poseer las siguientes características:

- Pueden representar una separación de aspectos del diseño. Un sistema puede desarrollarse por separado por diferentes grupos de desarrolladores.
- Las dos capas de aplicación de más alto nivel y sus subsistemas dentro del modelo de diseño suelen tener trazas directas hacia paquetes y/o clases del análisis.
- Pueden representar componentes de grano grueso en la implementación del sistema.
- Pueden representar productos de software reutilizados que han sido encapsulados en ellos.
- Pueden representar sistemas heredados (o parte de ellos) encapsulándolos.

Interfaz

Las interfaces se usan para especificar operaciones que proporcionan las clases y subsistemas de diseño.

Una clase que proporcione una interfaz debe proporcionar métodos que realicen las operaciones de la misma. Un subsistema que proporcione una interfaz debe contener clases de diseño u otros subsistemas que proporcionen la interfaz.

Son una forma de separar la especificación de la funcionalidad en términos de operaciones, haciendo independientes de ella a las clases que la implementan.

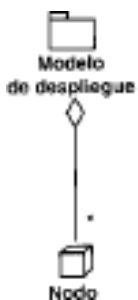
Definen interacciones permitidas entre los subsistemas.



- La descomposición del modelo de diseño en subsistemas, sus interfaces y las dependencias entre ellos, ya que constituyen la estructura fundamental del sistema.
- Clases del diseño fundamentales, como clases con traza con clases del análisis, clases activas, clases de diseño generales y centrarles; normalmente son clases abstractas.
- Realizaciones de caso de uso-diseño que describan alguna funcionalidad importante y crítica que debe desarrollarse pronto dentro del ciclo de vida del software, que impliquen muchas clases de diseño y por tanto tengan una cobertura amplia, posiblemente a lo largo de varios subsistemas.

Modelo de despliegue

Es un modelo de objetos que describe la distribución física del sistema en cuanto a la distribución de la funcionalidad en los nodos de cómputo. Puede describir diferentes configuraciones de red. Representa una correspondencia entre la arquitectura del SW y la arquitectura del HW. Es la descripción de la arquitectura.



- Cada nodo representa un recurso de cómputo (ej. procesador).
- Los nodos poseen relaciones que representan medios de comunicación entre ellos (ej. internet, intranet, bus, etc.).
- El modelo de despliegue puede describir diferentes configuraciones de red.
- La funcionalidad (los procesos) de un nodo se definen por los componentes que se distribuyen en ellos.
- El modelo de despliegue en sí mismo representa una correspondencia entre la arquitectura software y la arquitectura del sistema (el hardware)

Descripción de la arquitectura - vista del modelo de despliegue

La descripción de la arquitectura contiene una vista de la arquitectura del modelo de despliegue que muestra sus artefactos relevantes para la arquitectura.

Debido a su importancia, deberían mostrarse todos los aspectos del modelo de despliegue en la vista arquitectónica, incluyendo la correspondencia de los componentes sobre los nodos tal como se identificó durante la implementación.

Trabajadores del diseño

1. Arquitecto.
2. Ingeniero de casos de uso.
3. Ingeniero de componentes.



Arquitecto

Es el responsable de la arquitectura, la vista arquitectónica y de la integridad de los 2 modelos de salida, de diseño y despliegue, garantizando que sean correctos, consistentes y legibles como un todo, no hace el trabajo detallado. Los modelos son correctos cuando realizan la funcionalidad, y sólo la funcionalidad, descrita en el modelo de CU, en los requerimientos adicionales y en el modelo de análisis. El modelo de despliegue se diseña en el WF de Diseño, y se ejecuta en el WF de Despliegue.

- **Modelo de diseño:** Enfocado al software, tiene clases de diseño, componentes, interfaces, realizaciones de CU de diseño y la parte SPA de ese modelo de diseño que está representado por el artefacto llamado descripción de la arquitectura
- **Modelo de despliegue:** Enfocado al hardware, contiene los recursos de hardware, comunicaciones e infraestructura que hacen falta para que el sw que diseñamos pueda funcionar cumpliendo con los requerimientos, teniendo en cuenta la parte SPA. No creamos hardware sino nos enfocamos en los elementos que son necesarios para que nuestro sistema funcione.
- **Descripción de la arquitectura** (vista del modelo de diseño) □ Diagrama de componentes
- **Descripción de la arquitectura** (vista del modelo de despliegue) □ Diagrama de despliegue (SW y HW)

Ingeniero de casos de uso

Responsable de la realización e integridad de los CU-diseño, garantizando que cumplen con los requerimientos que se esperan de ellos. Una realización de CU-Diseño debe realizar correctamente el comportamiento de su correspondiente realización de CU-Análisis y del CU del modelo CU, y sólo esos comportamientos.

Toma las realizaciones de CU de análisis y las rediseña aplicando cambios teniendo en cuenta RFN y restricciones, construyendo los procesos de diseño (Realizaciones de CU de Diseño)



- Realizaciones de CU-Diseño: Proporciona una realización física de la realización de CU de análisis.

Ingeniero de componentes

Define y mantiene las operaciones, métodos, atributos, relaciones y requisitos de implementación de clases de diseño, garantizando que cumple lo que se espera de ellas en la Realización de CU.

Puede mantener también la integridad de subsistemas de diseño, interfaces y componentes (porción de código). Esto incluyen garantizar que sus contenidos (clases y relaciones) son correctos, que las dependencias de otros subsistemas y/o interfaces son correctas y mínimas, y que realizan correctamente las interfaces que ofrecen.

- **Clases de diseño:** Debe ser completa y suficiente, sencilla, alta cohesión y bajo acoplamiento
- **Subsistemas de diseño:** Forma de organizar los artefactos en piezas más manejables
- **Interfaces:** Especificar las operaciones que proporcionan las clases y los subsistemas de diseño

Comparación de con el WF de análisis:

Análisis	Diseño
Modelo Conceptual	Modelo Físico
Genérico respecto al diseño	Específico para una implementación
Tres estereotipos	Cualquier número de estereotipos
Menos formal	Más formal
Menos dinámico	Más dinámico
Más barato de desarrollar	Más Caro de desarrollar
Puede no mantenerse	Debe mantenerse en todo el ciclo de vida
Define una estructura que es entrada esencial para modelar el sistema.	Da forma al sistema mientras que intenta preservar la estructura definida por el modelo de análisis

Flujo de trabajo

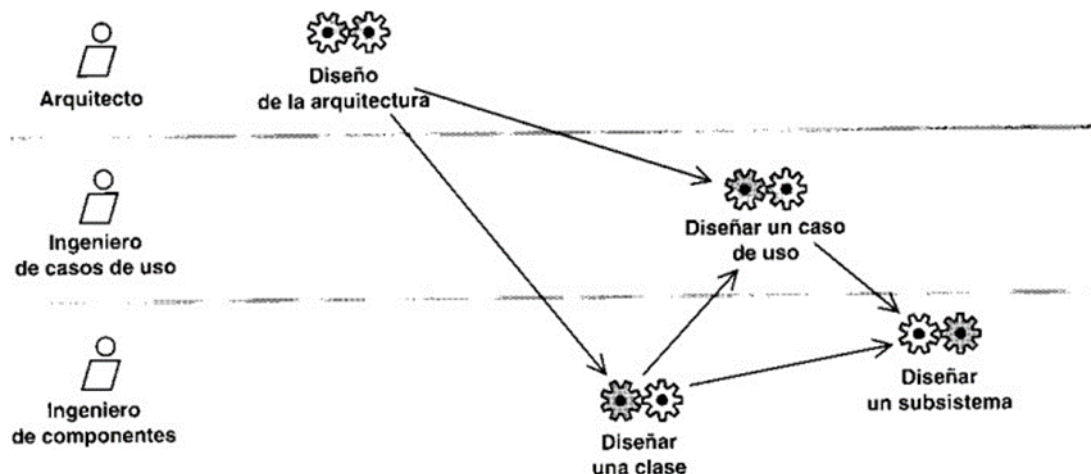
1. Los arquitectos inician la creación de los modelos de diseño y despliegue, y esbozan los artefactos relevantes.
2. Los ingenieros de casos de uso luego realizan cada caso en términos de clases y subsistemas, y sus interfaces.
3. Los ingenieros de componentes especifican a continuación los requerimientos, y los integran dentro de cada clase, bien mediante la creación de operaciones, atributos y relaciones

consistentes sobre cada clase, o bien mediante la creación de operaciones consistentes en cada interfaz que proporcione el subsistema.

4. A lo largo del flujo de trabajo de diseño, los desarrolladores identificarán, a medida que evolucione el diseño, nuevos candidatos para ser subsistemas, interfaces, clases y mecanismos de diseño genérico, y los ingenieros de componentes responsables de los subsistemas individuales deberán refinarlos y mantenerlos.

Los resultados de estas realizaciones establecen los requisitos de comportamiento para cada elemento que participe, y luego los ingenieros de componentes especifican los requisitos y los integran dentro de cada clase bien mediante la creación de operaciones, atributos y relaciones consistentes sobre cada clase, o bien mediante la creación de operaciones consistentes en cada interfaz que proporcione el subsistema.

A lo largo del flujo de trabajo de diseño, los desarrolladores identificarán, a medida que evolucione el diseño, nuevos candidatos para ser subsistemas, interfaces, clases y mecanismos de diseño genérico, y los ingenieros de componentes responsables de los subsistemas individuales deberán refinarlos y mantenerlos.



Actividades

Diseño de la arquitectura ► Arquitecto

Se busca esbozar los dos modelos trabajados (modelos de diseño y despliegue y su arquitectura), identificando los elementos:

1. **Ident. Nodos y sus configuraciones** (pueden aplicarse modelos de 3 capas, como MVC)

Las configuraciones de red habituales utilizan un patrón de tres capas en el cual los clientes se dejan en una capa, la funcionalidad de la base de datos en otra, y la lógica de negocio en una tercera.

Aspectos a tener en cuenta:

- Que nodos se necesitan. Cuál debe ser su capacidad en términos de potencia y tamaño de memoria.
- Qué tipo de conexiones debe haber entre los nodos. Que protocolos de comunicaciones se deben utilizar.
- Qué características deben tener las conexiones y los protocolos de comunicaciones, como el ancho de banda, disponibilidad, calidad.
 - Si se necesita copias de seguridad de datos, capacidad de procesos redundante, etc.

Teniendo conocimiento en los límites y posibilidades de los nodos y sus conexiones, el arquitecto puede incorporar ciertas tecnologías y servicios de replicación de datos que pueden facilitar la distribución del sistema. Cada configuración de red debería mostrarse en un diagrama de despliegue separado.

2. Ident. Subsistemas e interfaces

Los subsistemas pueden identificarse inicialmente como una forma de dividir el trabajo de diseño, o bien pueden irse encontrando a medida que el modelo de diseño evoluciona y va creciendo hasta convertirse en una gran estructura que debe ser descompuesta. Algunos subsistemas representan productos reutilizados y otros son recursos existentes en la empresa.

Los subsistemas son un medio para organizar el modelo de diseño en piezas más manejables y, la inclusión de estos tipos de subsistemas en el modelo de diseño permite analizar y evaluar las alternativas de reutilización.

• Identificación de subsistemas de aplicación:

En este paso se identifican los subsistemas de la capa específica de la aplicación y de la capa general de la aplicación, es decir, los subsistemas de las dos capas superiores. Si durante el

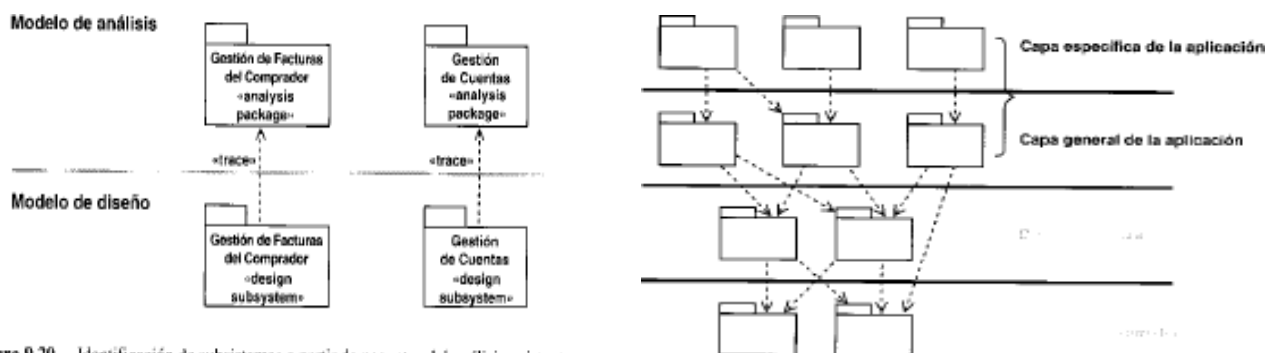


Figura 9.20. Identificación de subsistemas a partir de paquetes del análisis existentes.

análisis se hizo una descomposición adecuada en paquetes de análisis, se los puede utilizar para identificar los correspondientes subsistemas dentro del modelo de diseño.

• Identificación de subsistemas intermedios y subsistemas software

En el desarrollo de sistemas, la identificación de subsistemas intermedios y subsistemas software es crucial. El software del sistema y la capa intermedia son fundamentales, ya que sustentan todas las funciones, como el sistema operativo, sistemas de gestión de bases de datos, software de comunicaciones, entre otros. Durante las fases iniciales y de elaboración, se enfoca en seleccionar e integrar productos software, ya sean comprados o construidos. El arquitecto se asegura de que estos productos se alineen con la arquitectura y permitan una implementación eficiente. Es vital minimizar las dependencias de productos comprados para reducir riesgos futuros, ya que el control sobre su evolución es limitado. Tratar cada producto software comprado como un subsistema independiente con interfaces claras es una estrategia para gestionar estas dependencias.

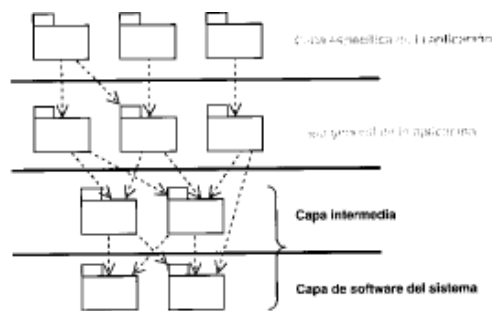


Figura 9.24. Los subsistemas de las capas intermedia y de software del sistema encapsulan productos software comprados.

• Definición de dependencias entre subsistemas

Deberían definirse dependencias entre subsistemas si sus contenidos tienen relaciones unos con otros. La dirección de la dependencia debería ser la misma que la dirección (navegabilidad) de la relación. Consideraremos Las dependencias entre los paquetes de análisis se corresponden con los subsistemas de diseño. Si utilizamos interfaces entre subsistemas las dependencias deberían ir hacia las interfaces no hacia los propios subsistemas. Cuando las dependencias atraviesan capas tienen impacto en la arquitectura por ende es útil hacerlas explícitas en el modelo de diseño y mostrarlas en diagramas de clases así los ingenieros de componentes y de casos de uso pueden hacer referencia a esos diagramas.

• Identificación de interfaces entre subsistemas

Las interfaces proporcionadas por un subsistema definen operaciones que son accesibles desde fuera del subsistema. Estas interfaces proporcionan clases u otros subsistemas recursivamente. Cuando un subsistema tiene una dependencia que apunta hacia él es probable que deba proporcionar una interfaz. Además, si existe un paquete del análisis que se pueda obtener mediante una traza del subsistema entonces cualquier clase del análisis referenciada desde el exterior del paquete puede implicar una interfaz candidata para el subsistema.

No es suficiente con identificar interfaces, también debemos identificar las operaciones que

deben definirse sobre cada una de ellas. Esto se hace mediante el diseño de los casos de uso en términos de subsistemas e interfaces en la actividad de diseño de casos de uso.

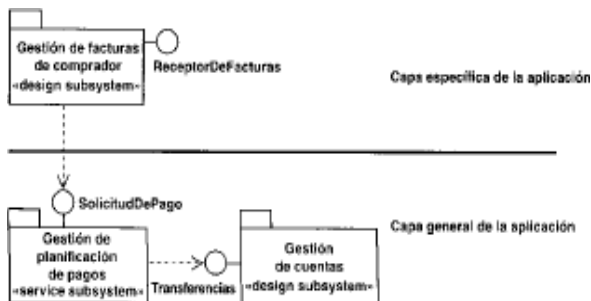
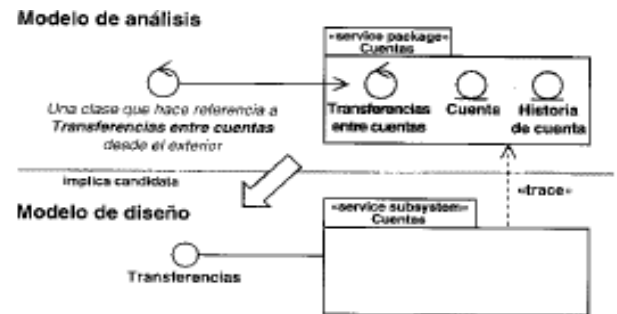


Figura 9.28. Las interfaces en las dos capas superiores del modelo de diseño.



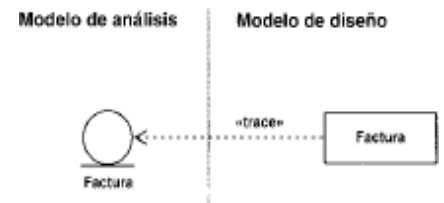
a 9.27. Una interfaz identificada inicialmente a partir del modelo de análisis.

3. Clases de diseño relevantes para la arquitectura

La mayoría de las clases del diseño se identificarán al diseñar las clases en la actividad de diseño de clases y se refinarán de acuerdo a los resultados obtenidos en la actividad del diseño de los casos de uso, por eso los desarrolladores deberían evitar el identificar demasiadas clases en esta etapa o el quedar atrapados en demasiados detalles. Sería suficiente con un esbozo inicial de las clases significativas para la arquitectura, como las clases activas.

- **Identificación de clases del diseño a partir de clases del análisis**

Se puede esbozar inicialmente algunas clases del diseño a partir de las clases del análisis significativas para la arquitectura y, también se pueden utilizar las relaciones entre esas clases para identificar las relaciones entre las clases del diseño.



- **Identificación de clases activas**

El arquitecto identifica también las clases activas necesarias en el sistema considerando los requisitos de concurrencia del mismo. Por ejemplo:

- Los requisitos de rendimiento, tiempo de respuesta y disponibilidad que tienen los diferentes actores en su interacción con el sistema (si un actor tiene requisitos exigentes en cuanto a tiempo de respuesta podemos dedicar un objeto activo a la gestión de ese actor tomando sus entradas y enviando las respuestas)
- La distribución del sistema sobre los nodos lo cual puede requerir un objeto activo por nodo y objetos activos aparte para gestionar la intercomunicación entre los nodos
- Otros requisitos sobre el arranque y terminación del sistema, progresión, evitar interbloqueos, reconfiguración de nodos.

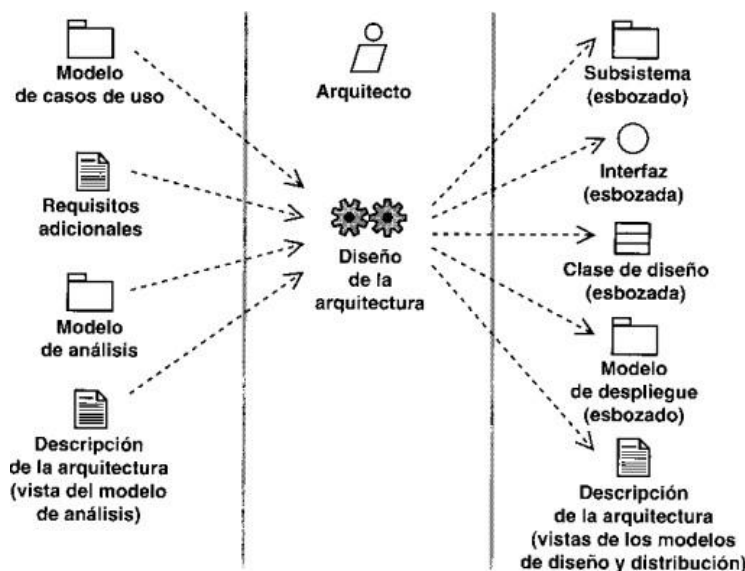
Cada clase activa se esboza mediante la consideración del ciclo de vida de sus objetos activos y de cómo deberían comunicarse, sincronizarse y compartir información los objetos activos. Después, éstos se asignan a los nodos del modelo de despliegue considerando la capacidad de los nodos, procesadores, tamaños de memoria, disponibilidad y ancho de banda de la conexión. La asignación de clases activas a los nodos en este paso es la entrada para la

asignación de componentes (ejecutables) a los nodos durante la implementación.

4. Ident. mecanismos de diseño que tratan requisitos comunes

Se estudian los requisitos comunes y se decide cómo tratarlos teniendo en cuenta las tecnologías de diseño e implementación disponibles. El resultado es un conjunto de mecanismos genéricos de diseño que pueden manifestarse como clases, colaboraciones, subsistemas. Los requerimientos a tratarse suelen estar relacionados con: persistencia, distribución transparente de objetos, características de seguridad, detección y recuperación de errores, etc.

La mayoría de los mecanismos genéricos deberían ser identificados y diseñados en la fase de elaboración. El arquitecto suele ser capaz de diseñar un conjunto de mecanismos que darán solución a los aspectos más difíciles del diseño haciendo que los casos de uso sean fáciles y directamente reutilizables en la fase de construcción.



Diseño de un caso de uso ► Ing. De casos de uso

1. Identificar clases de diseño y subsistemas participantes en el flujo de sucesos del caso de uso.

Se identifican las clases de diseño necesarias para realizar el caso de uso.

- Estudiar las clases del análisis que participan en la correspondiente realización de caso de uso-análisis.
- Estudiar los requerimientos especiales de la correspondiente realización de caso de uso-análisis, identificando las clases del diseño que los cumplen
- Como resultado deberíamos identificar las clases necesarias y asignar su responsabilidad a algún ingeniero de componentes.
- Se debe mostrar en un diagrama de clases las clases del diseño que participan en

una realización de caso de uso para mostrar las relaciones que se utilizan en la realización del caso de uso.

2. Distribuir el comportamiento del caso de uso entre los objetos del diseño que interactúan y/o entre los subsistemas participantes.

Debemos describir cómo interactúan los objetos del diseño correspondientes a las clases del diseño. Esto se hace mediante diagramas de secuencia que contienen instancias de los actores, los objetos del diseño y las transmisiones de mensajes entre éstos, que participan en el caso de uso.

3. Definir los requerimientos sobre las operaciones de las clases del diseño y/o sobre los subsistemas y sus interfaces.

Identificación de subsistemas e interfaces participantes

A veces es más apropiado diseñar un caso de uso en término de los subsistemas y/o interfaces que participan en él:

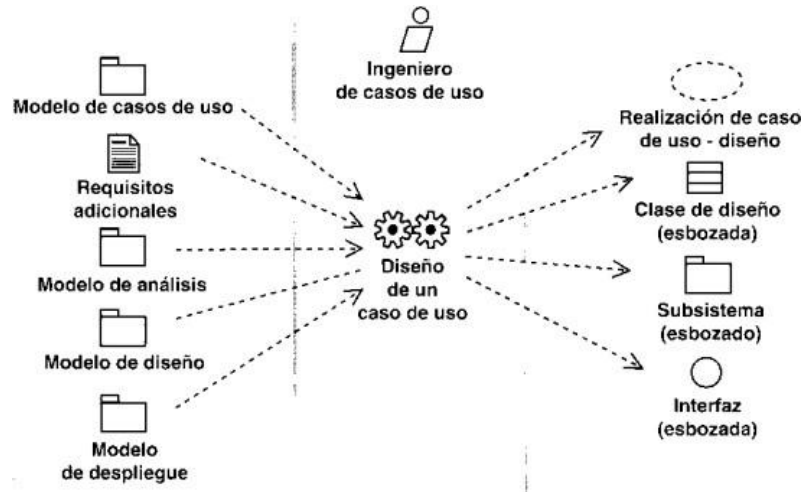
- Estudiar las clases del análisis que participen en las correspondientes realizaciones de caso de uso análisis. Identificar los paquetes del análisis que contienen a esas clases del análisis, si existen. Después, identificar los subsistemas del diseño que poseen una traza hacia esos paquetes del análisis.
- Estudiar los requerimientos especiales de las correspondientes realizaciones de caso de uso-análisis. Identificar las clases de diseño que realizan dichos requerimientos especiales, si existen. Después, identificar los subsistemas del diseño que contienen a esas clases.
- Debemos mostrar en un diagrama de clases asociado a la realización, los subsistemas que participan en una realización de caso de uso para mostrar las dependencias entre esos subsistemas y cualquier interfaz que se utilice en la realización de caso de uso.

Descripción de interfaces entre subsistemas

Cuando tenemos un esbozo de los subsistemas necesarios para realizar el caso de uso, debemos describir cómo interactúan los objetos de las clases que contiene en el nivel de subsistema. Lo haremos mediante el diagrama de secuencia, que contiene instancias de actores, subsistemas, y transmisiones de mensajes entre éstos.

4. Capturar los requisitos de implementación del caso de uso

En este paso se incluye en la realización de caso de uso todos los requerimientos identificados durante el diseño que deberían tratarse en la implementación (requerimientos no funcionales)



Diseño de una clase ► Ing. De Componentes

Se busca crear una clase de diseño que cumpla su papel en las realizaciones de caso de uso, y los requisitos no funcionales que se aplican a estos. Se tienen en cuenta atributos, operaciones, relaciones, métodos...

Esbozar una clase

Esbozar clases de diseño tomando como entrada clases de análisis (entidades, de control, boundary) y/o interfaces.

Cuando se toma como entrada una interfaz se asigna directamente una clase de diseño para que proporcione esa interfaz. Cuando se toma como entrada las clases del análisis se usan distintos métodos dependiendo del estereotipo de la clase:

- Diseñar clases boundary es dependiente de la tecnología de interfaz específica que se utilice. Por ejemplo, si se diseñan en Visual Basic éstas implicarían clases de diseño estereotipadas como forms.
- Diseñar clases de entidad que representen información persistente implica el uso de tecnologías de bases de datos específicas. Por ejemplo, se puede incluir la creación de clases de diseño para hacer la correspondencia con tablas en un modelo de datos relacional. Esto puede automatizarse parcialmente, aunque es delicado y puede requerir la adopción de una estrategia persistente lo cual puede requerir ciertos trabajadores como diseñadores de bases de datos y actividades de diseño y modelo de datos.
- Diseñar clases de control es una tarea delicada ya que encapsulan secuencias, coordinación de otros objetos o incluso pura lógica de negocio. Por lo tanto, es necesario tener en cuenta aspectos de distribución (si la secuencia se debe manejar por distintos nodos de una red las clases de diseño se separan en dichos nodos), aspectos de rendimiento

(puede no ser necesario tener una clase de diseño para la clase de control, otras clases del diseño de boundary o de entidad pueden realizar su tarea) y aspectos de transacción ya que las clases de control a menudo encapsulan transacciones y su diseño debería incluir tecnología de manejo de transacciones.

Identificar operaciones

Identificamos las operaciones que las clases del diseño van a necesitar y describimos esas operaciones utilizando la sintaxis los lenguajes de programación, incluyendo la visibilidad (public, private, protected, etc.).

Las entradas para esta actividad son:

- Responsabilidades de cualquier clase del análisis que tenga una traza con la clase del diseño. La responsabilidad puede implicar una o varias operaciones.
- Requerimientos especiales de cualquier clase del análisis que tenga una traza con la clase del diseño. Éstos pueden requerir algún mecanismo de diseño genérico o tecnología de base de datos.
- Interfaces que la clase de diseño necesita proporcionar junto a sus operaciones.
- Realizaciones de caso de uso-diseño en las que participa.

Identificar atributos

Identificamos los atributos requeridos por la clase de diseño y los describimos utilizando la sintaxis del lenguaje de programación. Un atributo especifica una propiedad de una clase de diseño y está a menudo implicado y es requerido por las operaciones de la clase.

- Atributos de las clases de análisis que tengan una traza con las clases del diseño.
- Los tipos de atributos están restringidos por el lenguaje de programación.
- Intentar reutilizar tipos de atributos existentes.
- Separar atributos en una clase si se necesita compartir el mismo por varios objetos o si hay muchos atributos o éstos son complejos.

Identificar asociaciones y agregaciones

Como los objetos del diseño interactúan unos con otros, estas interacciones necesitan asociaciones entre las clases correspondientes. Los ingenieros de componentes deben estudiar la transmisión de mensajes en los diagramas de secuencia para determinar qué asociaciones son necesarias. Se debe garantizar el bajo acoplamiento. Se deben modelar rutas de búsqueda óptimas teniendo en cuenta que los aspectos del rendimiento deben ser manejados en el diseño.

Identificar las generalizaciones

Las generalizaciones deben utilizarse con la misma semántica definida en el lenguaje de programación. Si el lenguaje de programación no admite la generalización o herencia, se deben utilizar asociaciones y/o agregaciones en su lugar.

Describir los métodos

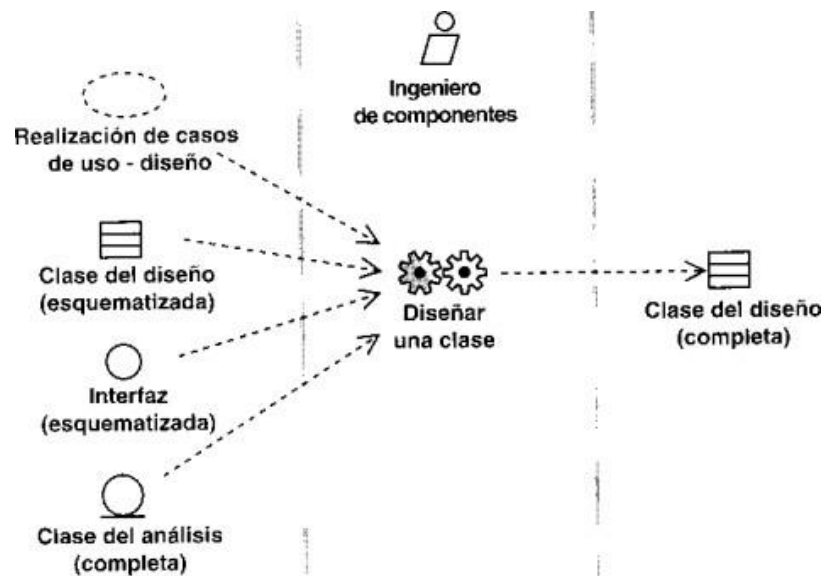
Los métodos pueden ser utilizados en el diseño para especificar como se deben realizar las operaciones. El método puede ser especificado mediante lenguaje natural o pseudocódigo, aunque la mayoría de las veces no son especificados en el diseño, sino en la implementación utilizando directamente el lenguaje de programación.

Describir estados

Se utilizan diagramas de estado para describir las diferentes transiciones de estado de un objeto del diseño, esto es porque los estados de un objeto determinan su comportamiento cuando reciben un mensaje. Cada diagrama de estado es una entrada para la implementación de la correspondiente clase del diseño.

Tratar requerimientos especiales

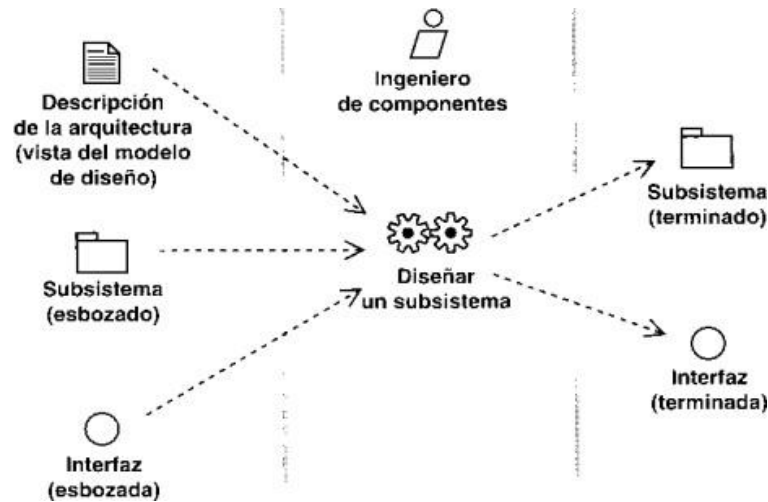
En esta fase debe ser tratado cualquier requerimiento especial que no haya sido considerado en pasos anteriores.



Diseño de un subsistema ► Ing. De Componentes

1. Garantizar que el mismo es lo más independiente posible.
2. Garantizar que el mismo proporciona las interfaces correctas.

- Garantizar que el mismo cumple su propósito correctamente de ofrecer una realización correcta de las operaciones tal y como se definen en las interfaces que proporciona.



Mantenimiento de las dependencias entre subsistemas

Deberían definirse y mantenerse dependencias en un subsistema respecto a otros cuando los elementos no contenidos en estos últimos estén asociados con elementos dentro de aquel. Hay que minimizar las dependencias entre subsistemas y/o interfaces. Las dependencias deberían declararse hacia las interfaces en vez de hacia los subsistemas si es que los mismos proporcionan dichas interfaces.

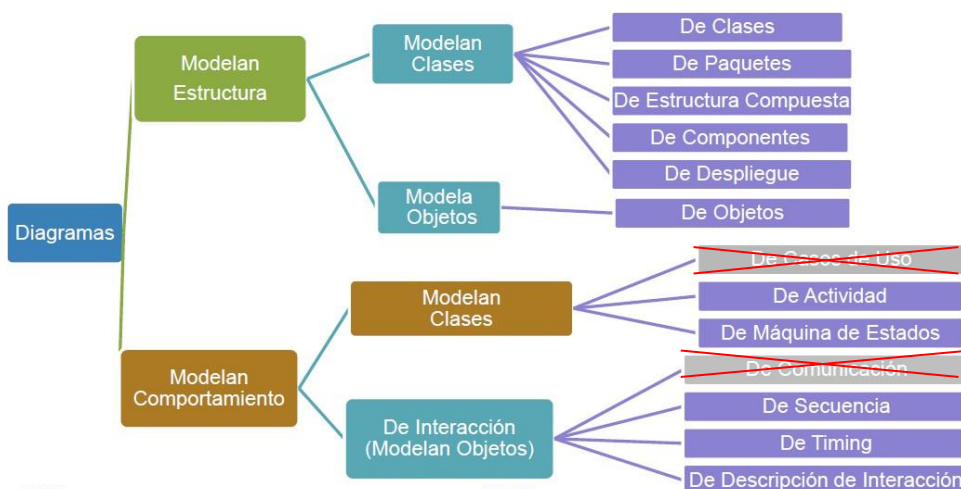
Mantenimiento de interfaces proporcionadas por el subsistema

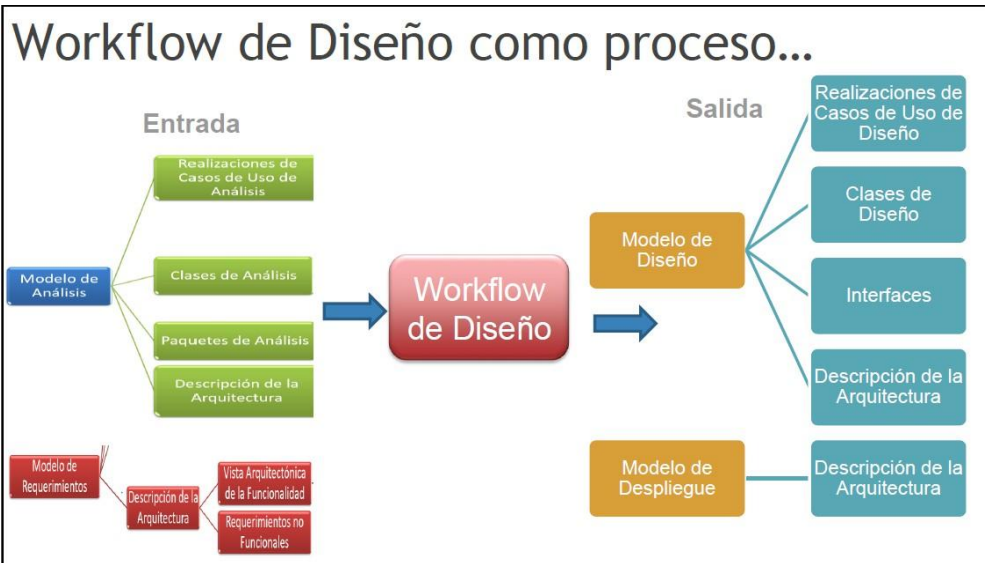
Las operaciones definidas por las interfaces que proporciona un subsistema deberán soportar todos los roles que cumple el subsistema en las diferentes realizaciones de caso de uso.

Mantenimiento de los contenidos de los subsistemas

Un subsistema cumple sus objetivos cuando ofrece una realización correcta de las operaciones tal y como están descritas por las interfaces que proporciona.

Diagramas de UML 2.0 en el Diseño





RESUMEN DEL DISEÑO

El principal resultado del diseño es el modelo de diseño que se esfuerza en conservar la estructura del sistema impuesta por el modelo de análisis y que sirve como esquema para la implementación.

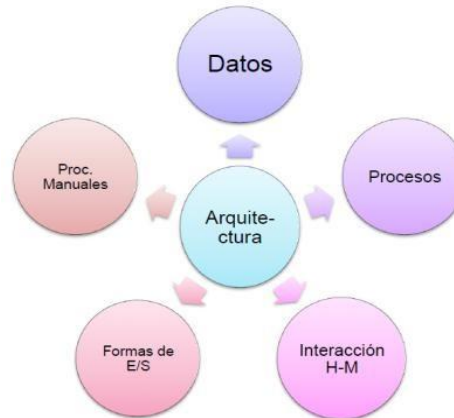
El modelo de diseño incluye:

- Subsistemas del diseño y subsistemas de servicio y sus dependencias, interfaces y contenidos.
- Clases del diseño, incluyendo las clases activas y sus operaciones, atributos y requisitos de implementación.
- Realizaciones de caso de uso-diseño, que describen cómo se diseñan los casos de uso en términos de colaboraciones dentro del modelo de diseño.
- La vista arquitectónica del modelo de diseño incluyendo sus elementos significativos de cara a la arquitectura.

El diseño también obtiene como resultado un modelo de despliegue, que describe todas las configuraciones de red sobre las cuales debería implantarse el sistema. El modelo de despliegue incluye:

- Nodos, sus características y sus conexiones.
- Una correspondencia inicial de clases activas sobre los nodos.
- La vista arquitectónica del modelo de despliegue que incluye a sus elementos relevantes para la arquitectura.

¿Qué aspectos deben diseñarse?



Se puede diseñar:

Arquitectura

Este aspecto se refiere al diseño general y estratégico del software. Incluye decisiones significativas para cumplir con los requerimientos funcionales y no funcionales del producto. La arquitectura define la relación entre los principales elementos estructurales del programa y se representa generalmente a través de vistas o diagramas UML. Es un modelo genérico que establece las bases sin entrar en detalles específicos. Toma los RNF SPA y los aplica al modelo de análisis. Centro de todos los aspectos a diseñar, es el diseño estratégico que hacemos del software, desde ella tomamos las decisiones significativas que luego detallaremos en el resto de los aspectos a diseñar. En otras palabras, la arquitectura define un diseño estratégico global o general que presenta la forma en la que se van a resolver los requerimientos, especialmente los no funcionales, pero, éstos no están aislados, sino que, condicionan la funcionalidad. Lo no funcional está embebido en lo funcional y, se condicionan mutuamente.

El diseño arquitectónico tiene que dar una respuesta de cómo hacer para bajar el modelo lógico/conceptual a una tecnología de implementación concreta, resolviendo las características o requerimientos no funcionales.

El diseño arquitectónico da respuesta a los requerimientos de calidad del software con decisiones estratégicas. Durante el diseño arquitectónico se utilizan patrones arquitectónicos porque es una forma de mejorar la calidad del software y ser más productivos y eficientes. Utilizar patrones es fuertemente recomendado ya que un patrón es una solución concreta a un problema bien identificado y, es una solución probada, ya se sabe que funciona.

Las salidas arquitectónicas más importantes y de mayor cantidad salen del WF de diseño. En el diseño se termina de armar la arquitectura de producto.

Base de datos

Aquí se transforman los requerimientos en estructuras de datos necesarias para implementar el software. Un componente clave en este diseño es el mapeo objeto-relacional (ORM), que se encarga de transformar los objetos en tablas de bases de datos y viceversa, permitiendo la persistencia de los datos. Implica analizar nuestras clases, definir cuál de esas clases necesitan persistencia, en que almacenamiento persistente la vamos a guardar, diseñar la BD y la estructura de los datos, y diseñar la persistencia (como hago para conectar las clases con la BD). La problemática de diseñar datos es muy importante porque las bases de datos son uno de los aspectos más condicionantes e importante para el funcionamiento del software ya que la parte más lenta es el acceso a datos.

Procesos (P. de diseño)

Consiste en transformar elementos estructurales de la arquitectura del programa en descripciones procedimentales de los componentes del software. Este diseño implica considerar aspectos como la concurrencia, distribución de procesos, performance y persistencia. Se utilizan patrones de diseño para la creación de Procesos. Realizaciones de casos de uso de análisis refinadas y transformadas en procesos, es decir, son las realizaciones de casos de uso de diseño. Se aborda con las realizaciones de caso de uso-diseño. Se toman las realizaciones de caso de uso-análisis, se ve qué aspectos son necesarios rediseñar o agregar/completar, es decir, se las refina en procesos eficientes y se obtienen las realizaciones de caso de uso-diseño.

Las realizaciones de caso de uso-diseño son los procesos que se van a ejecutar entre componentes relacionados entre sí para construir la funcionalidad que esperamos ya respetando los requerimientos no funcionales.

Interfaces -> interacción humano-máquina

Este aspecto se centra en cómo el software se comunica con otros sistemas, ambientes, dispositivos externos, y usuarios. Incluye el diseño de interfaces externas e internas, así como las interfaces de usuario, considerando la usabilidad y la experiencia del usuario (UI/UX).

El diseño de interacción humano-máquina es la disciplina relacionada con el diseño, evaluación e implementación de sistemas computacionales interactivos para uso humano. Hace referencia a todo lo que se tiene que tener en cuenta para completar el vínculo que el SW va a tener con la gente que lo va a usar.

Es muy importante ya que si el usuario no entiende cómo usar la aplicación se frustra y decide desinstalar la aplicación sin darle luego una segunda oportunidad.

Formas de entrada/salida

Se refiere a cómo se ingresa información al software y cómo se presentan las salidas. También tiene que ver con la comunicación de los usuarios con el software por otras vías alternativas que no es la pantalla de un dispositivo (papel, archivos, etc).

Tipos de procesamiento:

- **En lote (la transacción se procesa después de ocurrir)**
 - La transacción ocurrió, pero el software la procesa después (aunque sea 5seg u horas después). Se acumulan transacciones para procesarlas después. Tienen procesamiento diferido. Los procesos batch alivian la carga del HW en horarios pico de trabajo. La mayoría de las veces se requiere un soporte adicional (voto en papel, solicitud en papel) para acumularlas y luego procesarlas. Ejemplos• sistema de votación, procesos en los bancos (las solicitudes de tarjeta de crédito o préstamo se procesan al cierre del día laboral), cliring bancario: intercambio de cheques entre bancos.
- **En línea (la transacción se procesa cuando ocurre)**
 - La transacción se procesa en el momento en el que ocurre. Ejemplo: inscripción a examen o a materias.
- **En tiempo real (tiempo de respuesta muy estricto, pueden modificar el entorno en el que está integrado).**
 - Son sistemas en línea, pero que tiene la capacidad de modificar el ambiente donde el software está inmerso. Es muy estricto a los tiempos de respuesta (muy preciso; si un proceso debe durar un determinado tiempo, no puede durar más, no quiere decir que sea más rápido sino que debe cumplir el tiempo exacto que se solicita). Ejemplo: la alarma de incendio que si detecta fuego empieza a tirar chorros de agua (está modificando el ambiente).

Cada vez se usan menos las formas de entrada alternativas ya que cada vez hay más sistemas en línea pero, para los sistemas batch que se utilizan hasta hoy en día todavía deben diseñarse esas formas de entrada (campos e información a completar) y debe ser consistente con los datos a cargar en la interfaz y que luego estarán en la base de datos.

Las formas de salida tienen que ver con las formas de visualización de la información, nos interesa que el usuario tenga información que si no fuese por el sistema, de otra forma no contaría con dicha información. Cuando se diseñan las formas de salida se puede elegir entre presentarla en pantalla, en pdf, imprimirlas en papel, generarlas en otro archivo como puede ser un texto plano, Excel, etc para que luego el usuario pueda manipular esos datos.

Procedimientos manuales.

Describe cómo se integra el software al sistema de negocio. Se resuelven las problemáticas de cómo el software se adapta e inserta al entorno, incluyendo la adaptación del software al entorno y la planificación de contingencias como los "planes B".

De las actividades del modelo de negocio debemos ver cuáles necesitan soporte del sistema de información las cuales se complementan con procedimientos alternativos para cuando el procedimiento principal (el que es soportado por el software) falle (se corta la luz, no anda internet, se cae el sistema, etc).

Guía para evaluar un buen diseño

Cuando termino de realizar el diseño, ¿cómo sé que mi diseño es bueno, que es el que tiene que ser?

- El diseño deberá implementar todos los requerimientos explícitos del modelo de análisis y deberán ajustarse a todos los requerimientos implícitos que desea el cliente.
- El diseño deberá ser una guía legible y comprensible para aquellos que generan código y para aquellos que comprueban y consecuentemente, dan soporte al software.
- El diseño deberá proporcionar una imagen completa del software, enfrentándose a los dominios de comportamiento, funcionales y de datos desde una perspectiva de implementación.

Propósito de WF de diseño

- Adquirir comprensión en profundidad de los aspectos relacionados a los Requerimientos no funcionales y restricciones tecnológicas
- Crear una entrada para el Workflow de implementación
- Descomponer los trabajos de implementación en partes manejables, permitiendo la concurrencia entre equipos de desarrollo
- Capturar interfaces entre subsistemas
- Crear una abstracción de la solución de diseño sin restringirse a una tecnología



El diseño deberá poderse rastrear hasta el modelo de análisis.



El diseño no deberá inventar nada que ya esté inventado.



El diseño deberá «minimizar la distancia intelectual» entre el software y el problema.



El diseño deberá presentar uniformidad e integración.

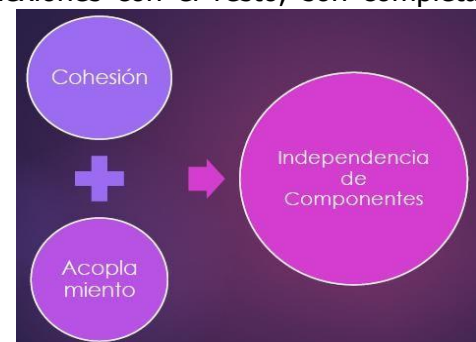


El diseño deberá estructurarse para admitir cambios.

Principios de diseño: se aplican a cualquier diseño independientemente del paradigma. Son los pilares que ayudan a construir un producto de software de calidad.

Acoplamiento Características de un buen diseño

- **Independencia de los componentes:** Se trata de que los componentes sean independientes unos de otros. Para medir el grado de independencia de los componentes de un diseño se aplican 2 principios del diseño básico: Alta Cohesión y bajo acoplamiento.
 - **Identificación y tratamiento de excepciones:** debe impactar lo menos posible en el SW.
 - **Prevención y tolerancia de defectos:** la tolerancia se refiere a la información del error al usuario y cómo recuperarse de esa situación imprevista mientras que la prevención, se refiere a tratar de evitar que los problemas terminen impactando negativamente en el sistema. Nos referimos a realizar las validaciones correctas, que no lleguen datos basura a la base de datos, etc. También debe tener bajo impacto en el software.
 - **Reutilización de código:** El costo y el tiempo son los dos factores más importantes en el desarrollo de software. La reutilización de código es una de las formas de reducir costos de desarrollo. Al dedicar menos tiempo, se puede entrar antes al mercado y si se reducen los costos se puede invertir más en marketing. Hacer que código existente funcione en otro contexto puede ser difícil y generar acoplamiento, dependencias que reducen la flexibilidad del código y hace que sea más difícil de reutilizar. Usar patrones de diseño es una forma de aumentar la flexibilidad de los componentes y hacerlos más fáciles de utilizar, lo cual se traduce en un buen diseño.
 - **Extensibilidad:** lo único constante en la programación son los cambios. Las aplicaciones deben ser diseñadas para posibles cambios futuros, ya sea cambios en el ambiente, nuevos requerimientos por parte del cliente, tendencias, etc.
-
- Dos componentes están altamente acoplados cuando existe mucha dependencia entre ellos.
 - Los componentes poco acoplados, tienen algunas dependencias, pero las interconexiones entre ellos son débiles.
 - Los componentes no acoplados no tienen interconexiones con el resto, son completamente independientes. Poco probable en un sistema



Cohesión

- Se refiere al grado de interconexión interna con el que se ha construido el componente.
- Un componente es cohesivo si todos sus elementos están orientados a la realización de una única tarea y son esenciales para llevarla a cabo.

Conceptos básicos de la OOP

La oop es un paradigma basado en el concepto de envolver piezas de datos y sus comportamientos

relacionados en paquetes especiales llamados objetos, contruidos a partir de un conjunto de planos llamados clases.

Una clase puede tener atributos, y las instancias de estas clases, es decir, los objetos, se comportan de manera similar mediante los métodos de la clase. Cada objeto puede tener un estado diferente.

Las clases pueden seguir una jerarquía; podemos tener una clase padre o superclase, que es una clase base que enumera atributos y comportamientos comunes de las clases hijas o subclases, que heredan el estado y comportamiento, y definen sólo lo que difiere de su clase padre. Las hijas pueden anular el comportamiento que heredan de los métodos de la clase base, reemplazarlo o modificarlo.

Hay 4 pilares fundamentales de este paradigma:

- **Abstracción:** trabajamos un modelo limitado, omite detalles irrelevantes
- **Encapsulamiento:** capacidad de un objeto de ocultar partes de él, exponiendo una interfaz limitada. Podemos hacer sus partes privadas o protegidas
- **Herencia:** capacidad de construir nuevas clases sobre las existentes, reutilizando código. Hablamos de reúso de caja blanca. Cada clase puede extender sólo una superclase, y en cambio, implementar varias interfaces.
- **Polimorfismo:** capacidad de un programa para detectar la clase real de un objeto y llamar a su implementación, incluso aunque su tipo real sea desconocido en el contexto actual.

Los objetos pueden relacionarse de diferentes formas:

Asociación: un objeto usa a otro. En UML se representa mediante una flecha simple apuntando al objeto utilizado.



Dependencia: variante de la asociación que implica que no hay un vínculo permanente entre los objetos. Normalmente implica que un objeto acepta otro como parámetro de método, crea instancias o utiliza otro objeto. Existe una dependencia entre dos clases si los cambios en la definición de una clase modificarán a la otra.



Composición: es una relación de "parte completa" entre dos objetos; uno se compone de una o más instancias del otro. Reúso de caja negra, se puede reusar el comportamiento de manera más transparente. Yo le hago una petición (DELEGACION) al objeto y el objeto me devuelve la respuesta y yo no sé qué código ejecuta para devolverme la petición. Debemos favorecer la composición sobre la herencia.



Agregación: variante de la composición menos estricta, un objeto contiene simplemente una referencia a otro. El componente puede existir sin el contenedor y se puede vincular a varios contenedores al mismo tiempo.

PRINCIPIOS PARA IMPLEMENTACIÓN EN POO

- Notación Camel Case.
- Código feliz.
- Métodos cortos.
- Fácil lectura -> Fácil mantenimiento.
- KISS (Keep It Simple, Stupid).
- Evitar el YAGNI (You Aren't Going to Need It).
- Evitar el "y ya que estamos..."
- Hacer la cosa más simple que podría funcionar.
- ¡No me hagas pensar!
- Escribir código para quien lo va a mantener.
- Principio del menor asombro: el nombre del método debería corresponder con lo que hace.
- Abrazar el cambio.
- Reusar.
- Usar convenciones de código.

Principios de diseño de software

- El diseño deberá poderse rastrearse hasta el modelo de análisis (trazabilidad – dirigido por CU). Se debe saber de dónde viene cada aspecto del diseño que estoy construyendo. Es la forma de garantizar que vamos a cumplir con los requerimientos.
- El diseño no deberá inventar nada que ya esté inventado. (Se debe hacer uso de patrones)
- El diseño deberá "minimizar la distancia intelectual" entre el software y el problema como si de la misma vida real se tratara. Tenemos que tratar de que el software sea lo más cercano conceptualmente posible a su realidad. Hay que tener en cuenta la terminología, respetar los nombres que el cliente usa y organizar la secuencia lógicamente en función de su realidad. Para diseñar, hay que ponerse del lado de quien usa el software.
- El diseño deberá presentar uniformidad e integración. (consistencia)

Principio de diseño: Es una técnica o herramienta básica que se aplica para construir SW y que nos permiten evitar tener un mal diseño, obteniendo SW más flexible, extensible y mantenible. Son guías de alto nivel, que aplican a cualquier lenguaje de programación OO. Su foco principal es mantener la cohesión alta, el acoplamiento bajo, facilitar los cambios y prevenir errores inesperados.

Programar hacia la interfaz, no hacia la implementación

Debemos buscar depender de abstracciones, para flexibilizar y ampliar el diseño sin romper ningún código existente.

Favorecer la composición sobre la herencia

Una subclase no puede reducir la interfaz de la superclase; debe implementar todos los métodos abstractos de la clase principal, incluso si no se van a usar

Al anular métodos, debemos asegurarnos de que el nuevo comportamiento sea compatible con el básico

La herencia rompe la encapsulación de la superclase; cada subclase conoce los detalles de la clase padre.

Las subclases están muy ligadas a la clase padre, en la que cualquier cambio puede romper la funcionalidad de las clases hijas

Intentar reutilizar código a través de herencia puede crear jerarquías de herencia paralelas

Podemos usar composición para evitar todos los problemas generados por la herencia.

DRY (Don't Repeat Yourself - No te repitas):

Busca la consolidación de la funcionalidad del sistema en una única porción de código para evitar la duplicación. En el código, este principio se aplica mediante la abstracción de elementos comunes en un solo lugar. Por ejemplo, al utilizar el **patrón State, todo el código está definido en la clase abstracta, evitando repeticiones innecesarias.**

Evitar duplicaciones de código repetido:

El principio de DRY implica evitar la repetición de código con el mismo propósito. Al definir funcionalidades o requerimientos en un único lugar, se crean abstracciones que permiten modificar el software sin afectar partes no relacionadas. Esto garantiza que cada requerimiento esté representado por una única porción de código, facilitando el mantenimiento y reduciendo el riesgo de inconsistencias.

Ubicar la información en un único lugar:

DRY se relaciona con la colocación de cada pieza de información en un lugar único. Al referenciar la información desde su ubicación centralizada, se evitan inconsistencias y duplicaciones. Este enfoque no se limita solo al código, sino que abarca todos los aspectos del sistema. La idea es que cada pieza de información y comportamiento del sistema tenga una ubicación clara y única para evitar esfuerzos innecesarios de mantenimiento.

YAGNI(No lo vas a necesitar)

Evitar la codificación de funcionalidades no solicitadas pero que se supone serán necesarias en un futuro, es pérdida de tiempo y nunca es seguro que se necesite realmente

Keep it simple stupid (KISS)

Evitar aumentar infinitamente la abstracción dado que aumenta la complejidad, no incluir grandes librerías con gran cantidad de funciones si solo haremos uso de unas pocas. Descomponer algo complejo en componentes simples, mayor flexibilidad, comprensión, calidad y facilidad de mantener

Principio de menor conocimiento (Ley de Demeter)

Cada unidad debe tener un conocimiento limitado sobre otras unidades, debe estar estrechamente relacionado, con el propósito de producir objetos débilmente acoplados, mayor legibilidad/cohesion

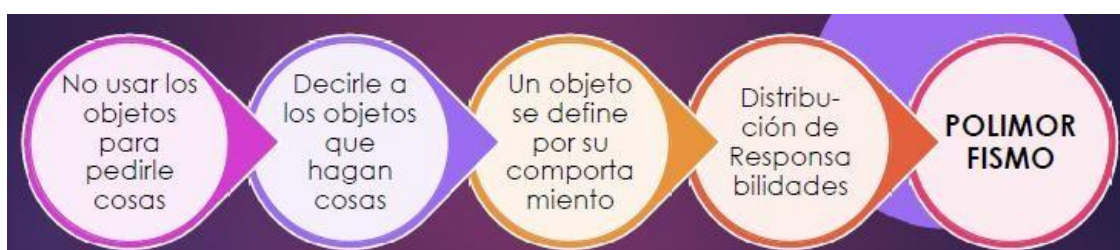
Tell, don't ask principle

Se fundamenta en la filosofía de la programación orientada a objetos (OO), donde los objetos colaboran para implementar comportamientos que no podrían lograr por sí solos. La clave de esta colaboración radica en la distribución de responsabilidades entre los objetos (DELEGACION), evitando el uso de clases que solo contengan métodos de obtener y establecer valores (getters y setters).

En esencia, este principio aboga por que un objeto, al necesitar realizar una tarea, no debería preguntar directamente por la información requerida, sino más bien ordenar a otro objeto que realice la acción necesaria y devuelva el resultado. La responsabilidad se asigna al objeto que posee la información pertinente, lo que contribuye a mejorar la cohesión de cada clase al encapsular el código con los datos que utiliza.

La esencia del TDA se refleja en la idea de decirle a los objetos qué hacer en lugar de preguntarles sobre su estado. Un objeto se define principalmente por su comportamiento, y el principio busca que los objetos colaboren en términos de comportamiento, priorizando la distribución adecuada de responsabilidades, lo cual se ve facilitado por el polimorfismo, una característica inherente al paradigma de objetos que permite diferentes implementaciones del mismo comportamiento en clases distintas.

El objeto que tiene la información debe responder la acción solicitada. **Relacionado al patrón experto. Hace lo que sabes y dame el resultado.**



Reutilización de software

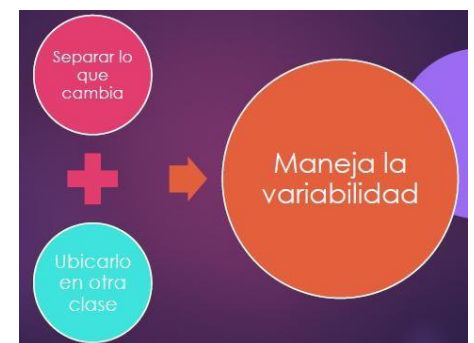
La ingeniería de software basada en la reutilización es un enfoque al desarrollo que trata de maximizar la reutilización del software existente, en unidades de diferentes tamaños: sistemas de aplicación, componentes, objetos y funciones. En ocasiones, reutilizar código adaptándolo a nuestra necesidad puede ser complejo, por lo que puede reutilizarse un concepto o idea, sin incluir detalles de implementación. Por ejemplo, los patrones de diseño son conceptos reutilizables.

Una ventaja de la reutilización es que se pueden reducir los costos del desarrollo, pero tenemos costos y problemas asociados. Puede costar el hecho de entender si un componente es o no adecuado para su reutilización, y la prueba para garantizar la confiabilidad. Las etapas de diseño e implementación del software pueden incluir actividades para buscar y evaluar componentes para reutilizar.

Encapsular lo que varia

También conocido como el principio SOLID Open/Close, y su objetivo principal es mejorar la flexibilidad, mantenibilidad y manejar la variabilidad en un sistema de software. Este principio busca separar las partes del código que son propensas a cambios y encapsularlas en objetos independientes. Al aplicar este principio, se logra un código más modular, flexible y mantenible, facilitando la adaptación del sistema a los cambios y variaciones en los requisitos.

Patrón que aplica: State, Strategy, una clase que encapsula el código que puede llegar a cambiar.



Encapsulación a nivel de método: Un método que calcula el total de un pedido con los impuestos incluidos. El tema de impuestos va a cambiar en un futuro y, además depende de cada país, provincia, ciudad, etc y el método de cálculo deberá ser cambiado varias veces.

```
1 method getOrderTotal(order) is
2   total = 0
3   foreach item in order.lineItems
4     total += item.price * item.quantity
5
6   if (order.country == "US")
7     total += total * 0.07 // US sales tax
8   else if (order.country == "EU"):
9     total += total * 0.20 // European VAT
10
11  return total
```

ANTES: el código del cálculo del impuesto

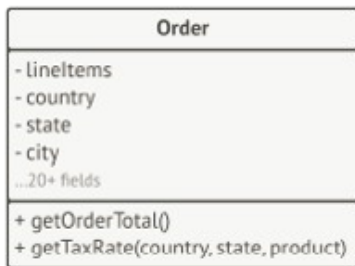
```
1 method getOrderTotal(order) is
2   total = 0
3   foreach item in order.lineItems
4     total += item.price * item.quantity
5
6   total += total * getTaxRate(order.country)
7
8   return total
9
10 method getTaxRate(country) is
11   if (country == "US")
12     return 0.07 // US sales tax
13   else if (country == "EU"):
14     return 0.20 // European VAT
15   else
16     return 0
```

DESPUÉS: los cambios relacionados con está mezclado

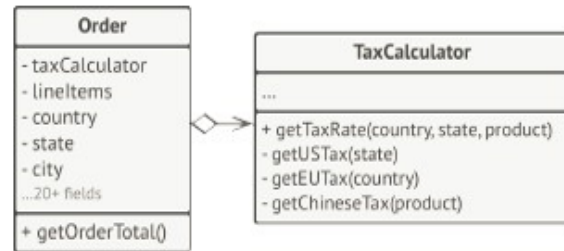
con el resto del código.

el impuesto queda aislado dentro de un único método.

Encapsulación a nivel de clase: si se añade cada vez más y más responsabilidades a un método que solía hacer algo sencillo termina nublando la responsabilidad principal de la clase contenedora. Si se extrae todo a una nueva clase se consigue una mayor claridad y sencillez.



ANTES: el cálculo del impuesto está en la clase Pedido



DESPUÉS: el cálculo del impuesto se esconde dentro de la clase Pedido. Los objetos de la clase Pedido delegan todo el trabajo relacionado con el impuesto a un objeto especial.

Programar hacia la interfaz

"Programa a una interfaz, no a una implementación. Depender de abstracciones, no de concreciones".

Enfatiza la importancia de desarrollar código que dependa de abstracciones en lugar de implementaciones específicas. Este enfoque, alineado con el principio "Segregación de Interfaces", tiene como objetivo aumentar la flexibilidad del código para facilitar su extensión sin descomponer las partes existentes. Busca minimizar el acoplamiento entre módulos o sistemas. Este principio promueve la transparencia con el cliente al establecer contratos claros, facilitando la comprensión y el uso de la funcionalidad proporcionada por los módulos, al tiempo que se mantiene la flexibilidad para futuras extensiones.

1. Determinar lo que necesita exactamente un objeto del otro, ¿qué métodos va a ejecutar?
2. Describir esos métodos en una nueva interfaz o clase abstracta.
3. Haz que la clase que es una dependencia implemente esta interfaz.
4. Ahora haz la segunda clase dependiente de esta interfaz en lugar de la clase concreta. Todavía puede funcionar con objetos de la clase original pero ahora la conexión es mucho más flexible.

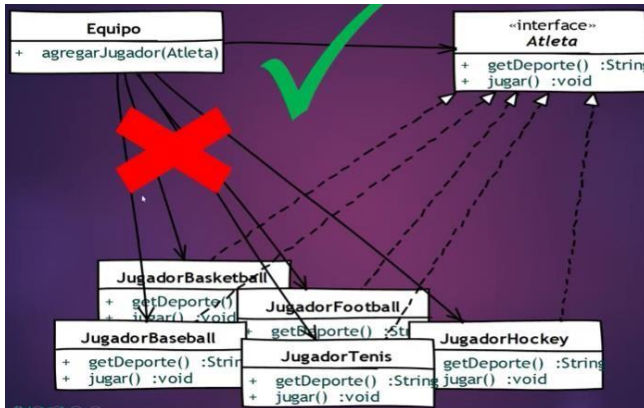
Las interfaces me aseguran que un objeto va a poder realizar una acción, entonces cualquier cliente que necesite de esa acción va a poder pedírselo a través de esa interfaz.

Cada clase que implemente una interfaz debe cumplir con los comportamientos de la misma.

Manipula objetos en términos de la interfaz definida por clases abstractas tiene dos ventajas:

- 1) Los clientes no tienen que conocer los tipos específicos de los objetos que usan, basta con que se adhieran a la interfaz que esperan los clientes.
- 2) Los clientes desconocen las clases que implementan dichos objetos, sólo conocen las clases abstractas que definen la interfaz.

Patrón que aplica: Factory Method



Favorecer la composición sobre herencia.

La herencia (Caja blanca), aunque aparentemente sencilla para reutilizar código entre clases, conlleva desventajas evidentes en programas complejos. Genera un fuerte acoplamiento entre las subclases y la superclase, rompiendo el encapsulamiento al exponer los detalles internos. La composición, en cambio, fomenta la reutilización a través de la delegación, minimizando el acoplamiento y preservando el encapsulamiento. Al utilizar la delegación para el reúso en lugar de la herencia, se aprovecha el polimorfismo, permitiendo que objetos encapsulados respondan dinámicamente a métodos con la misma firma. La composición resuelve dinámicamente en tiempo de ejecución, evita problemas de acoplamiento y encapsulamiento, y el comportamiento depende de las relaciones establecidas.

Problemas de la herencia:

- Una subclase no puede reducir la interfaz de la superclase. Tiene que implementar todos los métodos abstractos de la clase padre, incluso si no va a usarlos.
- Al sobrescribir métodos debes asegurarte de que el nuevo comportamiento sea compatible con el de la base.
- La herencia rompe el encapsulamiento de la superclase, porque los detalles internos de la clase padre se hacen visibles para la subclase.
- Las subclases están fuertemente acopladas a superclases. Cualquier cambio en una superclase puede descomponer la funcionalidad en una subclase.
- Intentar reutilizar código mediante la herencia puede conducir a la creación de jerarquías de herencia paralelas.

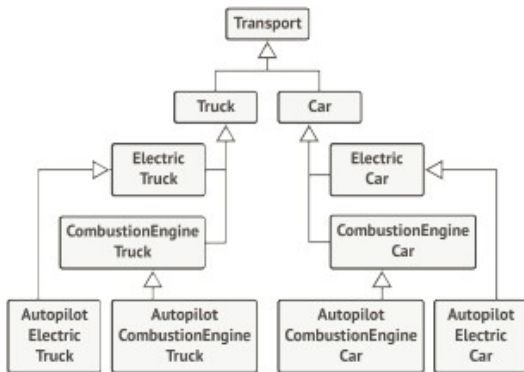
Existe una alternativa a la herencia, que se llama composición. Mientras la herencia representa la relación "es un" entre clases, la composición se basa en la relación "tiene un". Esto también se puede aplicar a la agregación, en la que un objeto puede contener una referencia al otro pero no gestiona su ciclo vital.

VENTAJA de la composición: se puede sustituir un comportamiento durante el tiempo de ejecución.

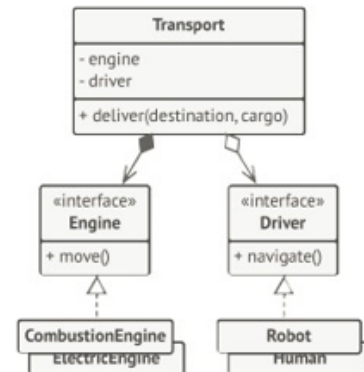
Patrón que aplica: Strategy

Ejemplo: se debe crear una aplicación de un catálogo para un fabricante de automóviles. La empresa

fabrica autos y camiones, pueden ser eléctricos o a nafta, manuales o de caja automática.



HERENCIA: extender una clase en varias dimensiones (tipo de automóvil x tipo de motor x tipo de conducción). Cada nueva variable resulta en la multiplicación de subclases. Se repite código porque una subclase no puede extender dos clases al mismo tiempo



COMPOSICIÓN: diferentes “dimensiones” de funcionalidad extraídas para ponerse dentro de sus propias jerarquías de clases. En lugar de que los objetos de auto implementen un comportamiento por su cuenta, pueden delegarlo a otros objetos

HERENCIA DE CLASES	COMPOSICIÓN DE OBJETOS (Asoc-Aggeg-Comp)
Ventajas • Se define en tiempo de compilación. • Es sencilla de utilizar (se implementa directamente en el lenguaje de programación). • Fácil modificar la implementación que está siendo utilizada.	Ventajas • Se define en tiempo de ejecución (con objetos que hacen referencia a otros objetos). • Requiere objetos con interfaces bien diseñadas, dado que a los objetos se accede a través de sus interfaces. • No rompe el encapsulamiento (por lo del punto anterior, las interfaces). • Cualquier objeto puede ser reemplazado por otro en tiempo de ejecución siempre que sean del mismo tipo. • Las dependencias de implementación son menores. • Se mantiene el encapsulamiento y cada clase se centra en una tarea. • El comportamiento del sistema depende de las relaciones que se están dando entre los objetos, en vez de estar definido en las clases. • Se conoce como reuso de caja negra, gracias al polimorfismo se puede hacer reuso de comportamiento, pero de una manera más transparente, se le hace una petición a un objeto y éste me devuelve la respuesta sin saber cómo se ejecutó el código. Se delega la responsabilidad.
Desventajas • No se pueden cambiar las implementaciones heredadas de las clases padre en tiempo de ejecución (se define en tiempo de compilación). • Se conoce como reuso de caja blanca porque las clases tienen que abrirse, rompe el encapsulamiento (los detalles del padre son expuestos a las subclases). • Alto acoplamiento (los cambios en las clases padre obligan a cambiar las subclases).	

Relaciones que tenemos en el SW:

- Herencia
- Realización
- Asociación (composición y agregación)



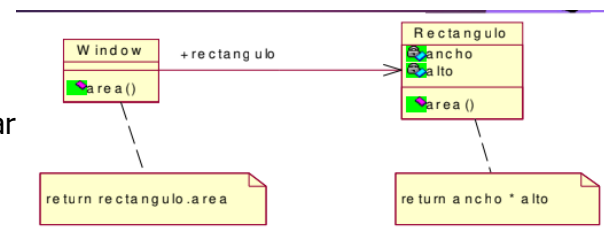
Se debe favorecer a la COMPOSICIÓN por sobre la herencia porque es una relación más beneficiosa. Ayuda a mantener la clase encapsulada y centrada en una sola tarea y se evita que crezca la estructura de clases y se vuelva inmanejable, dado que ahora tendremos más objetos y menos clases. El mecanismo que usamos para la composición se llama DELEGACIÓN, la cual funciona a través de peticiones donde se desentiende del comportamiento porque se lo estoy pidiendo a otra clase. El otro objeto hace lo que yo le pido, porque tiene el código para hacerlo y me devuelve una respuesta.

DELEGACIÓN

La delegación es un modo de lograr que la composición sea tan potente para la reutilización como lo es la herencia. La principal ventaja de la delegación es que se hace que sea fácil combinar comportamientos en tiempo de ejecución y cambiar la manera en que éstos se combinan. Es una alternativa a la herencia que no viola el principio de sustitución. La delegación tiene un inconveniente: el software dinámico y altamente parametrizado es más difícil de entender que el estático. Hay también ineficiencias en tiempo de ejecución. Formas de funcionamiento de la delegación:

*Pasándose a sí mismo por parámetro

*Le pasa los atributos que necesita el otro para poder calcular



Con esto se aplica:

- El principio de no te repitas, dado que el código está en un sólo lugar.
- No se rompe en encapsulamiento
- El principio de responsabilidad única.

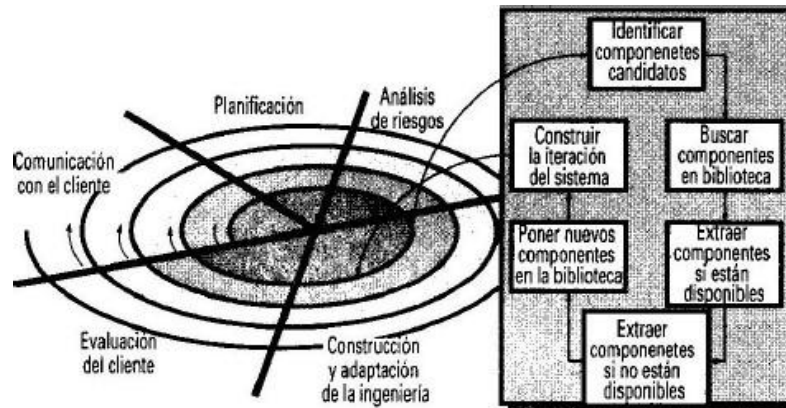
Entonces, la delegación es una herramienta muy buena para hacer reúso de caja negra. Varios patrones de diseño utilizan la delegación, algunos son el State y Strategy.

ESTRATEGIA DE ENSAMBLADO DE COMPONENTES

La estrategia de **Ensamblado de Componentes** es una decisión arquitectónica y de diseño de la solución final del software a construir, que implica desde decidir implementar por componentes, definir la granularidad del componente hasta su ensamblando final y prueba de integración.

- Es un Modelo Evolutivo de Desarrollo de Software.
- El modelo de ensamblaje de componentes incorpora muchas de las características del modelo en espiral.
- Es evolutivo por naturaleza, y exige un enfoque iterativo para la creación del software.

- Sin embargo, el modelo ensamblador de componentes configura aplicaciones desde componentes preparados de software (algunas veces llamados clases o COTS)



El desarrollo de software basado en componentes permite reutilizar piezas de código preelaborado que permiten realizar diversas tareas, conllevando a diversos beneficios como las mejoras a la calidad, la reducción del ciclo de desarrollo y el mayor retorno sobre la inversión.

¿En qué consiste la estrategia de ensamblaje de componentes?

- Comienza con la identificación de clases candidatas, examinando los datos que se van a manejar por parte de la aplicación y el algoritmo que se va a aplicar para conseguir el tratamiento, empaquetándolos en una clase.
- Las clases (componentes) creadas en los proyectos de ingeniería del SW anteriores se almacenan en una biblioteca de clases. Una vez identificadas las candidatas, se examina la biblioteca para determinar si ya existen.

Ingeniería de SW basado en componentes: Es un modelo evolutivo de desarrollo de SW basado en la reutilización de piezas de código. Es un paradigma de ensamblar componentes y escribir códigos para que estos funcionen. La complejidad de los sistemas computacionales actuales nos llevó a la reutilización de SW existente. El desarrollo de SW basado en componentes consiente en reutilizar piezas de código preelaborado (componentes) que encapsulan alguna funcionalidad expuesta a través de interfaces.

La ISBC es el proceso de definir, implementar e integrar los componentes independientes en los sistemas. La ISBC se centra en el diseño y construcción de sistemas computacionales que utilizan componentes de SW reutilizables. Trabaja bajo la filosofía de "comprar, no construir". Proporciona ventajas significativas, ya que con la reutilización pueden llevar a una reducción del 70% del tiempo de desarrollo y más del 80% del costo del proyecto. La modularidad que ofrece permite evolucionar de manera independiente los componentes hacia una mejor calidad, simplificando las pruebas y el mantenimiento del sistema.

Los componentes son abstracciones de alto nivel en comparación con los objetos y se definen mediante interfaces.

Fundamentos:

- 1. Componentes independientes que son completamente especificados por sus interfaces:** debería haber una clara separación entre la interfaz de los componentes y su implementación para que una implementación de un componente pueda reemplazarse por otro sin cambiar el sistema.
- 2. Estándares de componentes que facilitan su integración:** estos estándares definen en el nivel más bajo, cómo deberían especificarse las interfaces y cómo se comunican los componentes. Si los componentes cumplen con los estándares, entonces su funcionamiento es independiente del lenguaje de programación y, por lo tanto, los componentes escritos en diferentes lenguajes pueden integrarse en el mismo sistema.
- 3. Middleware que brinda soporte de software para la integración de componentes:** Para hacer que componentes independientes distribuidos trabajen en conjunto, es necesario soporte de middleware que maneje las comunicaciones de componentes. El middleware para soporte de componentes maneja eficientemente los conflictos de bajo nivel y permite enfocarse en problemas relacionados con la aplicación. Además, un producto middleware usado en este caso, puede proporcionar soporte para asignación de recursos, gestión de transacciones, seguridad y concurrencia.
- 4. Un proceso de desarrollo que se engrana con la ISBC:** Se necesita de un proceso de desarrollo que permita la evolución de requerimientos, dependiendo de la funcionalidad de los componentes disponibles.

Principios de diseño en la ISBC

CBSE está asentado sobre unos principios de diseño sólidos que soportan la construcción de software comprensible y mantenible:

1. Los componentes son independientes, para que no interfieran su funcionamiento no unos con otros.
2. Se ocultan los detalles de la implementación por lo que la implementación de componentes puede cambiar sin afectar al resto del sistema. (**EWV**)
3. Los componentes se comunican a través de interfaces bien definidas, de forma que, si estas interfaces se mantienen, un componente puede reemplazarse por otro que proporcione una funcionalidad adicional o mejorada. (**PTI**)
4. Las infraestructuras de componentes ofrecen varios servicios estándar que pueden usarse en varios sistemas de aplicación. Esto reduce los costos de desarrollo de aplicaciones. (**DRY**).

Problemas de la ISBC

- **Confiabilidad de los componentes:** los componentes son cajas negras y el código puede no estar disponible para los usuarios y no se puede garantizar la confiabilidad del componente dado que puede no tener documentado los modos de fallos que comprometen al sistema si se usa ese componente, su comportamiento no funcional puede no ser el esperado y, podría llegar a tener código que compromete la seguridad del sistema.
- **Certificación de componentes:** se deberían certificar los componentes para asegurar su confiabilidad, pero ¿Quién pagaría su certificación? ¿quién sería responsable si el componente no funciona de acuerdo a la certificación?
- **Predicción de propiedades emergentes:** al ser los componentes cajas negras, predecir sus propiedades emergentes es difícil. Por eso, cuando se integran componentes, el sistema podría tener propiedades no deseadas que limitan su uso.
- **Equilibrio de requerimientos:** se tiene que encontrar el equilibrio entre requerimientos ideales y componentes disponibles en el proceso de especificación y diseño del sistema. Se necesita de un método de análisis de equilibrio sistemático y más estructurado que ayude a seleccionar y configurar componentes.

Con el uso de ISBC, cumplimos con principios de diseño como componentes independientes, ocultar detalles de implementación, comunicación mediante interfaces para luego sustituir componentes, infraestructuras con servicios estándar para reutilización de código.

Componentes

Un componente puede definirse como una unidad de software independiente que puede estar compuesta por otros componentes y que se utiliza para crear un sistema de software. Las definiciones formales de componente anteriores son abstractas y no proporcionan una idea clara de lo que realmente hace un componente. Por eso, una forma útil es **considerar a un componente como un proveedor de servicios independiente:**

El componente es una entidad ejecutable independiente definida mediante sus interfaces, no necesitamos conocer su código fuente para usarlo.

Los servicios que ofrece se ponen a disposición mediante una interfaz.

Cuando un sistema necesita algún servicio, llama a un componente para proporcionar dicho servicio sin tener en cuenta donde se está ejecutando el componente o qué lenguaje de programación se ha utilizado para desarrollar el componente. Ej: un componente que proporciona un servicio de conversión de datos tal como el JSON.

Características de los componentes

Característica	Descripción
Estandarizado	Debe ajustarse a un estándar de componentes, que puede definir: interfaces, metadatos, documentación, composición e implementación.
Independiente	Debe ser posible componerlo e implementarlo sin tener que usar otros componentes.
Componible	Todas las interacciones externas deben darse mediante interfaces definidas públicamente. Debe permitir acceso externo a información sobre sí mismo. (atributos y métodos)
Implementable	Un componente debe ser independiente y debe ser capaz de funcionar como una entidad autónoma o que funcione sobre una plataforma de componentes que implemente el modelo de componentes. Significa que no debe compilarse antes de ser implementado.
Documentado	Se debe documentar y especificar la sintaxis, semántica de todas las interfaces de los componentes para que los usuarios puedan decidir si el componente satisface sus necesidades.

La visión de un componente como un proveedor de servicios resalta dos características críticas de un componente reutilizable:

1. El componente es una entidad ejecutable independiente que puede estar formado por uno o más componentes ejecutables. No necesita ser compilado y puede usarse con otros componentes.
2. Los servicios ofrecidos por un componente están disponibles a través de una interfaz y todas las interacciones son a través de esa interfaz. La interfaz del componente se expresa en términos de operaciones parametrizadas y su estado interno nunca se muestra.

Los componentes tienen dos interfaces relacionadas:

La interfaz "proporciona" define los servicios ofrecidos por el componente, es el componente API.

La interfaz "requiere" especifica qué servicios deben ofrecer otros componentes para que este opere correctamente, y sin ellos, no funcionará. Esto no compromete la independencia, ya que esta interfaz no indica cómo deben proporcionarse los servicios, sólo sabe que los necesita.



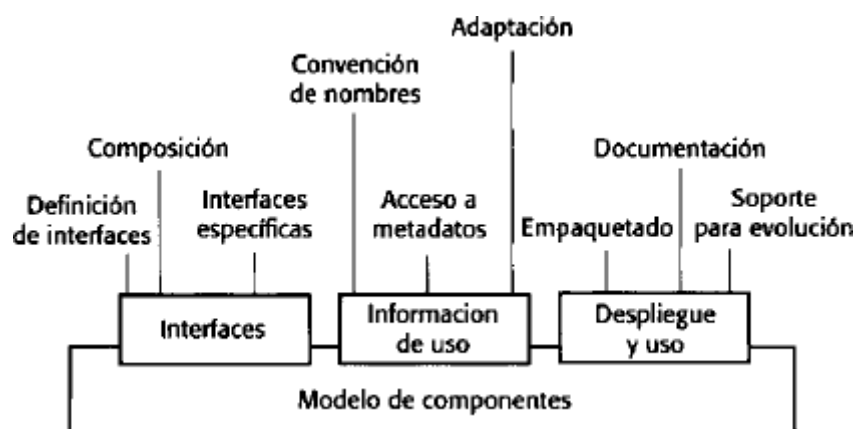
Modelos de Componentes

Un modelo de componentes es una definición de estándares para su implementación, documentación y

despliegue, y sirven para asegurarse de que estos pueden interoperar con otros. Los modelos más importantes son el modelo de componentes CORBA, el modelo Enterprise JavaBeans y el modelo COM+ (Modelo .NET) de Microsoft.

Este explica cómo deben implementarse los componentes:

- **Elementos relacionados con las Interfaces:** los componentes se definen al especificar sus interfaces, el modelo de componentes especifica cómo deben definirse las mismas y cómo debería ser el nombre de las operaciones, parámetros y excepciones que se incluyen en la definición de una interfaz, el lenguaje utilizado para definir las interfaces y si el modelo necesita interfaces específicas que se deben usar para integrar el componente con el modelo de componentes que proporciona servicios estándar.
- **Elementos relacionados con la información necesaria el uso del componente:** deben tener nombres únicos que los identifiquen. Los componentes son entidades genéricas y deben configurarse para ajustarse en un sistema de aplicación. El modelo de componentes puede especificar cómo pueden personalizarse estos componentes en cada caso. Los metadatos de un componente son datos acerca del componente en sí, tales como información acerca de sus interfaces y atributos. Los metadatos son importantes porque permiten a los usuarios del componente determinar qué servicios se proporcionan y requieren.
- **Elementos relacionados con la implementación:** el modelo de componentes incluye una especificación de cómo deben empacarse los componentes para su implementación como entidades ejecutables independientes. Además, como los requerimientos cambian o surgen nuevos, los componentes también deberán cambiarse o reemplazarse. Por eso, el modelo de componentes debe incluir reglas para regular cuándo y cómo se permite el reemplazo.



Soporte de Middleware

Los servicios proporcionados por la implementación de modelo de componentes se dividen en dos categorías:

1. Servicios de plataforma: son servicios fundamentales que permiten a los componentes

comunicarse entre sí e interoperar en un entorno distribuido. Ej: CORBA.

2. Servicios de soporte o servicios horizontales: son servicios comunes que probablemente requieran ser usados por muchos componentes diversos. La disponibilidad de estos servicios reduce los costos de desarrollo de componentes y evita incompatibilidades.

Para usar los servicios ofrecidos por la infraestructura de un modelo de componentes, los componentes se implementan en un "contenedor" estandarizado predefinido. Un contenedor es un conjunto de interfaces utilizadas para acceder a las implementaciones de los servicios de soporte. La inclusión del componente en el contenedor proporciona automáticamente el acceso a los servicios.

Procesos ISBC

Son procesos de software que brindan soporte a la ISBC, toman en cuenta posibilidades de reutilización y las actividades de proceso implicadas en el desarrollo y uso de componentes reutilizables. Podemos mencionar dos tipos:

1. **Desarrollo para reutilización:** busca producir uno o más componentes reutilizables
2. **Desarrollo con reutilización:** busca desarrollar nuevas aplicaciones REUTILIZANDO componentes existentes.

En el desarrollo con reutilización, no se sabe cuáles están disponibles, así que se necesita descubrir dichos componentes y diseñar un sistema para utilizarlos de la manera más efectiva. No puede tener acceso al código fuente del componente.

1. **Adquisición de componentes** es el proceso de adquirir componentes para reutilización o desarrollo de un componente reutilizable.
2. **La gestión de componentes** se ocupa de catalogar los componentes y almacenarlos y organizarlos para su uso.
3. **La certificación de componentes** comprueba componentes para asegurar que cumplen su especificación.

ISBC para reutilización - Desarrollo de componentes para reutilización

Es el proceso de desarrollar componentes reutilizables y ponerlos a disposición para reutilizarlos a través de un sistema de gestión de componentes, estos componentes pueden desarrollarse generalizando componentes ya existentes. Esto tiene un costo asociado, por lo tanto, en primer lugar, hay que decidir si un componente realmente va a reutilizarse y, en segundo lugar, hay que evaluar si el ahorro que va a significar el componente reutilizado justifica el costo de hacer que este componente sea reutilizable.

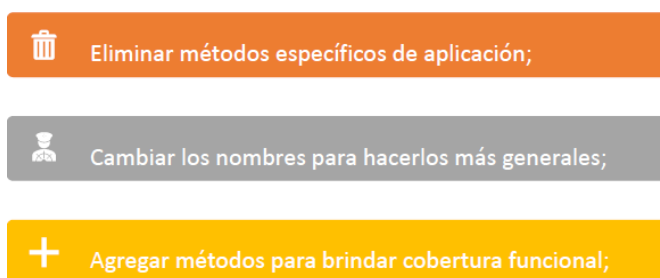
Para responder a la primera cuestión, es decir a la reutilización del componente debemos tener en cuenta:

- Debería implementar una o más abstracciones del dominio estables, es decir, un concepto

del dominio de aplicación que cambia muy lentamente. Ej: en un banco, las cuentas, los depósitos y las extracciones.

- Todas las excepciones que se pueden producir deberían publicarse en la interfaz del componente
- Deberían esconder la representación del estado.
- Deberían ser lo más independiente posible

Los cambios que pueden hacerse para que un componente sea más reutilizable incluyen:



Debe existir un equilibrio entre la reutilización y la usabilidad, a medida que se incrementa la generalidad de un componente, aumenta su reusabilidad ya que hacer que un componente sea reutilizable implica

proporcionar interfaces con operaciones que abarcan todas las formas en las cuales puede ser usado el componente. Hacer que un componente sea usable significa proporcionar una interfaz mínima sencilla

que sea fácil de comprender. La reusabilidad añade complejidad y por eso, dificulta la comprensibilidad. En cuanto al manejo de excepciones, el componente no debería hacerse cargo de esto, sino que debería devolver las mismas para que el software que aplique los componentes pueda manejarlos como desee.

Esto acarrea dos problemas:

- que se devuelvan todas las excepciones puede inflar las interfaces y complicarlas
- si el componente depende de las excepciones y de su manejo, podemos tener problemas con la funcionalidad

ISBC con reutilización

La ISBC con reutilización debe incluir actividades que encuentren e integren componentes reutilizables.



La validación de componentes consiste en probar los componentes para asegurarse de que se comportan como se espera. Consiste en desarrollar un conjunto de casos de prueba para un componente.

Diferencias con los procesos de software originales:

1. Los requerimientos se esbozan, no se especifican con detalle.
2. Se alienta a los usuarios y stakeholders a ser flexibles en la definición de requerimientos
3. Se necesita un conjunto completo de requerimientos para poder identificar tantos componentes como sean posibles que cumplan con los mismos.
4. Los requerimientos son refinados en etapas tempranas del proceso en función de los componentes disponibles. Si no se pueden satisfacer, se discute con los usuarios que, son flexibles a cambios de idea para que les entreguen un software más rápido y económico.
5. Hay una actividad adicional de identificación y refinamiento de componentes después del diseño arquitectónico dado que puede que algunos componentes pueden resultar no adecuados o pueden no trabajar adecuadamente con otros componentes seleccionados. Esto implica que puede haber cambios en los requerimientos también.
6. La composición de componentes sería el proceso de desarrollo, en donde se integran los componentes con la infraestructura del modelo de componentes. A veces se requiere escribir "código pegamento" para integrar las interfaces de los componentes incompatibles.

Características importantes de algunas etapas del proceso de CBSE:

- La etapa del diseño arquitectónico es muy importante dado que se define el modelo de componentes a utilizar, se establece la organización de alto nivel y se toman decisiones sobre la distribución y control del sistema. Mientras más robusta sea la arquitectura la reutilización resultará exitosa.
- La identificación de componentes es una etapa única. Ésta implica tres subactividades:
 - O Búsqueda de componentes: se puede buscar entre componentes disponibles localmente o que sean de proveedores de confianza.
 - O Selección de componentes: se seleccionan los componentes que cumplan con los requerimientos. Puede ser fácil si hay una correspondencia directa o puede complicarse incluso si hay requerimientos que requieren distintos grupos de componentes. Habrá que decidir qué grupo de componentes nos dan el mejor cumplimiento de los requerimientos.
 - O Validación de componentes: sirve para comprobar que los componentes se comportan como deberían hacerlo. Si el componente proviene de una fuente de confianza, la prueba individual no será necesaria y se lo prueba sólo cuando se lo integra con otros. Si viene de una fuente desconocida siempre se deberá probarlo antes de incluirlo en el sistema. La validación implica desarrollar un conjunto de casos de prueba y desarrollar software adicional de pruebas para ejecutar esas pruebas. Acá pueden presentarse dos problemas: puede que la documentación del componente no sea tan detallada como para escribir los casos de prueba y, además puede haber funcionalidades no deseadas que interfieren con el uso del componente pudiendo llegar a causar fallas graves del sistema.

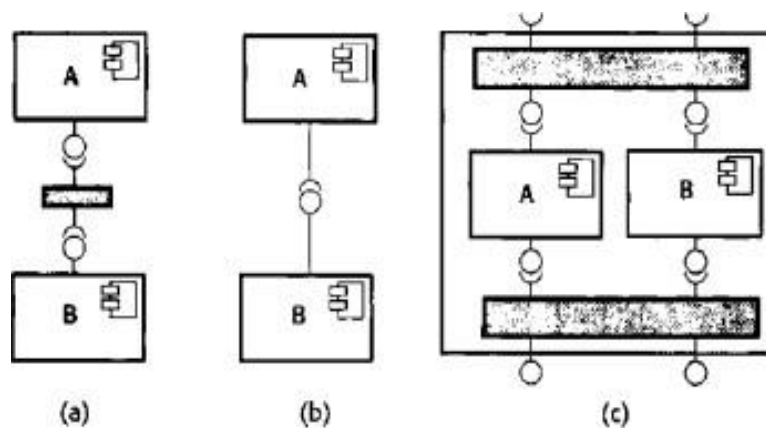
Composición de componentes

Proceso de integrar componentes con "código pegamento", escrito para crear un sistema u otro

componente.

Hay diferentes formas de hacerlo:

- **Composición secuencial (A):** se crea un nuevo componente a partir de dos componentes existentes, al llamarlos en secuencia. Se requiere código de pegamento adicional para llamar a los servicios del componente en el orden correcto y asegurar que los resultados entregados por el componente A sean compatibles con las entradas esperadas por el componente B. Este tipo de composición puede usarse con componentes que son elementos de programa o con componentes que son servicios.
- **Composición jerárquica (B):** ocurre cuando un componente, la interfaz "requiere", llama a los servicios de otro componente, el cual sería la interfaz "proporciona" y debe ser compatible con la interfaz de quien lo llamó. Puede no haber código pegamento.
- **Composición aditiva (C):** dos o más componentes se juntan para crear uno nuevo, con nueva funcionalidad combinada. Las interfaces del componente compuesto (resultante) se crean uniendo todas las interfaces de los componentes constituyentes eliminando las operaciones duplicadas si es necesario. Este tipo de componente puede usarse con componentes que son unidades de programa o con componentes que son servicios. A y B no son dependientes y no se llaman mutuamente.



Incompatibilidad de interfaces

Pueden presentarse problemas al utilizar más de dos componentes en conjunto.

- **Incompatibilidad de parámetro:** las operaciones de las interfaces tienen el mismo nombre, pero sus tipos o cantidad de parámetros son diferentes
- **Incompatibilidad de operación:** los nombres de las operaciones en las interfaces tienen diferente nombre
- **Operación incompleta:** la interfaz "proporciona" es un subconjunto de la interfaz "requiere" o viceversa

Pueden aparecer conflictos entre requerimientos funcionales y no funcionales, y conflictos entre las necesidades de entrega rápida y la evolución del sistema.

Será necesario decidir cuestiones tales como:

- ¿Qué composición de componentes es más efectiva para entregar los requerimientos funcionales?
- ¿Qué composición de componentes facilitará cambios futuros?
- ¿Cuáles serán las propiedades emergentes del sistema compuesto? Se trata de propiedades como rendimiento y confiabilidad. Sólo podrá valorarlas luego de implementado el sistema completo.

Beneficios

Estudios informan que el ensamblaje de componentes lleva a:

- Una reducción del 70% del tiempo de ciclo de desarrollo.
- Un 84% del costo del proyecto.
- Un índice de productividad del 26,2 comparado con la norma de industria del 19,9.

Aunque estos resultados están en función de la robustez de la biblioteca de componentes, no hay duda que el ensamblaje de componentes proporciona ventajas significativas para los ingenieros de software.

Ventajas

- Aumento de reutilización de software.
- Simplifica las pruebas. Pruebas unitarias antes de probar el conjunto completo de componentes ensamblados.
- Simplifica el mantenimiento del sistema. Cuando existe un débil acoplamiento entre componentes, el desarrollador es libre de actualizar y/o agregar componentes según sea necesario, sin afectar otras partes del sistema.
- Mayor calidad. Dado que un componente puede ser construido y luego mejorado continuamente por un experto u organización, la calidad de una aplicación basada en componentes mejorará con el paso del tiempo.

Patrones de Diseño (Solución concreta a un problema bien definido)

Patrón: Solución concreta a un problema bien identificado.

¿Qué es un patrón de diseño? Son genéricos, son soluciones habituales a problemas que ocurren con frecuencia en el diseño de software.

Cada patrón describe un problema que ocurre una y otra vez en un determinado entorno, así como la solución a ese problema.

Un patrón de diseño describe una estructura de diseño que resuelve un problema particular y recurrente de diseño dentro de un contexto, está pensado y descripto para funcionar siempre en cualquier lenguaje

de programación. Buscan dar una descripción que permita a un diseñador determinar si el patrón es aplicable al trabajo en cuestión, puede volverse a usar y sirve como guía para desarrollar un patrón distinto en funciones o estructura.

No hay que resolver cada problema partiendo de cero, hay que reutilizar soluciones que ya han sido útiles en el pasado. Los patrones resuelven estos problemas concretos y hacen que los diseños OO sean más flexibles, elegantes y reutilizables.

Estos patrones se aplican a nivel de código, es el nivel de patrón más bajo porque definen las clases y cómo se van a relacionar y que métodos se usarán para resolver un problema concreto de diseño. El problema mayormente se relaciona con cohesión o acoplamiento. No dependen de ningún lenguaje de programación en particular.

La diferencia con los patrones arquitectónicos es que éstos son a nivel de capas, mientras que los patrones de diseño son a nivel de clases concretas o componentes de código, es a un nivel alto de granularidad.

Los patrones de diseño:

- Refinan un subsistema o componente de un sistema software.
- Describen una estructura a la cual muchas veces recurrimos para resolver problemas de diseño
- Son patrones de escala media
- No afectan a la arquitectura de un sistema.
- No dependen del lenguaje de programación.
- Permiten resolver problemas complejos y direccionan la cooperación efectiva entre componentes.

Diferencias entre PATRONES E IDIOMAS

- Un idioma es un patrón de diseño adaptado o especializado a un lenguaje de programación específico y con recomendaciones específicas para ese lenguaje.
- La diferencia que existe entre los mismos es el alcance, ambos describen soluciones a problemas de diseño recurrentes. En el patrón de diseño tanto el problema como la solución son lo suficientemente genéricos para ser independientes del lenguaje de programación mientras que, en el idioma, es un patrón de diseño que es específico para un lenguaje de programación.

Elementos de un patrón de diseño

1. Nombre: describe con una o dos palabras un problema de diseño junto con sus soluciones y consecuencias.

o Incrementan el vocabulario de diseño: nos permite hablar con otros colegas, mencionarlos en documentación y tenerlos en cuenta. Más fácil pensar diseños y transmitirlos a otros.

O Nos permiten diseñar con abstracción de un nivel más alto.

2. Problema: describe cuando aplicar el patrón. Explica el problema y su contexto.

3. Solución: describe de forma abstracta los elementos que forman el diseño, sus relaciones, responsabilidades y colaboraciones.

O No describe ningún diseño concreto.

O Es como una plantilla que puede aplicarse en muchas situaciones diferentes.

4. Consecuencias: son los resultados, así como las ventajas e inconvenientes de aplicar el patrón.

O Son críticas para evaluar el patrón y entender los costos y beneficios de aplicarlo.

O Suele relacionar el equilibrio entre el espacio y el tiempo.

O Puede tratar asuntos de lenguajes e implementaciones.

5. Otros elementos: Alias, Participantes (clases afectadas en la implementación del patrón)

¿Por qué utilizar patrones de diseño?

Porque te enseña a resolver todo tipo de problemas utilizando principios del diseño orientado a objetos.

- **Facilita la comunicación** en el equipo de trabajo, ya que con el nombre (si el equipo lo conoce) identificamos el problema a resolver.
- **Facilitan la reutilización** de buenos diseños y arquitecturas.
- **Nos ayudan a elegir alternativas de diseño** que hacen que un sistema sea reutilizable, y a evitar fallas que dificultan dicha reutilización.
- Pueden **mejorar la documentación y mantenimiento** de sistemas existentes.
- Ayudan a lograr un **buen diseño** rápidamente (mejoran la calidad de diseño).

¿Qué es un patrón de diseño?

Patrón: cada patrón describe un problema que ocurre una y otra vez en un determinado entorno, así como la solución a ese problema, de tal modo que se pueda aplicar esta solución un millón de veces, sin hacer lo mismo dos veces.

Tipos de patrones de diseño

Los patrones de diseño son 23 y, se clasifican según su propósito:

1. **Patrones de creación:** Los patrones de diseño de creación abstraen el proceso de creación de instancias (de objetos). Ayudan a hacer un sistema independiente de cómo los objetos son creados, compuestos y representados.
2. **Patrones estructurales:** Explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que mantiene la flexibilidad y eficiencia de la estructura. Tratan la composición de clases y objetos
3. **Patrones de comportamiento:** se encargan de una comunicación efectiva y la asignación de responsabilidades entre objetos.

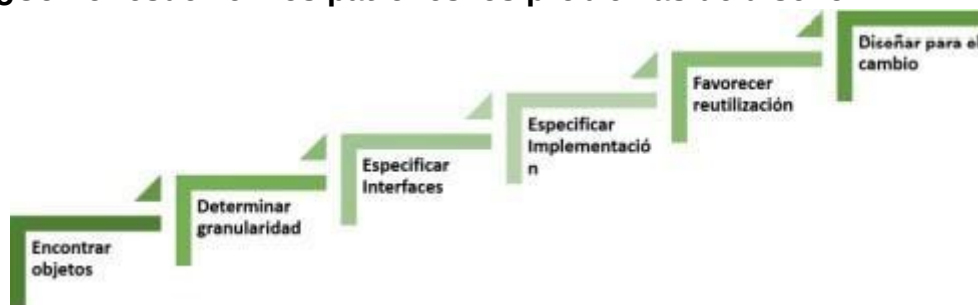
Ámbito:

- **Patrones de clase:** se ocupan de las relaciones entre clases y subclases, establecidas por herencia (relaciones estáticas)
- **Patrones de objetos:** tratan relaciones entre objetos, dinámicas, que cambian en tiempo de ejecución.

Los patrones de creación de clases delegan parte del proceso de creación de objetos a las subclases, mientras que los patrones de creación de objetos lo hacen en otro objeto. Los patrones estructurales de clase usan la herencia para componer clases, mientras que los de objetos describen formas de ensamblar objetos. Los patrones de comportamiento de clases usan herencia para describir algoritmos y flujos de control, mientras que los de objetos describen cómo cooperan un grupo de objetos para realizar una tarea que ningún objeto puede realizar por sí solo.

		PROPÓSITO		
		Creación	Estructural	Comportamiento
ÁMBITO	Clase	Factory Method	Adapter (de clases)	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter (de objetos) Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

¿Cómo resuelven los patrones los problemas de diseño?



Encontrar los objetos apropiados (Clases de fabricación pura)

Muchas veces, lo complicado del diseño orientado a objetos es descomponer un sistema en objetos, ya que entran en juego factores como el encapsulamiento, granularidad, dependencia, flexibilidad, rendimiento, reutilización, etc. Las abstracciones que surgen durante el diseño son fundamentales para flexibilizar el diseño. Los patrones ayudan a identificar esas abstracciones. no tan obvias implementadas mediante clases de fabricación pura (objetos boundary y objetos de control), clases que se inventan, se agregan para construir la solución, estas clases tienen que ver con el dominio de la solución y que son fundamentales para lograr un diseño flexible, bajar el acoplamiento y aumentar la cohesión. **Strategy y State.**

Clases de fabricación pura (las que te da el patrón), clases que forman parte del dominio de la solución.

Buscamos clases de fabricación pura, clases de diseño, que forman parte del dominio de la solución. Son clases que agregamos nosotros que no tienen una traza con el dominio del problema porque no se pueden rastrear, son inventos nuestros. Esto nos ayuda a tener una alta cohesión, bajo acoplamiento, SW flexible y extensible. Cada patrón va a ir agregando clases que tengan que ver con la solución.

Determinar la granularidad de los objetos

Debemos decidir qué debería ser un objeto. Los patrones de diseño nos ayudan a tomar decisiones de cuántas responsabilidades asignarle a una clase, cuanto queremos que haga cada clase (cuántos métodos debería tener). La granularidad está relacionada a las responsabilidades que voy a asignar a una clase, entonces, debemos decidir qué responsabilidades tendrá la clase. Si una clase respeta el principio de responsabilidad única (SRP), el nivel de granularidad es fina, es decir, la clase va a tener muy pocos métodos que tienen que ver con el concepto que modela, y una con granularidad gruesa no cumplirá la cohesión.

Builder.

Granularidad fina => Alta Cohesión (Responsabilidad mínima a las clases)

Granularidad gruesa => Baja cohesión (Las clases hacen muchas cosas)

Especificar las interfaces de los objetos

Se aplica el principio "Programar hacia la interfaz y no hacia la implementación". Se reemplaza el nivel de acoplamiento con una interfaz ya que nos conviene que las dependencias se hagan con abstracciones y no con concreciones

Los patrones nos ayudan a que aparezcan interfaces que nos permiten manejar el nivel de acoplamiento y manejar la transparencia para los clientes, sabiendo con quien se tiene que comunicar. Los patrones me ayudan a establecer lineamientos que cumplan con la segregación de interfaces.

Especificamos interfaces para que el cliente deje de comunicarse con cada clase concreta necesitando algo, entonces a través de una interfaz se mejora el nivel de acoplamiento comunicándose sólo con ella. Es que los patrones ayudan a identificar interfaces para manejar el nivel de acoplamiento y manejar la transparencia hacia el cliente, mejorarle la vida al cliente para simplificar a quien debe comunicarse (interfaz).

Realización => Ventaja de que se resuelve dinámicamente (tiempo de ejecución), no rompe encapsulamiento, no rehusó código (métodos vacíos)

Las interfaces son fundamentales en el diseño orientado a objetos, ya que los objetos sólo se conocen a través de su interfaz. La interfaz de un objeto no dice nada acerca de su implementación (dos objetos con implementaciones diferentes pueden tener interfaces idénticas).

Cada operación declarada por un objeto especifica el nombre, parámetros y retorno de la misma (**signatura** o **firma**). Al conjunto de todas las signaturas definidas por las operaciones de un objeto se le denomina **interfaz del objeto**, la cual caracteriza al conjunto completo de peticiones que se le pueden enviar al objeto.

Lo que tengo en una interfaz es la signatura de los métodos (nombre del método, parámetros y retornos) pero, está vacío, sin código. Lo que se reusa es la signatura del método y no su código, de esta forma se aplica el polimorfismo mediante una relación de realización. Con esto, se garantiza que se favorece a una clase que es la que necesita acceder a los servicios de la interfaz porque se relaciona con una sola clase y ahí se encuentra la signatura de los métodos independientemente de cómo cada clase concreta lo resuelve. No se rompe el encapsulamiento y se resuelve dinámicamente en tiempo de ejecución en el momento de atender la petición.

Ejemplo: Memento, Decorator, Proxy.

Especificar las implementaciones de los objetos (Como escribir el código que va en cada clase)

Cuando hablamos de implementación hablamos de código que va dentro de los métodos, se relaciona con el patrón State, ayuda con la especificación de implementaciones, cuando definís todos los comportamientos en la clase padre, definimos como se va a implementar determinado comportamiento que después los estados concretos van a tomar y lo van a tomar tal cual o lo redefinen.

Hay otros patrones que definen lineamientos de cómo va a ser el algoritmo que se va a implementar.

Ej: TM me define una forma de escribir un algoritmo que todas las clases hijas tienen que respetar la estructura de herencia para que el método plantilla funcione.

La implementación de un objeto queda definida por su clase, la cual especifica los datos, representación interna y operaciones que el objeto puede realizar. Los objetos se crean instanciando una clase, lo que ocupa espacio de almacenamiento para los datos del objeto y asocia las operaciones con esos datos.

Cuando se hace reusó de comportamiento/ de código el uso de la herencia queda justificado, pero para que la herencia este bien diseñada, el hijo se tiene que comportar como el padre. Hay algunos patrones que sólo tienen sentido si se usan con herencia, si no pierden el propósito

Una clase abstracta es aquella cuyo propósito principal es definir una interfaz común para sus subclasses. Esta delegará parte o toda su implementación a sus subclasses; las operaciones abstractas (definidas en una clase abstracta, pero no implementadas) se definen en las clases concretas.

- Muchos patrones dependen de la distinción entre Herencia de clases y herencia de interfaces.

Recordando:

Herencia de clases (generalización): define la implementación de un objeto en términos de la implementación de otro objeto. Es un mecanismo para compartir código y representación.

Herencia de interfaces (realización): describe cuando se puede usar un objeto en lugar de otro.

- Patrones de creación permiten que un sistema esté basado en términos de interfaces y no en implementaciones.
- Favorece reutilización
- Delegación.

Los patrones que especifican las implementaciones de objetos son: **Observer, State,**

Manipular los objetos solamente en términos de la interfaz definida por las clases abstractas tiene 2 ventajas:

1. Los clientes no tienen que conocer los tipos específicos de los objetos que usan, basta con que éstos se adhieran a la interfaz que esperan los clientes.
2. Los clientes desconocen las clases que implementan dichos objetos.

Favorecer la reutilización (Reutilizar código, sobre todo con caja negra) Estos patrones recomiendan **favorecer la composición sobre la herencia**. Solemos abusar de la herencia, por lo que es mejor aplicar reutilización anticipándonos a nuevos cambios para que el sistema pueda evolucionar de forma adecuada. Cada patrón permite que algunos aspectos de la estructura del sistema puedan cambiar independientemente de otros aspectos, facilitando el reusó, extensibilidad y mantenimiento.

- Hay que tener en cuenta las 3 relaciones que permiten reusar de alguna manera: **Caja blanca** (Herencia) y **Caja Negra** (Composición). Pero, estos patrones recomiendan favorecer la composición sobre la herencia. Tender al reuso de caja negra con delegación.
- Herencia vs composición (trabajan juntas).
- Delegación.
- La clave para la reutilización es anticiparse a los nuevos requisitos y cambios en los mismos, de modo que los sistemas evolucionen de forma adecuada.
- Cada patrón permite que algunos aspectos de la estructura del sistema puedan cambios independientemente.
- Facilitan reuso interno, extensibilidad, mantenimiento

Las técnicas más comunes para reutilizar funcionalidad en sistemas OO son el reuso de caja blanca y reuso de caja negra, las relaciones de clases son la herencia de clases/interfaces y la composición (asociación, agregación) de objetos.

Herencia: Rompe el encapsulamiento para que las clases hijas tomen estructura y

comportamiento del padre, hay una dependencia muy fuerte, relación entre clases, no se traslada a los objetos, en tiempo de compilación, la solución sea más rígida, menos flexible

Composición: Relación de todo a parte, se define en tiempo de ejecución se traslada a los objetos, no rompe el encapsulamiento, sw más flexible. Hace reuso de caja negra a través de la delegación. Delegación es aprovechar el código que está escrito en otra clase, sin tener que romper el encapsulamiento y sin tener que escribir el código en la clase contexto, invoca al método para que otra clase lo resuelva.

Una clase tiene chances de ser reutilizada si tiene un bajo acoplamiento, cohesión funcional. Relacionado con el principio DRY, TDA

Diseñar para el cambio (Diseñar aceptando que los productos de SW van a cambiar, por lo tanto contribuyen a diseñar estructuras flexibles y mantenibles)

Cada patrón permite que algunos aspectos de la estructura del sistema puedan cambiar independientemente de otros. Apunta a diseñar de forma que el diseño sea flexible, robusto y fácil de evolucionar, dado que los requerimientos cambian y la demanda de cambio en el software siempre va a existir. Un diseño que no tenga en cuenta el cambio sufre el riesgo de tener que ser rediseñado por completo en el futuro. Un diseño que no tenga en cuenta el cambio sufre el riesgo de tener que ser rediseñado por completo en el futuro. Cada patrón de diseño deja que algún aspecto de la estructura del sistema varíe independientemente de los otros, haciendo así al sistema más robusto y flexible frente a un tipo de cambio concreto.

Causas comunes de rediseño y patrones que lo resuelven:

Crear un objeto especificando su clase explícitamente; esto nos liga a una implementación concreta en vez de a una interfaz. Abstract factory, factory method, prototype

Herencia de clases frente a herencia de interfaces

La clase de un objeto define cómo se implementa un objeto, su estado interno e implementación de sus operaciones. Por el contrario, el tipo de un objeto sólo se refiere a su interfaz; si decimos que un objeto es instancia de una clase, decimos que el mismo admite la interfaz definida por la clase. La herencia de clases define la implementación de un objeto en términos de la implementación de otro objeto. La herencia de interfaces describe cuándo se puede usar un objeto en lugar de otro.

Programar para interfaces, no para implementación

Si la herencia se usa con cuidado, todas las clases que derivan de una clase abstracta compartirán su interfaz, podrán agregar o redefinir operaciones, por lo que todas las subclases podrán responder a las peticiones de la interfaz y convertirse en subtipos de la clase abstracta. Manipular los objetos sólo en términos de la interfaz definida por la clase abstracta tiene dos ventajas:

Los clientes no tienen que conocer los tipos específicos de los objetos que usan, siempre que estos se

adhieran a la interfaz que el cliente espera.

Los clientes desconocen las clases que implementan estos objetos, sólo conocen a la clase abstracta

Dependencia de operaciones concretas; esto nos liga a una única forma de satisfacer una operación. Chain of responsibility, command

Dependencia de plataformas HW o SW; las interfaces de los sistemas operativos y las APIs varían para diferentes plataformas, el software no debe depender de una plataforma concreta. Abstract factory, bridge

Dependencia de las representaciones o implementaciones de objetos; los clientes de un objeto no deberían conocer cómo se representa, almacena, localiza o implementa el mismo, ya que si el objeto varía, podría variar el cliente también. Ocultando estos datos, podemos evitar modificaciones en cascada. Abstract factory, bridge, memento, proxy

Dependencias algorítmicas; los algoritmos que puedan variar deben estar aislados, ya que si cambian, los objetos que dependan de él deberán cambiar también. Builder, iterator, strategy, template method, visitor

Fuerte acoplamiento; hace que las clases sean difíciles de reutilizar por separado. Abstract factory, bridge, chain of responsibility, command, facade, mediator, observer

Añadir funcionalidad mediante herencia; debemos añadir funcionalidades mediante definiciones en subclases y componiendo sus instancias con otras existentes. Bridge, chain of responsibility, composite, decorator, observer, strategy

Incapacidad para modificar las clases convenientemente; adapter, decorator, visitor.

Como los patrones y los frameworks tienen similitudes, mucha gente se pregunta en qué se diferencian, si es que se diferencian en algo. Son diferentes en tres aspectos fundamentales:

Los patrones de diseño son más abstractos que los frameworks. Los frameworks pueden plasmarse en código, pero solo los ejemplos de los patrones pueden ser plasmados en código.

Los patrones de diseño son elementos arquitectónicos más pequeños que los frameworks. Un framework típico contiene varios patrones de diseño, pero lo contrario nunca es cierto.

Los patrones de diseño están menos especializados que los frameworks. Los frameworks siempre tienen un dominio de aplicación concreto. Sin embargo, los patrones de diseño de este catálogo se pueden usar en prácticamente cualquier tipo de aplicación

Características de un buen diseño

- Independencia de los componentes, medida según el grado de cohesión y acoplamiento
- Identificación y tratamiento de excepciones

- Prevención y tolerancia de defectos
- Y debemos tener en cuenta la reutilización de código; menos tiempo de desarrollo significa entrar al mercado antes que los competidores. Existen 3 niveles; bajo nivel (reutiliza clases, bibliotecas...), nivel medio (patrones), alto nivel (frameworks).

Patrones de diseño

Patrones de creación

Los patrones de diseño de creación abstraen el proceso de creación y ocultan los detalles de cómo los objetos son creados, mejorando y flexibilizando tal proceso a través de la distribución de responsabilidades. Nos permiten manejar el que se crea, cómo se crea, quién lo crea y cuándo se crea. Ayudan a hacer un sistema independiente de cómo los objetos son creados, compuestos y representados.

Dos tipos:

- Patrón de creación de clase: usa la herencia (puede ser generalización o realización) para cambiar la clase que es instanciada.
- Patrón de creación de objeto: DELEGA la creación de la instancia a otro objeto. (Composición)

Hay dos temas recurrentes en estos patrones:

- Todos encapsulan el conocimiento sobre las clases concretas que usa el sistema.
- Todos ocultan como se crean y se asocian las instancias de estas clases.

Todo lo que el sistema conoce de los objetos son sus interfaces. Flexibilizar el proceso de creación (qué se crea, quién, cómo y cuándo se crea), en Distribuir responsabilidades en distintas clases que se van a hacer cargo de cosas distintas (No tener una clase de que se haga cargo de la creación o de todo)

Propósito específico: La intención de los patrones de creación es apuntar a la flexibilidad en el proceso de delegación haciendo distribución de responsabilidades, para ello los patrones crean nuevas clases de fabricación pura. Busca que haya flexibilidad en qué se crea, quién lo crea, cómo se crea y cuándo se crea.

Patrones de creación

1. **Abstract Factory:** proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin especificar sus clases concretas.
2. **Factory Method:** define una interfaz para crear un objeto, pero deja que las subclasses decidan qué clase instanciar. Delega en las subclasses la creación de objetos.

3. **Singleton**: garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.
4. **Builder**: separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones.
5. **Prototype**: especifica los tipos de objeto a crear por medio de una instancia prototípica, y crea nuevos objetos copiando de este prototipo.

BUILDER

Propósito:

Permite la construcción de objetos complejos paso a paso. Separa la construcción de un objeto complejo de su representación. Se utiliza cuando sabemos qué necesitamos crear, pero es necesario crearlo por partes y tener el control del proceso de creación. Se usa para la creación de objetos complejos, que tienen muchas partes (objetos compuestos) y que se mantiene un control en el proceso de creación (paso1, paso2, paso3, etc). El resultado es un objeto complejo, compuesto por partes.

Se hace una separación de intereses (el objeto que va a crear, el que lo usa, el momento de creación y responsable de creación), donde se distribuyen responsabilidades entre las clases de fabricación pura. Ejemplos de objetos complejos: Pantallas, reportes, estadísticas, etc.

Director (Clase de fabricación pura) responsable de tener la lógica en el orden de cómo crear por parte y en dirigir el proceso de construcción, da órdenes.

Constructor Concreto (ya existía) es el responsable de crear los objetos productos, el que hace (el que trabaja).

El actor elige el escenario y sabe que ConstructorConcreto va a usar en la dinámica y lo crea. Cada constructor concreto es el responsable de crear su producto concreto que es el que crea al final del proceso de creación. El **contexto** tiene una dependencia con el **IConstructor** para crear el constructor Concreto.

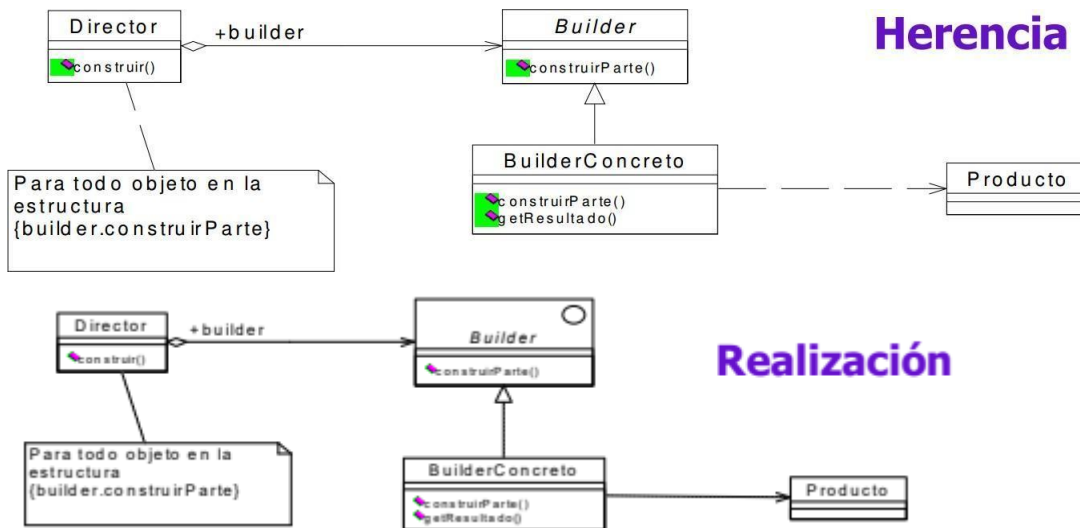
El contexto siempre delega la responsabilidad al Director de crear pasandole la lógica.

1ro Se crea el constructor Concreto y se lo pasa por parámetro al director cuando se lo crea, para que el director dirija el proceso de construcción.

Los constructores concretos implementan polimórficamente los métodos que están definidos en el IConstructor. Una vez finalizado el proceso de construcción, el contexto le pide al Constructor todo lo que creo, así lo puede utilizar.

Propósito: Separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones.

Herencia o Realización



Estructura:

- La interfaz del Constructor declara los pasos de construcción del producto, comunes a todos los tipos de constructores.
- Constructores Concretos proporcionan diferentes implementaciones de los pasos de construcción, sin necesariamente seguir una interfaz común
- Los productos son los objetos resultantes
- La clase Director define el orden en el que llamar a los pasos
- El Cliente debe asociar uno de los objetos constructores con el director

Beneficios y desventajas:

- + Podemos construir objetos paso a paso, aplazar pasos, ejecutarlos recursivamente. Tenemos un control más fino del proceso de construcción y de la estructura interna del producto resultante.
- + Podemos reutilizar el código de construcción
- + Principio de responsabilidad única, aislamos el código de construcción. (SOLID)
- + Permite variar la representación interna de un producto
- + La complejidad general del código aumenta al crear muchas clases

Principios de diseño OO se aplican en el patrón Builder()

Responsabilidad Única (RSP): Cada constructor concreto va a tener la responsabilidad de crear un único producto.

Programar hacia la interfaz (PTI): Desacoplar la dependencia haciendo que un director se comunique hacia una interfaz y no hacia los constructores concretos.

Open-Close(O/C): Si queremos agregar constructores debemos implementar el interfaz constructor.

Ordena no pregunte (TDA): El contexto le ordena.

FACTORY METHOD

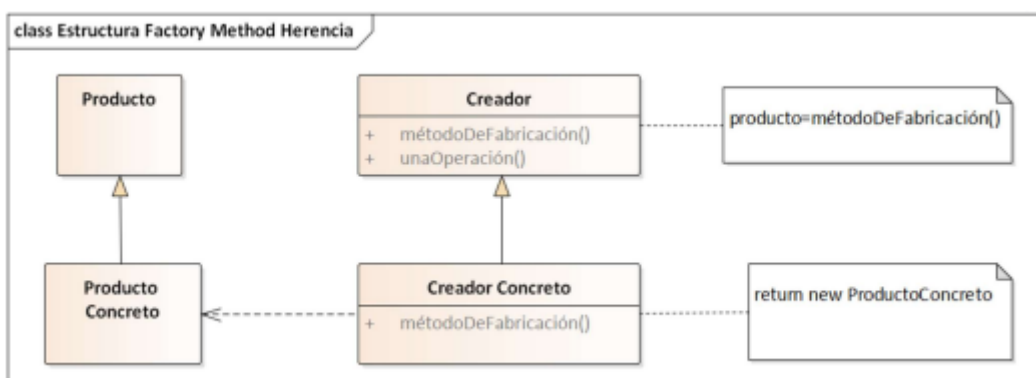
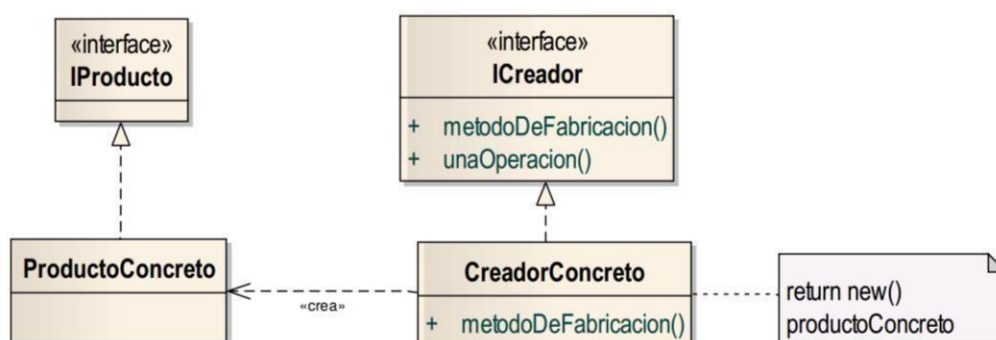
Centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la diversidad de casos particulares que se pueden prever, para elegir el subtipo que crear.

(Define una interfaz para crear un objeto, pero deja que la subclase decida qué clase instanciar. Delega en las subclases la creación de objeto)

Se puede usar en Herencia y Realización. Tiene la libertad o flexibilidad de postergar lo que más se pueda, la decisión de cuál es el producto concreto que tiene que crear, entonces se especializa el método de construcción de acuerdo al producto concreto que tiene que construir.

El producto concreto necesita de una interfaz común (a Diferencia del Builder), para todos los productos concretos, que se va a especializar según lo que necesitemos. El Builder crea objetos complejos a diferencia del Factory Method que crea objetos simples.

MetodoDeFabricacion() -> (Como template ya que el patrón se aplica a un MÉTODO!) Es un método genérico, en el cual lo implementa cada creador Concreto donde crea su producto específico (Por eso existe la dependencia del creador concreto y el producto que crea)



El método tiene toda la información para crear, y el retorno del método es el objeto creado. El contexto se puede convertir en la interfaz Creador.

Propósito:

Patrón que proporciona una interfaz para crear objetos en una superclase, y permite que las subclases alteren el tipo de objetos que se crearán. Así, podremos delegar la responsabilidad de creación de objetos a una clase, sin elegir qué objetos crear hasta la ejecución.

Solución:

El patrón sugiere un reemplazo de las llamadas directas a la construcción de objetos con llamadas a un método Factory. Los objetos devueltos por el método se denominan 'productos', deben compartir interfaz, y el método devolverá productos de esta interfaz.

Estructura:

El producto declara la interfaz común de los objetos a crear

Los productos concretos implementan la interfaz

La clase Creador declara el método de factoría, que debe devolver un objeto de la interfaz mencionada
CreadorConcreto anula el método de factoría base y lo adapta con el retorno que necesita.

Aplicabilidad:

Sirve para cuando desconocemos de antemano los tipos de objetos que debemos usar para trabajar. Esto es porque luego podremos ampliar el código, agregando nuevas clases a retornar, sin romper el código. También sirve para cuando deseamos proporcionar a los usuarios una forma de ampliar los componentes internos, y para cuando queremos ahorrar recursos del sistema reutilizando objetos existentes.

-Intenta flexibilizar la creación de un objeto

-Realización o herencia

-Objetos sencillos cuya creación se desea postergar a último momento

-Conjunto de creadores concretos que implementan lo declarado en una interfaz, cada creador tiene visibilidad sobre el producto que crea.

Beneficios y desventajas:

- + Evita acoplamiento entre creador y productos concretos
- + Principio de responsabilidad única: crear objetos. Principio de abierto/cerrado: podemos agregar objetos a crear sin modificar nada de código. (**SOLID**)
- + El código se vuelve más complicado al introducir muchas subclases

Cómo implementar:

1. Haga que todos los productos sigan la misma interfaz.
2. Cree un método de factoría vacío en la clase creadora, su tipo de retorno debe ser la interfaz común.
3. En el código del creador debemos reemplazar los constructores por llamadas al método
4. Cree subclasses que implementen el método, adaptándolo a sus necesidades.
5. Si después de implementar todo, el método de la clase padre queda vacío (sin ningún comportamiento base) puede transformarlo en método abstracto.

Principios de diseño OO se aplican en el patrón Factory Method()

Responsabilidad Única (RSP): Proporcionar una forma de encapsular la creación de objetos en una clase dedicada, a menudo llamada "fábrica". Esto permite que una clase tenga una única razón para cambiar: la creación de objetos. La creación y la lógica del objeto creado se separan, lo que facilita la gestión de cambios y mantenimiento.

Programar hacia la interfaz (PTI): Facilita la programación hacia la interfaz al definir una interfaz o clase base para la creación de objetos. Las clases concretas que implementan esta interfaz son responsables de crear instancias de objetos específicos. Al programar hacia la interfaz de la fábrica, el código cliente puede trabajar con objetos a través de su interfaz común sin preocuparse por las clases concretas reales que se están utilizando.

Open-Close(O/C): Al introducir una nueva clase concreta que implementa la interfaz de la fábrica, se puede agregar una nueva variante de objeto sin cambiar el código cliente existente. Esto promueve la extensibilidad del sistema sin requerir modificaciones en las clases existentes.

Patrones de estructura

Los patrones estructurales determinan cómo se combinan las clases y objetos para formar estructuras más grandes y complejas.

El propósito de estos patrones es ayudar a resolver problemas de estructura: lograr comunicación entre dos clases que sean incompatibles y añadir funcionalidad a los objetos dinámicamente (con asociación, agregación, composición).

Dos tipos:

- **Patrones estructurales de clase:** hacen uso de la herencia para componer interfaces o implementaciones.
- **Patrones estructurales de objeto:** describen formas de componer objetos para obtener nueva funcionalidad. Hacen uso de la composición.

Resuelven 2 problemas:

1. Simplificar estructuras muy complejas, resuelve necesidades puntuales de

comportamiento en los objetos pero que se pueda hacer dinámicamente, en tiempo de ejecución. Si o si usamos relaciones que se trasladen a los objetos, asociación, agregación, composición. (Composite)

2. Lograr compatibilidad entre clases incompatibles, cuando necesitamos que una clase acceda a servicios o beneficios de otra clase, pero no se ven. Se resuelve con la indirección (poner una clase en el medio que funcione como intermediario, haciendo que las 2 clases sean compatibles en términos de comunicación). (Adapter)

Indirección -> Significa en términos generales poner una clase en el medio que funcione como intermediario, intérprete o traductor dependiendo el problema que sea. Hace que las dos clases sean compatibles.

Patrones de estructura

1. **Adapter**: convierte la interfaz de una clase en otra distinta que es la que esperan los clientes. Permite que cooperen clases que de otra forma no podrían por tener interfaces incompatibles.
2. **Bridge**: desacopla una abstracción de su implementación, de manera que ambas puedan variar de forma independiente.
3. **Composite**: combina (compone) objetos en estructuras de árbol para representar jerarquías de todo-parte. Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.
4. **Decorator**: añade dinámicamente nuevas responsabilidades a un objeto, proporcionando una alternativa flexible a la herencia para extender funcionalidad.
5. **Facade**: proporciona una interfaz unificada para un conjunto de interfaces de un subsistema. Define una interfaz de alto nivel que hace que el subsistema sea fácil de usar.
6. **Flyweight**: usa comportamiento para permitir un gran número de objetos de granularidad fina de forma eficiente.
7. **Proxy**(apoderado): proporciona un sustituto o representante de otro objeto para controlar el acceso a éste.

ADAPTER

Se utiliza el patrón cuando existen clases incompatibles, adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla.

Solo se aplica con herencia.

Propósito:

Convierte la interfaz de una clase en otra interfaz esperada por los clientes, para resolver incompatibilidad de interfaces y que los objetos puedan colaborar. Intermediario entre dos clases que no pueden comunicarse

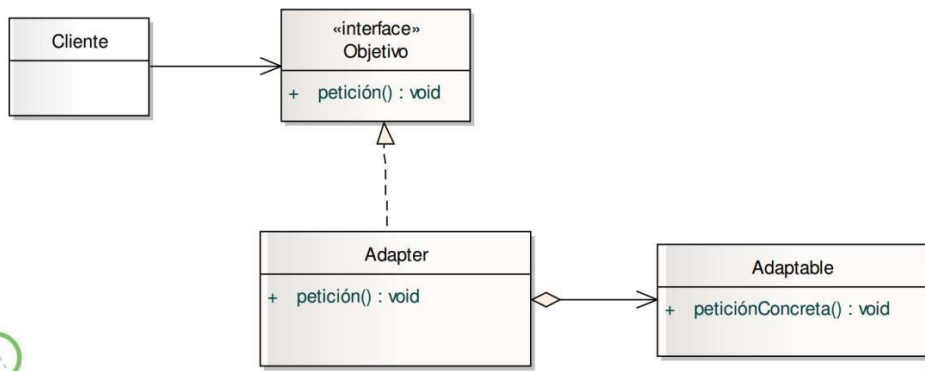
Problema:

Dos objetos con interfaces incompatibles deben trabajar juntos. Si modifico el código de alguno de los dos, podría romperlo.

Solución:

El patrón adaptador convierte la interfaz de un objeto para que otro objeto pueda entenderlo; envuelve uno de los objetos para ocultar la complejidad de la conversión.

1. El adaptador obtiene una interfaz, compatible con uno de los objetos existentes.
2. Con esta interfaz, el objeto existente puede llamar de forma segura a los métodos del adaptador.
3. Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, pero en el formato y el orden que espera el segundo objeto.



Estructura:

- El Cliente es una clase que contiene la lógica empresarial del programa
- La interfaz de cliente describe un protocolo a seguir por otras clases para colaborar con el cliente
- El servicio es una clase útil, que el cliente no puede usar por tener interfaz no compatible
- El Adaptador es una clase que puede trabajar con el cliente y el servicio; implementa la interfaz del cliente, envuelve el objeto de servicio. El mismo recibe llamadas del cliente y las traduce en llamadas al objeto de servicio en un formato que el servicio puede entender
- El código del cliente no se acopla a la clase de adaptador concreto siempre que funcione con el adaptador a través de la interfaz del cliente. Gracias a esto, puede introducir nuevos tipos de adaptadores en el programa sin romper el código de cliente existente.

Beneficios y desventajas:

- + Principio de responsabilidad única, podemos separar la interfaz o el código de la conversión de datos de la lógica del programa. Principio abierto cerrado, podemos introducir nuevos adaptadores sin romper el código existente, siempre que funcionen a través de la interfaz del cliente (**SOLID**)
- La complejidad general del código aumenta al crear muchas interfaces y clases nuevas

Principios de diseño OO se aplican en el patrón Adapter()

Segregación de interfaces: la interfaz intermedia tiene los comportamientos que le van a servir al

cliente y los que tiene que implementar el adaptador.

Principio de responsabilidad única: Puedes separar la interfaz o el código de conversión de datos de la lógica de negocio primaria del programa.

Principio de abierto/cerrado: Puedes introducir nuevos tipos de adaptadores al programa sin descomponer el código cliente existente, siempre y cuando trabajen con los adaptadores a través de la interfaz con el cliente.

COMPOSITE

Permite tratar objetos **compuestos** como si fueran **simples** (Individuales). Combina objetos en estructuras de árbol para representar jerarquías de **todo-parte** (Objetos individuales y compuestos). Debe existir una relación **jerarquía de tipo TODO-PARTE** (Agregación o Composición), que tenga al sumo 3 niveles o sea recursiva para poder aplicar el patrón.

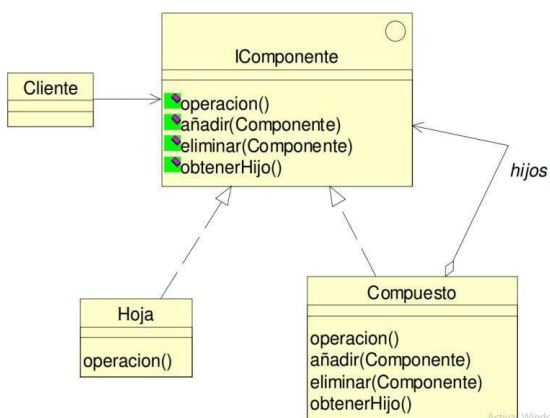
Motivación: Simplifica estructuras complejas, para poder relacionarse con esa jerarquía (cliente que necesita acceder a esos servicios) y resuelve incompatibilidades entre clases.

Son dos clases concretas que implementan la interfaz componente (IComponente). Tiene la facultad de cambiar la estructura base que viene del análisis, porque reemplaza clases por otras. Se debe rediseñar la estructura.

Se aplica con realización y herencia.

Propósito:

Permite componer objetos en estructura de árbol y luego trabajar con cada estructura como objeto individual. Permite ejecutar comportamientos de forma recursiva sobre todos los componentes del árbol. Permite al cliente manejar los objetos primitivos y compuestos de manera uniforme mediante interfaz.



Estructura:

- La Interfaz de componente describe operaciones comunes a elementos simples y complejos del árbol
- La Hoja es un elemento básico del árbol, sin subelementos

- El Contenedor es un elemento con subelementos, que no conoce las clases concretas de sus hijos, sino la interfaz compartida
- El Cliente trabaja con todos los elementos a través de la interfaz

-Único patrón que modifica estructura

-Resuelve problemas de compatibilidad o de estructura complejas

Aplicabilidad:

Cuando necesitemos implementar una estructura de objeto en forma de árbol

Cuando deseemos que el código del cliente trate a los elementos simples y compuestos de manera uniforme

Beneficios y desventajas:

- + Principio abierto cerrado, podemos introducir nuevos tipos de elementos sin romper el código existente (**SOLID**)
- + Podemos trabajar con estructuras de árbol complejas de manera más conveniente, usando la recursividad y polimorfismo
- Puede ser difícil proporcionar una interfaz común si los comportamientos de las clases difieren demasiado

Principios de diseño OO se aplican en el patrón Composite

Programar hacia la interfaz y no hacia la implementación: Hago más transparente la interfaz para el cliente, decido que me preocupa más la interfaz hacia el cliente que la forma que voy a implementar una determinada solución.

Principio de abierto/cerrado: Puedes introducir nuevos tipos de elementos en la aplicación sin descomponer el código existente, que ahora funciona con el árbol de objetos.

El composite no aplica el principio de diseño (Segregación de interfaces):

Segregación de interfaces: Porque define una interfaz que solo son aplicables para el compuesto y no para la hoja. Los declara en la interfaz Porque prefiere privilegiar el servicio que ofrece al cliente, que sea más transparente con quien se relaciona. La hoja hace una implementación trivial

Rompe un principio de diseño -> segregación de interfaz (No obligues a las clases concretas a implementar comportamiento que no debería implementar)

Hijo, hace una implementación trivial (Esto no lo podés hacer, o no hagas nada, o no podés agregar componentes), hace eso ya que el método no puede quedar vacío (**NO PUEDEN EXISTIR MÉTODOS VACÍOS EN CLASES CONCRETAS**)

Una solución sería que el cliente tenga un puntero a la hoja y al compuesto (Pero aumenta

acoplamiento, y rompe principio programar hacia la interfaz)

Deja de lado el principio segregación de interfaz para darle más lugar al programar hacia la interfaz, detrás de este principio está el concepto de transparencia para el cliente, se le da cierta información al cliente a través del composite, programar hacia la interfaz dice mejor pensar en el bienestar del cliente

Patrones de comportamiento

Los patrones de comportamiento tienen que ver con algoritmos y con la distribución y delegación de responsabilidades a objetos. No sólo describen patrones de clases y objetos, sino permiten manejar la comunicación entre objetos del sistema y el flujo de información entre ellos.

- Su propósito es ayudar a decidir en qué clases voy a ubicar cada método que necesito.
- Su objetivo es flexibilizar la relación entre los objetos.
- Se enfoca en las definiciones asociadas a los algoritmos que están dentro de las clases.

Dos tipos:

- **Patrones de comportamiento de clase:** usan la herencia para distribuir el comportamiento entre clases.
- **Patrones de comportamiento de objeto:** usan la composición de objetos en vez de la herencia.

Las claves son cooperaciones entre objetos para realizar tareas complejas que por sí solos no podrían realizar, reduciendo la dependencia entre objetos (Iterator, Observer, etc.) y, asociar comportamiento a objetos e invocar su ejecución (Command, Strategy, etc).

Los patrones de comportamiento apuntan a:

- Distribución de responsabilidades
- Cómo escribir los métodos
- Cómo asignar los métodos a las clases que conviene asignar
- Definir granularidad

Patrones de comportamiento

1. Chain of responsibility: evita acoplar el emisor de una petición a su receptor, dando a más de un objeto la posibilidad de responder la petición. Crea una cadena con los objetos receptores y pasa la petición a través de la cadena hasta que ésta sea tratada por algún objeto.

2. Command: encapsula una petición en un objeto, permitiendo así parametrizar a los clientes con diferentes peticiones, encolar o llevar un registro de las peticiones y poder deshacer las operaciones.

3. Interpreter: dado un lenguaje, define una representación de su gramática junto con un intérprete que usa dicha representación para interpretar sentencias del lenguaje.

- 4. Iterator:** proporciona un modo de acceder secuencialmente a los elementos de un objeto agregado sin exponer a su representación interna.
- 5. Mediator:** define un objeto que encapsula cómo interactúan una serie de objetos. Promueve un bajo acoplamiento al evitar que los objetos se refieran unos a otros explícitamente, y permite variar la interacción entre ellos de forma independiente.
- 6. Memento:** representa y externaliza el estado interno de un objeto sin violar el encapsulamiento, de forma que éste pueda volver a dicho estado más tarde.
- 7. Observer:** define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y se actualicen automáticamente todos los objetos que dependen de él.
- 8. State:** permite que un objeto modifique su comportamiento cada vez que cambie su estado interno. Parecerá que cambia la clase del objeto.
- 9. Strategy:** define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo usan.
- 10. Template Method:** define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos. Permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.
- 11. Visitor:** representa una operación sobre los elementos de una estructura de objetos. Permite definir una nueva operación sin cambiar las clases de los elementos sobre los que opera.

TEMPLATE METHOD

Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.

-Solo Herencia

Se aplica a nivel de 1 método, llamado método plantilla, entonces el propósito es garantizar que un algoritmo se va a ejecutar exactamente en orden que yo quiero que se ejecuten las operaciones primitivas que yo tengo que ejecutar.

Este patrón se usa cuando queremos garantizar que todas las subclases que van a heredar el método plantilla, implementan el algoritmo que está contenido, en el orden de invocación de las operaciones que está definido en el método.

Propósito:

Patrón que define el esqueleto de un algoritmo en la superclase, y permite que las subclases anulen pasos específicos del algoritmo sin cambiar la estructura. Este patrón sugiere que se descomponga el

algoritmo a implementar en una serie de pasos, y que convierta esos pasos en métodos. Luego, en el método plantilla, hará llamadas a los métodos. Los métodos llamados pueden ser abstractos o tener una implementación. Para usar el algoritmo, el cliente debe proporcionar su propia subclase que implemente todos los pasos abstractos y anule los innecesarios.

Tendremos los **pasos abstractos (implementados por cada subclase)**, los **opcionales (tienen implementación predeterminada, pero podemos anularlos si se vuelven innecesarios)** y **pasos ganchos (paso opcional con cuerpo vacío)**.

Estructura:

- La Clase abstracta declara métodos que actúan como pasos de un algoritmo, y el método plantilla que llama a estos métodos en un orden específico
- Las Clases concretas pueden anular uno o todos los pasos

Aplicabilidad:

desee permitir que clientes amplíen sólo pasos particulares de un algoritmo, y no todo el algoritmo ni su estructura. Cuando tenga varias clases que contengan algoritmos muy parecidos. Sin embargo, es posible que deba modificar todas las clases involucradas si en un futuro cambia el algoritmo

El orden es el mismo para todas las subclases del método plantilla implementado (el que lleva el orden), pero cada subclase dentro de cada operación hace algo distinto. Tengo que garantizar la estructura de ejecución de las operaciones (Proteger con método plantilla), pero le doy libertad a cada clase concreta de que las operaciones las resuelva de manera diferente.

El método plantilla se define en la clase PADRE como público final, para que las hijas no lo puedan modificar, pero sí modificar polimórficamente las operaciones que se invocan desde el método plantilla.

Si no hay un algoritmo con n° operaciones que invocar para resolver un problema con un orden en particular, no hace falta utilizar el patron.

Un método plantilla (template) invoca los siguientes tipos de métodos:

- Operaciones abstractas del padre.
- Operaciones concretas del padre.
- Invocando, operaciones concretas de los hijos.
- Método hook (Método enganche), lo tiene definido el padre para que todos los hijos lo puedan usar o ignorar en función de lo que necesiten (es un comportamiento por defecto). También puede ser vacío.

Beneficios y desventajas:

- + Puede permitir que los clientes anulen sólo ciertas partes de un algoritmo
- + Puede extraer código duplicado en una superclase

- Algunos clientes pueden estar limitados por el esqueleto del algoritmo
- Puede violar el Principio de sustitución de Liskov, suprimiendo una implementación de paso predeterminada en la clase padre a través de una subclase
- Son difíciles de mantener al agregarle más pasos

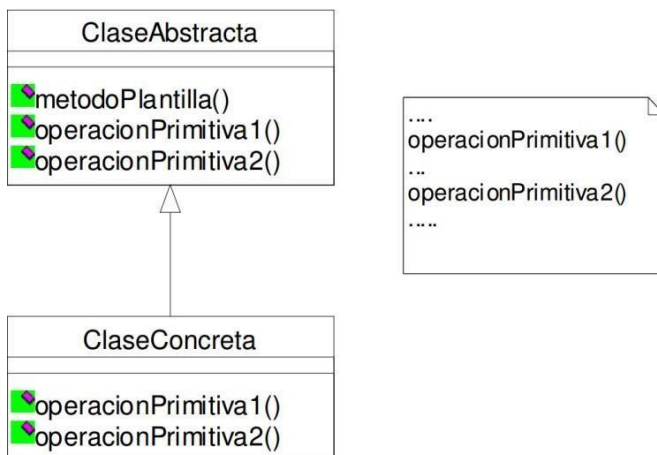
Estructura de control invertida, relacionada con el principio **Inversión de dependencia**

Las invocaciones en la herencia son del hijo al padre (hijo --I> padre), principalmente los hijos invocan operaciones del padre, acá se invierte el orden natural. Se plantea que dentro del metodoPlantilla() que está en el PADRE, tengo invocaciones a las operaciones concretas de las clases hijas.

Principio de hollywood (No nos llamen, nosotros te llamamos) -> La clase padre llama a la clase hija. Se da en tiempo de compilación.

En términos de objetos, la clase concreta (Hija) es instanciada y hereda el método plantilla, invocando la operación a sí misma. Ya que la clase abstracta no se puede instanciar.

En términos de estructura estamos aplicando la inversión de dependencia, porque estamos haciendo una invocación de un método de la clase hijo en el método plantilla que está en el padre.



En la clase concreta no se ve el método plantilla xq solo se visualiza lo que se redefine. Primero se implementan los métodos de enganche y luego los que se redefinen.

Principios de diseño OO se aplican en el patrón Template Method()

LSP: Analiza si está bien armada la herencia. (hay q mejorarlo, pero lo cumple)

DIP: Principio de inversión de dependencia. Le da más potencia al método plantilla y que permite que en el método plantilla haya declarado invocaciones a sus clases hijas (naturalmente la herencia no lo permite), aca el padre apunta al hijo y en herencia es al revés. Permite que el TM pueda armar una plantilla completa, con invocaciones a clases concretas.

OBSERVER

Define una dependencia de uno-a-muchos entre objetos, de forma que cuando cambia el estado un **objeto (Sujeto)**, **todos sus dependientes (Observadores)** automáticamente son notificados y actualizados. Trabajar con actualización de componentes, que el ámbito son los observadores, frente al cambio de estado del sujeto. Nivel bajo o casi nulo de acoplamiento, entre los observadores y sujeto. Están desacoplados, se pueden agregar o quitar de forma fácil.

Un ejemplo puede ser el Modelo - Vista, donde el modelo tiene los datos y se modifican, tiene que notificar y actualizar a los observadores que son las vistas.

Se puede aplicar este patrón con la forma **Pull** (se lo notifica y se actualiza cuando lo desea, es decir busca los datos que necesita). En cambio, el **Push** (se pasa la información forzosamente para que actualice)

-Realización o herencia

Propósito:

Patrón que permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando. Define una dependencia de uno a muchos entre objetos; cuando cambia el estado de un objeto, todos sus dependientes son notificados y actualizados. Necesitamos que todos los suscriptores implementen la misma interfaz, para que el publicante se comunique con ellos a través de ella.

Estructura:

- El Publicante emite eventos de interés para otros objetos, como cambiar su estado o ejecutar un comportamiento. Contiene una infraestructura de suscripción, cuando ocurre un nuevo evento, el publicante revisa la lista de suscriptores y llama al método de notificación declarado en la interfaz común de suscriptor en cada objeto de suscriptor.
- La interfaz de suscriptor declara la interfaz de notificación
- Los suscriptores concretos, que implementan la interfaz, realizan algunas acciones en respuesta a las notificaciones del publicante.
- El publicante puede enviar datos como parámetro por el método de notificación
- El cliente crea objetos de suscriptor y publicante por separado, y luego registra a los suscriptores para las actualizaciones del publicante (o los des-registra)
- Puede existir un Gestor de cambios que mantenga la relación suscriptor-publicante

Aplicabilidad:

- Cuando los cambios de estado de un objeto requieran cambiar otros objetos desconocidos de antemano
- Cuando algunos objetos deban observar a otros durante tiempo limitado o casos específicos

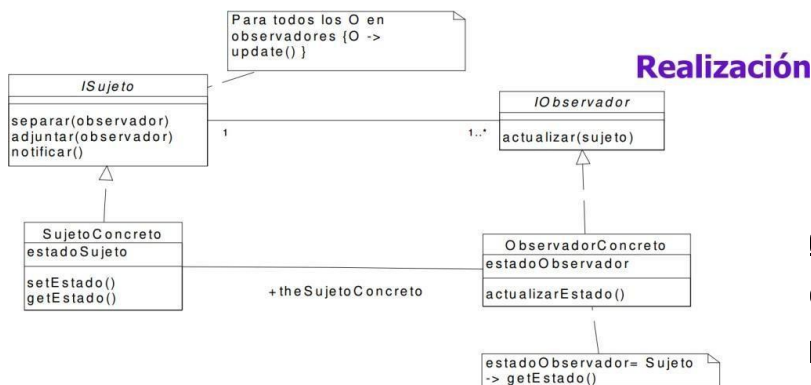
Beneficios y desventajas:

- + Puede establecer relaciones entre objetos en tiempo de ejecución
- Los suscriptores son notificados en orden aleatorio

Consecuencias:

- Permite modificar independientemente subjects y observers.
- Acoplamiento abstracto y mínimo entre subject y observer.
- Pueden reutilizarse por separado.
- Observers pueden añadirse sin modificar el subject.
- Subject no necesita conocer las clases concretas de observers.}
- Soporte para comunicación del tipo broadcast:
- El subject no especifica al receptor concreto de la notificación.
- Es posible añadir y eliminar observers en cualquier parte.
- Actualizaciones inesperadas.
- Interfaz de observer simple requiere que los observers tengan que deducir el ítem cambiado.
- **Reusabilidad:** al separar el sujeto de sus observadores se permite reutilizar sujetos y observadores independientemente.
- **Flexibilidad:** se puede agregar y quitar observadores en cualquier momento sin afectar al sujeto ni a otros observadores, ya que el sujeto siempre ejecuta el método notificar() para todos los observadores existentes.
- **Disminuye el acoplamiento:** el sujeto conoce a sus observadores en las clases Madre, lo cual reduce el acoplamiento entre las clases instanciables, ya que la relación es abstracta.

Es posible asegurar la consistencia entre sujeto y observadores aplicando el patrón Template Method, haciendo que al finalizar la actualización del sujeto se llame al método notificar(), evitando el riesgo de llamarlo cuando aún no se ha producido la actualización.



ver():

ingo el comportamiento que tiene q ver con plementen y con el observador igual.

Programar hacia la interfaz y no hacia la implementación (PTI): reducimos el acoplamiento entre sujetos y observadores porque se comunican a través de la interfaz.

Abierto para la extensión y cerrado para la modificación (OCP): Puedo introducir nuevas clases observadores sin tener que modificar el sujeto (solamente implementando la interfaz de observadores) y

viceversa.

STATE

Tiene un solo estado en un momento determinado del tiempo. Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.

Se usa cuando tengo clases que dependen del estado en el que se encuentra, por lo cual delega el comportamiento a la estructura del estado. Multiplicidad 1 a 1

Resuelve el problema del switch case, if a través del polimorfismo.

El contexto es el que delega la responsabilidad a la clase Estado abstracto (Invoca el mismo comportamiento en la clase Estado), es decir, se relaciona con la abstracción e instancia un objeto concreto.

El estado concreto es el que hace reutilización de código o lo redefine, todo el código está definido en la clase Estado abstracta, por eso el State funciona con la Herencia.

Un método de estado en la clase contexto (ej cancelar()) invoca el comportamiento que está definido en la clase estado con un método (estado.cancelar()), pasándose a sí mismo (el objeto contexto) como parámetro para saber su estado actual (apuntando a un estado concreto). Si no se pasa a sí mismo como parámetro le tiene que pasar la información al estado para que lo pueda resolver. Le delega la responsabilidad al objeto en el que se encuentra (estado actual)

Solo herencia

Permite que un objeto altere su comportamiento cuando cambia su estado interno, como si el mismo cambiara su clase. Está ligado al concepto de máquina de estados finita, y sugiere que se creen nuevas clases para todos los estados posibles de un objeto, extrayendo el comportamiento específico del estado en estas clases. El objeto original, llamado Contexto, almacena una referencia a un objeto de estado que representa su estado actual, y delega todo el trabajo relacionado con el estado a ese objeto. Todas las clases de estado deben seguir la misma interfaz, y el contexto debe trabajar con ella.

Estructura:

1. El Contexto almacena una referencia de tipo interfaz de estado, que se representa por medio de una clase de estado y representa el estado actual.
2. La interfaz de estado declara los métodos específicos de cada estado
3. Los Estados concretos proporcionan sus implementaciones para cada método
4. Tanto el contexto como los estados concretos pueden establecer el siguiente estado del contexto

Aplicabilidad:

- Cuando tenga un objeto que se comporte de manera diferente según su estado actual, tenga

muchos estados y el código del estado cambie con frecuencia.

- Cuando tenga una clase contaminada con condicionales masivos para definir el comportamiento según el estado de la clase
- Cuando tenga mucho código duplicado en estados y transiciones

Beneficios y desventajas:

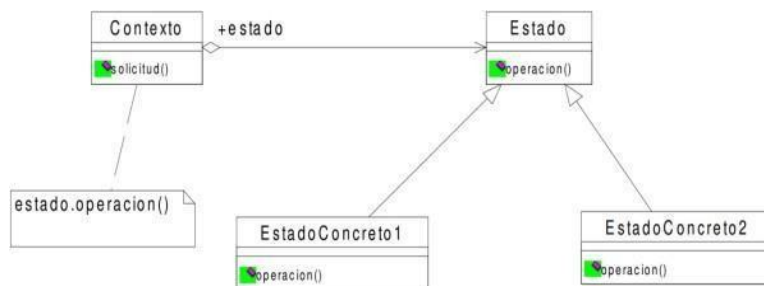
+ Principio de responsabilidad única; organice el código relacionado con estados particulares en clases separadas. Principio abierto-cerrado; podemos introducir nuevas clases de estados sin tener que cambiar el código del contexto ni otros estados. **SOLID**

+ Simplifique el código del contexto eliminando condicionales

- Aplicarlo puede ser excesivo si tenemos pocos estados o pocos cambios de estado
- La desventaja es que se genera un gran crecimiento de clases, o sea una clase por cada estado.

Consecuencias:

- Coloca todo el comportamiento asociado a un estado particular en una clase
- Ayuda a evitar estados inconsistentes
- Transiciones de estado más explícitas
- Incrementa el número de objetos



Principios de diseño OO se aplican en el patrón State ()

SRP: Se reduce la responsabilidad en el Contexto y se crea cada Estado Concreto para mantener una sola responsabilidad.

OCP: Si se necesita cambiar se agregan nuevos estados Concretos (se extiende) para manejar la variabilidad y comportamiento. Se crea una jerarquía de clases nuevas para no modificar la clase contexto que es la clase original.

LSP: Los Concretos se pueden intercambiar con el Abstracto, ya que todos son capaces de responder a sus métodos.

DRY: El código común se mantiene en la implementación trivial del padre, evitando redefinirlo en todas las hijas.

EWV: Los Concretos encapsulan la lógica específica de cada uno, aislando los cambios del resto, sin afectarlos.

TDa: El Contexto delega el trabajo al Estado, que resolverá la lógica del pedido.

STRATEGY

Se define una familia de algoritmos, en el cual encapsula cada uno de ellos en clases y elegir cuál estrategia utilizar en tiempo de ejecución.

Se puede resolver con herencia y realización.

Alguien le informa al contexto la información específica para ver qué estrategia utilizar en ese momento (actor que selecciona algo, por parámetro, etc), el contexto puede tener más de una estrategia habilitada al mismo tiempo en la estructura (Multiplicidad 1 a *).

El contexto le delega a la estrategia. En herencia cada estrategia concreta implementa lo que heredó de la estrategia abstracta, en realización las estrategias concretas implementan el comportamiento polimórfico que está declarado en la interfaz como un método vacío. Las clases concretas no pueden tener métodos vacíos. **Conviene utilizar realización ya que cada estrategia concreta es diferente a la otra y no hay reutilización de código.**

El contexto crea la interfaz concreta con un new() y se pasa como parámetro o pasa la información para que la estrategia pueda resolver el problema.

La delegación está en que el contexto tiene una invocación al método de la Estrategia.

Propósito:

Permite definir una familia de algoritmos, poner a cada uno en una clase separada y hacer que sus objetos sean intercambiables. Permite variar los algoritmos de forma independiente a los clientes que los usan. Sugiere que tome una clase que haga algo específico de diferentes maneras y extraiga todos esos algoritmos en clases separadas, llamadas Estrategias, que implementan la misma interfaz de estrategia. La clase original, llamada Contexto, debe tener un campo para almacenar una referencia a una estrategia, y delega el trabajo a un objeto estrategia. El Cliente elige la estrategia deseada.

-Alguien externo determina que estrategia usar

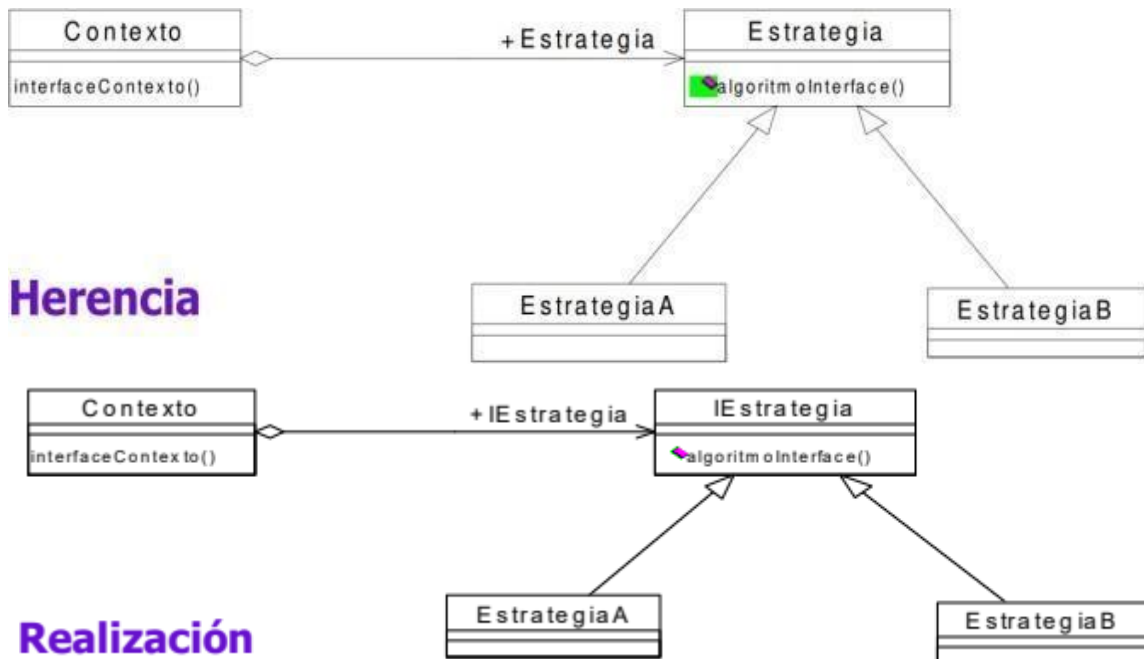
-Una clase requiere ejecutar una operación de diferentes maneras

Estructura:

- El Contexto mantiene una referencia a una estrategia concreta y se comunica con ella a través de la interfaz de estrategias
- La interfaz de estrategia es común en todas las estrategias concretas
- Las estrategias concretas implementan diferentes variaciones de un algoritmo que el contexto

usa

- El contexto llama al método de ejecución de la estrategia mediante el objeto estrategia
- El cliente crea un objeto de estrategia específico y lo pasa al contexto



Aplicabilidad:

- Cuando desee utilizar diferentes variantes de un algoritmo y poder cambiarlo en tiempo de ejecución
- Cuando tenga muchas clases similares que sólo difieran en la forma en que ejecutan un comportamiento
- Cuando su clase tenga un operador condicional masivo
- Para aislar lógica empresarial de detalles de implementación

Beneficios y desventajas:

- + Puede cambiar el algoritmo a usar en tiempo de ejecución
- + Puede aislar los detalles de implementación del algoritmo
- + Puede reemplazar la herencia con composición
- + Principio abierto-cerrado; puede introducir nuevas estrategias sin cambiar el contexto
- Los clientes deben conocer las diferencias entre las estrategias para seleccionar la adecuada

Consecuencias:

- Define una familia de algoritmos relacionados
- Una alternativa a crear subclases de la clase Contexto
- Elimina condicionales masivos
- El cliente puede elegir estrategias
- Se incrementa el número de objetos

Principios de diseño OO que se aplican en el Strategy ()

OCP: el Contexto está abierto para extensión, ya que se pueden agregar más Estrategias. [ok]

ISP: Les da a las clases concretas los métodos que necesita implementar, cada estrategia es específica (alta cohesión). [ok]

DRY: la parte común del algoritmo se mantiene en el contexto, evitando duplicaciones. El contexto tiene la llamada del método, a través de la delegación el código de ese método lo tiene la estrategia que lo resuelve.

ECV: la parte que varía del algoritmo se encapsula en las Estrategias. Ej: cálculos que saquen el iva (puede cambiar el iva)

PTI: se establece una interfaz única y todas las Estrategias se implementan hacia ella para utilizarse en el Contexto.

COI: se usa composición para que el contexto mantenga referencias a las Estrategias, en vez de una clase base con el comportamiento general e hijas con la parte que cambia. Se analiza el contexto con la estrategia. Aplicamos delegación en contexto y estrategia, reutilizamos código xq solo hacemos la llamada.

Conceptos del diseño

División de problemas

Este es un concepto de diseño que sugiere que cualquier problema complejo puede manejarse con más facilidad si se lo subdivide en elementos más fáciles de resolver, se aplica la estrategia divide y vencerás.

Modularidad

Forma de división de problemas. El software se divide en componentes o módulos que se integran para satisfacer los requerimientos del problema. A mayor cantidad de módulos, menor será el esfuerzo y tiempo que nos lleve un proyecto.

Ocultamiento de información

El principio de ocultamiento de información sugiere que los módulos deben diseñarse y especificarse de forma tal que la información contenida en ellos sea inaccesible para los módulos que no necesiten de ella. Esto implica que la modularidad efectiva se logre cuando los módulos independientes intercambian únicamente la información necesaria para que el software funcione.

Independencia funcional

Resultado de la separación de problemas, de los conceptos de abstracción y ocultamiento de información. Se busca desarrollar módulos que tengan "aversión" a la interacción excesiva con otros módulos. Cada módulo debe resolver un subconjunto específico de requerimientos, con una interfaz

sencilla. Los módulos independientes son más fáciles de mantener y se pueden reutilizar, se reduce la propagación de errores.

Se mide con el uso de los conceptos *cohesión* y *acoplamiento*.

Refinamiento

Estrategia que indica que se debe elaborar un programa mediante refinamientos sucesivos de los detalles del procedimiento. Es un proceso de elaboración, comenzando con una descripción de la información, dando cada vez más detalles hasta llegar a la implementación. Ayuda a revelar esos detalles que se ocultan en niveles más altos de abstracción.

Aspectos

A veces pueden presentarse preocupaciones que abarcan todo el sistema, estas son llamadas aspectos, representan una preocupación de interferencia.

Rediseño

Técnica de reorganización que simplifica el diseño de un componente sin cambiar su comportamiento, mejorando su estructura interna, eliminando redundancias o código mal usado o innecesario. Se busca corregir fallas, para obtener un software más fácil de integrar, probar y mantener.

Conceptos de diseño orientado a objetos

Clases de diseño

Conforme avanza la etapa de diseño, se pueden desarrollar cinco tipos de clases de diseño, cada una representa una capa distinta de la arquitectura del diseño:

Clases de usuario de la interfaz: definen abstracciones necesarias para la interacción humano-máquina.

- **Clases del dominio de negocios:** identifican atributos y métodos requeridos para implementar algunos elementos de este dominio.
- **Clases de proceso:** implantan abstracciones de negocio de bajo nivel, requeridas para administrar por completo las clases de dominio.
- **Clases persistentes:** representan almacenamientos de datos, persisten fuera de la ejecución.
- **Clases de sistemas:** implementan las funciones de administración de software que lo integra en el contexto que corresponda.

Se definen 4 características de una clase de diseño bien formada:

Completa y suficiente; una clase de diseño debe ser el encapsulado total de todos los atributos y métodos que necesite, ni más ni menos.

Primitivismo; los métodos asociados a una clase de diseño deben centrarse en el cumplimiento de un

servicio para la clase, sin repetir.

Mucha cohesión

Poco acoplamiento

El *modelo de diseño* puede verse en dos dimensiones distintas: la dimensión del proceso, que indica la evolución del modelo de diseño mientras se ejecutan sus tareas, y la dimensión de la abstracción, que representa el nivel de detalle cuando cada elemento del modelo de análisis se transforma en uno de diseño, y se mejora de forma iterativa.

Elementos del diseño de datos

El diseño de datos crea un modelo de datos que se representa en un nivel de abstracción alto. Estos datos luego se refinan de forma progresiva hasta representaciones específicas listas para su implementación.

Elementos del diseño arquitectónico

Dan la visión general del software. Proviene de tres fuentes: 1) información sobre el dominio de la aplicación, 2) los elementos específicos del modelo de requerimientos, y 3) la disponibilidad de estilos arquitectónicos y sus patrones.

Se los suele ilustrar como un conjunto de sistemas interconectados.

Elementos de diseño de la interfaz

Permiten que la información fluya hacia dentro y fuera del sistema, y cómo están comunicados los componentes que forman parte de la arquitectura. Permiten la comunicación y colaboración de diferentes partes del software.

Hay tres elementos importantes: 1) interfaz de usuario, 2) interfaces externas con otros sistemas, y 3) interfaces internas que involucran a otros componentes del diseño.

Elementos del diseño en nivel de componentes

Su diseño describe por completo los detalles internos de cada componente, definiendo estructuras de datos para todos los objetos de datos locales y detalles algorítmicos para todo el procesamiento que tiene lugar dentro de un componente. Sus detalles se modelan con diagramas UML y pseudocódigo.

Elementos del diseño del despliegue

Indican la forma en la que se acomodarán la funcionalidad del software y los subsistemas dentro del ambiente físico de la computación que lo apoyará. Un diagrama de despliegue muestra el ambiente de computación sin especificar detalles de configuración.

Estrategias de prototipado

Una estrategia de prototipado es una elección de modelo de proceso (ciclo de vida) que se recomienda elegir a la hora de implementar un proyecto complejo, con dominio no familiar, que usará una tecnología desconocida o que tiene un costo de rechazo muy alto. Requiere el uso de prototipos en el diseño, la implementación y la validación de requerimientos.

Se puede usar de diferentes maneras en el proceso de desarrollo:

- En el proceso de ingeniería de requerimientos, para obtenerlos y validarlos.
- En el proceso de diseño, ayuda a explorar soluciones y apoyar al diseño de interfaces de usuario.
- En el proceso de pruebas.

Permiten a los usuarios ver cómo el sistema apoya a su trabajo. Pueden adquirir nuevas ideas para los requerimientos y encontrar áreas fuertes y débiles en el software. Puede revelar errores y omisiones en los requerimientos propuestos.

Prototipo

Consisten en una primera versión de un nuevo tipo de producto, incompleto dado que se muestran solo algunas características, es un modelo del sistema que se construye para comprender mejor el problema y sus posibles soluciones.

Objetivos de los Prototipos

- a) Aclarar los requerimientos de los usuarios
- b) Verificar la factibilidad del diseño del sistema

Características:

- Funcionalidad limitada
- Poca fiabilidad
- Características de operación pobres
- Pocos días de desarrollo

Beneficios

- Mejora la usabilidad del sistema
- Una mejor concordancia entre el sistema y las necesidades del usuario
- Ayuda al cliente a establecer claramente los requerimientos
- Mejora la calidad del diseño
- Mejora el mantenimiento
- Reducción en el esfuerzo de desarrollo

- Aumentan la productividad
- Planifican el desarrollo
- Entusiasmo de los usuarios respecto a los prototipos.

Desventajas/Riesgos/Críticas:

- Funcionalidad limitada.
- Poca fiabilidad.
- Características de operación pobres.
- Alto costo: 10% del presupuesto del proyecto
- El cliente cree que es el sistema funcional.
- Peligro de familiarización con malas elecciones iniciales.
- Difícil de administrar: se necesita mucha experiencia.

Proceso de desarrollo de prototipos

1. Establecer objetivos del prototipo
2. Definir su funcionalidad: qué incluir y qué excluir
3. Desarrollarlo
4. Evaluarlo: analizar los resultados de su uso

¿Cuándo es conveniente usar prototipos?

Siempre, pero especialmente cuando...

- El Área de aplicación no está bien definida, es decir, el dominio es riesgoso (bien por su dificultad o por falta de tradición en su aplicación).
- El costo del rechazo por parte de los usuarios, por no cumplir sus expectativas, es muy alto.
- Es necesario evaluar previamente el impacto del sistema en los usuarios y en la organización.
- Se usan nuevos métodos, técnicas, tecnología.
- No se conocen los requerimientos o es necesario realizar una evaluación de los requerimientos
- Cuando los costos de inversión son altos.
- Cuando hay factores de riesgo alto asociados al proyecto.

¿Cómo se utilizan?

Se presenta al cliente para que experimente, ayudándolo a establecer claramente los requisitos.

- En la ingeniería de requerimientos: ayuda a obtener y validar los requerimientos.
- En el diseño: ayuda a explorar soluciones de SW particulares y apoyar el diseño de interfaces de usuario.
- En el proceso de pruebas: para ejecutar pruebas back-to-back con el sistema que se entregará al cliente.
- Permiten a los usuarios ver como el sistema apoya a su trabajo: pueden adquirir nuevas ideas para los requerimientos, encontrar áreas fuertes y débiles en el SW, revelar errores y omisiones.

- Ayuda a los desarrolladores a: validar correcciones de la especificación, aprender sobre problemas que se presentarán en el diseño e implementación, mejorar el producto, examinar viabilidad y utilidad de la aplicación.

Uso de los Prototipos:

- Se presenta al cliente un prototipo para su experimentación
- Ayuda al cliente a establecer claramente los requisitos.
- Ayuda a los desarrolladores a:
 - Validar corrección de la especificación
 - Aprender sobre problemas que se presentarán durante el diseño e implementación del sistema.
 - Mejorar el producto.
 - Examinar viabilidad y utilidad de la aplicación.

Clasificación de prototipos

1. Según su propósito:

- **Prototipado de interfaz de usuario:** modelos de pantallas. Les permite a los usuarios ganar experiencia directa con la interfaz para juzgar su usabilidad.
- **Prototipado funcional (operacional):** implementa algunas funciones, y a medida que se comprueba que son las apropiadas, se corrigen, refinan y se añaden otras.
- **Prototipos arquitectónicos:** sirven para evaluar decisiones arquitectónicas de infraestructura, tecnología e integración.
- **Modelos de rendimiento:** evalúan el rendimiento de una aplicación crítica (no sirven al análisis de requerimientos).

2. Según su utilidad:

- **Rápidos o desechables** - Sirven a análisis y validación de requerimientos, se redacta la ERS y se descarta, se genera un problema si este prototipo no se desecha y se convierte en el producto final
- **Evolutivos** – Comienza con un sistema relativamente simple que implementa los requerimientos más importantes o mejor conocidos. Va cambiando en cuanto se descubren nuevos requerimientos y finalmente, se convierte en el sistema requerido. Se usa en el desarrollo Web y en aplicaciones de comercio electrónico

3. Según su alcance:

- Vertical - desarrolla totalmente algunas funciones
- Horizontal - desarrolla parcialmente todas las funciones

Técnicas de prototipado

- **Prototipación conducida por guiones:** Desarrollar un conjunto de guiones y pantallas utilizando una herramienta. Cuando el usuario interactúa, la pantalla cambia al siguiente display.
- **Programación visual:** utilizar un lenguaje diseñado para desarrollo rápido tal como Visual Basic.
- **Prototipado basado en internet:** utilizar un navegador web y guiones asociados.

Estrategias para el Desarrollo de Prototipos

Las estrategias para el desarrollo de prototipos son:

- **Prototipos para pantallas:** El elemento clave es el intercambio de información con el usuario.
- **Prototipos para procedimientos de procesamiento:** El prototipo incluye solo procesos sin considerar errores.
- **Prototipos para funciones básicas:** Solo se desarrolla el núcleo de la aplicación, es decir solo los procesos básicos.

Tareas de los usuarios: Identificar la finalidad del sistema, describir la salida, describir los requerimientos de datos, utilizar y evaluar el prototipo, identificar mejoras necesarias y documentar características no deseables.

Creencias erróneas sobre el tema Prototipos

Los errores sobre el tema de prototipos son:

- El desarrollo del prototipo es trivial
- Es sólo para aplicaciones pequeñas
- Es sólo para aplicaciones sencillas
- La participación del usuario es simbólica

Prototipación de interfaz de usuario:

- La ayuda de la prototipación es permitirle a los usuarios ganar experiencia directa con la interfaz.
- Sin esa experiencia directa, es imposible juzgar la usabilidad de una interfaz.
- La Prototipación puede ser un proceso de dos etapas:
 - Primeramente en el proceso, se pueden utilizar prototipado en papel;
 - El diseño luego es refinado y luego se desarrollan prototipos automatizados incrementalmente sofisticados conforme se van desarrollando.

Prototipado en papel:

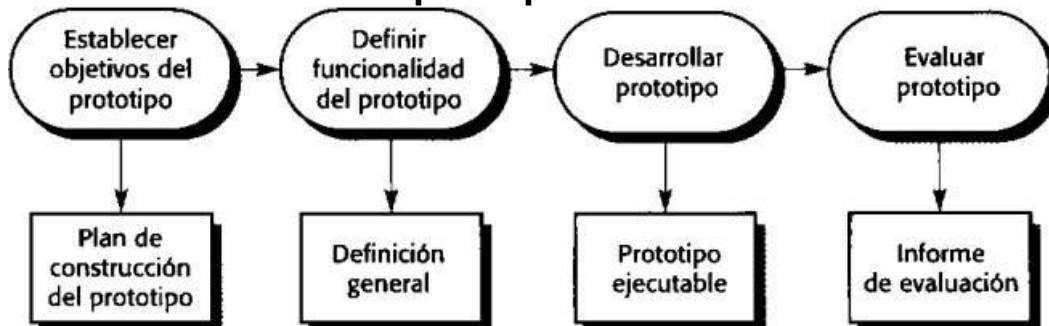
- Escenarios de recorrida utilizando sketches, de la interfaz.
- Uso de storyboards para presentar una serie de interacciones con el sistema.
- El Prototipado en Papel es una forma efectiva de obtener reacciones del usuario a una propuesta de diseño.

Evaluación de interfaz de usuario

- Alguna evaluación de un diseño de interfaz de usuario debería realizarse para evaluar su nivel de adecuación.
- Una evaluación a escala completa es muy cara e impráctica para la mayoría de los sistemas.
- Idealmente, una interfaz debería ser evaluada contra una especificación de usabilidad. Sin embargo, es raro que tales especificaciones se construyan.

Proceso de desarrollo de prototipos

Proceso de desarrollo de prototipos



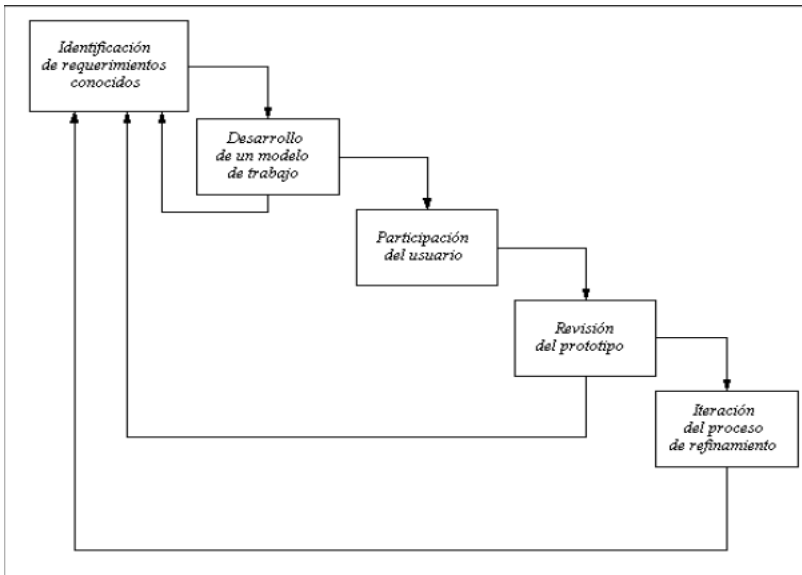
1. **Establecer objetivos del prototipo:** los objetivos del prototipo deben ser claros desde el principio, ya que se puede malinterpretar la función del prototipo y no se obtengan los beneficios esperados.
2. **Definir funcionalidad del prototipo:** decidir que incluir y que excluir del prototipo. Se podría dejar a un lado los requerimientos no funcionales y centrarse en la funcionalidad, reducir estándares, obviar manejo de errores, etc. aunque depende del tipo de prototipo que se esté haciendo.
3. **Desarrollar prototipo:** construir el prototipo.
4. **Evaluar prototipo:** analizar los resultados del uso del prototipo.

Etapas del Modelo de Prototipos

Las etapas del modelo de prototipos son:

- 1- Identificación de requerimientos conocidos
- 2- Desarrollo de un modelo de trabajo
- 3- Participación del usuario
- 4- Revisión del prototipo
- 5- Iteración del proceso de refinamiento

El modelo de prototipos o construcción de los mismos se puede graficar de la siguiente manera:



Diseño de la interfaz de usuario

Interacción Humano Computadora (IHC): disciplina relacionada con el diseño, evaluación e implementación de sistemas computacionales interactivos para uso humano, y el estudio de los principales fenómenos que los rodean.

DATO VS INFORMACIÓN

- No todos los datos procesados son información, muchas veces se tiene un solo dato sin procesar y es información. No tiene nada que ver con el nivel de procesamiento para que algo sea dato o información
- Lo que determina eso es el contexto y la utilidad (si a mi me interesa y llega de una manera que la pueda interpretar, esa sí sirve).
- Las características que nos determina que algo es información son: oportuna, adecuada, ilustrativa y es que se pueda hacer algo con ella, que sea útil. Al momento de capturar requerimientos, cuanta más información se tenga se pueden tomar mejores decisiones

Un sistema de software debe tener, además de utilidad (concepto relacionado a la funcionalidad, si hace lo que está destinado a hacer), buena usabilidad: medición cualitativa de la facilidad y eficiencia con la que un humano emplea las funciones del producto. Si no la tiene, el usuario podría frustrarse y dejar de usarlo. Por eso, para comenzar con el diseño de la interfaz, se desarrollaron dos conceptos: interfaz gráfica de usuario (GUI) y las reglas doradas.

Evolución de la IHM

El hombre comenzó usando internet como sitios estáticos, con contenido que no variaba, sólo accedía a la información. Luego, la internet evolucionó y se crearon los sitios dinámicos, aplicaciones web, donde el usuario interactuaba con el sistema. Por último, nacieron los sitios sociales donde existe el contenido dinámico público modificado por mucha gente y se dan interacciones entre hombres.

Reglas doradas:

1. Dejar el control al usuario

Podemos mencionar algunos principios de diseño que lograrían esto:

- Definir modos de interacción de manera que no se obligue al usuario a realizar acciones no deseadas
- Dar una interacción flexible, opciones
- Permitir que la interacción del usuario sea interrumpible y reversible
- Facilitar la interacción a medida que aumenta la habilidad, permitir que se personalice
- Ocultar tecnicismos internos al usuario ocasional
- Diseñar la interacción directa con objetos que aparezcan en la pantalla

2. Reducir la carga de memoria del usuario

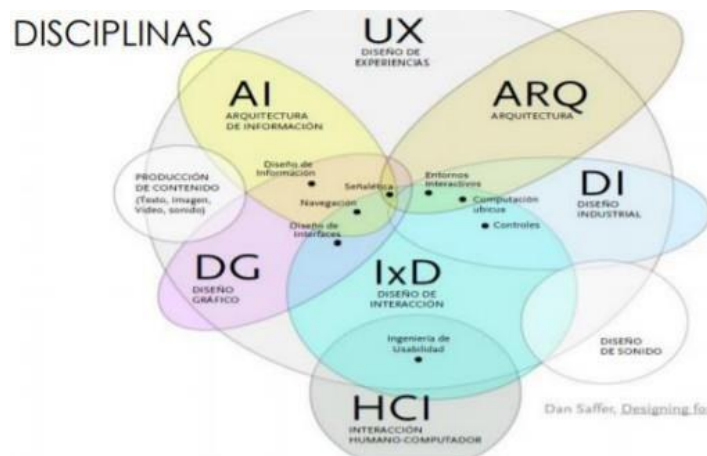
- Reducir la demanda de memoria a corto plazo
- Hacer que lo preestablecido sea significativo
- Definir atajos intuitivos
- La distribución visual de la interfaz debe basarse en una metáfora del mundo real
- Revelar información de manera progresiva

3. Hacer que la interfaz sea consistente

- Permita que el usuario coloque la tarea en curso en un contexto significativo
- Mantener la consistencia en toda la familia de aplicaciones
- Si los modelos interactivos anteriores han creado expectativas en el usuario, no haga cambios a menos que sea necesario

Una interfaz de usuario comprende los dispositivos E/S y el software que los administra, y se incluye cualquier otro aspecto que trate con el uso humano de las computadoras. La interacción va evolucionando conforme a la demanda que tiene la sociedad respecto al software que necesita.

La interacción humano-máquina es la disciplina relacionada con el diseño, evaluación e implementación de sistemas computacionales interactivos para el uso humano, y con el estudio de los principales fenómenos que los rodean. Debemos ocuparnos de la arquitectura de la información, es decir, qué información vamos a mostrar y cómo. Hay que ocuparse del diseño de la experiencia del usuario (UX) que va más allá de la pantalla.

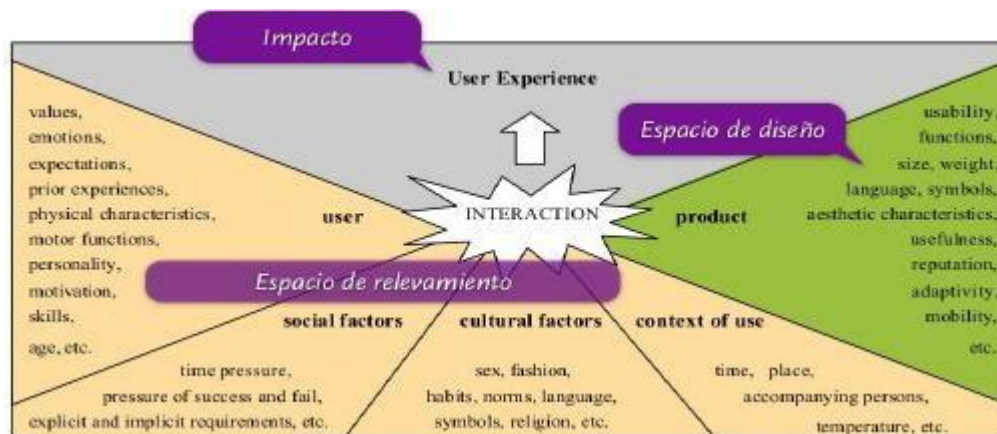


La **user experience (UX)** es el proceso de proporcionar la máxima satisfacción al cliente para mejorar la experiencia general de usuario cuando interactúa con el software. Se centra en la navegación de una aplicación y la estructuración de sus componentes para crear un buen flujo de uso.

La **user interface (UI)** es el proceso de mejorar la presentación e interactividad en el software, enfocándose en el aspecto del mismo y la interacción con el usuario.

Factores que componen la UX:

- Aspectos del usuario (personalidad, valores, emociones)
- Factores sociales (presión del tiempo, estrés. Requerimientos de la sociedad)
- Factores culturales (sexo, moda, religión, lugar del mundo en donde viven)
- Colores (qué transmite cada uno)
- Auto explicación del software, debe ser fácil de aprender a usar ya que los manuales no se leen.



Debemos lograr un equilibrio entre las tres funciones del diseño: presentación, información y uso. Lo que el usuario ve es la presentación que implica expresiones estéticas como figuras, colores y movimientos, con la información que yo necesito mostrar y con el uso que quiero hacer de ese producto de software.

El diseño de UI/UX ayuda a ganar la confianza de los usuarios y hace que usen el software, dándoles lo que buscan. Sin embargo, también el diseño de interfaces se vuelve muy abstracto y más complicado de lograr.

Paradoja de la productividad: las grandes inversiones tecnológicas no han producido los incrementos de productividad esperados.

Facilidad de uso; los usuarios no desean leer manuales ni perder el tiempo aprendiendo a usar el sistema.

Hay algunos *componentes centrales de UX Design*, que son los siguientes:

- **Arquitectura de información**

Busca proporcionar a sus usuarios una navegación fácil

- **Diseño de interacción**

Busca crear el diseño conceptual con el que los usuarios interactúan con el producto

- **Usabilidad**

Facilidad de uso, que incluye desde utilidad para el usuario hasta control de errores y estética. Se pueden medir en rangos según diferentes atributos, como aprendizaje, velocidad de funcionamiento, robustez, recuperación de errores, adaptación... Incrementa la productividad, se usan menos tiempos de aprendizaje y menos errores, y elimina costos de cambio.

- **Wireframing**

Busca crear una muestra de la aplicación para probar características y aspecto

- **Diseño visual**

Determina cómo se define la marca de la empresa, intenta hacer que la apariencia de la aplicación impacte en los usuarios.

COMPLEJIDAD DEL DISEÑO DE IUS

“Es fácil hacer las cosas difíciles. Es difícil hacer las cosas fáciles”

- Dificultades de los diseñadores para comprender las tareas del usuario.
- Especificaciones iniciales incompletas o ambiguas.
- La comprensión completa de un sistema se adquiere a través de su uso.
- La interfaz debe satisfacer las necesidades, experiencia y expectativas de los usuarios previstos.
- Amplia diversidad de usuarios, con diferentes características.
- Los programadores tienen dificultades en pensar como los usuarios

Complejidad inherente de las tareas y los dominios. Es difícil lograr, pero SI fáciles de usar, si las aplicaciones poseen muchas funciones.

HEURÍSTICAS DE NIELSEN

1. **Visibilidad del estado del sistema**: darle al usuario la visibilidad de lo que el sistema está haciendo para que conozca qué está pasando en el sistema (Ej: la ruedita de carga)
2. **Relación entre el sistema y el mundo real**: los términos utilizados para diseñar el sistema y

las interfaces de usuario son los mismos que se usan para referirse a los objetos en el mundo real. Hay que usar el vocabulario del negocio.

3. **Libertad y control por parte del usuario:** permitir al usuario volver atrás. Darle al usuario la sensación de que tiene el control de lo que está sucediendo. Ej: si el usuario ingresa a una pantalla por error, permitir que salga de allí sin tantas vueltas.
4. **Consistencia y estándares:** Mantener siempre las mismas imágenes, iconos y colores. Se debe respetar los estándares y mantener consistencia en todo el sistema. Por ej: estandarizar que el botón aceptar esté a la derecha y el cancelar a la izquierda.
5. **Prevención de errores:** Tratar de frenar al usuario antes de que cometa un error (ej: notificaciones de confirmación antes de eliminar algo, antes de efectivizar la compra).
6. **Reconocer antes que recordar:** Evitar mucho texto, tiene que haber poca explicación y usar algo que el usuario pueda reconocer antes que ponerse a leer un texto para saber qué es lo que tiene que hacer.
7. **Flexibilidad y eficiencia en el uso:** Permitir a usuarios nuevos ir por un camino guiado y a usuarios más expertos permitir el uso de shortcuts (atajos) y un uso más avanzado y rápido del sistema.
8. **Diseño estético y minimalista:** No sobrecargar la pantalla con muchos componentes que al usuario no le hacen falta. Centrarse en la esencia de lo que se quiere mostrar. Ej: para una página de venta online se debería colocar solo la foto del producto y una descripción, y no colocar iconos del clima, el precio del dólar.
9. **Ayuda a los usuarios a reconocer, diagnosticar y recuperarse de los errores:** Hay que mostrar la causa del error y la solución, cómo hacer para recuperarse de ese error. No hacerlo es poco amigable para el usuario.
10. **Ayuda y documentación:** Los sistemas deben disponer de algún espacio de documentación que aclare sobre el producto, que ayude a realizar las acciones que necesita sobre la interfaz. Por ej: el icono de información (i) que muestra cómo funciona el componente.

8 REGLAS DE SHNEIDERMAN (CoARDEDeCoM)

1. **Buscar siempre la coherencia:** Se relaciona con la heurística de Nielsen de **consistencia y estándares**. Se basa en el uso de iconos familiares, colores, jerarquía, utilizando el conocimiento previo del usuario, sin que tengan que aprender algo nuevo, para que puedan realizar lo que desean más rápidamente.
2. **Permitir el uso de short-cuts (Atajos):** Relacionado con la heurística de Nielsen de **flexibilidad y eficiencia** en el uso. Mientras el usuario adquiere experiencia, puede navegar y usar el interfaz más rápido y sin esfuerzo.

3. **Dar realimentación de información:** Se relaciona con la heurística de Nielsen de **visibilidad y estado del sistema**. Para cada acción que realiza el usuario debe haber una respuesta del sistema en un tiempo adecuado (ruedita cuando está cargando) el usuario debe saber dónde está y qué está pasando todo el tiempo.
4. **Diseñar diálogos que tengan un fin:** Se relaciona con la heurística de Nielsen del **diseño estético y minimalista**. Se debe pensar en el objetivo que tiene el texto a mostrar. No se debe incluir cosas que no hagan falta, por ej: origen y destino es suficiente para la interfaz de compra de boletos.
5. **Permitir manejos simples de los errores:** Se relaciona con la heurística de Nielsen de **ayudar los usuarios a reconocer, diagnosticar y recuperarse de los errores**. Cuando ocurre un error, mostrarle al usuario cuál es la manera más fácil de salir de ese error y continuar utilizando el sistema.
6. **Permitir deshacer las acciones con facilidad:** Permitir al usuario volver atrás y arrepentirse de algo que hizo. Se relaciona con la heurística de Nielsen de **libertad y control** por parte del usuario.
7. **Permitir que el centro de control sea interno:** Se relaciona con las heurísticas de Nielsen de dar **retroalimentación y visibilidad del estado del sistema, libertad y control** por parte del usuario y **prevención de errores**. Una interfaz debe situar el poder en el usuario, hacerle creer que él tiene el control.
8. **Reducir la carga de la memoria inmediata:** Se relaciona con la heurística de Nielsen de **reconocer antes que recordad**. No hacerle recordar al usuario lo que vio en una pantalla anterior, brindarle información que le permita no tener que recordarlo. Son los breadcrumbs. Ej: cuando estás comprando por internet y te aparece siempre el monto a pagar para no tener que volver atrás para chequear o recordar el precio.

Diseño de la persistencia

Introducción

Las bases de datos se almacenan frecuentemente sobre almacenamiento secundario (discos rígidos), proveyendo maneras eficientes de acceder a los datos. Otra necesidad proviene de almacenar objetos mayores que una ejecución de programa (que el objeto sobreviva a la ejecución que lo creó). Esta capacidad se llama persistencia. La persistencia frecuentemente significa que los objetos se copian desde una memoria primaria rápida y volátil a una memoria secundaria, lenta y persistente.

La persistencia la hemos hecho presente en la arquitectura como un componente “Persistencia” que hacía de vínculo entre la lógica de negocio que debía ser mantenida por un tiempo y la base de datos relacional. El origen de la problemática de necesitar el diseño de la persistencia son los paradigmas.

Nosotros trabajamos desde los requerimientos con un paradigma orientado a objetos cuya abstracción principal era la clase. Mientras que las bases de datos que se utilizan hoy en día pertenecen al paradigma estructurado.

El paradigma estructurado tiene el código organizado en módulos (abstracción clave) y, para manejar datos, las bases de datos.

Implica analizar nuestras clases, definir cuál de esas clases necesitan persistencia, en que almacenamiento persistente la vamos a guardar, diseñar la BD y la estructura de los datos, y diseñar la persistencia (como hago para conectar las clases con la BD).

Cuando necesitamos que nuestros objetos sean persistentes y no tenemos una BD OO, surge nuestra necesidad de buscarle una solución al paradigma.

Los objetos persistentes son aquellos que requieren almacenamiento persistente. Tenemos diferentes mecanismos para lograrlo:

- Bases de datos de objetos
- Bases de datos relacionales; se requieren servicios especiales que hagan que funcionen con objetos
- Otros (ficheros, XML, etc)



Cuando necesitamos que nuestros objetos sean persistentes y no tenemos una BD OO, surge nuestra necesidad de buscarle una solución al paradigma.

Concepto de Objeto: Comportamiento (su rol, qué es lo que va a hacer en el dominio), estado (valor que pueden tomar los atributos), identidad (permite que sea único, depende de algo externo, es extrínseca, no depende del valor de los atributos). En el modelo de entidad relación la identidad si depende del valor del atributo

DBMS

Necesitamos un sistema de gestión de bases de datos (DBMS) que maneje el almacenamiento en la base de datos. El programador solo quiere una vista lógica de la base de datos y no tomar decisiones sobre

cómo se hace el almacenamiento físico.

Un DBMS debería proveer:

- **Concurrencia:** permite a múltiples usuarios trabajar con una base de datos común simultáneamente.
- **Recuperación:** si ocurre una falla (hardware o software), el DBMS debería ser capaz de volver la base de datos a un estado uniforme de datos.
- **Facilidad de consulta:** el DBMS debería soportar una manera fácil de acceso a los datos en la base de datos.

¿Qué objetos necesitan persistencia?

Los modelos de análisis y de requerimientos pueden estudiarse para decidir qué objetos van a ser persistentes. **Los objetos de entidad** son los candidatos importantes para el almacenamiento persistente.

Mecanismos de almacenamiento y objetos persistentes

Bases de datos de objetos: Si se usa este tipo de bases de datos para almacenar y recuperar objetos, entonces no es necesario ningún servicio de persistencia. Características: soportan objetos complejos, cada objeto debe tener una identidad independientemente de los atributos, deben soportar el encapsulamiento de datos y el comportamiento, debería soportar un mecanismo de estructuración en forma de clases o tipos, debería soportar la herencia, debería soportar sobreescritura y ligadura tardía, debería ser extensible para poder agregar nuevos tipos y debería tener completitud, esto es, el lenguaje de manipulación debería ser capaz de expresar cada función calculable.

Bases de datos relacionales: debido al predominio de este tipo de BD, es necesario que se utilicen. En este caso surgen problemas de incompatibilidad entre la representación de los datos orientada a registros y la orientada a objetos por lo que se requiere un servicio especial para establecer la correspondencia O-R (mapping O-R).

Otros: a veces se desea almacenar los datos con otro mecanismo de almacenamiento o formato como archivos comunes, estructuras xml, bases de datos jerárquicas, etc. De igual forma en este caso también se presentan incompatibilidades y, por lo tanto también se requieren servicios especiales que hagan que funcionen con objetos.

Modelos de datos

Los DBMSs han evolucionado mediante un número de generaciones incluyendo modelos de datos jerárquicos, de red, relacionales y orientados a objetos. El modelo de datos dominante en aplicaciones industriales en la actualidad es el modelo relacional.

Modelo de entidad/relación

En una base de datos relacional, los datos se almacenan en tablas. Los tipos a ser usados en las tablas son mayormente los tipos primitivos (caracteres, enteros, reales, etc.). Esto trae problemas a la hora de almacenar objetos.

El modelo de entidad/relación se basa en:

- **Relaciones:**

- O Una relación es la abstracción de un conjunto de asociaciones que existen entre las instancias de dos entidades. Las **relaciones entre tablas son lógicas, con claves foráneas, a diferencia de las clases que son relaciones físicas a través de punteros y es unidireccional.**

- O **Tienen sentido bidireccional.** Esta es una **diferencia con las clases, las cuales tienen un vínculo con puntero.** En este modelo la relación es en dos direcciones y tenemos una matriz con filas y columnas. Cada tabla sería una clase y cada fila sería un objeto.

- **Cardinalidad:**

- O Indica **para una instancia de una entidad A con cuántas instancias de la entidad B se relaciona.**

- O **Es lo que en las clases es la MULTIPLICIDAD.**

- **Opcionalidad:**

- O Indica para una instancia de una entidad A, si la relación con instancias de la entidad B, es opcional u obligatoria.

- O Sería lo que es 0..1 en la relación entre las clases

- **Reglas de integridad:**

- O **Integridad relacional:** Ningún componente del atributo identificador en una entidad aceptará NULOS

- O **Integridad referencial:** Un modelo de datos no debe contener valores en sus atributos referenciales para los cuales no exista un valor concordante en el/los atributos identificadores en la entidad objetivo.

- O **Clave primaria:** Un atributo a (posiblemente compuesto) de la relación R es una clave candidata de R, si y solo si satisface las siguientes propiedades:

- **Unicidad:** no existen dos tuplas de R con el mismo valor de a, en un momento dado.
 - **Minimalidad:** si a es un atributo compuesto, no puedo eliminar un componente de a sin destruir la propiedad de unicidad.

- O **Relaciones:** En una base de datos relacional, nunca registraremos información de algo que no podamos identificar. Para las claves primarias compuestas: cada valor individual de la clave primaria debe ser no nulo en su totalidad. Esta regla se aplica a las relaciones base únicamente. Se aplica sólo a las claves primarias.

O **Claves ajenas:** La clave ajena (FOREIGN KEY, en inglés) es un atributo (quizás compuesto) de una relación R2 cuyos valores deben concordar con los de la clave primaria de alguna otra relación R1.

Problema de Impedancia

Se refiere al problema de la correspondencia entre los paradigmas, en el paradigma OO la información se almacena en los objetos y, las bases de datos relacionales pertenecen al paradigma estructurado. Por lo tanto, necesitamos transformar nuestra estructura de información de objeto a una estructura orientada a tablas (modelo relacional). Esto crea un fuerte acoplamiento entre la aplicación y el DBMS. Este problema se denomina problema de impedancia.

Se basa en los siguientes problemas:

1. **Las bases de datos relacionales no soportan comportamiento:** los métodos no son soportados, todos los métodos que hacen la integración de la clase y que es lo más importante que tiene la clase no se puede guardar en una BD relacional.
2. **Las bases de datos relacionales solo almacenan tipos de datos primitivos y no objetos complejos:** estos tipos de datos los define el proveedor del motor de base de datos y, no se pueden definir tipos propios de datos y, las clases no son ni más ni menos que un tipo de dato propio, es normal que los objetos se compongan de más objetos: se soluciona utilizando múltiples tablas, y relacionándolas con sus PK. Hay que desarmar la estructura de clases y mapearlas a cada una de las columnas de las tablas.

La vinculación entre el modelo de clases y las bases de datos relacionales es el MAPEO o DISEÑO DE LA PERSISTENCIA.

No existe una correspondencia directa entre estos dos sistemas, por lo que hay que crear un esquema de correspondencia (mecanismos de mapeo) para resolverlo. Se debe decidir la forma en que se almacenarán los atributos complejos de los objetos y las relaciones entre ellos.

En resumen:

- Problema al almacenar objetos complejos (comportamiento y atributos con referencia a otros objetos). Los datos deben ser primitivos.
- Se debe violar el encapsulamiento para extraer el estado del objeto: atributos que deberían ser privados e inaccesibles desde el exterior tienen que exponerse para almacenarlos en la BD.
- Problema para representar la herencia: Las relaciones de herencia y asociación no son soportadas nativamente por las BDR y se deben implementar mecanismos para simularlas: encontrar una manera de representar la PK y muchas veces los objetos no tienen atributos que sirvan, por lo que se debe añadir uno y esconderlo de los clientes, ya que no es parte del objeto

en sí, sino una herramienta para integrar con la BDR.

- Genera gran acoplamiento entre la aplicación y el DBMS: se debe hacer que la menor cantidad de partes posibles conozcan la interfaz del DBMS.

MAPEO DE CLASES A BASES DE DATOS RELACIONALES

Mapeo de CLASES/OBJETOS en Tablas

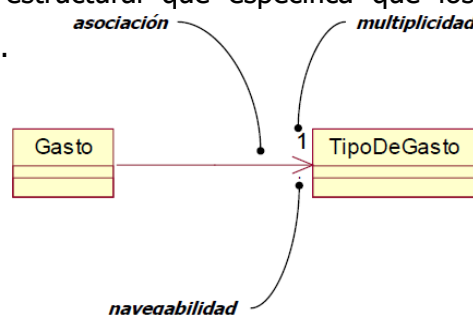
El primer consiste en determinar qué clases y atributos de la clase deben almacenarse en la base de datos. En general se procede a tomar el **modelo de clases de diseño** (resultante de haber aplicado patrones en donde ya tiene atributos refinados identificados con sus tipos y con la visibilidad necesaria, los atributos tienen que ser privados). Luego, se hace un descarte de las clases que no necesitan persistencia. Las clases que sí necesitan persistencia son las del negocio que se llega casi al modelo de dominio, sólo que no tenían el nivel de detalle con el que cuentan ahora. A esa clase de negocio se le asigna una tabla y se descarta el comportamiento. A cada atributo se lo coloca en una columna de la tabla. La tabla se llama igual que la clase para mantener la trazabilidad o el vínculo.

Una clase se mapea en tablas de la siguiente manera:

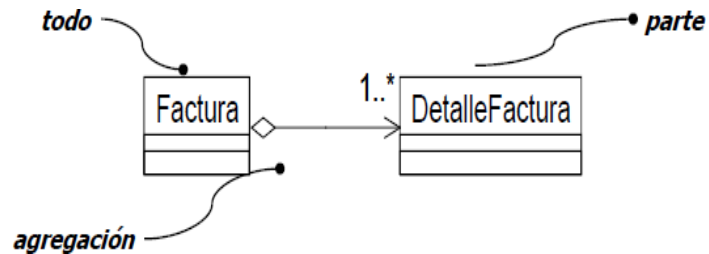
1. Asignar una tabla para la clase.
2. Cada atributo primitivo se transformará en una columna en la tabla. Si el atributo es complejo, agregamos una tabla adicional para el atributo, o distribuimos el atributo en varias columnas de la tabla de la clase.
3. La columna de la clave primaria será el identificador único de la instancia. El identificador debe ser preferentemente invisible al usuario y generadas automáticamente por máquina.
4. Cada instancia de la clase será representada por una fila en la tabla.
5. Cada asociación de conocimiento con una cardinalidad mayor que 1 (0..N por ejemplo)se transformará en una nueva tabla. Esta nueva tabla conectará las tablas que representan los objetos que van a ser asociados mediante el identificador de la clave primaria. En algunos casos, podemos tener la relación representada como una columna (atributo) en la tabla del objeto asociado.

Relaciones entre clases que requieren persistencia

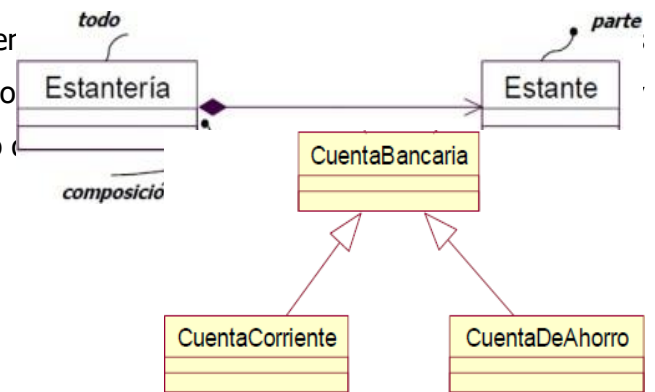
- Asociación: Es una relación estructural que especifica que los objetos de un elemento se conectan a los objetos de otro.



- Agregación: Es un tipo especial de asociación, que representa una relación completamente conceptual entre un "todo" y sus "partes".



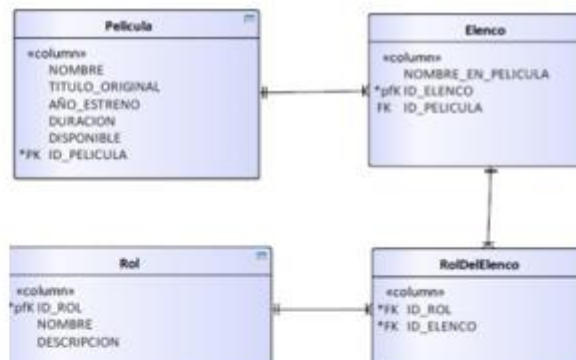
- Composición: Es una variación de la agregación simple, con una fuerte relación de pertenencia y vidas coincidentes de la parte con el todo.
- Generalización: Es una relación entre un elemento específico de ese elemento (subclase o hijo) comportamiento, o modificar el comportamiento



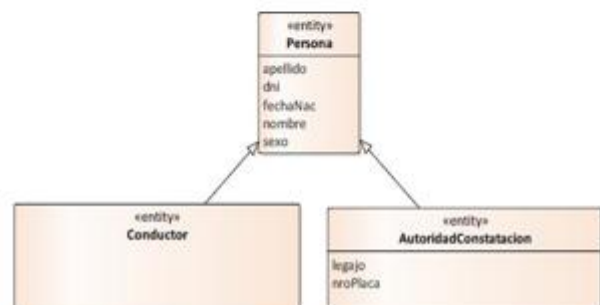
Opciones para el mapeo de asociación (lo cual incluye agregación y composición)



3. Muchos a muchos: se crea una tabla intermedia asociativa. Es necesaria por el problema de la clave foránea.



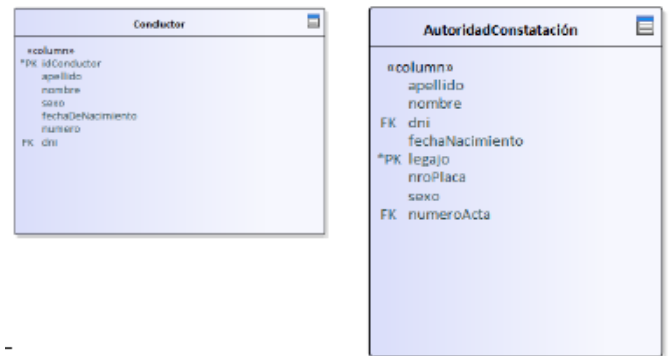
4. Uno a uno: las relaciones uno a uno de ambos lados no debería existir porque en ese caso se debería fusionar todo en una sola tabla. En este caso sí se puede porque del lado del usuario tenemos opcionalidad 0..1.



A partir de la siguiente herencia, vemos cómo serían las opciones

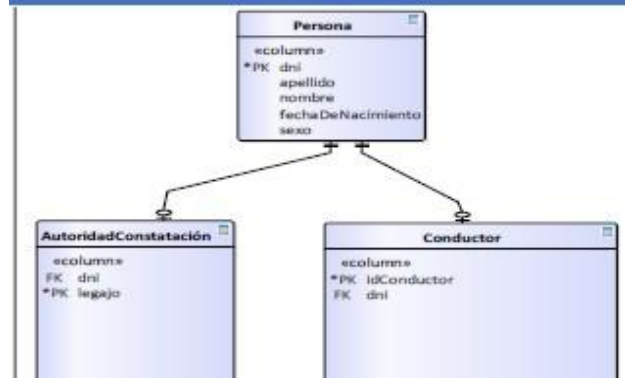
1. **Eliminar el padre:** significa eliminar la herencia. Quedan tablas únicas para los objetos hijos. Se copian todos los atributos de la clase padre. Ninguna tabla representa la clase abstracta. Es más rápida ya que los datos están en una sola tabla y no hay que hacer joins. El tamaño de la tabla aumenta ya que se duplican columnas (los atributos comunes van a repetirse). Puede traer problemas de consistencia y redundancia.

Eliminar la herencia – eliminando al padre



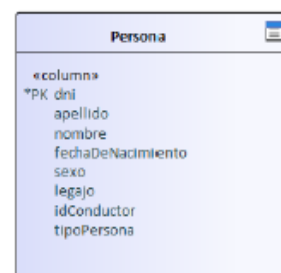
2. **Simularla:** es una jerarquía supertipo-subtipo. La clase abstracta está en su propia tabla y las tablas de los hijos hacen referencia a ella. Reduce la redundancia, pero hacer JOINS que pueden reducir la performance.

Simular la herencia



3. **Eliminar los hijos:** se ponen todos los atributos de todas las clases hijas en una sola tabla. No se recomienda porque no hay trazabilidad, hay baja cohesión y se desperdicia almacenamiento ya que muchos atributos quedarán en blanco.

Eliminar la herencia – eliminando los hijos



Para elegir una u otra opción hay que analizar el volumen de las tablas, la cantidad de registros y cómo va a ir creciendo el modelo.

Modelos de persistencia

Como solución al problema de que los objetos deben ser persistentes y, a su vez tenemos al problema de impedancia, sino que tenemos que ver a qué clase de nuestro software le damos la responsabilidad de que haga este trabajo. Para ello, se utiliza un **framework de persistencia**, el cual es un conjunto de tipos de propósito general, reutilizable y extensible que proporciona funcionalidad para dar soporte a los objetos persistentes. Un servicio de persistencia o subsistema realmente proporciona el servicio y se creará con un framework de persistencia.

En general, un framework:

- Es un conjunto extensible de clases e interfaces que colaboran para proporcionar servicios de la parte central e invariable de un subsistema lógico.
- Contiene clases concretas y (especialmente) abstractas que definen las interfaces a las que ajustarse, interacciones de objetos en las que participar, y otras variantes.
- Pueden requerir que el usuario del framework defina subclases del framework para utilizar, adaptar y extender los servicios del framework.
- Tiene clases abstractas que podrían contener tanto métodos abstractos como concretos.
- Confía en el Principio de Hollywood "No nos llame, nosotros lo llamaremos". Esto quiere decir que, las clases definidas por el usuario recibirán mensajes de las clases predefinidas en el framework.
- Ofrecen un alto grado de reutilización, mucho más . que con clases individuales.
- Debería proporcionar funciones para:
 - Almacenar y recuperar objetos en un mecanismo de almacenamiento persistente.
 - Confirmar y deshacer (commit y rollback) las transacciones.

Un servicio de persistencia se escribe normalmente para que trabaje con BDR lo cual también se conoce como servicio de correspondencia O-R. Generalmente, un servicio de persistencia tiene que traducir los objetos a registros y guardarlos en una base de datos y traducir los registros a objetos cuando recuperamos de la base de datos.

En particular, estas son las funciones principales que debe proporcionar y se conoce como desmaterialización y materialización respectivamente.

Materialización y Desmaterialización

- **Materialización:** una o varias filas de tablas (registros) de la base de datos son convertidos a objetos en memoria.
 - Materialización inmediata: recupera los objetos al instante.
 - Materialización perezosa: recupera los objetos cuando se los necesita realmente. Se puede implementar utilizando un proxy virtual.

- **Desmaterialización:** proceso inverso, un objeto se graba en la base de datos en forma de uno o varios registros (filas) de tablas.

Los dos modelos de persistencia que tenemos son:

1. SuperObjeto Persistente

2. Esquema de Persistencia

Súper Objeto Persistente

Se crea una **super clase de fabricación** pura con los comportamientos necesarios para implementar la persistencia (materialización y desmaterialización), aquellas clases que requieran de persistencia deben heredar los comportamientos de esta super clase. Se llama esquema de correspondencia directa porque no se usan intermediarios, se agregan los métodos a esa clase. En este modelo, se crea una clase abstracta llamada SuperObjeto Persistente (u objeto a almacenar). Todos los objetos persistentes heredan de la misma. Desde el punto de vista arquitectónico el SuperObjetoPersistente se encuentra en la capa LN y cada clase persistente sabe cómo guardarse y recuperarse de la BDR.

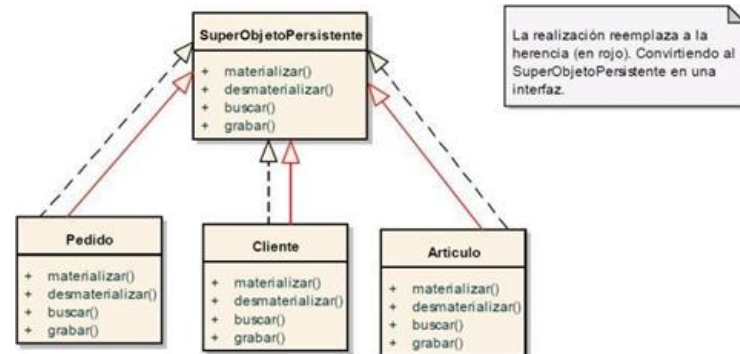
Cada clase del dominio tiene la responsabilidad de saber cómo hacerse persistente en la base de datos.

• Ventajas:

- O No crece la estructura de clases
- O Fácil de implementar

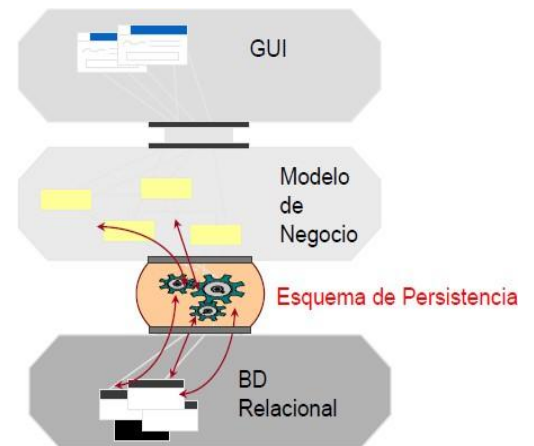
• Desventajas:

- O Si la clase trae herencia del dominio y también tiene que heredar la persistencia, hay muchos lenguajes de programación que no soportan herencia múltiple, sólo herencia simple. Aunque se podría solucionar con una realización e interfaz para el SuperObjeto Persistente.
- O No se cumple el principio de diseño **Open/Close** ni tampoco el principio de **Responsabilidad Única**. Una clase del dominio no debería saber cómo hacerse persistente porque no modela ese concepto.
- O Se rompe la cohesión. (hay baja cohesión cuando debería ser alta)
- O Desde el punto de vista arquitectónico, se presenta el problema del **acoplamiento entre la capa de negocio y la capa de base de datos**. (hay alto acoplamiento cuando debería ser bajo)



Esquema de persistencia

Se agrega un componente de persistencia para resolver el mapeo sin tocar las clases, compuesto por clases de fabricación pura, que tienen el rol de establecer vínculos entre la capa de LN y la capa de Datos, sin que los objetos se vean afectados por el problema de persistencia. El esquema de persistencia resuelve el problema de materializar, desmaterializar y los métodos necesarios para la persistencia, se crean clase de fabricación pura para cada clase del dominio que deba ser persistente, dichas clases se ubican dentro del esquema de persistencia



Se basa en un esquema **de correspondencia indirecta**.

Es lo que se usa desde la arquitectura y lo que se recomienda.

Se llama esquema de correspondencia indirecta porque ponemos

“algo” en el medio, ese algo, es el esquema de persistencia. Para que funcione, se agrega en el esquema de persistencia una clase que funcione como intermediara para cada clase de negocio.

Este esquema tiene 2 niveles, el primero son las interfaces que definen los lineamientos y después una clase de persistencia de BD para cada clase del dominio que necesite ser persistente. Se encarga de hacer el mapeo. En las interfaces está la decisión arquitectónica de cómo manejar la materialización, desmaterialización, identificación de un objeto con un ID, etc. Cada clase concreta tiene que implementar el comportamiento de acuerdo a los atributos que tiene la clase abstracta.

Para materializar y desmaterializar se debe guardar en la misma fila, manteniendo la referencia entre el puntero del objeto producto y el puntero ID de la fila en la tabla donde ese objeto estaba desmaterializado.

En este esquema se aplica el Principio de Hollywood (Inversión de dependencia), xq si no lo hiciéramos, las clases de negocio tendrán la responsabilidad de invocar al esquema de persistencia, entonces seguirán igual de acopladas. Para que esto no pase aplicamos el principio DIP donde una clase de control dentro del esquema invoca la persistencia, y cuando esto pasa el esquema de persistencia toma el control, donde busca el objeto que hay que hacer persistente y lo hace persistente sobre la base de datos, no nos llames, nosotros te llamamos, de esta forma mantengo la cohesión de las clases de dominio porque no las tengo que tocar, la persistencia se encarga de eso. Tiene bajo acoplamiento

Es un conjunto reutilizable y generalmente expansible de clases que prestan servicios a los objetos persistentes. Debe traducir los objetos a registros y guardarlos en una BD y luego los traduce a objetos cuando los recupera de la BD.

Se les quita la responsabilidad a las clases del negocio.

- **Ventajas:**

- O Se respeta el principio de Responsabilidad Única y el open Close porque las clases del dominio no se tocan
- O Mantiene la alta cohesión
- O Soporta extensibilidad
- O Mejora la reusabilidad
- O Mantiene bajo el acoplamiento ya que aplica el principio de Inversión de Dependencias (Principio de Hollywood). El control lo toman las clases del esquema de persistencia que van y buscan a los objetos y los materializan/desmaterializan.

- **Desventajas:**

- O Hay un aumento considerable de clases

Patrones de persistencia

- **Patrón Identificador de Objetos:** propone asignar un identificador de objeto a cada registro y objeto. Es único, vincula objetos con registros evitando duplicados en una tabla sin equivocación. Es una forma consistente de relacionar objetos con registros y asegurar que la materialización repetida no de como resultado objetos duplicados.
- **Patrón Representación de Objetos como Tablas:** propone la definición de una tabla en una BDR por cada objeto persistente.
- **Patrón fachada:** permite el acceso al servicio de persistencia mediante fachada, proporcionando una interfaz uniforme a un subsistema. La fachada no hace el trabajo, solo lo delega en objetos del subsistema. Se necesita una operación para recuperar un objeto dado un OID. También se necesita conocer el tipo del objeto que se va a materializar.
- **Patrón Conversor (Mapper) o Intermediario (Broker):** propone crear una clase responsable de materializar y desmaterializar los objetos. Agrega un intermediario por cada clase de dominio (genera el doble de clases). Utiliza objetos para establecer la correspondencia con los objetos persistentes. Enfoque de correspondencia indirecta.
- **Materialización con el método plantilla:** con el patrón TemplateMethod encapsulamos en una sola clase el comportamiento común de la materialización. El punto de variación es la manera de crear el objeto a partir del almacenamiento.
- **Patrón Gestión de Caché:** es conveniente mantener los objetos materializados almacenados en una caché local para mejorar el rendimiento de y dar soporte a las operaciones de gestión de transacciones, con su OID como clave. El conversor primero busca en la caché para evitar materializaciones innecesarias.
- **Estados transaccionales y el Patrón State:** los objetos persistentes pueden insertarse, eliminarse

o modificarse. Operar sobre un objeto persistente no provoca una actualización inmediata sobre la BD, más bien debe ejecutar una operación commit explícita. La respuesta a una operación depende del estado de la transacción del objeto (estado de los objetos para saber si se modificaron en función de su estado almacenado).

- **Diseño de una transacción con Patrón Command:** una transacción es una unidad de trabajo (conjunto de tareas) cuya terminación es atómica (se realizan todas o ninguna). En los servicios de persistencia se incluyen la inserción, actualización y eliminación de objetos, las cuales necesitan que se ordenen justo antes de la ejecución. La idea clave es representar cada tarea de la transacción como un objeto con un método ejecutar () polimórfico. Con el patrón se busca crear un objeto por cada operación a realizar en la transacción, haciendo una cola de operaciones (cola de objetos) permitiendo hacer un rollback (deshacer los cambios), si es necesario.

- ❖ **Materialización perezosa mediante un Proxy Virtual:** es un proxy para otro objeto (al que se quiere acceder) que materializa el sujeto real cuando se referencia por primera vez. Es una referencia inteligente. Se basa en la suposición de que los proxies conocen el OID de sus sujetos reales, y al requerir la materialización, se lo usa para identificar y recuperar el sujeto real.

Mapeo

- 1_Cada clase representa 1 tabla
- 2_Cada atributo representa una columna, en caso de no ser atómico debe descomponerse en varias
- 3_La columna de la PK será único identificador
- 4_Cada instancia de la clase es representada por filas

Modelos de persistencia

1. SuperObjetoPersistente
 - Fácil de implementar
 - No incrementa la estructura c.
2. Esquema de Persistencia
 - Mantiene cohesión
 - Soporta extensibilidad
 - Mejor reusabilidad

Principios SOLID

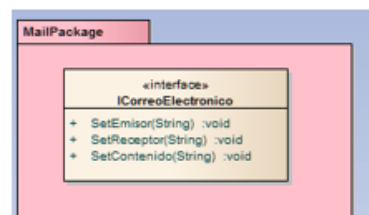
Los principios SOLID son cinco principios enfocados a la elaboración de software de calidad, asociados a la programación orientada a objetos y al diseño. Ayudan a mantener y ampliar el software en el tiempo, y a que el código sea legible y fácilmente testeable, ya que nos obligará a sacar muchas interfaces, y esto favorecerá el uso de Mocking y prepara el código para cambios propios de los tests. Los principios son:

S -> Single responsibility principle.

No debería haber nunca más de una razón para cambiar una clase. Una clase debería tener una única responsabilidad, de forma tal que cuando cambie algún requisito, dicho cambio sólo afecte a dicha clase por una razón. Debe tener un solo rol dentro del SW, relacionado con el concepto que esa clase modela que define su responsabilidad. Cada parte del SW debería tener una única razón para cambiar. Se busca tener una cohesión funcional. Así, en caso de mantenimiento, frente a un cambio sabremos exactamente a dónde ir para implementar la modificación. Además, si una clase cumple varias responsabilidades, frente a un cambio en una sección de la clase podríamos romper el código de la misma clase que no esté relacionado con el cambio agregado. Este principio se basa en tener alta cohesión. El precio de tener alta cohesión es el surgimiento de nuevas clases de fabricación pura. **Podríamos relacionarlo con el patrón Experto de los patrones GRASP y State, Strategy, Builder, Adapter.** Busca reducir la complejidad y evitar romper el código. Responsabilidad: Obligación que tiene un objeto en cuanto a su comportamiento, donde implementamos métodos para llevarla a cabo.

Ejemplo: Supongamos que tenemos un paquete de software para la gestión de correos electrónicos. En algún lugar de dicho paquete, antes del envío de los correos en sí, queremos establecer los distintos parámetros que, por ahora, son: Emisor, Receptor, Contenido.

La interpretación del enunciado, de forma sencilla podría ser:



Y el código:

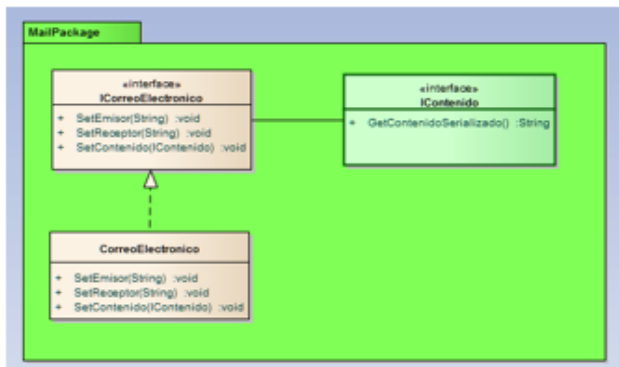
```

public interface ICorreoElectronico
{
    void SetEmisor(String emisor);
    void SetReceptor(String receptor);
    void SetContenido(String contenido);
}

public class CorreoElectronico : ICorreoElectronico
{
    public void SetEmisor(String emisor)
    {
    }
    public void SetReceptor(String receptor)
    {
    }
    public void SetContenido(String contenido)
    {
    }
}
  
```

Esto incumple el SRP dado que hay más de una razón para el cambio de la clase. Observemos que el contenido del Email es un String, si cambiara el tipo de contenido, habría que cambiar el código cada vez que el contenido cambia. También se podría necesitar agregar más campos además del emisor, receptos y contenido, con lo cual, sería necesario cambiar la clase Correo Electrónico.

SOLUCIÓN



```

interface ICorreoElectronico
{
    void SetEmisor(String emisor);
    void SetReceptor(String receptor);
    void SetContenido(IContenido content);
}

interface IContenido
{
    String GetContenidoSerializado(); // para serializar el contenido
}

class CorreoElectronico : ICorreoElectronico
{
    public void SetEmisor(String sender) { }
    public void SetReceptor(String receiver) { }
    public void SetContenido(IContenido content) { }
}
    
```

Si en vez de un contenido String utilizamos una interfaz IContenido que suponga un contrato para hacer viable cualquier tipo de contenido cada vez que nos pidan que añadamos un tipo de contenido sólo tendremos que modificar dicha interfaz y/o las clases que lo implementen y ya no a la clase de Correo Electrónico y a su interfaz.

O -> Open-close principle.

Las entidades de software deberían estar abiertas para extensión y cerradas para modificación. Ante la necesidad de un cambio no debería cambiar el SW existente, sino agregar algo nuevo que trabaje con el anterior sin modificar su funcionamiento. Cambia el comportamiento de una clase mediante herencia, polimorfismo y composición, no modificando el código propio de la misma. Así, podemos evitar romper el código existente frente a una implementación de algo nuevo. Frente a la situación de cambio de requerimientos o de comportamientos variables, en lugar de abrir la clase para que hagan algo nuevo, creamos otra sub clase que soporte el nuevo comportamiento demandado. Extender significa agregar clases nuevas. Este principio tiene la ventaja de que también se cumple con el SRP moviendo comportamiento a una clase distinta que se ocupe del mismo.

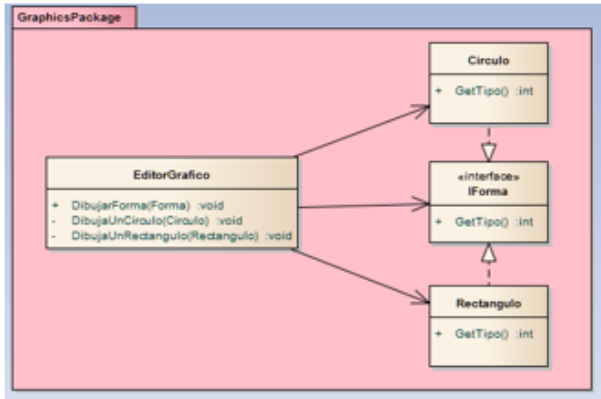
Patrón que aplica: State, strategy, observer, builder, adapter, composite



Ej: Tengo una clase pintor y quiero agregar un nuevo estilo de pintura, extendiendo con clases nuevas para soportar el comportamiento y aplicando polimorfismo redefino el método pintar(), que es igual a todas las clases, entonces polimórficamente implemento el pintar según que es lo que necesito.

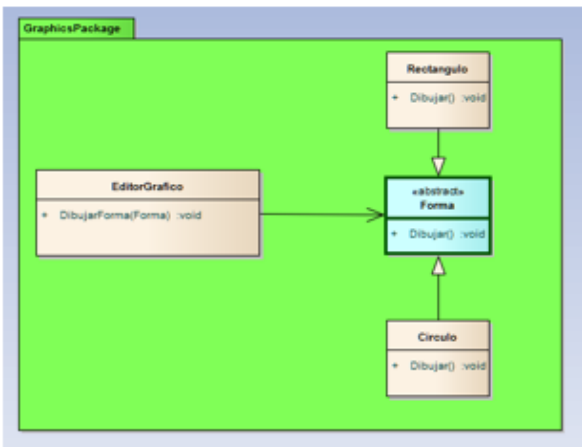
Ejemplo: Diseñar un editor gráfico capaz de dibujar un círculo y un rectángulo

ORIGINAL



Esto incumple el OCP por varias razones, primero, la clase EditorGrafico no está abierta a la extensión, y si quisiéramos añadir una nueva figura geométrica habría que crear la nueva clase y habría que modificar el método público y añadir uno privado para el pintado de dicha nueva forma... y además si nos fijamos en SRP, añadir una nueva forma sería un motivo para el cambio y añadir una nueva funcionalidad como por ejemplo BorrarForma, EditarForma o cualquier otra cosa que queramos que haga nuestro editor gráfico, sería un segundo motivo para el cambio... Si nos piden que añadamos alguna otra forma, tendrán una responsabilidad, la de dibujarse a sí mismas.

SOLUCION



Esto es más simple, más sencillo, más robusto, más cerrado a las modificaciones pues un círculo siempre se pinta igual y un rectángulo lo mismo, pero a la vez abierto a la extensión porque podemos añadir las formas que queramos sin tener que modificar código anterior en el EditorGrafico. Siempre que haya comportamientos que dependan del tipo pensar en si se puede implementar con una clase abstracta.

L -> Liskov substitution principle.

Las funciones que utilicen punteros a clases base deben ser capaces de usar objetos de clases derivadas de éstas sin saberlo. Las subclases deben comportarse adecuadamente cuando sean usadas en lugar de sus clases base. Una subclase debería poder ser usada como clase base sin tener ningún problema y sin romper el sistema ya que ambas clases deberían ser compatibles en todo momento.

Este principio es una prueba conceptual para conocer si la herencia está bien pensada. Una herencia está bien pensada cuando es una herencia de comportamiento, cuando tengo comportamiento que se va a ejecutar diferente en las subclases gracias al polimorfismo. El comportamiento de los hijos no se tiene que transformar en algo que no tiene nada que ver con lo que era ese comportamiento en el padre.

El principio de sustitución tiene un conjunto de requisitos formales para las subclases y específicamente para sus métodos.

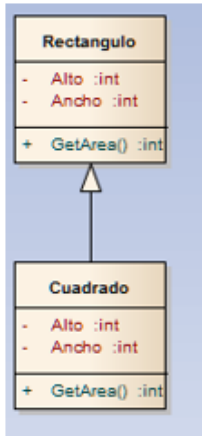


Ej: Un Robot Padre que hace Café.

Herencia bien diseñada: Un Robot Hijo que hace Capuccino (es un café que se hace de forma distinta)

Herencia mal diseñada: Un Robot Hijo que hace Agua (no se comporta como el padre)

Ejemplo: Un cuadrado es un rectángulo con la peculiaridad de que sus lados miden exactamente lo mismo, su diagrama de clases y código es el siguiente:



```

public class Rectangulo
{
    public int Alto { get; set; }
    public int Ancho { get; set; }
    public int GetArea()
    {
        return Alto * Ancho;
    }
}

public class Cuadrado : Rectangulo
{
    public override int Alto
    {
        get { return base.Alto; }
        set { base.Alto = base.Ancho = value; }
    }

    public override int Ancho
    {
        get { return base.Ancho; }
        set { base.Ancho = base.Alto = value; }
    }
}

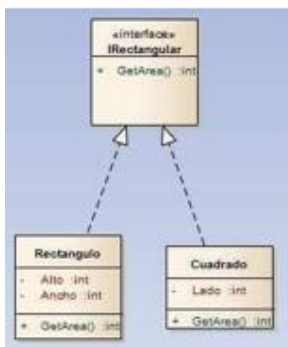
public class Prueba
{
    private static Rectangulo GetNewRectangulo()
    {
        // Podría ser cualquier objeto que también sea un rectángulo...
        // por ejemplo un cuadrado... ¿No?
        return new Cuadrado();
    }

    // Y asumire que puede darle valores al alto y al ancho...
    r.Alto = 2;
    r.Ancho = 3;

    Console.WriteLine(r.GetArea());
    // Y al ver la consola el usuario dice: WTF!
}
    
```

El principio LSP no se cumple. ¿Se ha comportado adecuadamente la subclase en sustitución de la clase padre? Puede que sea cierto que un cuadrado es un caso particular de un rectángulo en muchos aspectos pero, ¿en el caso del área no se comportan igual?

SOLUCIÓN



```

public interface IRectangular
{
    void SetAlto(int alto);
    void SetAncho(int ancho);
    int GetArea();
}

public class Rectangulo : IRectangular
{
    private int _alto;
    private int _ancho;

    public void SetAlto(int alto)
    {
        _alto = alto;
    }

    public void SetAncho(int ancho)
    {
        _ancho = ancho;
    }

    public int GetArea()
    {
        return _alto * _ancho;
    }
}

public class Cuadrado : IRectangular
{
    private int _lado;

    public void SetAlto(int alto)
    {
        _lado = alto;
    }

    public void SetAncho(int ancho)
    {
        _lado = ancho;
    }

    public int GetArea()
    {
        return _lado * _lado;
    }
}
    
```

Debido a que el comportamiento de un cuadrado no es coherente con el del rectángulo, la mejor solución es no tener un cuadrado heredando de un rectángulo sino crear una interfaz de la que hereden tanto cuadrado como rectángulo...

Los tipos de parámetros en un método de una subclase debe coincidir o ser más abstractos que los tipos de parámetros en el método de la superclase.

El tipo de retorno en un método de una subclase debe coincidir o ser un subtipo del tipo de retorno en el método de la superclase.

Los requisitos para un tipo de retorno son inversos a los requisitos para los tipos de parámetros.

Un método en una subclase no debe generar tipos de excepciones que no se espera que arroje el método base.

Los tipos de excepciones debe coincidir o ser subtipos de los que el método base ya puede lanzar.

Una subclase no debería fortalecer las condiciones previas.

Una subclase no debería debilitar las condiciones posteriores.

Deben conservarse las invariantes de una superclase. Esta es probablemente la regla menos formal de todas. Las invariantes son condiciones en las que un objeto tiene sentido.

Una subclase no debería cambiar los valores de los campos privados de la superclase

I -> Interface segregation principle.

El objetivo es no forzar a los clientes a interactuar o relacionarse con interfaces muy complejas, con cosas que no necesita (dependen de métodos que no utiliza), ni forzar las clases concretas a implementar comportamiento que no necesitan o no pueden. (Alta cohesión en interfaces). Mantén las interfaces pequeñas y cohesivas, que puedan coexistir unas con otras. Podemos hacer que una clase implemente más de una interfaz, a diferencia de la herencia que por lo general permite que una clase herede de una única clase. Se relaciona una clase concreta con una interfaz, que la implementa. Se ve a la interfaz como un contrato con cada cliente, que debería ser lo más pequeño posible (pocos métodos), para que ese cliente acceda al comportamiento declarado en la interfaz.

Debemos aprovechar esto, y hacer que cada interfaz sea cohesiva, asignarle la responsabilidad que corresponde a la interfaz, no mezclar métodos y comportamientos que no están relacionados, y hacer que la clase implemente todas las interfaces que necesitamos. Una clase concreta puede implementar todas las interfaces que quiera.

Patrón que aplica: Strategy, observer, adapter

Es necesario en este caso, hacer una distinción entre los siguientes conceptos:

- **Interfaz:** es un tipo especial de clase abstracta (una clase abstracta es una clase padre de una herencia). Sólo puede tener métodos abstractos. La idea es separar la especificación de funcionalidad (la interfaz) de su implementación por una clase o componente que es la que realmente va a ejecutar ese código. Una interfaz no se puede instanciar, se habla de "contratos", se especifica el comportamiento, pero no está implementado el código que especifica ese comportamiento. Las interfaces siempre están asociadas a una clase que es concreta y que tiene implementado en un método lo que en la interfaz está declarado.
- **Clase Abstracta:** Son clases que tiene algunos (o todos) sus métodos abstractos, o sea sin implementar. Esos métodos deben ser implementados por las clases hijas. Las clases abstractas no se pueden instanciar. Puede tener métodos concretos también.

- **Método abstracto:** tiene sólo la signatura, por dentro está vacío, no tiene código.
- **Método concreto:** es un método implementado.

La relación que se da entre las clases concretas y la interfaz, es la relación de **REALIZACIÓN**, la cual se representa con una línea punteada con punta de flecha rellena. La realización funciona cuando una clase concreta implementa comportamiento que está declarado en una interfaz. Una clase concreta puede implementar comportamiento de todas las interfaces que necesite sin ningún límite. Esto es una diferencia con la que los lenguajes de programación solo soportan herencia simple (una clase hija sólo puede tener un padre). Además, se basa en no obligar a las clases concretas a implementar comportamiento que no necesitan. Se deben segregar las interfaces gruesas hasta crear otras más detalladas y específicas, que sean cohesivas para que las clases clientes puedan implementar solamente aquellos métodos que necesitan de verdad.

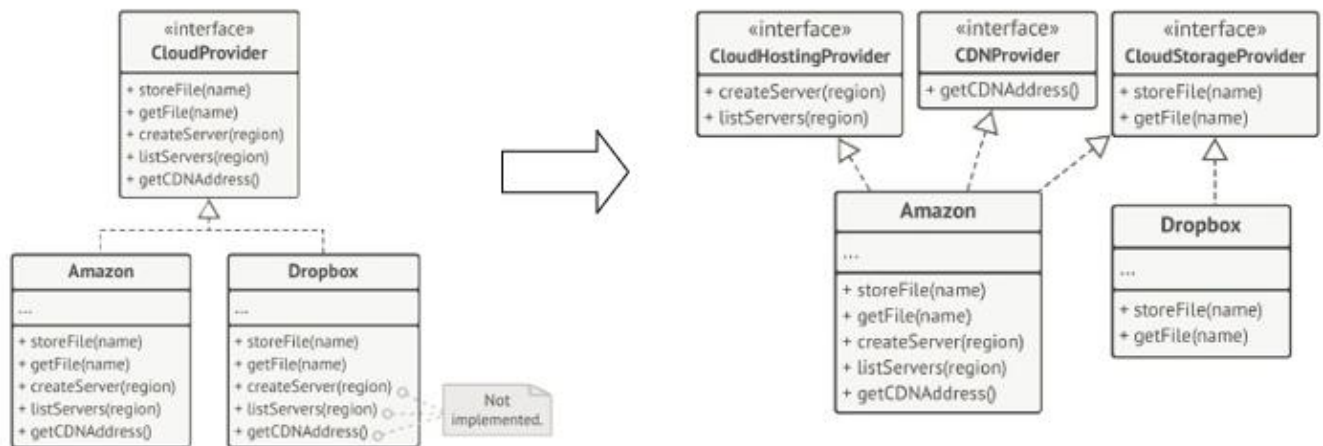


Esta relación de realización, la **clase concreta está obligada a implementar todos los métodos que están declarados en la interfaz** por lo tanto existe este principio en donde dice que las interfaces no deben estar muy cargadas.

Con la segregación de interfaces voy a terminas con más cantidad de interfaces, pero más específicas que le sirven a algunos clientes, pero las clases usan todos esos métodos.

Cada interfaz debe tener un nivel muy alto de COHESIÓN.

Ejemplo: Tenemos una biblioteca que facilita la integración de aplicaciones con proveedores de computación en la nube. La primera versión sólo soportaba Amazon y creímos que todos iban a usar esas mismas funciones y servicios de Amazon, pero cuando incluimos al proveedor DropBox nos dimos cuenta que había muchos métodos que no los usaba, pero que los tenía que implementar igual, lo cual no es beneficioso. La mejor solución es dividir la interfaz en partes. Las clases capaces de implementar la clase original con varios métodos, podrán ahora implementar varias interfaces específicas mientras que otras clases sólo pueden implementar aquellas interfaces que tienen los métodos que necesitan.



APIS de login...

D -> Dependency inversion principle – Principio de Inversión de Dependencias.

"Las clases de alto nivel no deben depender de clases de bajo nivel. Ambas deben depender de abstracciones. Las abstracciones no deben depender de detalles. Los detalles deben depender de abstracciones"

Se busca evitar que los módulos de alto nivel dependan de los módulos de menor nivel, sino desacoplar poniendo una interfaz en el medio, entonces los módulos de bajo nivel proveen los servicios que requiere el módulo de alto nivel, pudiendo los de bajo nivel intercambiarse según las peticiones del módulo de alto nivel.

Al diseñar software, podemos distinguir normalmente entre dos niveles de clases:

- Las clases de bajo nivel implementan operaciones básicas, como trabajar con un disco, transferir datos por una red, conectar con una base de datos, etc.
- Las clases de alto nivel contienen la lógica de negocio compleja que ordena a las clases de bajo nivel que hagan algo.

El principio de inversión de dependencia sugiere cambiar la dirección de esa dependencia. Para ello:

1. Se deben describir interfaces para operaciones de bajo nivel en las que se basarán las clases de alto nivel, preferiblemente en términos de negocio.
2. Ahora puedes hacer las clases de alto nivel dependientes de estas interfaces, en lugar de clases concretas de bajo nivel.
3. Una vez que las clases de bajo nivel implementan esas interfaces, se vuelven dependientes del nivel de la lógica de negocio invirtiendo la dirección de la dependencia original.

Este principio suele ir de la mano con el OCP: puedes extender las clases de bajo nivel para utilizarlas con distintas clases de lógica de negocio sin descomponer clases existentes.

El principal beneficio de la Inversión de Dependencias es el bajo acoplamiento. **Nos dice que no hay que depender de concreciones sino de abstracciones**, hay que plantear las relaciones entre clases a un nivel más alto.

Otra forma de representar el principio, es decir que las clases no deberían depender de clases que están en un nivel inferior de capas. (Patrón arquitectónico).

Para poder aplicar este principio en general, aparecen las interfaces o alguna clase abstracta (herencia, pero se debe usar lo menos posible), se debe trabajar con la relación de realización.

Para que un componente o clase dependa de abstracciones y no de concreciones, se usan INTERFACES. Las derivaciones de este principio son: *Inversión de Dependencias

*Principio de Hollywood ("no nos llame, nosotros lo llamamos"). Se da vuelta la relación.

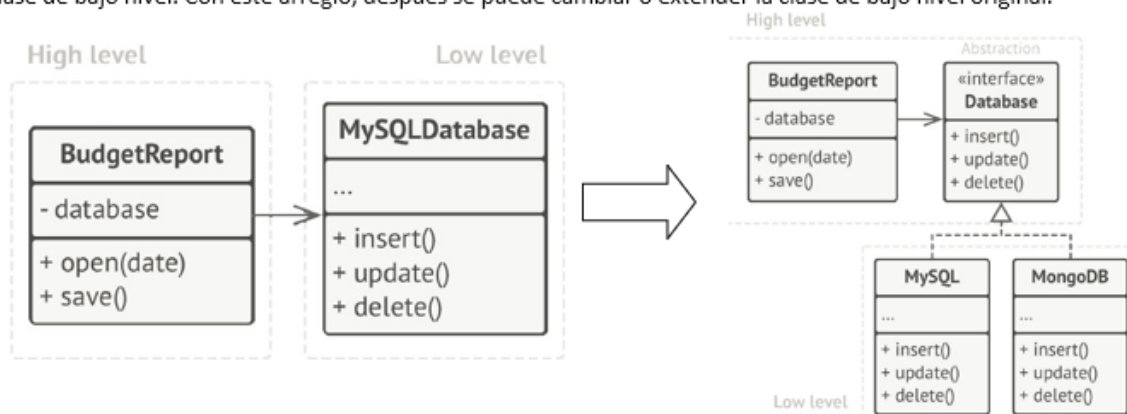
A veces, las personas diseñan primero clases de bajo nivel (operaciones primitivas como acceso a disco y base de datos), y sólo entonces comienzan a trabajar en las de alto nivel. Con este enfoque, las clases de lógica empresarial tienden a volverse dependientes de clases primitivas de bajo nivel. El principio de inversión de la dependencia sugiere cambiar la dirección de esta dependencia:

1. Para empezar, debe describir interfaces para operaciones de bajo nivel en las que se basan las clases de alto nivel, preferiblemente en términos comerciales.
2. Ahora puede hacer que las clases de alto nivel dependan de esas interfaces, en lugar de clases concretas de bajo nivel. Esta dependencia será mucho más suave que la original.
3. Una vez que las clases de bajo nivel implementan estas interfaces, se vuelven dependientes del nivel de lógica empresarial, invirtiendo la dirección de la dependencia original.

Interfaz: es el software más fácil de extender, es la construcción de código que tiene dos roles:

- Definir comportamiento que aplica a múltiples tipos
- Hacer foco principal en las clases que usan esos tipos.

Ejemplo: En este ejemplo, una clase de alto nivel, que se encarga de informes presupuestarios, utiliza una clase de base de datos de bajo nivel para leer y almacenar su información. Esto hace que cualquier cambio en la clase de bajo nivel, como ser el caso del lanzamiento de una nueva versión del servidor de la base de datos, puede afectar a la clase de alto nivel, que no tiene por qué conocer los detalles de almacenamiento de datos. Esto se puede solucionar creando una interfaz de alto nivel que describa operaciones de leer/escribir y haciendo que la clase de informes utilice esa interfaz en lugar de la clase de bajo nivel. Con este arreglo, después se puede cambiar o extender la clase de bajo nivel original.



Patrón que aplica: Template Method

Apunta a que implementes una relación de manera inversa a lo que naturalmente se está acostumbrado a que funcione, **esto nos permite manejar dependencias entre componentes de forma que no dependas de componentes que están en un nivel jerárquico más bajo, entonces para evitar esto (llamado indirección) agregamos una interfaz o algo en el medio que nos permita**

establecer ese vínculo de manera inversa que naturalmente se había pensado.

Ej: La relación en el TM es del hijo al padre, en este caso para poder implementar la plantilla el padre apunta al hijo (invierte la dependencia). Se introduce este recurso para que funcione al revés de lo que funciona.

Ej: Esquema de persistencia: Cuando tengo comunicación entre capas, la relación natural es del cliente (presentación) hacia la base de datos (de arriba hacia abajo), en este caso para que el esquema funcione con un buen nivel de cohesión permitimos que el esquema de persistencia le haga una petición a la lógica de negocio (lo natural sería que la LN pida y la BD suba la rta)

Conceptos:

❖ **Interdependencia Funcional:** la funcionalidad de un componente debe ser cohesiva y centrarse en una y sólo una función. El concepto de interdependencia funcional es resultado de la separación de problemas y de los conceptos de abstracción y ocultamiento de información. Se logra desarrollando módulos donde cada uno resuelva un subconjunto específico de requerimientos y tenga una interfaz sencilla. El SW con modularidad eficaz, es decir, con módulos independientes, es más fácil de desarrollar porque su función se subdivide y las interfaces se simplifican. Los módulos independientes son más fáciles de mantener (y probar) debido a que los efectos secundarios causados por el diseño o por la modificación del código son limitados, se reduce la propagación del error y es posible obtener módulos reutilizables. La independencia se evalúa con el uso de dos criterios:

Cohesión: un módulo cohesivo ejecuta una sola tarea, por lo que requiere interactuar poco con otros componentes en otras partes del programa.

Acoplamiento: es un indicador de la independencia relativa entre módulos. En el diseño de SW, debe buscarse el mínimo acoplamiento posible.

❖ **Reusabilidad:** grado en el que un programa puede volverse a utilizar en otras aplicaciones (se relaciona con el empaque y el alcance de las funciones que lleva a cabo el programa).

❖ **Modularidad:** construir SW que tenga modularidad eficaz, cada módulo debe centrarse exclusivamente en un aspecto bien delimitado del sistema: debe ser cohesivo en su función o restringido en el contenido que representa. Los módulos deben estar interconectados sencillamente: poco acoplamiento. El modularidad es la manifestación más común de la división de problemas. El SW se divide en componentes con nombres distintos y abordables por separado, llamados módulos, que se integran para satisfacer los requerimientos del problema.

Principios de diseño vinculados con el concepto de Cohesión: S – I; TDA- EWW

Delegación: Significa pedirle a alguien que haga algo por mí, confiar y darle el 100% de la responsabilidad para que lo realice y me devuelva el resultado. Mecanismo que me permite polimórficamente aprovechar un código que está escrito en otra clase y obtener el resultado sin tener que romper el encapsulamiento de esta clase (Caja negra), sin tener que escribir el mismo código, solo hago la invocación. El contexto de relación que permite la delegación es la asociación, agregación y composición. Delegar la responsabilidad a un objeto es diferente a pedirle colaboración. Lo usan patrones como State, Strategy, composite, etc.

Es lo que nos permite hacer reutilización con composición en lugar de herencia. Un objeto le delega una tarea a un delegado, resolviendo una petición entre 2 objetos o más. La idea es componer un objeto con instancias de otros, y permitirle que le pida a sus delegados que resuelvan cosas por él, logrando tener un "pseudo objeto" más complejo. La relación es sumamente flexible ya que los delegados pueden cambiar en tiempo de ejecución. Si se utilizan interfaces en lugar de implementaciones, el objeto contenedor no tiene por qué saber cómo se van a resolver sus peticiones, solo las delega y otro le entregará los resultados

Características	HERENCIA	REALIZACIÓN (Interfaces)	COMPOSICIÓN
Propósito	Extender una clase base con clases hijas que reutilicen su comportamiento y agreguen el propio.	Definir una interfaz (contrato) común, para que otras clases puedan cumplirlo y así unirse bajo un contrato común. Me define que se puede hacer, no cómo se hace	Contener clases dentro de otras clases, para que las contenedoras puedan usar los métodos expuestos por las contenidas.
Características	- Relación fuerte de generalización en donde un objeto especificado se basa en la especificación de otro (comparten código). Una clase padre comparte sus métodos y atributos con sus clases hijas. - Esta relación permite definir una implementación en términos de otra, lo que se conoce como reutilización de caja blanca, ya que la estructura interna de las clases es visible (viola encapsulamiento). - No requiere interfaz ya que puede acceder a los miembros del padre.	- Relación débil entre clasificadores en donde un objeto clasificador establece un contrato que garantiza que otro lo cumplirá. Se utiliza entre interfaces y colaboraciones. - Se utiliza para representar la relación entre una interfaz y la clase que la implementa. Describe cuándo se puede usar un objeto en lugar de otro. - Clases donde los métodos no tienen comportamiento, consiste solo en la firma (nombre, atributos, retorno, tipo) - No puede tener comportamiento, todos sus métodos son abstractos	- Relación todo-parte, a diferencia de la agregación, las partes no tienen vida independiente fuera del todo, cada parte pertenece a al menos un y sólo un todo. <i>En agregación una parte se puede compartir entre los todos.</i> - La funcionalidad se obtiene componiendo objetos, requiriendo que estos tengan interfaces bien definidas. Esto se conoce como reutilización de caja negra, porque los detalles internos de los objetos no son visibles. - Requiere que la clase tenga una interfaz bien definida para poder usarla.
Polimorfismo	Si	Si	No directamente por sí sola.
Tipo de Relación	es un	es un	tiene un
Se puede modificar en	Compilación (cuando programo). Trabaja con clases	Compilación	Ejecución (el sistema se ejecuta). Trabaja con Objetos
Permite <u>reuso</u> de código	Si	No	Si
Permite encapsulamiento	No	N/A	Si
Acoplamiento	Alto	Bajo	Medio/Bajo
Multiplicidad	1..1 En general solo se puede heredar una clase	<u>1..*</u>	<u>1..*</u> Se puede agregar muchas clases, obteniendo así múltiple reutilización
Ventajas	* Fácil de implementar, porque se tiene acceso al contenido de la clase padre. * Permite cambiar las <i>implementaciones</i> heredadas de las clases * Aumenta la reutilización de código	* Mucho más flexible y robusta que la herencia * Proporciona un mecanismo para establecer contratos	* Se define en tiempo de <i>ejecución (flexible)</i> * No rompe el encapsulamiento * Posee bajo acoplamiento * más consistente
Desventajas	* Rompe el encapsulamiento * Produce alto acoplamiento * No permite modificar las implementaciones en tiempo de ejecución, ya que se define en tiempo de <i>compilación (rígida)</i>	* No proporciona reutilización de código	* No es polimórfica por sí misma * Construir un sistema basado en composición es mucho más costoso y difícil de implementar * Más difícil de implementar porque trabaja únicamente con lo que la clase expone, a través de la delegación.

Diagramas utilizados en el WF de Diseño: TODOS MENOS EL DE CASOS DE USO

UNIDAD 3

- 1. Diseño Arquitectónico**
- 2. Patrones arquitectónicos**
- 3. Vistas arquitectónicas**
- 4. Documentación de la arquitectura**
- 5.**

Intro

Una principal característica del PUD es ser centrado en la arquitectura ya que es un aspecto muy importante. En el desarrollo tenemos un modelo de requerimientos y un modelo de análisis como entrada en el workflow de diseño, la arquitectura es lo primero que hay que resolver porque, al ser el planteo estratégico de cómo vamos a estructurar y organizar los elementos de software del producto para que satisfaga los requerimientos, hay que ir pensar en la arquitectura desde el primer paso (WF de requerimientos) destacando los aspectos significativos para la arquitectura.

No se puede entrar a hablar de diseño arquitectónico sin hacer antes una introducción a lo que es la arquitectura.

Concepto de ARQUITECTURA

- Es el planteo estratégico de la solución que necesitamos construir para satisfacer los requerimientos que se han establecido para el producto.
- Plantea cómo se van a estructurar y organizar los elementos de software del producto para satisfacer los requerimientos.
- Conjunto de decisiones significativas tomadas para cubrir los requerimientos no funcionales y garantizar que la funcionalidad que se pidió se desarrolle en los términos que se pidió.
- La arquitectura es lo difícil de cambiar ya que es la base de las capas en las que se hace el SW, esta primera capa define cómo van a funcionar las demás. Es un diseño global que luego se va a profundizar en los distintos aspectos que se diseñan (datos- proc. Manuales – procesos – formas de E/S – interacción H-M).

Diseño arquitectónico

El diseño arquitectónico es un **proceso** de asignación de modelos de requerimientos esenciales a una tecnología específica. Es el plan del diseño, donde se toman las decisiones de cómo vamos a resolver esos requerimientos.

La arquitectura del software de un programa es la estructura del sistema, lo que comprende a los

componentes del software, sus propiedades externas visibles y las relaciones entre ellos. Es una estructura o esqueleto general de la solución que queremos construir, que nos permite satisfacer los requerimientos que se nos presentan.

"La arquitectura de software de un programa o sistema informático es la/s estructura/s del sistema, que comprenden elementos de software, las propiedades visibles de esos elementos y las relaciones entre ellos."

"La arquitectura del software va más allá de los algoritmos y las estructuras de datos; diseñar y especificar la estructura general del sistema surge como un nuevo tipo de problema. Los problemas estructurales incluyen la organización general y estructura de control global; protocolos de comunicación, sincronización, y acceso a datos; asignación de funcionalidad a elementos de diseño; distribución física; composición de elementos de diseño; escala y rendimiento; y selección entre alternativas de diseño".

En otras palabras, es la relación de elementos o componentes que pueden estar en distintos modelos que explican lo que el sistema hace y cómo se tienen que organizar respecto a la estructura general de un sistema teniendo en cuenta su aplicación a una tecnología específica en la cual se va a desempeñar. Además, la arquitectura de software define un lineamiento de trabajo para que todos los miembros del equipo de trabajo sigan y respete dichos lineamientos.

Podemos ver que, entre las **tres** definiciones anteriores existen puntos en común, aunque en la tercera definición se incluyen ejemplos concretos que están implícitos en las primeras dos. Estas definiciones permiten extraer algunas características fundamentales de las arquitecturas de software. Estas, junto con algunos enfoques clave, se describen a continuación.

- **La arquitectura define estructura**

Gran parte del tiempo de un arquitecto se preocupa por cómo particionar una aplicación en un conjunto de componentes, módulos, objetos o cualquier unidad de partición de software interrelacionados que funcionen. Una arquitectura debe ser diseñada para cumplir los requerimientos específicos y restricciones de la aplicación a la cual está destinada.

Por ejemplo: un requerimiento para un sistema de gestión de información puede ser que la aplicación esté distribuida en múltiples sitios y, una restricción es que cierta funcionalidad e información deben residir en cada sitio. Otro ejemplo, una funcionalidad de la aplicación debe ser accesible desde un navegador web. Ambos ejemplos, imponen restricciones estructurales (sitio específico, alojar en servidor web).

Al particionar una aplicación, el arquitecto asigna responsabilidades a cada componente constituyente. Estas responsabilidades definen las tareas que se le pueden confiar a un componente que funcionará dentro de la aplicación. De esta manera, cada componente juega un papel específico en la aplicación, y el conjunto de componentes que comprende la arquitectura colabora para proporcionar la funcionalidad requerida.

El diseño conducido por la responsabilidad (Responsability-driven design) es una técnica de la orientación a objetos que se puede utilizar eficazmente para ayudar a definir los componentes clave en una arquitectura.

Un problema estructural clave para casi todas las aplicaciones es minimizar las dependencias entre componentes, creando una arquitectura débilmente acoplada a partir de un conjunto de componentes altamente cohesivos.

Existe una dependencia entre componentes cuando un cambio en uno fuerza potencialmente un cambio en otros. Eliminando dependencias innecesarias, los cambios se localizan y no se propagan a través de una arquitectura.

Las dependencias excesivas hacen que sea difícil realizar cambios en los sistemas, más caro probar los cambios, aumentan tiempos de construcción y dificultan el desarrollo simultáneo basado en equipos.

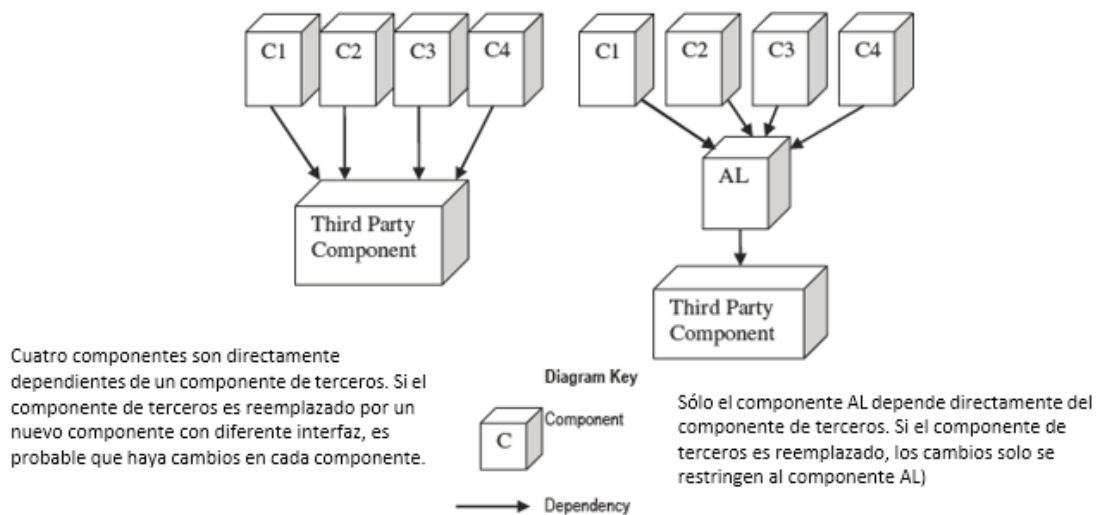


Fig. 1. Dos ejemplos de dependencia de componentes

• La arquitectura especifica componentes de comunicación

Cuando una aplicación es dividida en un conjunto de componentes, se vuelve necesario pensar cómo esos componentes comunican datos y controlan información. Los componentes en una aplicación pueden existir en el mismo espacio de direcciones y comunicarse a través de llamadas a métodos sencillas. Se pueden ejecutar en diferentes hilos o procesos y comunicarse a través de mecanismos de sincronización. O varios componentes pueden necesitar ser simultáneamente informados cuando un evento ocurre en el entorno de la aplicación. Existen muchas posibilidades.

Un cuerpo de trabajo conocido como patrones o estilos arquitectónicos han catalogado un número de

estructuras utilizadas exitosamente que facilitan ciertos tipos de componentes de comunicación. Estos patrones son planos arquitectónicos reutilizables que describen la estructura e interacción entre los componentes participantes.

Cada patrón tiene características bien definidas que lo hacen apropiado su uso para satisfacer tipos particulares de requerimientos. El poder de los patrones arquitectónicos se deriva de su utilidad y capacidad para transmitir información de diseño. Cuando una arquitectura se basa en patrones, también resulta fácil para los miembros del equipo comprender un diseño, ya que el patrón infiere estructura de componentes, comunicaciones y mecanismos abstractos que deben proporcionarse.

• **La arquitectura direcciona los requerimientos no funcionales**

Los requerimientos no funcionales no aparecen en los casos de uso. En vez de definir qué hace la aplicación, ellos se ocupan en cómo la aplicación provee la funcionalidad requerida.

Los arquitectos deben comprender los requerimientos funcionales y crear una plataforma que los respalde y satisfaga simultáneamente los requisitos no funcionales.

• **La arquitectura es una abstracción**

Una de las descripciones más útiles, pero a menudo inexistente, de una perspectiva arquitectónica es algo que se conoce coloquialmente como arquitectura de mercado. Esta es una página, una representación típicamente informal de la estructura del sistema e interacciones. Muestra los componentes principales, sus relaciones y tiene algunas etiquetas y cuadros de texto bien elegidos que representan filosofías de diseño plasmadas en la arquitectura. Una arquitectura de mercado es un excelente vehículo para facilitar la discusión de las partes interesadas durante el diseño, construcción, revisión, y por supuesto el proceso de venta. Es fácil de entender y explicar y sirve como punto de partida para un análisis más profundo.

Una arquitectura de mercado cuidadosamente elaborada es particularmente útil porque es una descripción abstracta de la aplicación. En realidad, cualquier descripción arquitectónica debe emplear la abstracción para ser comprensible por los miembros del equipo y partes interesadas del proyecto. Esto significa que los detalles innecesarios son suprimidos o ignorados para centrar la atención y el análisis en las cuestiones arquitectónicas más destacadas. Esto generalmente se hace describiendo los componentes en la arquitectura como cajas negras, especificando sólo sus propiedades visibles externas. Por supuesto, describir la estructura y el comportamiento del sistema como abstracciones de caja negra es normal para quienes utilizan técnicas de diseño orientadas a objetos.

Uno de los mecanismos más poderosos para describir una arquitectura es la descomposición jerárquica. Componentes que aparecen en un nivel de descripción se descomponen con más detalle en la documentación de diseño adjunta. Los diferentes niveles de descripción en la jerarquía tienden a ser de interés para diferentes desarrolladores en un proyecto. La arquitectura divide claramente las

responsabilidades de cada equipo, definiendo las dependencias entre ellos.

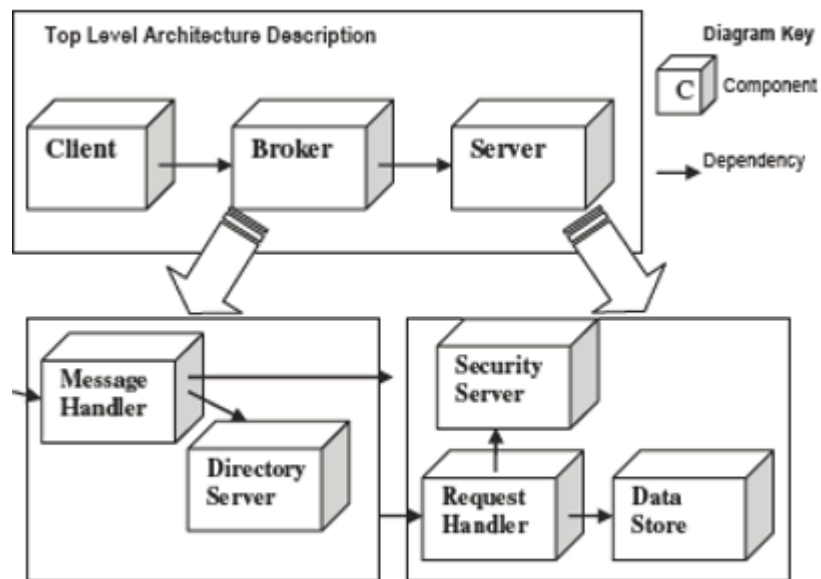
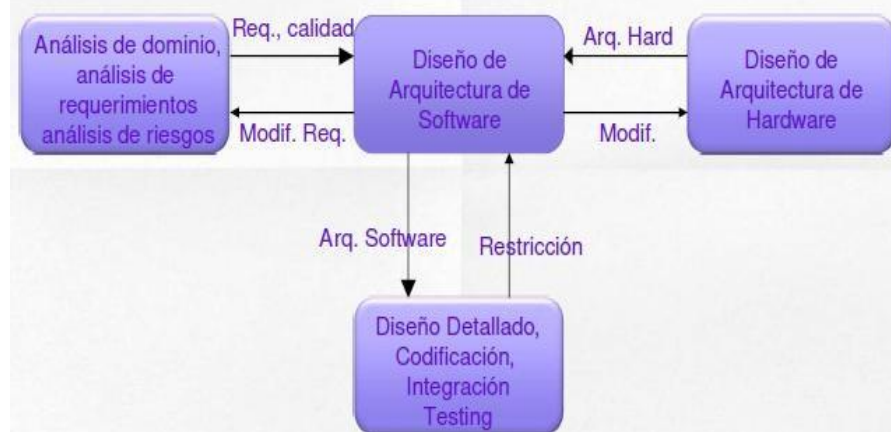


Fig. 2. Describing an architecture hierarchically

La arquitectura en contexto

Actualmente, no hay empresas u organizaciones que no tengan nada de sistema. Entonces, nos ubicamos con una situación de contexto existente, ya hay una arquitectura de software en esa organización y a nosotros nos representa una entrada de información además del modelo de requerimientos y de análisis.

Dicha arquitectura de hardware existente, ingresa como una **RESTRICCIÓN**. El producto nuevo debe convivir con los demás productos existentes en la empresa y, por eso la arquitectura de software debe tener en cuenta ese contexto. Al momento de plantear los requerimientos no funcionales, puede ser necesario modificar la arquitectura de software para cumplir dichos requerimientos, es por esto que se da un ida y vuelta. Todo implica negociaciones ya que, si no se puede modificar el HW entonces tendremos que modificar los requerimientos y amoldarnos a la realidad del HW existente. El término "negociación" está muy relacionado a la arquitectura y al rol del arquitecto. Una vez definido todo, tenemos lo que se conoce como **ARQUITECTURA DE SOFTWARE** y esta es la base, el lineamiento que nos va a permitir avanzar en las demás actividades del desarrollo (diseño detallado, codificación, integración, testing).



Rol del arquitecto

El arquitecto es el trabajador en los 3 primeros workflows: requerimientos, análisis y diseño. Es una figura reciente en la ingeniería de software. También suele llamarse líder técnico. Responsable de definir el documento de la arquitectura (Documento principal), lineamiento de diseño, lineamiento que el equipo va a implementar, prototipo arquitectónico (válida si ciertas decisiones arquitectónicas funcionan o no). Comunicarse y negociar con muchos involucrados

- **Enlace:**

Función: Los arquitectos actúan como intermediarios entre clientes y equipos técnicos, colaborando con analistas de negocios y requerimientos.

Coordinación: Facilitan la comunicación entre equipos de ingeniería y se relacionan con la gerencia, ventas y otros, traduciendo terminologías para las partes interesadas.

- **Ingeniería de software:**

Habilidades esenciales: Los arquitectos, como ingenieros de software, deben tener habilidades de diseño destacadas.

Promoción de buenas prácticas: Fomentan prácticas de ingeniería sólidas, documentando y comunicando claramente sus diseños, planes y decisiones.

Impacto consciente: Comprenden las implicaciones de sus decisiones y colaboran con equipos de prueba, documentación y lanzamiento.

- **Conocimiento tecnológico:**

Profundidad: Los arquitectos poseen un conocimiento profundo de los dominios tecnológicos relevantes para sus proyectos.

Influencia: Participan en la evaluación y selección de componentes y tecnologías, siguiendo los avances y aplicando nuevos estándares y productos útiles.

- **Gestión de riesgos:**

Previsión: Los arquitectos son cautelosos, identificando y evaluando riesgos asociados con decisiones de diseño y tecnológicas.

Documentación y gestión: Registran y gestionan riesgos junto con patrocinadores y gestión del proyecto, desarrollando estrategias de mitigación y comunicándolas a los equipos de ingeniería.

- Revisar y negociar los requerimientos
- Documentar la arquitectura
- Comunicar la arquitectura, asegurando que los interesados la comprendan
- Direccionar los RNF a la arquitectura
- Configurar la arquitectura de HW
- Trabajar con el administrador del proyecto, ayudando en la planificación, estimación, distribución de

las tareas y la calendarización del proyecto

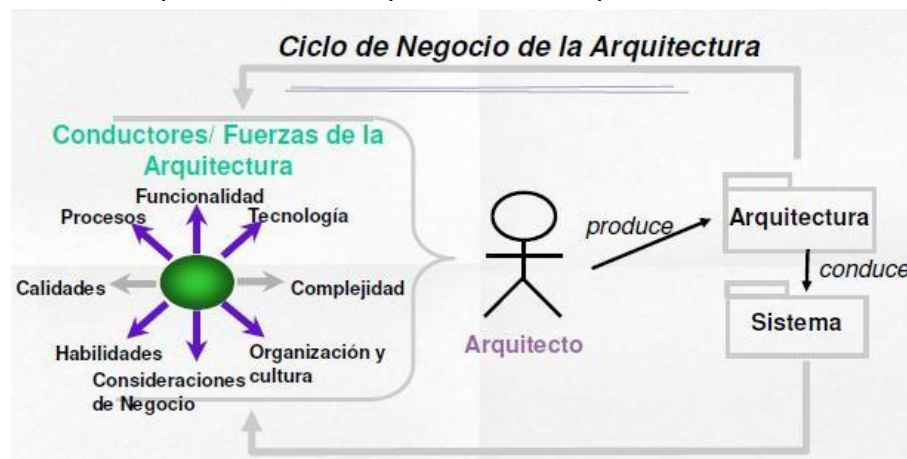
- Se asegura que la arquitectura sea respetada

Es una representación que permite:

1. Analizar la efectividad del diseño para cumplir los requerimientos,
2. Considerar alternativas arquitectónicas en una etapa en la que esto es posible, y
3. Reducir los riesgos asociados a la construcción del software.

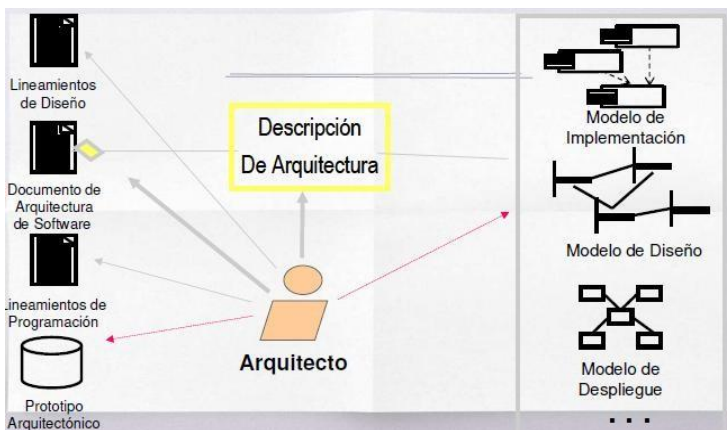
REVISAR Y NEGOCIAR LOS REQUERIMIENTOS

El arquitecto es el responsable de lograr un equilibrio entre todo lo que involucra una arquitectura. No solo se involucran cuestiones técnicas, sino que también forma parte de la arquitectura la interacción con las personas que van a usar ese producto de software. También hay que tomar decisiones y para ello, es necesario conocer la organización, la cultura de dicha organización, las consideraciones del negocio, qué habilidades tienen las personas que van a utilizar el producto. Cuando estas cuestiones no son tenidas en cuenta y sólo se limita la arquitectura a cuestiones de tecnología, HW, SW y comunicaciones, es cuando los productos no cumplen las expectativas de los clientes y fracasan. Una vez que se logra delinear esa arquitectura, esa arquitectura es la que debe CONDUCIR hacia el sistema a entregar al cliente.



DOCUMENTAR LA ARQUITECTURA

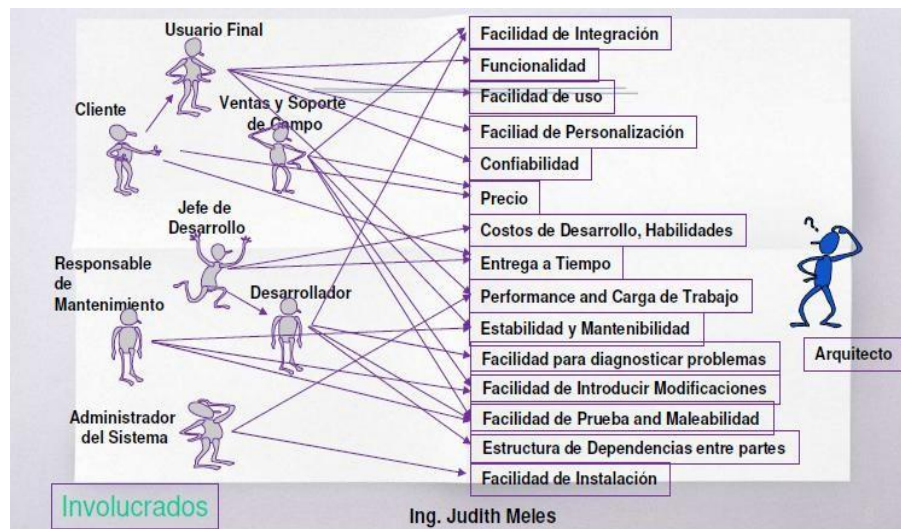
Lo que produce el arquitecto se conoce genéricamente como Documento de Arquitectura. La arquitectura se describe a través de las vistas arquitectónicas. Para construir dichas vistas, hacemos uso de los diagramas de UML. Complementariamente, el documento de la arquitectura de SW debe tener definiciones generales de cómo se va a construir el producto y tiene que tomar decisiones de diseño que se espera que el resto de los trabajadores respete. Cuando se completó la descripción de la arquitectura, se sigue con el resto de los modelos. (diseño detallado, implementación y despliegue).



COMUNICA LA ARQUITECTURA

Cuando hablamos del arquitecto como un negociador, lo debe ser realmente porque hay muchas personas que tienen necesidades y expectativas sobre el producto, pero cada uno tiene las suyas y son diferentes. La imagen a continuación intenta mostrar esto, cada persona le da un peso distinto a sus necesidades y expectativas propias que son diferentes a las de otra persona.

Tomando el punto de vista de usuario final, él va a querer que el producto cumpla la funcionalidad, sea fácil de usar, que se pueda personalizar, que pueda confiar en los resultados del sistema y, visto desde el lado del desarrollador, éste quiere que sea fácil de probar, que entienda la arquitectura, que sea fácil de construir y la pueda implementar. El arquitecto tiene la responsabilidad de negociar con todas las personas involucradas que tienen expectativas diferentes sobre el producto y armar un producto que deje contentos a todos.



Ventajas de diseñar y documentar la arquitectura

- **Comunicación con los involucrados:** la arquitectura es una presentación que sirve para usarse como enfoque para la discusión de un amplio número de participantes.
- **Análisis del sistema:** sirve para aclarar la arquitectura del sistema, para saber si se cumplen los requerimientos críticos.
- **Reutilización a gran escala:** es posible desarrollar arquitecturas de línea de productos donde la misma arquitectura se reutilice mediante una amplia gama de sistemas relacionados.

Como la arquitectura afecta a muchas personas, es importante que quede evidencia de las decisiones que tomamos. Para eso se documenta la arquitectura. La arquitectura se documenta, registra, modela o plasma en algún lugar para luego poder comunicarla a los demás. La documentación de la arquitectura es necesaria ya que, cuando el sistema evolucione, se entienda por qué se resolvió de esa manera en ese momento y porque es probable que, en el proyecto de evolución, dicha arquitectura se transforme en una arquitectura de línea de producto (por ej: Word y Excel, ahora es toda una línea de producto llamada paquete Office).

Dificultades asociadas a los requerimientos no funcionales

- No hay reglas ni lineamientos para determinar cuándo se obtuvo una solución óptima.
- No hay un modelo estándar que vaya a funcionar en todos los casos.
- Tiene buenas y malas soluciones, no soluciones correctas e incorrectas.
- Problemas relacionados con requerimientos no funcionales pueden ser síntomas de problemas mayores.
- Hay dos objetivos que deben poseer: deben ser objetivos y testeables.

Áreas de requerimientos no funcionales

- **Restricciones técnicas:** restringen opciones de diseño especificando cierta tecnología a usar. No son negociables. Ejemplo: "solo tenemos diseñadores Java", "la base de datos debe correr con Windows XP".
- **Restricciones de negocio:** también restringen opciones de diseño, pero por razones del negocio. La mayoría de las veces son no negociables. Ej: "las licencias son muy costosas, nos vamos a una versión de código abierto", "hay que desarrollar interfaz con el producto X que ya existe en la empresa".
- **Atributos de calidad del producto:** definen los requerimientos de una aplicación en términos de escalabilidad, disponibilidad, facilidad de cambio, portabilidad, usabilidad, rendimiento, etc. Éstos abordan cuestiones de interés para los usuarios de la aplicación como así también a otras partes interesadas como el propio equipo de proyecto o el patrocinador del proyecto.

La arquitectura debe explícitamente resolver las tres áreas.

Norma ISO/IEC 25000

Para la ESPECIFICACIÓN, CALIFICACIÓN Y EVALUACIÓN de los requerimientos no funcionales utilizamos la norma ISO 25000.

Esta norma define cómo deben especificarse y evaluarse los requerimientos no funcionales.

• Objetivos:

- o Especificación de requisitos de calidad de software
- o Evaluación y medición de la calidad del software

• Modelos:

- o Calidad en uso (ISO 25010:2011)

- o Calidad de producto (ISO 25010:2011) • Nos centramos en este modelo
- o Calidad de datos (ISO 25012:2008)

Los requerimientos no funcionales deben especificarse de manera MEDIBLE y VERIFICABLE.

CALIDAD DE PRODUCTO

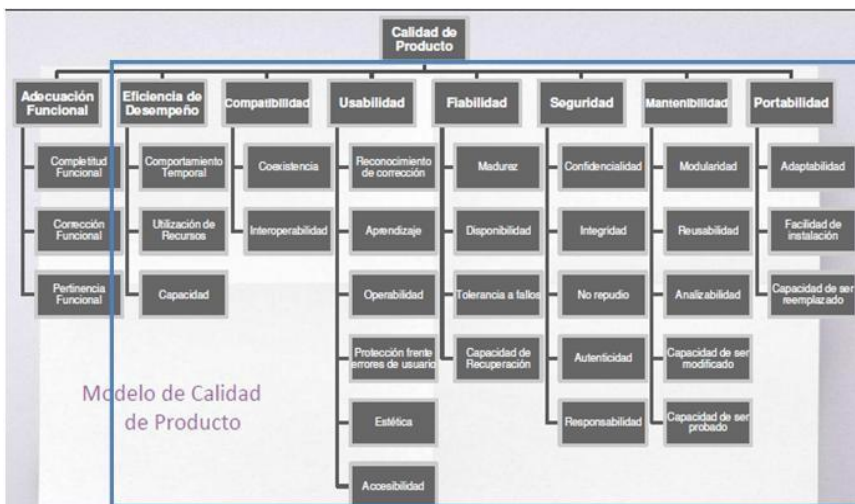
Es el modelo de la ISO a utilizar para guiar la identificación, evaluación y medición de los RNF.

¿De qué se trata?

Es relativa a propiedades estáticas del software y propiedades dinámicas de sistemas de computadoras.

- o Propiedades estáticas: se pueden medir sin ejecutar el sistema. Ej: el sistema tiene 8gb de RAM.
- o Propiedades dinámicas: sólo se pueden verificar cuando el sistema está en funcionamiento, cuando se está ejecutando. Ej: impresión de una factura en cierto tiempo.

MODELO DE CALIDAD DE PRODUCTO



Nos centramos en las características del recuadro:

- **Eficiencia de desempeño**: Relativo a la cantidad de recursos utilizados bajo condiciones establecidas.
 - o **Comportamiento temporal**: el grado con el cual la respuesta, los tiempos de procesamiento, tasas de rendimiento de un producto o sistema cuando ejecuta sus funciones cumple con los requerimientos.
 - o **Utilización de recursos**: el grado con el cual las cantidades y los tipos de recursos utilizados por un producto o sistema cuando ejecuta sus funciones cumple con los requerimientos.
 - o **Capacidad**: el grado con el cual los límites máximos de los parámetros de un sistema o producto cumplen con los requerimientos. Los parámetros pueden ser usuarios concurrentes, ancho de banda, rendimiento de las transacciones y tamaño de la base de datos.
Ejemplo: el tiempo de respuesta de procesamiento de información que genera el radar de un auto que se maneja solo debe ser de 15 milisegundos.
- **Compatibilidad**: el grado con el que un producto, sistema o componente realiza las funciones

requeridas compartiendo el mismo entorno de SW o HW.

- o **Coexistencia:** el grado con el cual un producto puede ejecutar sus funciones requeridas mientras comparte el entorno y los recursos con otros productos sin impactar en el deterioro de los otros productos.

- o **Interoperabilidad:** el grado con el cual dos o más productos, sistemas o componentes pueden intercambiar información y utilizarla.

Ej: un celular, reloj o cualquier otro dispositivo o sistema debe tener interoperabilidad con un gps.

- **Usabilidad:** el grado con el cual un producto o sistema puede ser utilizado por usuarios específicos para cumplir sus objetivos con efectividad, eficiencia y satisfacción en un contexto de uso específico. Puede especificarse y medirse en términos de las características de la calidad en uso.

- o **Reconocimiento de corrección:** el grado con el cual los usuarios pueden reconocer que un producto o sistema es apropiado para sus necesidades. La habilidad de reconocimiento depende de la impresión inicial y de la documentación asociada al producto o sistema.

- o **Aprendizaje:** el grado con el cual un producto o sistema puede ser utilizado por usuarios específicos para cumplir objetivos de aprendizaje establecidos para ejecutar el producto o sistema con efectividad, eficiencia, libre de riesgos y satisfacción en un contexto determinado de uso.

- o **Operabilidad:** el grado con el cual un producto o sistema posee atributos que lo hacen fácil de operar y controlar.

- o **Protección frente a errores de usuario:** el grado con el cual un producto o sistema protege a los usuarios de cometer errores.

- o **Estética:** el grado con el cual una interfaz de usuario permite una interacción placentera y satisfactoria a los usuarios.

- o **Accesibilidad:** el grado con el cual un producto o sistema puede ser utilizado por personas en el sentido amplio de características y capacidades para lograr un objetivo establecido en determinado contexto de uso. Ejemplos: asistentes inteligentes, se le habla al sistema para pedirle que haga algo (siri, google assistant, etc). Touch de pantalla adaptada al uso de guantes de astronautas, botones físicos para control de un sistema (en un contexto muy específico).

- **Fiabilidad:** el grado con el que el sistema, producto o componentes ejecuta funciones específicas bajo condiciones específicas por un período de tiempo determinado.

- o **Madurez:** el grado con el cual un sistema, producto o componente, cumple las necesidades de confiabilidad durante la operación normal.

- o **Disponibilidad:** el grado con el cual un sistema, producto o componente está operativo y accesible cuando es requerido su uso. Externamente, la disponibilidad puede definirse como el porcentaje de tiempo durante el cual el producto, sistema o componente está arriba. También se

define como una combinación de madurez (que gobierna la frecuencia de las fallas), la tolerancia a fallas y la capacidad de recuperación (que gobiernan el tiempo que el sistema está bajo luego de cada falla).

o **Tolerancia a fallos:** grado con el que un sistema, producto o componente opera según lo previsto a pesar de las fallas en el software o hardware.

o **Capacidad de recuperación:** el grado con el cual en un evento de interrupción o falla, un producto o sistema puede restaurar la información afectada y restablecer el estado deseado del sistema.

Ejemplo: el canal HBO GO se cayó cuando se iba a estrenar GOT en vivo, es un ejemplo de que no se cumple el requerimiento no funcional de fiabilidad. En cambio, la plataforma Netflix si lo cumple ya que casi nunca se cae y opera en 203 países teniendo más de 110 millones de suscriptores.

Tácticas de solución:

Administrar la disponibilidad con calidad requiere de:

1. Detección de fallas mediante monitoreo.
2. Preparación para la recuperación y reparación: respaldo.
3. Reintroducción a la recuperación una vez corregido: sincronización.
4. Prevención: monitoreo de procesos.

• **Seguridad:** el grado en que un producto o sistema protege información y datos de modo que personas u otros productos o sistemas tengan el nivel de acceso a datos apropiado a su tipo y nivel de autorización.

o **Confidencialidad:** el grado con el cual un producto o sistema asegura que los datos son accesibles sólo a aquellos autorizados.

o **Integridad:** el grado con el cual un producto, sistema o componente previene el acceso no autorizado modificaciones a un programa de computadoras o datos.

o **No repudio:** el grado con el que acciones o eventos pueden ser probados que existieron de manera que esas acciones o eventos no puedan ser luego repudiados.

o **Autenticidad:** el grado con el cual pueda establecerse la identidad de una persona o un recurso pueda ser demostrada.

o **Responsabilidad:** el grado con el cual una acción de una entidad puede trazarse unívocamente hacia esa entidad.

Ejemplo: movypark expuso los datos de las tarjetas de crédito de los usuarios. Este es un caso donde no se cumple el RNF de seguridad.

• **Mantenibilidad:** el grado de efectividad y eficiencia con el que un producto o sistema puede

modificarse para quienes intentan mantenerlo. La capacidad de facilitar la actividad de mantenimiento.

o **Modularidad:** el grado con el cual un sistema o programa de computadoras está compuesto por componentes discretos de manera que el cambio en uno de sus componentes tiene un impacto mínimo en el resto.

o **Reusabilidad:** el grado con el cual un activo puede ser utilizado en más de un sistema o en la construcción de otros activos.

o **Analizabilidad:** el grado de eficiencia o eficacia con el que es posible evaluar el impacto de un producto o sistema que requiere ser modificado una o varias de sus partes o diagnosticar un producto acerca de sus fallas y deficiencias o identificar las partes que necesitan ser modificadas.

o **Capacidad de ser modificado:** el grado en el que un producto o sistema puede ser eficiente y eficazmente modificado sin introducir defectos o degradar la calidad actual del producto.

o **Capacidad de ser probado:** grado de eficiencia y eficacia con que los criterios de prueba pueden establecerse para un sistema, producto o componente y las pruebas puedan ejecutarse para establecer si los criterios se cumplen.

Ejemplo: infinitas funcionalidades de instagram con sus actualizaciones, android ya que soporta muchos cambios.

• **Portabilidad:** el grado de eficacia y eficiencia con el que un sistema, producto o componente puede ser transferido de un hardware, software u otro entorno operativo de uso a otro.

o **Adaptabilidad:** el grado con el cual un producto o sistema puede ser efectivo y eficientemente adaptado para otro hardware, software o entornos operacionales de uso diferentes o evolucionados. La escalabilidad de capacidades internas (campos, tablas, volumen de transacciones, etc).

o **Facilidad de instalación:** el grado de efectividad y eficiencia con el que un producto o sistema puede ser instalado o desinstalado exitosamente en determinado entorno.

o **Capacidad de ser reemplazado:** el grado con el cual un producto puede reemplazar a otro producto de software específico con el mismo propósito y en el mismo entorno.

Ejemplo: consolas de juego GAMEBOY y GAMEBOY COLOR. Son compatibles con versiones anteriores.

¿Se puede ejecutar fácilmente una aplicación en una plataforma de software/hardware diferente a la que ha sido desarrollada? La portabilidad depende de las opciones de tecnología de software utilizadas para implementar la aplicación, y las características de las plataformas que necesita para ejecutarse. Las bases de código fácilmente portátiles tendrán sus dependencias de plataforma aisladas y encapsuladas en un pequeño conjunto de componentes que puede ser reemplazado sin afectar el resto de la aplicación.

No siempre se pueden cumplir todos los requerimientos no funcionales al mismo tiempo porque son muy

costosos de implementar y mantener. Hay que hacerle entender al cliente que hay que priorizar los requerimientos.

Al tener más de un RNF crítico, comienzan los conflictos:

1. Si quiero tener buen desempeño, tengo componentes de granularidad alta que afectan la mantenibilidad
2. Si introduzco datos redundantes, mejora la disponibilidad, pero hago más compleja la seguridad
3. La localización de aspectos de seguridad relacionados significa más comunicación, es decir, peor desempeño.

MODELADO DE LA ARQUITECTURA

- El modelo arquitectónico mapea los requerimientos funcionales del análisis a una arquitectura tecnológica.
- Debe tratar con los requerimientos no funcionales.
- No existe la solución "perfecta", el modelado arquitectónico debe escoger la solución óptima para el conjunto de circunstancias existentes.
- Debe considerar información sobre:
 - o Volúmenes de datos
 - o Funcionalidad más demandada del negocio
 - o Distribución geográfica
 - o Distribución de procesamiento de datos
 - o Dónde se guardarán los datos
 - o Cuáles procesos se ejecutarán en qué procesadores y que tanta comunicación se requerirá entre ellos.

Son las decisiones que debemos ir tomando y que es importante ir documentando porque son cosas que antes no habíamos tenido en cuenta. Antes no hablábamos de volúmenes de datos habíamos hablado de clases que crean objetos, pero no habíamos visto si iba a ser muy grande o cómo iba a ser el crecimiento en la BD. Debemos preguntarnos: ¿la distribución de los datos será centralizada o distribuida? ¿El software por quién será usado? Donde van a estar físicamente/geográficamente las personas? Todo esto antes no se había pensado. El ingeniero en sistemas debe hacerse responsable de lo nuevo y de lo viejo del sistema. Si la arquitectura no está documentada, después cuando se quiere modificar/evolucionar el sistema o hacerlo crecer se complica porque no sabemos cómo funciona el SW internamente.

SALIDAS DEL MODELO ARQUITECTÓNICO

Son las decisiones que fueron documentadas:

- Distribución geográfica de los requerimientos de computación.
- Componentes de hardware para las máquinas clientes y para los servidores.
- Configuración y cantidad de niveles de hardware.
- Mecanismos y lenguajes de comunicación de la red.
- Sistema operativo.
- Paradigma de desarrollo
- Lenguaje de presentación (front-end)
- Lenguaje de fondo (back-end)
- Sistema de administración de base de datos
- Ubicación/es de los procesos
- Ubicación/es de los datos físicos
- Estrategias de sincronización de los datos distribuidos físicamente

Elección del modelo arquitectónico y los requerimientos no funcionales

Aunque cada sistema es único, los sistemas en el mismo dominio de aplicación tienen normalmente arquitecturas similares que reflejan los conceptos fundamentales del dominio. Cuando se diseña una arquitectura de sistema, debe decidirse que tienen en común el sistema y las clases de aplicación más amplias, con la finalidad de determinar cuánto conocimiento se puede reutilizar de dichas arquitecturas de aplicación.

La arquitectura de un sistema de software puede basarse en un patrón o en un estilo arquitectónico particular. Un patrón arquitectónico es una descripción de una organización del sistema. Éstos captan la esencia de una arquitectura que se usó en diferentes sistemas de software.

Debido a la estrecha relación entre los requerimientos no funcionales y la arquitectura de software, el estilo y la estructura arquitectónica particulares que se elijan para un sistema dependerán de los requerimientos no funcionales del sistema:

- **Desempeño (performance: tiempo de respuesta en relación a los recursos de HW que se consumen):** si es un requerimiento crítico, esto sugiere que la arquitectura se debe diseñar para localizar operaciones críticas dentro de un número reducido de subsistemas con poca comunicación (hasta donde sea posible) entre estos subsistemas, es decir, **componentes de granularidad gruesa para reducir componentes de comunicación.**

La performance es una de las cualidades de una aplicación que puede cuantificarse y validarse fácilmente. Cualquiera sea la razón, cuando la performance importa, realmente importa. Un requerimiento de calidad de performance (desempeño) define una métrica que establece la cantidad de trabajo que debe realizar una aplicación en un tiempo determinado y/o plazos que deben cumplirse para

su correcto funcionamiento. Los sistemas militares o robóticos tienen duras restricciones de tiempo real ya que si una salida es producida un milisegundo más tarde pueden ocurrir cosas terribles.

La performance, usualmente se manifiesta en las siguientes medidas:

- **Rendimiento:** es una medida de la cantidad de trabajo que una aplicación debe realizar en una unidad de tiempo. El trabajo es típicamente medido en transacciones por segundo (tps) o mensajes procesados por segundos (mps). Es importante comprender con precisión qué se entiende por requisito de rendimiento. Hay una distinción crucial entre rendimiento medio en cierto periodo de tiempo (por ejemplo, un día en una empresa) y rendimiento máximo. Ejemplo: una aplicación para realizar apuestas en eventos. La mayor parte del tiempo, una aplicación de este tipo hace poco trabajo y, por lo tanto, tiene un promedio bajo de rendimiento que es fácilmente alcanzable. Sin embargo, los cinco minutos previos a que comience el evento se deben procesar cientos de apuestas cada segundo. Si la aplicación no es capaz de procesar estas apuestas a medida que se realizan, la empresa pierde dinero y los usuarios se sienten muy descontentos. Por tanto, para este escenario, la aplicación debe diseñarse para cumplir con el rendimiento máximo, no el promedio.

- **Tiempo de respuesta:** es una medida del tiempo de latencia o espera que una aplicación demora en procesar una transacción. El tiempo de respuesta es frecuentemente asociado con el tiempo al que una aplicación le toma responder a alguna entrada. Una rápida respuesta permite al usuario trabajar más efectivamente y por ende, es bueno para las empresas. Es importante distinguir entre tiempos de respuesta garantizados y tiempos de respuesta promedio. Algunas aplicaciones, necesitarán que todas sus peticiones sean respondidas en un límite de tiempo específico, esto es tiempo de respuesta garantizado. Otras, solo especificarán un tiempo de respuesta promedio, lo cual permite lo que permite mayores latencias cuando la aplicación está muy ocupada. Es muy importante especificar el tiempo de respuesta máximo permitido como: " El 95% de las solicitudes deben ser procesadas en menos de 4 segundos y ninguna debe tomar más de 15 segundos". Ejemplo: una aplicación de soporte de gestión a una gran tienda. Cuando un ítem es escaneado, una respuesta rápida del sistema con el precio del artículo significa que un cliente puede ser atendido rápidamente. Esto hace al cliente y a la tienda felices, y esto es bueno para todos los involucrados.

- **Plazos:** cualquier aplicación que tenga una ventana de tiempo límite para completar una tarea tendrá un requerimiento de plazo de desempeño. Estos plazos se asocian con los sistemas batch (procesamiento en lote). Ej: el sistema de pronóstico del tiempo que toma 36 horas para producir el pronóstico para el día siguiente. Un sistema de pagos debe acreditar dichos pagos en las cuentas en un día dado. Si no se cumple ese plazo, las personas no reciben el pago cuando lo esperaban y genera reclamos.

Si la performance es crítica (re contra necesario), estricta en que sea una performance determinada, se tiene que reducir lo que hace lento al software, como lo son las interfaces, los accesos entre capas o

subsistemas que se comunican. Esas comunicaciones son las que realentizan al SW, en particular, el acceso a la BD.

Tener componentes de granularidad gruesa quiere decir: tener más cantidad de componentes en un subsistema para que haya menos subsistemas y por ende, menor comunicación entre ellos (bajo acoplamiento). En definitiva: MENOS subsistemas que tienen MÁS componentes y que hacen MUCHAS cosas.

- **Seguridad:** si es un requerimiento crítico, esto sugiere utilizar una estructura en capas para la arquitectura con los recursos más críticos protegidos con capas internas y un alto nivel de validación aplicado a esas capas.

La seguridad a nivel arquitectónico se refiere a entender los requerimientos de seguridad para una aplicación y diseñar mecanismos para soportarlos. Los requerimientos más comunes relacionados a la seguridad son:

- **Autenticación:** las aplicaciones pueden verificar la identidad de sus usuarios y las de otras aplicaciones con las cuales se comunica.
- **Autorización:** usuarios autenticados y las aplicaciones han definido derechos de acceso a los recursos del sistema. Algunos usuarios pueden tener acceso de solo lectura a los datos de la aplicación mientras que otros pueden tener acceso de lectura y escritura.
- **Encriptación:** los mensajes enviados a /desde la aplicación son encriptados.
- **Integridad:** esto asegura que el contenido de un mensaje no es alterado durante el tránsito.
- **No repudio:** El remitente de un mensaje tiene comprobante de entrega y el receptor está seguro de la identidad del remitente. Esto significa que ninguno de los dos puede posteriormente refutar su participación en el intercambio de mensajes.

Existen tecnologías bien conocidas y ampliamente utilizadas que respaldan estos elementos de seguridad de la aplicación. La capa de conexión segura (SSL- Secure Socket Layer) y las Infraestructuras de Clave Pública (PKI) se utilizan comúnmente en aplicaciones de Internet para proporcionar autenticación, cifrado y no repudio. Sistemas operativos y bases de datos proporcionan seguridad basada en inicio de sesión para autenticación y autorización. Si una aplicación consta de varios componentes que todos deseen gestionar la seguridad, se deben diseñar soluciones adecuadas que normalmente localizan la gestión de seguridad en un solo componente que aprovecha la tecnología más adecuada para satisfacer los requisitos.

Si la seguridad es crítica, se debe identificar el subsistema a proteger. Lo más crítico a proteger son los datos, entonces hay que ubicarlos lo más lejos que se pueda del usuario. Sobre esa capa de datos, se colocan capas de validaciones y algoritmos de protección, logueos, autenticación que hacen difícil llegar hasta allí. Apunta al acceso permitido por usuarios autorizados.

- **Protección:** si es un requerimiento crítico, sugiere que la arquitectura debe diseñarse para incluir **operaciones relacionadas con la protección se localicen en un solo subsistema o en un número reducido de subsistemas**. Esto reduce costos y problemas de validación.

La protección tiene que ver con la funcionalidad, qué cosas se pueden ver y hacer con el sistema y qué cosas no se puede hacer.

- **Disponibilidad:** si es un requerimiento crítico, debe diseñarse una arquitectura que incluya **componentes redundantes de tal forma que puedan reemplazarse y actualizarse** sin detener el sistema.

La disponibilidad está relacionada con la confiabilidad de una aplicación. Si una aplicación no está disponible para usar cuando sea necesario, entonces es poco probable que cumpla con sus requerimientos funcionales. La disponibilidad es relativamente fácil de especificar y medir. Las fallas en las aplicaciones hacen que no estén disponibles. Las fallas impactan en la confiabilidad de una aplicación, que generalmente se mide por el tiempo medio entre fallos. El tiempo que dura cualquier período de indisponibilidad es determinado por la cantidad de tiempo que se tarda en detectar la falla y reiniciar el sistema. La replicación o redundancia de componentes es una estrategia probada para lograr una alta disponibilidad. Cuando falla un componente, la aplicación puede continuar ejecutándose utilizando réplicas que todavía están funcionando. Esto puede conducir a una degradación del rendimiento mientras el componente defectuoso está inactivo, pero la disponibilidad no es comprometida. La recuperabilidad está estrechamente relacionada con la disponibilidad. Una aplicación es recuperable si tiene la capacidad de restablecer los niveles de desempeño requeridos y recuperar datos afectados después de una falla del sistema. Un sistema de base de datos es el ejemplo clásico de un sistema recuperable. Cuando un servidor de base de datos falla, no está disponible hasta que se haya recuperado. Esto significa reiniciar el servidor y resolver las transacciones que estaban en curso cuando ocurrió el fallo. Temas interesantes para aplicaciones recuperables son cómo se detectan fallas y comienza la recuperación (preferiblemente de forma automática), y cuánto tiempo lleva recuperarse antes de que se restablezca el servicio completo. Durante el proceso de recuperación, la aplicación no está disponible y, por lo tanto, el tiempo medio para recuperarse es una métrica importante a considerar.

La disponibilidad significa que, yo quiero usar un sistema y el sistema funciona. Esta característica se cumple con redundancia, teniendo lo mismo 2 o 3 veces en espejo. Al cumplir con esta característica, se sacrifica otra ya que tener un sistema redundante afecta a la performance del mismo. No se puede tener un valor óptimo en ambas. La disponibilidad y la performance son las características más pedidas en un producto, pero, entran en conflicto. Para garantizar una, se sacrifica la otra.

- **Mantenibilidad:** si es un requerimiento crítico, la arquitectura debe diseñarse utilizando componentes

autocontenidos de granularidad fina que puedan cambiarse con facilidad. Los productores de datos separados de los consumidores y las estructuras de datos compartidas, deben evitarse.

Esta es una medida de lo fácil que es mantener una aplicación una vez implementada. El soporte generalmente implica diagnosticar y corregir problemas que ocurren durante el uso de la aplicación. Sistemas mantenibles tienden a proporcionar facilidades explícitas para el diagnóstico, como aplicaciones de registro de errores que graban la causa de la falla. También están contruidos en un módulo para que las correcciones de código se puedan implementar sin inconvenientes graves de uso de la aplicación.

La mantenibilidad no es valorada por el usuario hasta que llega el momento de realizar cambios. Se la suele posponer ya que se prioriza lo urgente. Para que un software sea mantenible, se necesitan componentes con alta cohesión y bajo acoplamiento.

Se necesitan componentes que hagan pocas cosas, en lo posible una única cosa para que sean reemplazables (cohesión alta) y relacionarse con la menor cantidad de componentes posibles (acoplamiento bajo).

La expresión "si es un requerimiento crítico" significa que ese requerimiento de calidad se debe priorizar si es realmente importante para el cumplimiento de las expectativas del producto. Esta característica debe ser llevada a un valor óptimo, entonces, se analiza cómo lograrlo.

Los atributos de calidad son parte de los requerimientos no funcionales de una aplicación, los cuales capturan cómo los requerimientos funcionales de una aplicación son alcanzados. Para que sean significativos, los atributos de calidad deben ser específicos sobre cómo una aplicación debería satisfacer una necesidad determinada. Un problema común que suele encontrarse en documentos de arquitectura es una declaración general como "La aplicación debe ser escalable". Esto es demasiado impreciso y realmente no sirve mucho para nadie. Es vital definir atributos de calidad concretos como "Debe ser posible escalar el desarrollo de unos 100 iniciales usuarios dispersos geográficamente a 10000 sin un incremento en el esfuerzo o costo de instalación y configuración". Esto es preciso y significativo. Como arquitecto, esto apunta un camino hacia un conjunto de soluciones y tecnologías concretas que facilitan el despliegue y la instalación con cero esfuerzos.

Modificabilidad:

Los cambios en un sistema de software forman parte de la vida del mismo, por lo tanto, es una buena práctica tener en cuenta los cambios durante el diseño de la arquitectura. Cuanta más flexibilidad se pueda incorporar a un diseño por adelantado, después los cambios serán menos dolorosos y menos costosos. El requerimiento de modificabilidad es una medida de lo fácil que puede ser cambiar una

aplicación para satisfacer nuevos requisitos funcionales y no funcionales. Predecir la modificabilidad requiere una estimación del esfuerzo y / o costo para realizar un cambio. Solo se sabe con certeza cuánto costará un cambio después de que se haya realizado. Luego, se puede saber cuán buena fue la estimación. Las medidas de modificabilidad solo son relevantes en el contexto de una arquitectura dada. Luego, evaluar la modificabilidad requiere que el arquitecto afirme posibles escenarios de cambio que capturen cómo pueden evolucionar los requerimientos. Los cambios pueden incluso especificarse en el plan de proyecto para versiones posteriores. Sin embargo, las posibles modificaciones deberán obtenerse de las partes interesadas de la aplicación, y diseñado de la experiencia del arquitecto. Algunos cambios pueden ser el cambio de un componente, que el sistema deba correr en otro sistema operativo, incluir nuevas funciones, etc.

Para cada escenario de cambio, el impacto del cambio anticipado en la arquitectura se puede evaluar. Este impacto rara vez es fácil de cuantificar, ya que la mayoría de las veces, la solución que se está evaluando no existe. En muchos casos, lo mejor que se puede lograr es un análisis de impacto de los componentes en la arquitectura que necesitarán modificación, o una demostración de cómo la solución puede adaptar la modificación sin cambio. Finalmente, con base en estimaciones de costo, tamaño o esfuerzo para los componentes afectados, se puede hacer alguna cuantificación del costo de un cambio. Los cambios aislados a componentes individuales o subsistemas débilmente acoplados es probable que sea menos costoso de fabricar que los que causan efectos dominó a través de la arquitectura. Si un probable cambio parece difícil y complejo de realizar, esto puede mostrar una debilidad en la arquitectura que podría justificar mayor consideración y rediseño.

Integración:

La integración se refiere a la facilidad con la que una aplicación puede ser incorporada en un contexto de aplicación más amplio. El valor de una aplicación o componente se puede aumentar considerablemente si su funcionalidad o los datos se pueden utilizar en otras formas que el diseñador no anticipó originalmente. Las estrategias más difundidas para proporcionar integración son mediante la integración de datos o proporcionando una interfaz de programación de aplicaciones (API).

La integración de datos implica almacenar los datos que una aplicación manipula de manera que otras aplicaciones puedan acceder. Esto puede ser tan simple como usar un base de datos relacional estándar para el almacenamiento de datos, o quizás implementando mecanismos para extraer los datos en un formato conocido como XML o un archivo de texto separado por comas (.csv) que otras aplicaciones pueden consumir. Con la integración de datos, la forma en que los datos son utilizados (o abusados) por otras aplicaciones está prácticamente fuera del control del propietario de los datos originales. Esto se debe a que la integridad de los datos y las reglas de negocio impuestas por la lógica de la aplicación, se pasa por alto. La alternativa es que la interoperabilidad sea lograda a través de una API. En este caso, los datos propios de la aplicación son escondidos detrás de un conjunto de funciones que facilitan el

control de acceso a datos externo. De esta manera, las reglas de negocio y la seguridad pueden ser aplicadas en la implementación de la API. La única forma de acceder a los datos e integrarse con la aplicación es mediante el uso de la API proporcionada. La elección de la estrategia de integración no es sencilla. La integración de datos es flexible y simple. Las aplicaciones escritas en cualquier idioma pueden procesar texto, o acceder a bases de datos relacionales usando SQL. Crear una API requiere más esfuerzo, pero proporciona un entorno mucho más controlado, en términos de corrección y seguridad, para la integración. Como siempre, la mejor elección de estrategia depende de lo que desee lograr y qué limitaciones existen.

Escalabilidad:

Comencemos con una definición representativa de escalabilidad: "Qué tan bien funcionará una solución a un problema cuando el tamaño del problema aumente". Esto es útil en un contexto arquitectónico. Nos dice que la escalabilidad es sobre cómo un diseño puede afrontar a requerimientos de la aplicación que tengan que ver con aumentos de tamaño. Se necesita saber qué se espera que crezca para que sea un atributo de calidad concreto. Aquí hay unos ejemplos:

o **Carga de solicitudes/peticiones:** Basado en una combinación definida de solicitudes en una plataforma de hardware dada, una arquitectura de una aplicación de servidor puede diseñarse para admitir 100 tps en carga máxima, con un tiempo de respuesta medio de un segundo. Si esta solicitud de carga creciera diez veces, ¿puede la arquitectura soportar este aumento de carga?

En el mundo perfecto y sin capacidad de hardware adicional, mientras aumenta la carga, el rendimiento de la aplicación debe permanecer constante (es decir, 100 tps), y el tiempo de respuesta por solicitud debe aumentar solo de forma lineal (es decir, 10 segundos). Una solución escalable permitirá una capacidad de procesamiento adicional para aumentar el rendimiento y reducir el tiempo de respuesta. Esta capacidad adicional se puede implementar de dos formas diferentes, una es agregando más CPUs (y probablemente más memoria) a la máquina en la que se ejecutan las aplicaciones (scale up- escalabilidad vertical), la otra forma es distribuir la aplicación en varias máquinas (scale out- escalabilidad horizontal) El scale up funciona bien si una aplicación tiene varios subprocesos o varias instancias de procesos individuales que se pueden ejecutar juntas en la misma máquina. Este último, por supuesto, consumirá memoria adicional y más recursos, ya que los procesos son vehículos pesados y hambrientos de recursos para lograr concurrencia.

La escalabilidad horizontal funciona bien si se requiere poco o idealmente ningún trabajo adicional para gestionar la distribución de solicitudes entre las múltiples máquinas. El objetivo es mantener cada máquina igualmente ocupada, ya que la inversión en más hardware se desperdicia si una máquina está completamente cargada y otras inactivas. La distribución de la carga uniformemente entre varias máquinas se conoce como balanceo de carga.

Es importante destacar que, para cualquier enfoque, la escalabilidad debe lograrse sin modificaciones a la arquitectura subyacente (aparte de la configuración inevitable cambia si se utilizan varios servidores). En realidad, a medida que aumenta la carga, las aplicaciones exhibirán una disminución en el rendimiento y un posterior aumento exponencial del tiempo de respuesta. Esto sucede por dos razones. En primer lugar, el aumento de carga provoca una mayor contención por recursos como CPU y memoria por los procesos y subprocesos en la arquitectura del servidor. En segundo lugar, cada solicitud consume algún recurso adicional (espacio de búfer, bloqueos, etc.) en la aplicación y, finalmente, este recurso se convierte en agotado y limita la escalabilidad.

o **Conexiones simultáneas:** Se puede diseñar una arquitectura para admitir 1000 usuarios simultáneos.

¿Cómo responde la arquitectura si este número crece significativamente? Si un usuario conectado consume algunos recursos, es probable que haya un límite para la cantidad de conexiones que se pueden admitir de manera efectiva.

o **Tamaño de datos:** En pocas palabras, ¿cómo se comporta una aplicación a medida que aumentan en tamaño los datos que procesa? Por ejemplo, una aplicación de mensajería, tal vez un chat, puede diseñarse para procesar mensajes de un tamaño medio esperado.

¿Qué tan bien reaccionará la arquitectura si el tamaño de los mensajes aumenta significativamente?

En una línea ligeramente diferente, una solución de gestión de la información puede estar diseñada para buscar y recuperar datos de un repositorio de un tamaño específico. ¿Cómo se comportará la aplicación si el tamaño del repositorio crece, en términos de tamaño y / o número de artículos?

o **Despliegue:** ¿Cómo es el esfuerzo que implica hacer el despliegue o modificar una aplicación para una creciente base de usuarios? Esto incluiría el esfuerzo de distribución, configuración y actualización con nuevas versiones. Una solución ideal sería proporcionar mecanismos automatizados que pueden implementar y configurar dinámicamente una aplicación para un nuevo usuario, capturando información de registro en el proceso. De hecho, esto es exactamente cómo muchas de las aplicaciones son distribuidas hoy en día en internet.

Otros atributos de calidad:

- **Testeabilidad:** ¿Qué tan fácil o difícil es probar una aplicación? Las decisiones tempranas de diseño pueden afectar en gran medida la cantidad de casos de prueba que se requieren. Como regla general, cuanto más complejo es un diseño, más difícil es la prueba. La simplicidad tiende a promover la facilidad de las pruebas. Asimismo, escribir menos de su propio código incorporando componentes previamente probados reduce el esfuerzo de prueba.

Conflictos arquitectónicos

- Utilizar componentes de granularidad gruesa mejora la performance, pero reduce la mantenibilidad. (gana la performance)
- La introducción de datos redundantes mejora la disponibilidad, pero hace más difícil la seguridad. (y la performance) Hay más lugares en donde se necesita controlar la seguridad y se tienen que agregar más capas de validación, esto hace que la performance se degrade, siendo más lento el sistema.
- La localización de aspectos de seguridad relacionados usualmente significa más comunicación, por lo tanto, degrada la performance.

Descripción del proceso de diseño arquitectónico

No hay un proceso estándar para diseñar arquitectura. Cada organización y equipo de trabajo adopta su propio proceso. No es necesario que todos tengan el mismo proceso, pero sí es necesario tener un proceso para hacer las cosas. No importa si es el mejor o si es el mismo que usa todo el mundo (esto garantiza que sirve y funciona).

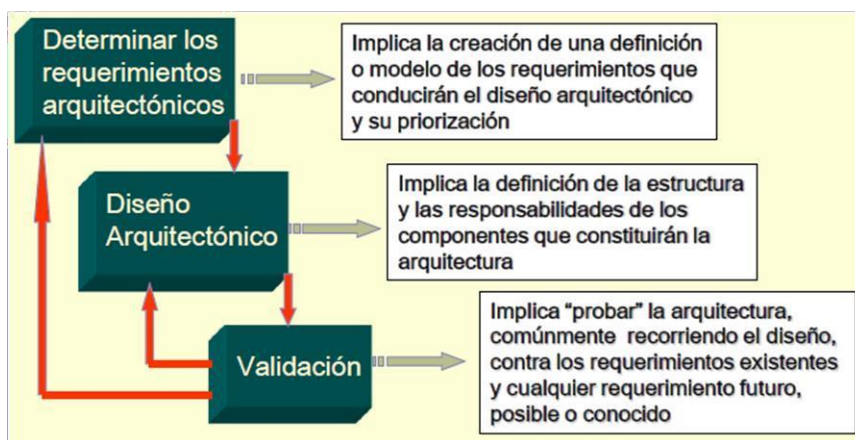
El rol del arquitecto es mucho más que simplemente encargarse de la actividad de diseño de software. El arquitecto debe:

- **Trabajar con el equipo de requerimientos:** este equipo estará enfocado en la obtención de los requerimientos funcionales para los interesados de la aplicación. El arquitecto cumple un rol importante entendiendo las necesidades generales del sistema y asegurando que los atributos de calidad apropiados son explícitos y comprensibles.
- **Trabajar con varias partes interesadas de la aplicación:** los arquitectos juegan un rol de enlace asegurándose que las necesidades de las partes interesadas de la aplicación son entendidas e incorporadas al diseño. Por ejemplo, además de los requerimientos del usuario del negocio para la aplicación, los administradores del sistema requerirán que la aplicación pueda ser fácilmente instalada, monitoreada, administrada y actualizada.
- **Liderar el equipo de diseño técnico:** definir la arquitectura de la aplicación es una actividad de diseño. El arquitecto dirige un equipo de diseño, que comprende diseñadores del sistema (o en grandes proyectos, otros arquitectos) y líderes técnicos para producir el plano de la arquitectura.
- **Trabajar con el administrador de proyecto:** el arquitecto trabaja cercanamente con el administrador del proyecto ayudando con la planificación del proyecto, la estimación, asignación y programación de las tareas.

Con el objetivo de guiar al arquitecto hacia la definición de la arquitectura de la aplicación, es útil definir un proceso definido de ingeniería de software. Se trabaja con un proceso que es rápido, corto, sencillo y llevadero.

Este proceso no es lo mismo que el WF de diseño. El WF de diseño permite diseñar TODO, incluso la arquitectura. **Este proceso es específicamente para diseñar la arquitectura.**

La siguiente figura muestra un proceso arquitectónico de tres pasos iterativos que pueden ser usados para guiar las actividades durante el diseño. En ella se describe brevemente cuáles son los pasos y qué implica cada uno.



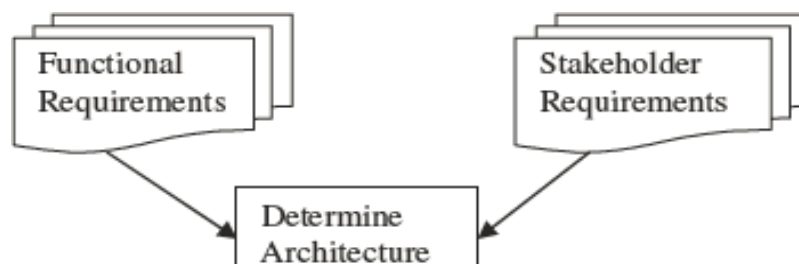
Este proceso de diseño de arquitectura es inherentemente iterativo. Una vez que el diseño es propuesto, la validación puede mostrar si el diseño necesita modificación o si ciertos requerimientos deben definirse y comprenderse mejor. Ambos conducirán a mejoras en el diseño, una validación posterior y así hasta que el equipo de diseño esté satisfecho de que se cumplan los requisitos.

1. **Determinar los requerimientos arquitectónicos**

Antes de que una solución arquitectónica pueda ser diseñada, es necesario tener una idea bastante buena de los requerimientos arquitectónicos para la aplicación. Los requerimientos arquitectónicos a veces también suelen llamarse requerimientos significativos para la arquitectura, los cuales son esencialmente la calidad y los requerimientos no funcionales para la aplicación.

1.1 Identificar requerimientos arquitectónicos

Las principales fuentes de los requerimientos arquitectónicos son el documento de requerimientos funcionales y otros documentos que capturan varias necesidades de las partes interesadas. La salida de este paso es un documento que determina los requerimientos arquitectónicos para la aplicación.



1.2 Priorización de requerimientos arquitectónicos

Implica tomar como entrada los requerimientos (funcionales y no funcionales), evaluarlos y determinar cuáles son significativos para la arquitectura. Con esos requerimientos me quedo para la arquitectura, ya que la condicionan.

Podemos dividir estos requerimientos en tres categorías de prioridad:

- **Alta:** La aplicación debe soportar este requerimiento, conducen el diseño de arquitectura
- **Medio:** Necesitará ser soportado en algún punto, pero no necesariamente en el primer release.
- **Bajo:** Estos requerimientos son parte de la lista de deseos. Las soluciones que los incluyen son deseables, pero no son conductores del diseño.

La priorización se vuelve complicada dado que ciertos requerimientos entran en conflicto.

Por ejemplo:

- La reutilización de componentes es la solución frente a una rápida comercialización, ya que hacer componentes generalizados y reutilizables siempre lleva más tiempo y esfuerzo.
- Gasto mínimo en productos COTS frente a reducir el costo y esfuerzo de desarrollo, ya que comprar estos productos significa que vas a programar menos código, pero cuestan dinero.

No existe una solución sencilla para estos conflictos. Es parte del trabajo del arquitecto discutirlos con las partes interesadas pertinentes y proponer posibles escenarios de solución que satisfagan adecuadamente ambos requisitos sin molestar a alguien o tener consecuencias no deseadas en la aplicación. Los requisitos en conflicto pueden incluso terminar teniendo la misma prioridad.

Hay que relacionar PRIORIZACIÓN con ITERACIÓN porque si el ciclo de vida fuese en cascada no tendría sentido la priorización porque se hace todo junto. De igual forma, si todo fuese prioridad alta, no tendría sentido tampoco.

Un requerimiento es de prioridad alta si es necesario desde el primer momento y debe resolverse en la 1era iteración, no se lo puede postergar. Un requerimiento es de prioridad media si se lo puede implementar en las subsiguientes iteraciones y, es de prioridad baja si entra dentro de la lista de deseos.

2. Diseño arquitectónico

Implica la definición de la estructura y las responsabilidades de los componentes que constituirán la arquitectura. Elegimos qué patrones arquitectónicos (framework) usaremos para armar el esqueleto del producto, son patrones de alto nivel (Nivel de capas o componentes). Los patrones

se aplican (significa que hay que construir los componentes que implementa estos patrones). Un patrón es una plantilla genérica. (Patrones arquitectónicos, los cuales nos servirán para resolver requerimientos). Luego, distribuimos los componentes, los creamos y aplicamos las soluciones arquitectónicas elegidas. De acá sale la documentación de la arquitectura y las vistas de arquitectura.

La etapa del diseño en sí tiene dos pasos, que son iterativos. El primero involucra elegir una estrategia general para la arquitectura basado en patrones arquitectónicos.

El segundo incluye especificar los componentes individuales que componen la aplicación, mostrando cómo encajan en el marco general y asignarles responsabilidades. La salida es un conjunto de vistas arquitectónicas que capturan el diseño de la arquitectura y un documento de diseño que explica el diseño, las razones clave de algunas de las principales decisiones de diseño, e identifica los riesgos involucrados.

2.1 Elegir el framework de arquitectura

La elección del framework de arquitectura se conforma con un conjunto de Patrones, esto es porque:

- La mayoría de las aplicaciones responden a un número pequeño de arquitecturas probadas y bien comprendidas
- Utilizar soluciones conocidas, minimiza los riesgos.
- No hay una fórmula mágica para diseñar un framework de arquitectura. Un pre-requisito es conocer los patrones arquitectónicos para abordar ciertos atributos de calidad.

2.2 Distribuir componentes

Una vez que se ha seleccionado el framework de arquitectura, basado en uno o más patrones arquitectónicos, la siguiente tarea es definir los componentes principales que comprenderá el diseño. El framework define la comunicación general para los componentes, debe identificarse lo siguiente:

- Identificar los principales componentes de la aplicación y cómo se insertan en el framework.
- Identificar la interfaz o los servicios que soporta cada componente.
- Identificar las responsabilidades del componente, definiendo qué será invocado para hacer cuando reciba una solicitud.
- Identificación de dependencias entre componentes.
- Identificar particiones en la arquitectura que son candidatas para la distribución a través de servidores en una red.

Los componentes de la arquitectura son las principales abstracciones que existirán en la aplicación. Por lo tanto, probablemente no sea una sorpresa que el diseño de componentes tenga mucho en común con las técnicas de diseño orientado a objetos ampliamente utilizadas. De hecho, los diagramas de clases y

paquetes se utilizan a menudo para representar componentes en una arquitectura.

Algunas pautas para el diseño de componentes son:

- Minimizar las dependencias entre componentes. Los cambios en un componente no se deben propagar a través de muchos componentes. Recordar: cada vez que cambie algo, tendrá que volver a probarlo.
- Diseñar componentes que encapsulen un conjunto de responsabilidades altamente cohesivas.
 - Aislar las dependencias entre el middleware y cualquier tecnología de infraestructura COTS.

La menor cantidad de componentes que dependen de un middleware específico y llamadas a API de componentes COTS, más fácil será cambiar o actualizar el middleware u otros servicios de infraestructura.

- Utilice la descomposición para estructurar componentes jerárquicamente.
- Minimice las llamadas entre componentes, ya que pueden resultar costosas si se distribuyen los componentes.

Es la construcción de componentes y el acoplamiento, esta tarea da origen a las vistas arquitectónicas (Modelan la arquitectura) y la documentación sobre las decisiones que hemos tomado en la arquitectura.

3. Validación

Durante el proceso arquitectónico, el objetivo de la fase de validación es incrementar la confianza del equipo de diseño en que la arquitectura cumpla con su propósito. La validación debe alcanzarse con los requerimientos de tiempo y presupuesto del proyecto. La validación del diseño de la arquitectura conlleva duros desafíos, ya que el diseño propuesto de la arquitectura no puede ser ejecutado ni probado para ver si cumple con los requerimientos. Por ende, hay dos técnicas principales que tienen un uso probado: **probar manualmente la arquitectura usando escenarios de prueba** y **construir prototipos** que crea un arquetipo de la aplicación deseada. El objetivo de ambos es identificar posibles defectos y debilidades en el diseño para que puedan mejorarse antes de que comience la implementación.

Implica probar la arquitectura, recorriendo el diseño contra los requerimientos, para determinar si cumple con los requerimientos, se puede probar:

- Utilizando escenarios (manualmente)
- Construir prototipos que crean un arquetipo de la app deseada

3.1 Usar escenarios de prueba

Se definen los escenarios y se aplican estímulos con impacto en la arquitectura, para luego trabajar en la respuesta a los estímulos.

Implica el testeo manual de la arquitectura usando escenarios de prueba. Los escenarios son una técnica desarrollada para detectar problemas relacionados con la arquitectura mediante evaluación y pruebas manuales. Los escenarios se relacionan con los atributos de calidad y tienen como

objetivo destacar las consecuencias de las decisiones arquitectónicas que se encapsulan en el diseño. Los escenarios son artefactos relativamente simples. Se define algún tipo de estímulo que tendrá impacto en la arquitectura. Luego, el escenario implica averiguar cómo responde la arquitectura a ese estímulo. Si la respuesta es deseable, entonces es considerado satisfecho por la arquitectura. Si la respuesta es indeseable, o difícil de cuantificar, entonces un defecto o al menos un área de riesgo en la arquitectura puede haber sido descubierto.

Ejemplo: Atributo de calidad: Disponibilidad

Estímulo: los mensajes se pueden perder si un servidor falla antes de que sean entregados.

Respuesta: los mensajes se almacenan en un servidor hasta que la conexión se reestablezca. Los mensajes sólo se perderán si el servidor falla antes de que se restaure la conexión.

La implicación es que los mensajes no se conservan en el disco, probablemente por motivos de rendimiento. La pérdida de mensajes en algunos contextos de aplicación puede ser aceptable.

Si no es así, este escenario pone de relieve un problema que puede obligar al diseño a adoptar mensajería persistente para evitar la pérdida de mensajes.

Validación: Uso de Escenarios

- Técnica desarrollada por el SEI.
- Los escenarios están relacionados con aspectos arquitectónicos tales como atributos de calidad, y ellos ayudan a destacar consecuencias de las decisiones arquitectónicas que están encapsuladas en el diseño.
 - ✦ Los escenarios son artefactos simples.
 - ✦ Implican la definición de estímulos que tendrán impacto en la arquitectura.
 - ✦ Se trabaja la respuesta de la arquitectura a esos estímulos.
 - ✦ Destacan las implicancias de la arquitectura en las decisiones de diseño.

Ejemplos de Escenarios

Atributo de Calidad	Estímulo	Respuesta
Disponibilidad	La conexión de red para los mensajes de los clientes falla.	Los mensajes son almacenados en el servidor MOM hasta que sea restablecida la comunicación.
Modificabilidad	Un conjunto nuevo de componentes de análisis de datos debe estar disponible en la aplicación	La aplicación debe reconstruirse con librerías nuevas y los archivos de configuración deben actualizarse en cada escritorio para que sean visibles en la barra de herramientas de GUI.
Seguridad	No se recibieron pedidos en una sesión de usuario por 10 minutos	El sistema trata la sesión como potencialmente insegura e invalida las credenciales asociadas a la sesión. El usuario debe registrarse nuevamente en la aplicación.

Prototipando

Esta segunda técnica se implementa porque algunos escenarios no son fáciles de responder basándose solo en una descripción de diseño. Se construyen prototipos que crean un simple arquetipo de la aplicación deseada, con esto se tiene la capacidad para evaluar con más detalle los requerimientos que se deben satisfacer a través de prototipos de prueba.

Los prototipos son versiones mínimas o reducidas de la aplicación deseada, creada específicamente para probar algunos aspectos del diseño mal entendido con alto riesgo.

Los prototipos se utilizan normalmente para dos propósitos:

1. Prueba de concepto: ¿Se puede construir la arquitectura tal como se diseñó de forma que puede satisfacer los requisitos?

2. Prueba de tecnología: ¿La tecnología (middleware, aplicaciones integradas, bibliotecas, etc.) seleccionada para implementar la aplicación se comportan como lo esperado?

En ambos casos, los prototipos pueden proporcionar evidencia concreta sobre preocupaciones que, de otro modo, son difíciles o imposibles de validar de otra forma.

Por ejemplo, el arquitecto puede saber que los sistemas de cola y correo electrónico son fácilmente capaces de soportar cinco mil mensajes en cinco minutos, ya que estas soluciones se utilizan en otra aplicación similar. Por lo tanto, no sería necesario construir esto como parte del prototipo. Sin embargo, el rendimiento de las interacciones con el uso de APIs es desconocido y, por lo tanto, deben probarse para ver si pueden procesar cinco mil mensajes en cinco minutos. Una vez creados y probados los prototipos, la respuesta de la arquitectura al estímulo en el escenario se puede responder con un alto grado de confianza.

Los prototipos deben usarse con prudencia para ayudar a reducir los riesgos inherentes en un diseño. A pesar de su utilidad, es necesaria una advertencia sobre la creación de prototipos.

Los esfuerzos de creación de prototipos deben controlarse y gestionarse cuidadosamente. Idealmente un prototipo debe desarrollarse en uno o dos días, una semana o dos como máximo.

Los prototipos de prueba de tecnología y prueba de concepto se desechan después de que han cumplido su propósito. Son un medio para lograr un fin, así que no debes adquirir vida propia y se convierten en un fin en sí mismos.

DOCUMENTAR LA ARQUITECTURA

La documentación de la arquitectura suele ser un tema espinoso en los proyectos de IT. Es común que haya poca o ninguna documentación que cubra la arquitectura en muchos proyectos. A veces, si hay alguna, está desactualizada, es inapropiada y básicamente no es muy útil. En el otro extremo hay proyectos que tienen masas de información relacionada a la arquitectura capturada en varios documentos y herramientas de diseño. Algunas veces esto es invaluable, pero a veces está

desactualizado, es inapropiado y no muy útil. **Entonces**, la experiencia nos dice que documentar arquitecturas no es un trabajo sencillo.

Pero hay muchas buenas razones por las que queremos documentar nuestras arquitecturas:

- Las partes interesadas pueden comprender y evaluar el diseño. Comunicación entre ellos.
- Podemos entender el diseño cuando volvemos a él después de un período de tiempo.
- Otros en el equipo del proyecto y la organización de desarrollo pueden aprender de la arquitectura al entender el pensamiento detrás del diseño.
- Podemos hacer un análisis del diseño, quizás para evaluar su posible desempeño, o generar métricas estándar como acoplamiento y cohesión.

Además, nos sirve para:

- Dejar plasmado las decisiones importantes
- Poder justificar a elección del framework elegido y los patrones
- Ver con claridad qué prioridad tienen los RNF.
- Cómo se da respuesta a los RNF críticos.
- Reutilización a gran escala
- Expresión del sistema y su evolución
- Evaluación y comparación de arquitectura de una forma consistente.
- Planificación, administración y ejecución de las actividades del desarrollo del sistema.
- Expresión de las características persistentes y soporte de los principios que guían un cambio aceptable.
- Verificación de la implementación.

Sin embargo, documentar las arquitecturas es problemático porque:

- No existe un estándar de documentación de arquitectura universalmente aceptado.
- Una arquitectura puede ser compleja y documentarla de forma comprensible es lenta y no trivial.
- Una arquitectura tiene muchas vistas posibles. Documentar todo lo potencialmente útil requiere mucho tiempo y es costoso.
- Un diseño de arquitectura a menudo evoluciona a medida que el sistema se desarrolla gradualmente y se obtienen más conocimientos sobre el dominio del problema. Mantener los documentos de arquitectura al día, es a menudo una actividad pasada por alto, especialmente cuando el tiempo y el cronograma presiona en un proyecto.

¿Qué documentar?

No hay una respuesta simple aquí. Lo que se vaya a documentar depende de varios factores, a saber:

- Complejidad de la arquitectura que se está diseñando: si es una aplicación con una arquitectura simple probablemente con un diagrama general de los principales componentes junto a un diagrama de estructura de los mismos y un esquema de la base de datos (generada automáticamente) será suficiente y, además este nivel de documentación es rápido de producir.

- Tiempo de vida de la aplicación: si el sistema cumplirá una función a corto plazo probablemente no necesita mucha documentación, pero si es largo plazo sí.

Necesidades de las partes interesadas de la aplicación: La documentación de la arquitectura sirve para comunicaciones importantes. En un equipo pequeño, la comunicación interpersonal a menudo es buena, por lo que la documentación puede ser mínima. En equipos más grandes, y especialmente cuando los grupos no comparten la misma oficina o edificio, la documentación se vuelve de vital importancia para describir elementos del diseño como:

- Interfaces de componentes; Restricciones de los subsistemas; Escenarios de prueba; decisiones de compra de componentes de terceros; Estructura del equipo y dependencias del horario; Servicios externos que ofrecerá la aplicación.

La documentación requiere tiempo para desarrollarse y cuesta dinero. Por tanto, es importante pensar detenidamente qué documentación será más útil dentro del contexto del proyecto, de producir y mantener esto como documentos de referencia clave para el proyecto.

Las herramientas más predominantes para documentar la arquitectura son las aplicaciones de Office usando la notación de diagramas de bloques y flechas, por eso se acusa que esto causa el estado de las prácticas de documentación actual. Para la documentación se utilizan vistas arquitectónicas y, el lenguaje de modelado UML 2.0 ayuda a generar estas vistas. Usando estas técnicas y herramientas de apoyo, no es demasiado difícil ni costoso generar documentación útil y valiosa.

VISTAS ARQUITECTÓNICAS

Las vistas arquitectónicas es una de las salidas más importantes que tiene el proceso de diseño arquitectónico ya que la arquitectura se modela a través de vistas.

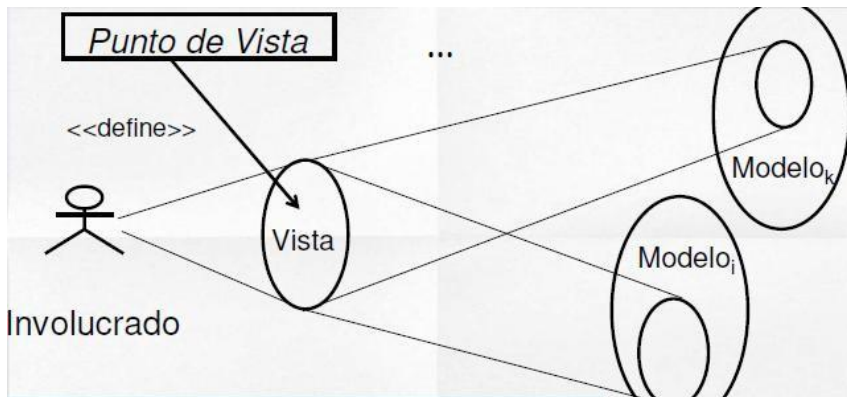
Además, permiten documentar la arquitectura.

Estas vistas efectivamente representan la arquitectura del producto y se construyen sobre la base de la identificación y priorización de los RNF significativos (1er paso del proceso de diseño arquitectónico).

CONCEPTOS

- **Modelo:** Una descripción completa de un sistema desde una perspectiva particular y a un nivel de abstracción específico.
- **Punto de vista:** Una definición (o descripción) teórica de una vista. Es el contenido, significado y representación (lenguajes, técnicas de modelado). Describe teóricamente qué se va a mostrar en esa vista. Se muestra lo que es de particular interés para un involucrado en el sistema.
- **Vista:** Una vista es una proyección del sistema desde una perspectiva en particular, se puede construir tomando elementos desde un modelo (modelo de análisis, req, diseño, etc) o de varios. Las vistas están pensadas para un involucrado en particular y sus intereses. El interesado ve lo que necesita

o quiere ver y se oculta la demás información. Se toma de la totalidad de un modelo y se queda sólo con las partes que responden a la motivación que se tiene para mostrar en esa vista. La vista es más lo que oculta que lo que muestra ya que cuando yo elijo qué elementos quiero mostrar para un interés en particular (perspectiva), es imposible mostrar todos los detalles de un sistema en una vista. Se usa para manejar la complejidad y para hacer más sencillo lo que queremos comunicar.



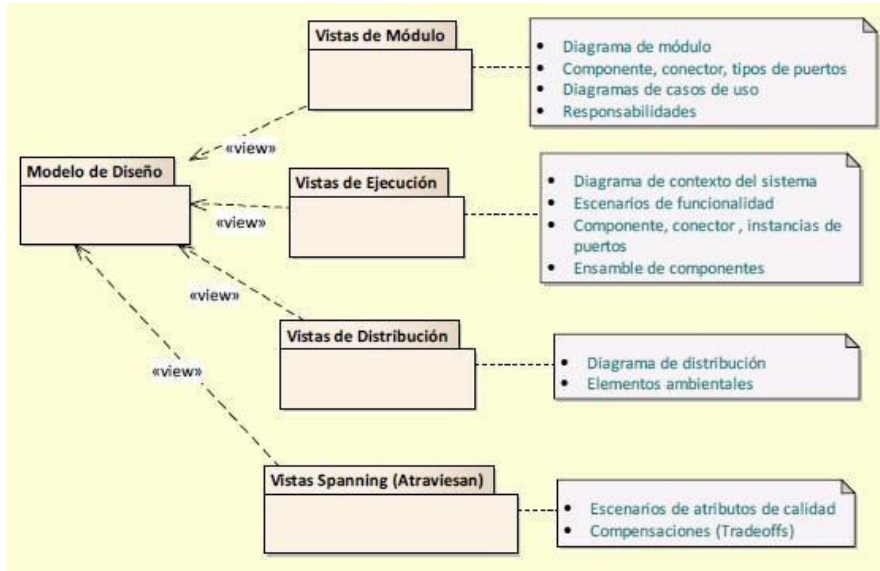
Tipos de vistas

Cada tipo de vista es un conjunto de vistas que pueden ser fácilmente conciliadas unas con otras. Las vistas que no pueden ser conciliadas pertenecen a tipos de vista diferentes.

- **Vista de módulo:** contiene vistas de los elementos que se pueden ver en tiempo de compilación. Vista estática de cómo organizamos el código. Definiciones de tipos de componentes, puertos y conectores, clases e interfaces. (Patrón Layered) – Diagrama de módulo, de casos de uso.
- **Vista de ejecución:** contiene vistas de los elementos que se pueden ver en tiempo de ejecución. Incluye escenarios de funcionalidad, listas de responsabilidad y ensambles de componentes. Instancias de componentes, conectores y puertos. (Patrón Publish-Suscribe, Patrón Broker, Patrón Messaging, Patrón Process Coordinator. Son los patrones de SW) – Diagrama de contexto del sistema.
- **Vista de distribución:** contiene vistas de elementos relacionados con la distribución del software en el hardware. (Patrón N-Tier, Patrón Map-Reduce, Mirrored, Farm (granja) y Rack – Son patrones de HW) – Diagrama de distribución- elementos ambientales.
- **Vista de spanning:** contiene cuestiones de confiabilidad, seguridad, performance, etc. (escenarios de atributos de calidad)

Los tipos componentes existen en el tipo de vista de módulo y las instancias de componentes (objetos) existen en la vista de ejecución, tal como ocurre con las clases y objetos.

Proveer vistas separadas ayuda a comprender cada asunto en forma aislada, pero también es necesario comprender la arquitectura como un todo.



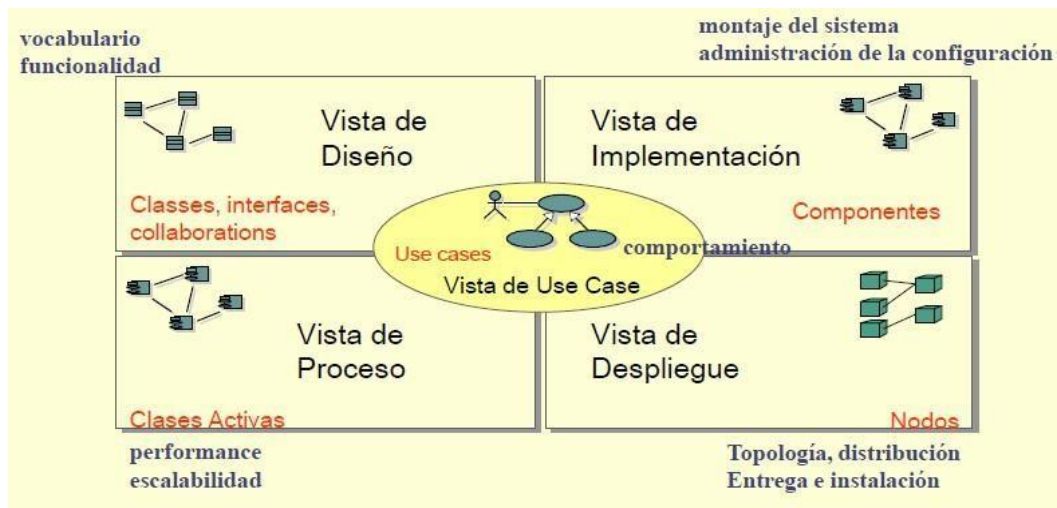
VISTAS EN EL PROCESO UNIFICADO DE DESARROLLO (PUD)

Estas 4 vistas están unidas por la Vista de Casos de Uso (vista de la funcionalidad) la cual contiene los casos de uso significativos para la arquitectura. Básicamente, capturan los requisitos para la arquitectura y, por lo tanto, están relacionados con más de una vista en particular. A partir de la Vista de Casos de Uso (vista de la funcionalidad) se van a construir las demás vistas. Con esto se aplica la característica de PUD "Conducido por casos de uso" porque de cada caso de uso, se va derivando la traza con los otros modelos que nos interesa mostrar y evolucionar.

Una arquitectura de software representa un artefacto de diseño complejo. Una forma de describir y comprender una arquitectura es basándonos en diferentes vistas:

- **Vista lógica (De diseño):** describe los elementos significativos de la arquitectura y las relaciones entre ellos. Describe cómo será provista la funcionalidad del sistema, qué clases, objetos o componentes voy a utilizar para darle soporte a la funcionalidad. La vista lógica captura la estructura de la aplicación usando diagramas de clases o equivalentes.
- **Vista del proceso:** se centra en describir la concurrencia y elementos de comunicación de una arquitectura. En las aplicaciones de TI, las principales preocupaciones están describiendo componentes multiproceso o replicados, y los mecanismos de comunicación síncronos o asíncronos utilizados.
- **Vista física (De despliegue):** Esto describe cómo los principales procesos y componentes se asignan al hardware de las aplicaciones. Muestra cómo el software será alojado en los diferentes componentes de hardware. Podría mostrar, por ejemplo, cómo se distribuye la base de datos y los servidores web de una aplicación en varias máquinas servidor.

- **Vista de implementación (componentes):** Esto captura la organización interna de los componentes de código, normalmente cuando se mantienen en un entorno de desarrollo o herramienta de gestión de la configuración. Describe también las dependencias de los módulos de implementación (código fuente, bibliotecas, componentes de terceros).



Una vista es una proyección de un modelo desde una perspectiva en particular. Un buen arquitecto de software tiene habilidades esenciales como ser un buen enlace entre clientes y equipo técnico, realizar buena ingeniería de software, tener gran conocimiento tecnológico y llevar una buena gestión de riesgos.

El modelo arquitectónico debe tratar con los requerimientos no funcionales y mapear las funcionalidades a una arquitectura tecnológica, escogiendo la solución óptima y no la perfecta (ya que no existe).

Entonces:

VISTA ARQUITECTÓNICA	ESTÁTICA	DINÁMICA
FUNCIONALIDAD – CASOS DE USO	DIAGRAMA DE CASOS DE USO SALE DEL WF DE REQUERIMIENTOS	DIAGRAMA DE SECUENCIA Y/O COMUNICACIÓN SE CONSTRUYE EN EL WF DE ANÁLISIS
DISEÑO	DIAGRAMA DE CLASE O DE COMPONENTES. SE USA EL DE COMPONENTES SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO
PROCESO	DIAGRAMA DE COMPONENTES SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO
DESPLIEGUE	DIAGRAMA DE DESPLIEGUE SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO
IMPLEMENTACIÓN	DIAGRAMA DE COMPONENTES SE CONSTRUYE EN EL WF DE DISEÑO	DIAGRAMA DE SECUENCIA SE CONSTRUYE EN EL WF DE DISEÑO

El cuadro debe leerse de la siguiente forma para cada caso: "La vista arquitectónica de funcionalidad estática se construye con diagramas de casos de uso", "La vista arquitectónica de funcionalidad dinámica se construye con el diagrama de secuencia y/o comunicación". Se procede de igual forma para las demás vistas.

Ahora nos preguntamos, ¿hace falta construir todas las vistas?, ¿hacen falta más vistas?

Las vistas son modelos simplificados para mostrar el contexto. No todos los sistemas requieren todas las vistas. Por ejemplo, si tenemos:

- Un solo procesador: saltar la vista de despliegue.
- Procesos simples: saltar la vista de procesos.
- Programas muy pequeños: saltar la vista de implementación.

También se pueden agregar otras vistas como: vista de los datos, vista de seguridad.

La vista de datos puede incluirse ya que se trabaja con una BD de otro paradigma entonces es útil mostrar qué decisiones se toman respecto a los datos para pasarlos del paradigma de objetos al modelo de datos relacional (persistencia). La vista de seguridad puede incluirse si la seguridad es un requerimiento muy crítico para la aplicación entonces la vista muestra cómo se resuelven los aspectos de seguridad.

El énfasis del PUD es adaptar el proceso a lo que el proyecto necesita para no hacer de más y no hacer de menos.

Las vistas arquitectónicas son GENERALES, muestran algunos componentes de SW que a mí me interesa destacar y garantizar su presencia. Mientras menos elementos tengan es mejor porque se comprende más, se puede comunicar mejor y es más fácil de explicar. Es un diseño a alto nivel, estratégico de cómo queremos que el producto resuelva los RF y los RNF.

Patrones arquitectónicos

Un patrón arquitectónico se puede considerar como una descripción abstracta de buena práctica, que debe describir una organización de sistema que fue probada y tuvo éxito. Debe incluir información sobre cuándo es adecuado usarlo, sus fortalezas y debilidades. Existen los platónicos (idealizados) y los embebidos (se los ve en sistemas reales).

- **Patrones de bajo nivel y detallados.** IDIOMAS. Se refiere a patrones específicos para un lenguaje de programación. (Aplicación de un patrón en un lenguaje en particular)
- **Patrones de nivel medio** (patrones de diseño, genéricos para cualquier lenguaje)
 - Proponen soluciones a nivel de clases, como mejorar lo que hace una clase mediante el uso de patrones. Sirven para cualquier lenguaje de programación
- **Patrones de alto nivel** (patrones arquitectónicos)
 - Proponen soluciones a un nivel de módulos, subsistemas y capas, a un nivel más global, muchas veces para implementar patrones arquitectónicos se usan patrones de diseño. Tiene un nivel de abstracción más alto. Afectan a todo un subsistema, a toda una capa o

todo el producto de SW.

El estilo arquitectónico es la unión de los patrones que usaremos para un producto; es un concepto diferente a patrones arquitectónicos, ya que un producto suele tener un único estilo que usa muchos patrones.

Un **patrón arquitectónico** es una descripción abstracta de buena práctica, que se ensayó y se puso a prueba en diferentes sistemas y entornos. Debe describir cuándo es y no adecuado usarlo, así como sus fortalezas y debilidades.

Un **framework arquitectónico** es el conjunto de patrones arquitectónicos que se utilizan para estructurar una aplicación. No se contrapone a la idea de que hay un estilo arquitectónico predominante. El framework serían los 3 o 4 patrones que tiene una aplicación (ej: Layered, N-Tier, Publish & Suscribe).

Patrones vs. Estilos arquitectónicos: suele ser difícil distinguirlos. A veces se los ve como sinónimos.

- Patrones arquitectónicos: están a una escala menor que los estilos. Múltiples patrones pueden aparecer en un diseño.
- Estilos arquitectónicos: Es el patrón que tiene más peso dentro del conjunto de patrones que tiene un producto de software ya que estructura y determina el comportamiento del producto. Un sistema usualmente tiene un único estilo arquitectónico dominante.

Platónicos vs. Embebidos

Es la diferencia entre lo que los patrones plantean teóricamente y lo que pueden hacer en la realidad de la aplicación.

- Platónicos: son patrones idealizados, los que se ven en los libros y rara vez de igual forma en el código.
- Embebidos: se los ve en sistemas reales y suelen violar las restricciones estrictas de los platónicos, lo que implica una gran compensación.

Muchas veces, se ve la necesidad de modificar el patrón para que cumplan con ciertos requerimientos no funcionales que son críticos (es el caso de los embebidos). Se le hace un ajuste al planteo teórico para resolver una necesidad puntual.

Patrones

1. Layered (estructura en capas).
2. Repositorio (o modelo centrado, Repository).
3. Modelo Vista Controlador (MVC, Model View Controller).

4. Arquitectura Cliente servidor
5. Messaging. (Arquitectura comunicando).
6. Publicar y Suscribir (Publish-Suscribe).
7. Tubería y Filtro (Pipe & Filter).
8. Lote-Secuencial (Batch-Sequential).
9. Broker (Arquitectura agente).
10. Coordinador de Proceso (Process Coordinator).

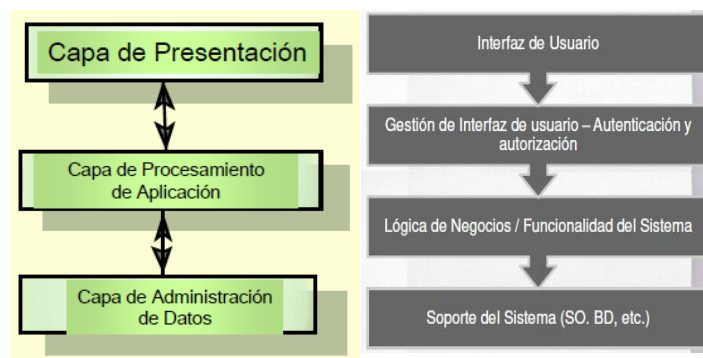
1. Layered (Vista estática/Vista de modulo)

Consiste en una pila de capas, donde cada capa actúa como una máquina virtual de la capa de arriba. Ayuda a separar la funcionalidad del sistema, independizar componentes, cada capa se apoya sólo en los servicios ofrecidos por la capa de abajo inmediata. (Como organizaremos el código). La restricción significa que las capas subsiguientes más abajo están ocultas. En una forma de lograr la separación y la independencia. Aquí, la funcionalidad del sistema está organizada en capas separadas, y cada capa se apoya solo en las facilidades y los servicios ofrecidos por la capa inmediatamente debajo de ella (las capas solo pueden usar a las capas que se encuentran directamente debajo de ellas). es parte del **tipo de vista módulo**

Calidad resultante: trata directamente los atributos de modificabilidad, portabilidad y reusabilidad. Mientras su interfaz no varíe, una capa puede sustituirse por otra equivalente; o cuando las interfaces de capa cambian o se agregan nuevas facilidades a una capa, sólo resulta afectada la capa adyacente. Las capas superiores dan más oportunidades de sustitución frente a un posible problema de performance.

Variantes:

- Algunas evitan la restricción de modo que las capas puedan comunicarse con capas de más abajo.
- Otra variante es el uso de capas compartidas, donde cada capa puede usar estas capas verticales.



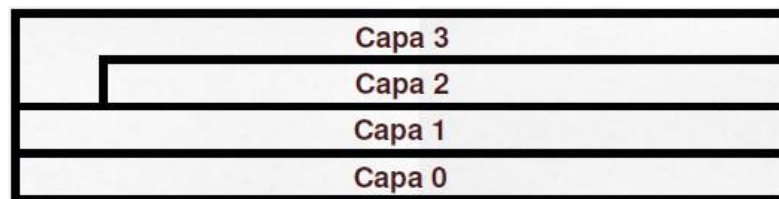
Notas: el estilo estratificado puede variar considerablemente en su forma platónica de su forma embebida. En la práctica se pueden saltar capas hacia capas inferiores, lo cual provoca negar los atributos de calidad que son su beneficio. Aun así, es beneficiosa dado que las capas agrupan módulos de funcionalidad coherente.

Propósito del patrón: lograr el mejor nivel de cohesión y acoplamiento posible, decidiendo en qué capa voy a ubicar cada componente de software. Trata los atributos de modificabilidad, portabilidad, reusabilidad. Mientras la interfaz no varíe, una capa puede ser sustituida por otra equivalente.

<i>Descripción</i>	Organiza el sistema en capas con funcionalidad relacionada con cada capa. Una capa da servicios a la capa de encima.
<i>Cuando se usa</i>	Se usa al construirse nuevas facilidades encima de los sistemas existentes, cuando el desarrollo se dispersa a través de varios equipos de trabajo, cuando existe un requerimiento de capa multinivel.
<i>Ventajas</i>	Permite la sustitución de capas completas (mientras se conserve la interfaz). En cada capa pueden incluirse facilidades redundantes para aumentar confiabilidad.
<i>Desventajas</i>	En la práctica suele ser difícil ofrecer una separación limpia entre capas, donde posiblemente una capa de nivel superior tenga que interactuar con capas del nivel inferior no adyacente. El rendimiento suele ser un problema debido a los múltiples niveles de una solicitud de servicios mientras se procesa en cada capa.

CAPA: Es el nombre que se le da desde la arquitectura a un subsistema. Dentro de una capa hay componentes. ¿Qué componentes ponemos dentro de cada capa? Este patrón nos ayuda a resolver esta pregunta.

- Es el patrón más común, tanto que muchos desarrolladores asumen que todos los sistemas deberían estar estratificados.
- Se encuentran sistemas forzados a ser estratificados.
- Aplica a elementos de código, es parte del tipo de vista de módulo. Es una vista estática, se muestran los componentes y dónde los voy a ubicar, pero no se muestra la comunicación o vínculo.
- En el siguiente gráfico se transforma el patrón platónico en uno embebido. La capa 3 sólo se debería comunicar con la capa 2 pero en este ejemplo se está comunicando con la capa 1.



Este patrón muestra una vista estática. Se ve al software desde un punto de vista estructural, no se ve la ejecución (en el primer gráfico no deberían estar las flechas). Se están definiendo lineamientos para ver cómo acomodamos los componentes de software de forma de privilegiar los dos principios básicos: alta cohesión y bajo acoplamiento.

El bajo acoplamiento está dado porque el patrón establece que cada capa se comunica única y exclusivamente con las capas adyacentes. Esto responde al principio de diseño "No hables con extraños".

2. Repository

Consiste en un conjunto de componentes independientes que interactúan con un modelo central (llamado también almacenamiento de datos o repositorio) en vez de interactuar entre ellos. Son

altamente modificables debido a la independencia de componentes, y extensibles; es sencillo agregar componentes.

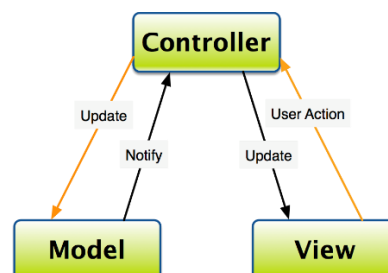
Está relacionado con el patrón MVC.

Calidad resultante: los sistemas repositorio son altamente modificables, debido a la independencia de los componentes; son extensibles, ya que es muy fácil agregar componentes.

<i>Descripción</i>	Todos los datos de un sistema se gestionan en un repositorio central, accesible a todos los componentes del sistema. Los componentes no interactúan directamente, sino tan sólo a través del repositorio.
<i>Cuando se usa</i>	Se usa cuando se tiene un sistema donde los grandes volúmenes de información generados deban almacenarse durante mucho tiempo. Puede usarse también en sistemas dirigidos por datos, en los que la inclusión de datos en el repositorio active alguna acción.
<i>Ventajas</i>	Los componentes pueden ser independientes, no necesitan conocer la existencia de otros componentes. Los cambios hechos por un componente se pueden propagar hacia todos los componentes. La totalidad de datos se puede gestionar de una manera consistente (todo está en un solo lugar). Es extensible, se pueden agregar vistas y controladores más tarde.
<i>Desventajas</i>	El repositorio es un punto de falla único (los problemas en el repositorio afectan a todo el sistema). Puede ser ineficiente la organización de la comunicación a través del repositorio. Difícil de distribuir en forma eficiente.

3. Model view controller

<i>Descripción</i>	Separa presentación e interacción de los datos del sistema. El sistema se estructura en tres componentes lógicos que interactúan entre sí. El modelo maneja los datos del sistema y las operaciones asociadas a esos datos. La vista define y gestiona como se presentarán los datos al usuario (refleja el estado del modelo). El controlador dirige la interacción del usuario y pasa esas interacciones a vista y modelo (recibe peticiones de la vista sobre el modelo, interacciona con el modelo, devuelve el resultado a la vista).
<i>Cuando se usa</i>	Se usa cuándo existen múltiples formas de interactuar con los datos. También se utiliza cuando se desconocen los requerimientos futuros para la interacción y presentación.
<i>Ventajas</i>	Permite que los datos cambien de manera independiente de su representación y viceversa. Soporta diferentes representaciones de los mismos datos, y los cambios en una representación se muestra en todos ellos. <i>Incrementa flexibilidad y reutilización.</i>
<i>Desventajas</i>	Puede implicar código adicional y complejidad de código cuándo el modelo de datos y las interacciones son simples.



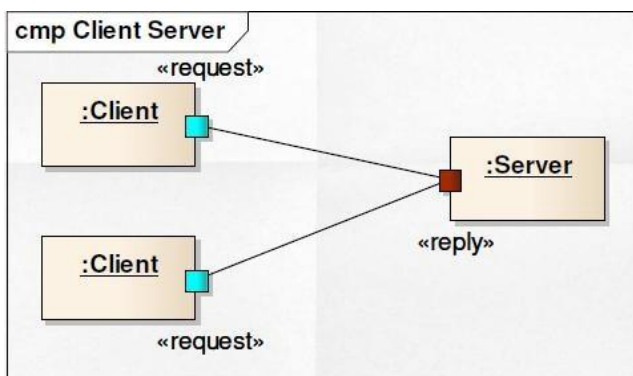
Modelo: Objeto aplicación, maneja los datos del sistema y las operaciones asociadas a esos datos. Frente a cambios, el modelo se encarga de avisar a las vistas que dependen de él

Vista: Objeto presentación en pantalla (interfaz de usuario), define y gestiona cómo se presentarán los datos al usuario (refleja el estado del modelo). La vista se actualiza a sí misma. Se pueden crear varias vistas de un modelo, para ofrecer diferentes presentaciones.

Controlador: Dirige la interacción del usuario y pasa esas interacciones a vista y modelo (recibe peticiones de la vista sobre el modelo, interacciona con el modelo, devuelve el resultado a la vista). Define cómo la interfaz reacciona antes la entrada del usuario
Separa presentación e interacción de los datos del sistema. Los tres componentes interactúan entre sí para dar una respuesta.

4. Arquitectura Cliente Servidor

- La aplicación es modelada como un conjunto de servicios que son provistos por servidores y un conjunto de clientes que usan esos servicios.
- Los clientes conocen a los servidores y deben saber cómo buscarlos, pero los servidores no necesitan conocer a los clientes.
- Clientes y Servidores son procesos lógicos.
- Los clientes inician la comunicación, hacen peticiones y esperan una respuesta.
- Los servidores no conocen la identidad de los clientes hasta que se han contactado. Los servidores atienden las peticiones
- El rol de los clientes y servidores no es intercambiable.
- Es una arquitectura **sincrónica**
- Como los servidores se pueden replicar, se usa cuando la carga de un sistema es variable.
- Si bien es una arquitectura distribuida, puede implementarse también en una sola computadora.
- Es parte de la **vista de ejecución (runtime)**.
- El uso más común es para explotar la capacidad gráfica de la PC y al mismo tiempo proteger los datos en un host central.
- Ejemplos: colas de impresión. Las aplicaciones como Word, Excel son clientes y la cola de impresión de Windows es el servidor.



Separación de preocupaciones: la lógica de presentación, negocios y manejo de datos está dividida en niveles.

Comunicaciones síncronas: cada nivel espera una respuesta del otro nivel antes de continuar.

Implementación flexible: cada nivel puede implementarse para que se ejecuten en la misma máquina o su propia máquina.

Atributos de calidad asociados

Disponibilidad: los servidores de cada nivel se pueden replicar

Manejo de fallas: al tener servidores replicados, ante un error, la solicitud del cliente se redirige a otro servidor de forma transparente para el usuario

Modificabilidad: cada nivel está encapsulado y puede ser modificado

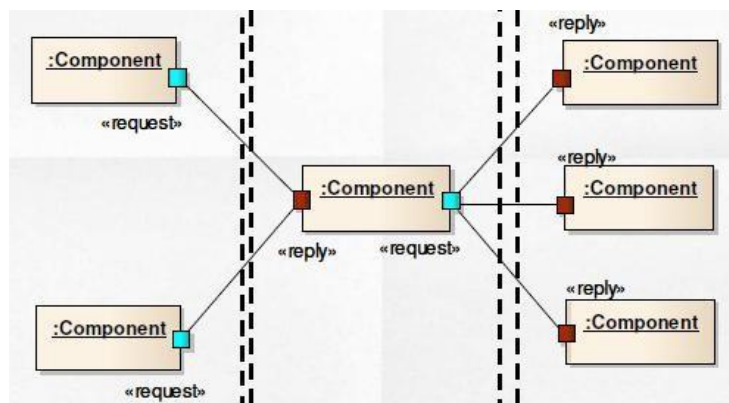
Rendimiento: puede haber muchos procesos simultáneos en cada servidor

Escalabilidad

Variantes:

- Los conectores pueden ser sincrónicos o asincrónicos.
- Puede haber límites en el número de clientes o servidores.
- Las conexiones pueden ser con o sin estado (sesiones).
- La topología del sistema puede ser estática o dinámica.
- Una variante permite al servidor, luego de que el cliente establezca contacto por primera vez, enviarle actualizaciones posteriores.
- Otra variante es el estilo N-Capas (N-Tier) que utiliza dos o más instancias del estilo Cliente-Servidor para formar una serie de capas.
 - o Los pedidos fluyen en una única dirección
 - o Las capas tienen responsabilidades funcionales exclusivas
 - o Este estilo es un híbrido entre la vista de ejecución y la de distribución, dado que las capas suelen (no siempre) asociarse a componentes de hardware distintos.

Arquitectura Cliente Servidor N-Tier (n- capas)



Propiedades clave del patrón:

- Separación de Intereses: capas diferentes y claramente divididas para presentación, negocios y manejo de datos.
- Comunicaciones sincrónicas: la comunicación entre capas es pedido-respuesta sincrónica. Los pedidos emanan en una única dirección del nivel del cliente a los niveles web y de lógica de negocio al nivel EIS.

Cada capa espera la respuesta del otro nivel antes de proseguir.

- Distribución flexible: no hay restricciones para la distribución multi- capas de la aplicación. Todas las capas pueden correr en la misma máquina o en su propia máquina. En aplicaciones WEB, la capa del cliente usualmente corre en un browser de una PC de escritorio de un usuario, comunicándose remotamente a través de Internet con componentes a nivel web.

Consideraciones:

- Este patrón es una variante embebida sobre la versión platónica del patrón.
- Esta variante surge ya que al introducir la arquitectura servidor en las capas, la capa de presentación funciona como cliente, pero las capas intermedias suponen un problema porque deben atender peticiones de las capas de arriba y deben hacer peticiones a las capas de abajo. Esto no es posible ya que cada capa debe cumplir un único rol. Es por eso que se plantea esta variante en donde las capas intermedias pueden ser Cliente y Servidor a la vez.
- En esta variante la capa superior (Capa de Presentación) SIEMPRE funciona como cliente y la capa inferior (Capa de Acceso a Datos) SIEMPRE funciona como Servidor. Sólo las capas intermedias pueden cumplir ambos roles al mismo tiempo.
- Funciona con comunicaciones sincrónicas y pertenece a la vista de ejecución (runtime) ya que podemos ver cómo se comunican los componentes.
- Si se quiere lograr la alta disponibilidad se deben replicar los servidores en cada capa.
- Este patrón generalmente es apropiado cuando una aplicación debe admitir un número potencialmente grande de clientes y solicitudes simultáneas, y cada solicitud toma un intervalo de tiempo relativamente corto (de unos milisegundos a unos segundos) para ser procesado.

5. Messaging

Los cambios en los formatos de los mensajes de los receptores pueden causar cambios en las implementaciones de los emisores. Los formatos de mensajes auto-descriptivos reducen esta dependencia. El cliente envía la petición a la cola y le delega a ésta el envío, continuando con sus actividades.

- Esta arquitectura desacopla emisores de receptores utilizando una cola de mensajes intermedia.
- La cola es el componente que acumula peticiones que van a ser entregadas en distintos componentes de SW que necesitan recibir esa información.
- El emisor envía un mensaje al receptor y sabe que eventualmente será entregado, aunque la red esté caída o el receptor no esté disponible.
- **Pertenece a la vista de ejecución (runtime)**

Propiedades claves:

- **Comunicaciones asíncronas:** clientes envían solicitudes a una cola, donde se almacenan hasta que una aplicación los remueve. Luego que el cliente escribió el mensaje en la cola, continúa sin esperar una respuesta.
- **Calidad de Servicio (QoS) Configurable:** la cola es configurable para una entrega de mayor velocidad, no confiable o de menor velocidad o una entrega confiable. Las operaciones de la cola pueden coordinarse con transacciones de base de datos. Se utiliza middleware para configurar la cola: prioridad en la entrega, cantidad de reintentos, etc.
- **Bajo acoplamiento:** no hay vínculo directo entre clientes y servidores. El cliente es inconsciente de cuál servidor recibe el mensaje y, el servidor es inconsciente de qué cliente vino el mensaje.

Consideraciones:

- Las aplicaciones que pueden dividir el procesamiento de un pedido en pasos discretos, conectados por colas, son una extensión del patrón messaging. Este es el patrón Pipe & Filter.
- Provee una solución resistente para aplicaciones en las cuales la conectividad es transitoria, debido a la poca confiabilidad de la red y de los servidores. En este caso, los mensajes se mantienen en la cola hasta que el servidor se conecta y los elimina.
- Se puede implementar para pedido-respuesta sincrónico utilizando un par de colas pedido-respuesta.
- Se deben acumular peticiones las cuales serán procesadas con algún criterio de ordenamiento (FIFO, LIFO, etc). Se usa cuando no se quiere perder información ya que está pensado para comunicaciones asincrónicas, si se produce una interrupción, la información queda acumulada como petición en la cola, no se pierde y cuando se reestablece la conexión se efectiviza la entrega de peticiones.

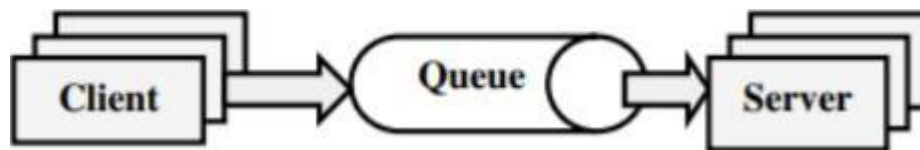
Si quiero garantizar la alta disponibilidad se debe replicar la cola.

La desventaja es que el bus de eventos puede afectar la performance del sistema.

Se puede implementar para pedido-respuesta sincrónico usando colas

pedido-respuesta. Además, si agregamos que se pueda dividir el procesamiento de un pedido en pasos discretos conectados por colas tendremos un patrón Pipe & filter.

Descripción	Emisores envían mensajes a receptores, donde los mensajes son almacenados en una cola.
Cuando se usa	Provee una solución resistente para aplicaciones en las cuales la conectividad es transitoria, tanto debido a la red como a la poca confiabilidad de los servidores. Es apropiada cuando el cliente no necesita una respuesta inmediata después de enviar el mensaje.
Ventajas	Promueve el bajo acoplamiento, permitiendo alta modificabilidad, dado que emisores y receptores no se conectan.
Desventajas	Los cambios en los formatos de los mensajes de los receptores pueden causar cambios en las implementaciones de los <u>emisores</u> pero Los formatos de mensajes <u>auto-descriptivos</u> reducen esta dependencia.



Atributos de calidad:

- Disponibilidad: podemos tener instancias de servidores replicadas.
- Manejo de fallas
- Modificabilidad
- Rendimiento: la cola de mensajes puede entregar miles de mensajes por segundo
- Escalabilidad

6. Publish-subscribe

Es parte de la vista de ejecución (runtime). Componentes independientes publican eventos y otros se suscriben a ellos. Desacopla publicadores de suscriptores. Ambos se relacionan a través de tópicos.

Componentes estáticos y vínculos(dinámico) es decir comunicación entre componentes

Frente a un cambio de estado (Algún valor en algún atributo se modificó; hay un evento que hace que cambie de estado) en un componente haya otros componentes que manifestaron interés en ese cambio de estado, se tienen que suscribir.

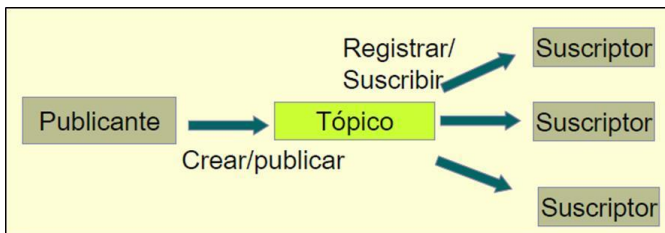
- Los publicantes ignoran el motivo global por el cual el evento es publicado, los suscriptores ignoran por qué o quién publica el evento y dependen sólo del evento, no de quién lo publica. A su vez, los publicantes son inconscientes del consumo de los eventos.
- Cada tópico puede tener más de un publicante y los publicantes pueden aparecer y desaparecer dinámicamente, lo que le da flexibilidad sobre configuraciones estáticas.
- Los suscriptores pueden suscribirse y des- suscribirse dinámicamente a un tópico.
- Funciona sobre la base de la aceptación de los suscriptores.
- Es un patrón muy utilizado hoy en día, en la mayoría de las aplicaciones que usamos a diario.
- Pull y push, pull notifica que tiene que actualizar y push actualiza forzosamente

Busca mantener actualizados a los interesados sobre algún cambio de estado de un componente.

No hay vínculo directo entre publicantes y suscriptores. Esto implica tener bajo acoplamiento.

Los buses de eventos varían en las propiedades que soportan:

- **Durables:** garantizan que cualquier mensaje que aceptan no se perderá durante una falla (los eventos se escriben en un almacenamiento confiable).
- **Entrega en orden:** garantizan la entrega en orden o priorizada de eventos.
- **Entrega en lotes:** los eventos se acumulan en lotes para evitar cúmulos de eventos similares.

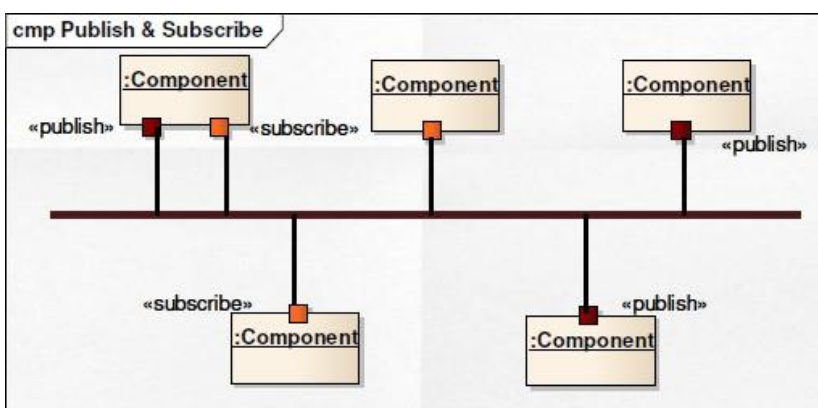


Propiedades clave del patrón:

Mensajería muchos a muchos: los mensajes publicados son enviados a todos los suscriptores que están registrados en el tópico. Muchos publicantes pueden publicar en el mismo tópico, y muchos suscriptores pueden escuchar en el mismo tópico.

Calidad de Servicio (QoS) Configurable: Además de mensajes seguros/no seguros, el mecanismo de comunicación subyacente puede ser punto-a-punto o multicast/broadcast. El primero envía un mensaje distinto para cada suscriptor del tópico, el último manda un mensaje que todo suscriptor recibe.

Bajo acoplamiento: No hay vínculo directo entre publicantes y suscriptores. Los publicantes no saben quién recibe su mensaje y los suscriptores no saben cuál publicante envía el mensaje.



Atributos de calidad:

- Disponibilidad
- Manejo de fallas
- Modificabilidad: se pueden agregar y quitar editores y suscriptores sin cambiar la arquitectura
- Rendimiento
- Escalabilidad

Descripción	Componentes independientes publican eventos y otros se suscriben a ellos.
Cuando se usa	En arquitecturas muy flexibles y adecuadas para aplicaciones que requieren mensajes asíncronos uno-a-muchos, muchos-a-uno, muchos-a-muchos componentes.

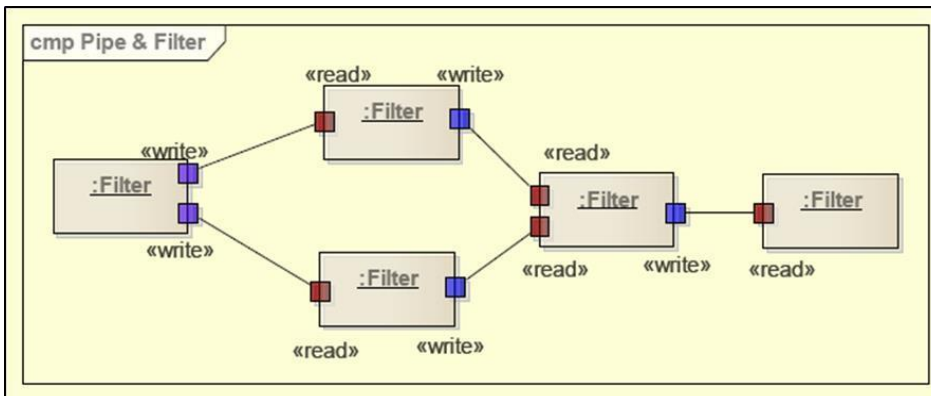
Consideraciones:

- Las arquitecturas basadas en publish-suscribe son muy flexibles y adecuadas para aplicaciones que requieren mensajes asíncronos (no se espera una respuesta) uno-a-muchos, muchos-a-uno, muchos-a-muchos entre componentes. Puede configurarse como síncrono. Al ser asíncrono, no se pierde información, si hay pérdida de conexión, la información queda guardada en el tópico y luego se rehace el intento de entregar los mensajes tantas veces como esté configurado ya que este patrón funciona con Middleware (se puede configurar la cantidad de veces en intentar enviar la notificación, prioridad, formato del mensaje, etc).
- Si se quiere garantizar la alta disponibilidad, en este caso, se debe replicar el tópico para que haya redundancia.
- En cuanto al tiempo de respuesta (eficiencia de desempeño) si hay una comunicación punto a punto, el proceso de notificar se vuelve más lento que si la comunicación fuese multicast (le llega a todos los suscriptores al mismo tiempo). Es por eso que este patrón está pensado para comunicaciones multicast o broadcast que es mucho más eficiente en términos de desempeño y de la demora en la cantidad de mensajes que pueden entregarse a los suscriptores por unidad de tiempo.
- El principal beneficio es desacoplar los componentes que producen los eventos de quienes los consumen, lo que hace la arquitectura mantenible y evolucionable.
- Los componentes pueden asumir el rol de publicantes, de suscriptores o eventualmente ser al mismo tiempo, publicante para un tópico y suscriptor para otro.

7. Pipe & filter

Los datos fluyen de un componente a otro a través de una tubería, y se transforman conforme se desplazan en la secuencia. Cada paso se implementa como un transformador, y los datos fluyen hasta que se convierten en salida. Cada componente de procesamiento (filtro) es discreto y realiza un tipo de transformación.

Descripción	Los datos fluyen (como en una tubería) de un componente a otro para su procesamiento. El procesamiento de datos se organiza de forma que cada componente de procesamiento (filtro) sea discreto y realice un tipo de transformación de datos.
Cuando se usa	Se suele utilizar en aplicaciones de procesamiento de datos (ya sean <u>Batch</u> –en lotes- o en transacciones), donde las entradas se procesan en etapas separadas para generar salidas relacionadas.
Ventajas	Es fácil de entender y soporta reutilización de transformación. El estilo de flujo de trabajo coincide con la estructura de muchos procesos empresariales. La evolución al agregar transformaciones es directa. Puede implementarse como sistema secuencial o como uno concurrente.
Desventajas	El formato para transferencia de datos debe acordarse entre las transformaciones que se comunican. Cada transformación debe analizar sus entradas y sintetizar sus salidas al formato acordado. Esto aumenta la carga del sistema y dificulta la reutilización de transformaciones funcionales que usen estructuras de datos compatibles. Son inapropiadas para aplicaciones interactivas.



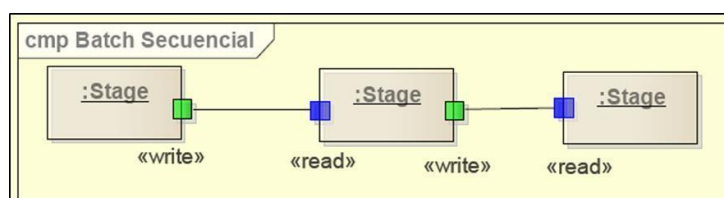
8. Batch-sequential

Los datos fluyen de una etapa a otra y son procesados incrementalmente. Cada etapa completa todo su procesamiento antes de escribir su salida (a diferencia del pipe & filter).

Parecido al anterior:

- Cada etapa es independiente, depende de los datos que toma y no de la etapa anterior
- Las etapas no interactúan entre sí
- Es una serie de pasos lineales
- No se hace trabajo en los conectores

Poseen alta modificabilidad y buen rendimiento. Fácil de entender. El problema es que la salida final está completamente terminada o está ausente; afecta a la usabilidad. No se puede ejecutar trabajo en paralelo a menos que influyan muchos trabajos a través del sistema.

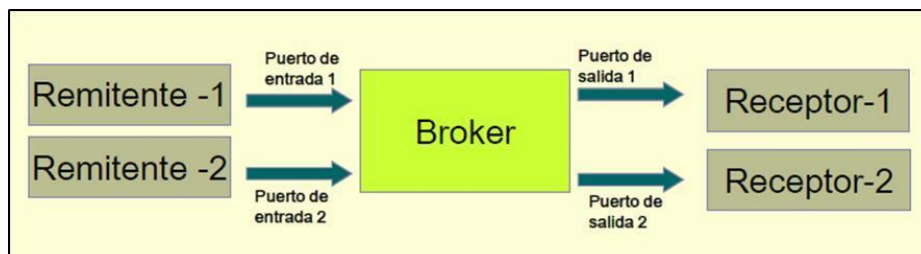


Descripción	Los datos fluyen de una etapa a otra y es procesado incrementalmente.
Ventajas	Alta modificabilidad, ya que cada etapa es independiente de las otras. Buen rendimiento. Conceptualmente simple dado que una etapa está ejecutándose por vez.
Desventajas	La salida final está completamente terminada o ausente, lo que impacta en la usabilidad del sistema (a diferencia del Pipe & Filter). No se puede ejecutar trabajo en paralelo a menos que fluyan múltiples trabajos a través del sistema.

9. Broker

Se usa para aplicaciones en las que los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino. El agente desacopla el remitente del receptor.

- **Hub and spoke architecture:** el bróker (intermediario) actúa como un concentrador de mensajería y los remitentes y receptores se conectan como radios. Las conexiones con el bróker se realizan a través de puertos asociados con un formato de mensaje específico.
- **Realiza el enrutamiento de mensajes:** el bróker incorpora lógica de procesamiento para entregar un mensaje recibido en un puerto de entrada a un puerto de salida. La ruta de entrega puede estar codificada.
- **Realiza la transformación de mensajes:** la lógica del bróker transforma el tipo de mensaje de origen recibido en el puerto de entrada al tipo de mensaje de destino requerido en el puerto de salida.



Consideraciones:

- Los agentes son adecuados para aplicaciones en las cuales los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino

El agente desacopla el remitente del receptor, permitiéndoles producir o consumir su formato nativo, y centraliza la definición de la transformación lógica en el agente para facilitar la comprensión y modificación.

- Se utiliza cuando hay un problema de compatibilidad y, la mayoría de las veces está involucrado alguien ajeno a nuestro sistema que no podemos controlar
- **Pertenece a la vista de ejecución o runtime.**
- Se debe replicar el bróker si se quiere garantizar la alta disponibilidad
 - Funciona con comunicaciones asíncronas y el objetivo es no perder información. Se puede configurar tanto la calidad de servicio como para comunicaciones síncronas, aunque no se recomienda.

Cuando se usa	Son adecuados para aplicaciones en las cuales los componentes intercambian mensajes que requieren grandes transformaciones entre los formatos de origen y destino. Ej: interpretar información de sensores de alarmas, de impresoras fiscales o de sistemas externos.
Ventajas	El agente desacopla el remitente del receptor, permitiéndoles producir o consumir su formato nativo, y centraliza la definición de la transformación lógica en el agente para facilitar la comprensión y modificación. (bajo acoplamiento)

La diferencia que existe entre los patrones: Publish & Suscribe, Messaging y Broker es el propósito que persigue cada uno es decir, cuál es el problema que resuelve. Luego, los tres funcionan con comunicaciones asíncronas y se le puede configurar la calidad de servicio como así también para que funcionen con comunicaciones síncronas.

Atributos de calidad:

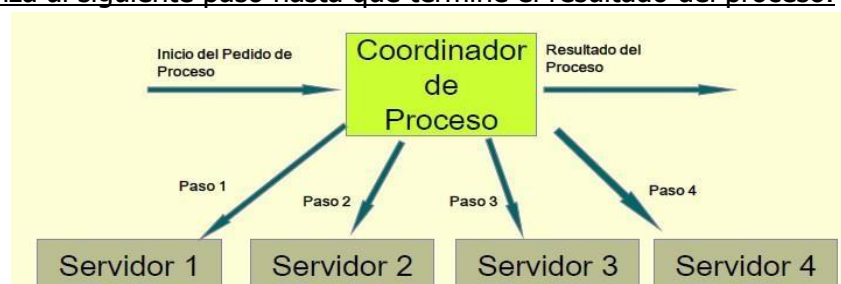
- Disponibilidad
- Manejo de fallas: validan y descartan mensajes en formato incorrecto
- Modificabilidad: separan la lógica del enrutamiento de mensajes y la transformación de los mismos.
- Rendimiento: pueden convertirse en cuello de botella.
- Escalabilidad

10. Process coordinator

Se utiliza para resolver peticiones muy complejas que pueden separarse en pasos. Se encapsulan las etapas de procesamiento en varios servidores distintos que resuelven cada uno una parte. Luego se incluye un coordinador que es quien sabe en qué orden y qué servidores resolverán la petición. Se reduce así la complejidad y se desacoplan los componentes, ya que los servidores son ignorantes de dónde se los utiliza y el coordinador no sabe cómo resuelve cada servidor su tarea, sólo que lo hace.

Se utiliza para procesos de negocios complejos. Un proceso de negocio complejo es quien tiene varios pasos, y también tiene una lógica de orden de ejecución de esos pasos, con una única salida.

Estructuramos cada uno de estos servidores para que resuelvan cada paso, y al coordinador que va a invocando los pasos en funcion a la logica que tiene definida. Cuando recibe la respuesta de las peticiones, avanza al siguiente paso hasta que termine el resultado del proceso.



Es usado comúnmente para implementar procesos de negocio complejos. Encapsulando la lógica de proceso en un solo lugar es más sencillo cambiar y administrar la performance del mismo.

- **Encapsulamiento de procesos:** el coordinador de proceso encapsula la secuencia de pasos necesarios para completar un proceso de negocio. La secuencia puede ser arbitrariamente compleja. El coordinador de proceso es un solo punto de definición para el proceso de negocio, haciéndolo más fácil de entender y modificar. Recibe una solicitud de iniciación de proceso, llama a los servidores en el orden definido por el proceso y emite los resultados.
- **Bajo acoplamiento:** los componentes del servidor son inconscientes de su rol en el proceso de negocio general, y del orden de los pasos del proceso. Los servidores simplemente definen un conjunto de servicios que pueden ejecutar y el coordinador los llama cuando es necesario como parte del proceso de negocio.
- **Comunicaciones flexibles:** las comunicaciones entre el coordinador y los servidores pueden ser sincrónicas o asincrónicas. Para las comunicaciones **sincrónicas**, el coordinador espera hasta que el servidor responda. Para las comunicaciones **asincrónicas**, el coordinador provee una llamada o respuesta cola/tópico, y espera hasta que el servidor responde utilizando el mecanismo definido.

Consideraciones:

- El patrón coordinador de proceso es utilizado comúnmente para implementar procesos de negocio complejos que deben hacer peticiones a componentes de servidores diferentes. Encapsulando la lógica de proceso en un lugar, es más fácil cambiar, administrar y monitorear la performance del proceso.
- Tiene una arquitectura centralizada, ya que el propósito de este patrón es ubicar en un componente de software llamado coordinador de proceso, la responsabilidad de manejar la lógica de un proceso de negocio complejo.
- Está pensado para trabajar con comunicaciones asíncronas aunque se puede configurar para síncronas.

<i>Cuando se usa</i>	Es utilizado comúnmente para implementar procesos de negocio complejos que deben publicar requerimientos a algunos componentes servidores diferentes. Ej: tareas que se deben corroborar en AFIP, proceso para sacar un préstamo hipotecario, proceso de sacar un autoplan , cambio de identidad, etc.
<i>Ventajas</i>	Encapsulando la lógica de proceso en un lugar, es más fácil cambiar, administrar y monitorear la performance del proceso. Bajo acoplamiento ya que los servidores no se conocen entre sí Se configura el coordinador y es el que da la orden de los pasos a seguir El resultado es único y lo genera el coordinador de proceso
<i>Desventajas</i>	El coordinador de proceso se puede convertir en un cuello de botella ya que es un acumulador que debe atender demasiadas peticiones y puede degradar la performance Si hay fallas puede resolverse con redundancia.

Atributos de calidad:

- Disponibilidad: el coordinador es un solo punto de falla; debe replicarse.
- Manejo de fallas: complejo

- **Modificabilidad:** los servidores pueden cambiar su implementación sin afectar al coordinador ni a otros servidores.
- **Rendimiento:** para lograrlo, el coordinador debe poder manejar muchas solicitudes concurrentes.
- **Escalabilidad:** el coordinador puede replicarse para escalar la aplicación.

Diagrama de Componentes: Se usa para visualizar aspectos estáticos de los componentes físicos y sus relaciones p/especificar detalles de su construcción

Arquitectura de Sistemas Distribuidos

Introducción

Prácticamente todos los grandes sistemas informáticos son en la actualidad sistemas distribuidos. Un sistema distribuido es un sistema en el que el procesamiento de información se distribuye sobre varias computadoras, en contraste con los sistemas centralizados en que todos los componentes del sistema se ejecutan en una sola computadora. Es una colección de computadoras independientes que aparecen al usuario como un solo sistema coherente. Por lo tanto, existen cuestiones específicas que deben tenerse en cuenta al momento de diseñar este tipo de sistemas.

Un sistema distribuido debe ser transparente. Esto significa que los usuarios ven el sistema como un solo sistema cuyo comportamiento no resulta afectado por la forma en que se distribuye el sistema. En la práctica, es imposible hacer un sistema por completo transparente (demoras en la red, fallas del nodo remoto, etc.).

Lo que define si un sistema es distribuido o no es donde se ejecutan el procesamiento, por lo que existen:

- **Sistemas personales:** no son distribuidos y han sido diseñados para ejecutarse en una computadora personal o estación de trabajo. No comparten recursos de hardware.
- **Sistemas embebidos:** Sistemas creados para un hardware específico. (Ej: software de autos o celulares). Está el software integrado en el HW. Tuvo su auge importante por la IA y los altos niveles de automatización. Son desarrollados por ingenieros electrónicos o de computación. Se ejecutan en un único procesador o en un grupo integrado de procesadores.
- **Sistemas distribuidos:** el sistema de software se ejecuta en un grupo de procesadores cooperativos integrado conectados por una red. Son los más utilizados hoy en día y apunta a la integración.

Características de los sistemas distribuidos:

1. **Compartición de recursos:** Permite compartir recursos hardware y software —como discos, impresoras, ficheros y compiladores que se asocian con computadoras de una red. Esto permite

aprovechar recursos que no disponemos físicamente y sí están en otro lugar, lo cual aporta a la capacidad de procesamiento.

2. **Apertura:** Son sistemas abiertos, lo que significa que se diseñan sobre protocolos estándares que combinan software y equipos de diferentes vendedores. Esto facilita la integración.
3. **Concurrencia:** varios procesadores operando al mismo tiempo sobre diferentes computadoras de la red sin conflictos. Estos procesos pueden (aunque no necesariamente) comunicarse con otros durante su funcionamiento normal.
4. **Tolerancia a fallas:** La disponibilidad de varios recursos y el potencial para reducir información significa que los sistemas distribuidos pueden ser tolerantes a algunos fallos de funcionamiento del hardware y del software. En la mayoría de los sistemas distribuidos, se puede proporcionar un servicio degradado cuando ocurren fallos de funcionamiento; una completa pérdida de servicio sólo ocurre cuando existe un fallo de funcionamiento en la red.
5. **Escalabilidad:** los sistemas pueden crecer incrementando recursos para cubrir nuevas demandas. La escalabilidad de un sistema refleja su disponibilidad para entregar una alta calidad de servicio conforme aumentan las demandas del sistema. Hay tres dimensiones de escalabilidad:
 - Tamaño: Debe ser posible agregar más recursos
 - Distribución: Dispersar recursos sin reducir rendimiento
 - Manejabilidad: Debe poder administrarse un sistema conforme aumenta su tamaño

Desventajas de los sistemas distribuidos

1. **Complejidad:** son más difíciles de comprender y probar. Hay demasiados elementos que pueden fallar. Mover los recursos de una parte del sistema a otra puede afectar de forma radical al rendimiento del sistema.
2. **Seguridad:** Se puede acceder de muchas computadoras, hay mucha vulnerabilidad y muchos elementos para atacar. Esto hace difícil asegurar la integridad de los datos.
3. **Manejabilidad:** Las máquinas pueden ser de distintos tipos y con diferentes sistemas operativos. Los defectos pueden propagarse de una máquina a otra con consecuencias inesperadas, lo que implica que es más difícil de gestionar y administrar, se requiere más esfuerzo.
4. **Impredecibilidad:** los sistemas distribuidos tienen una respuesta impredecible. La respuesta depende de la carga total en el sistema, de su organización y de la carga de la red. Como todos ellos pueden cambiar con mucha rapidez, el tiempo requerido para responder a una petición de usuario puede variar drásticamente de una petición a otra.

Ataques de los que deben defenderse los sistemas distribuidos

1. Intercepción: un atacante intercepta comunicaciones entre partes del sistema, provocando que haya poca confidencialidad.
2. Interrupción: los servicios son atacados y no pueden entregarse como se esperaba. Se bombardea al

servicio con peticiones ilegítimas para que no pueda atender las legítimas.

3. Modificación: el atacante cambia los datos o el servicio del sistema.

4. Fabricación: el atacante genera información que no debe existir y luego la usa para conseguir ciertos privilegios. Además, hace que el sistema maneje información poco confiable o fuera de la realidad.

Modelos de interacción en sistemas distribuidos

- Interacción procedimental: una computadora solicita un servicio conocido a otra computadora y espera que la otra computadora le responda. Sincrónico.
- Interacción basada en mensajes: la computadora emisora define en un mensaje la información y lo envía a otra computadora. No es necesario esperar la respuesta. Normalmente se implementa con un componente que crea un mensaje, el cual detalla los servicios requeridos por otro componente. Asincrónico.

Middleware

Los componentes en un sistema distribuido pueden implementarse en diferentes lenguajes de programación y pueden ejecutarse en tipos de procesadores completamente diferentes. Los modelos de datos, la representación de la información y los protocolos de comunicación pueden ser todos diferentes. Un sistema distribuido, por lo tanto, requiere software que pueda gestionar estas partes distintas, y asegurar que dichas partes se puedan comunicar e intercambiar datos. El término middleware se usa para hacer referencia a ese software; se sitúa en medio de los diferentes componentes distribuidos del sistema.

Es el software que permite gestionar y garantizar que las diversas partes de un sistema distribuido puedan comunicarse e intercambiar datos. Se llama middleware porque se encuentra en el centro de las partes del sistema distribuido.

Por lo general, se implementa como un conjunto de librerías, que se instala en cada computadora distribuida.

El middleware es comúnmente "off the shelf", es decir software de propósito general (enlatado) en lugar de software hecho a medida. Ejemplos: comunicación con bases de datos, gestores de transacciones, convertidores de datos y controladores de comunicación.

En un sistema distribuido brinda dos tipos de soporte:

- Soporte de interacción: el middleware coordina las interacciones entre diferentes componentes del sistema. Puede soportar conversión de parámetros si se usan lenguajes de programación diferentes.
- Provisión de servicios comunes: el middleware proporciona implementaciones de reutilización de servicios que pueden requerir varios componentes en el sistema distribuido. por ejemplo: Servicios de Seguridad (autenticación y autorización), Servicios de Notificación y Nomenclatura, Servicios de Gestión

de Transacción

Evolución de Arquitecturas de Sistemas Distribuidos

1) Arquitectura Maestro-Esclavo (Master-Slave) o Arquitectura Multiprocesador

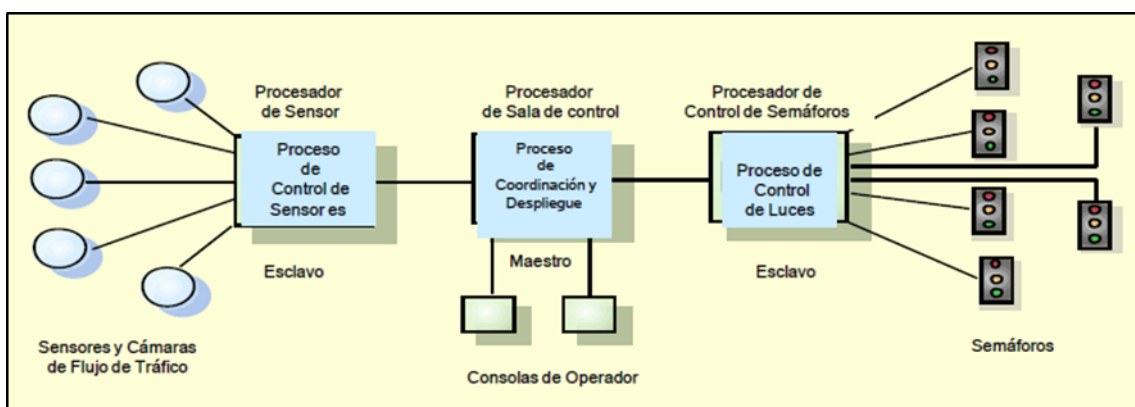
El modelo más simple de un sistema distribuido es un sistema multiprocesador en el que el sistema software está formado por varios procesos que pueden (aunque no necesariamente) ejecutarse sobre procesadores diferentes. Este modelo es común en sistemas grandes de tiempo real. El uso de múltiples procesadores mejora el rendimiento y adaptabilidad del sistema.

Se usan en sistemas de tiempo real donde es importante cumplir con los tiempos de procesamiento, donde puede haber procesadores separados para la adquisición de datos del entorno del sistema, procesamiento de datos y actuador de gestión.

El proceso "maestro" es el responsable de la computación, coordinación, comunicación y control de los procesos "esclavos". Los procesos "esclavos" se dedican a acciones específicas, como adquisición de datos de sensores.

Se usa cuando es posible predecir el procesamiento distribuido que se requiere y cuando se puede asignar fácilmente el procesamiento a los procesadores "esclavos". Esta situación es común en los sistemas de tiempo real.

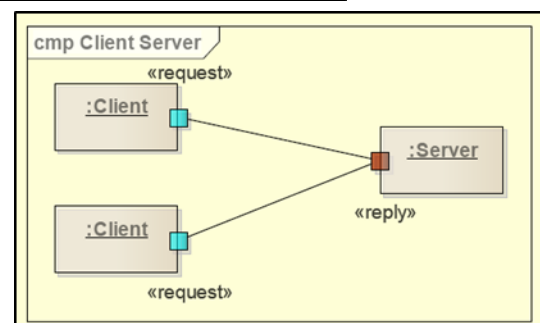
Ejemplo: sistema de control de tráfico. Un conjunto de sensores distribuidos recogen información sobre el flujo de tráfico y la procesan localmente antes de enviarla a una sala de control. Los operadores toman decisiones usando esta información y dan instrucciones a un proceso de control de diversas luces de tráfico.



2) Arquitecturas cliente-servidor

La aplicación es modelada como un conjunto de servicios que son provistos por servidores y un conjunto de clientes que usan esos servicios. Los clientes piden servicios a los servidores.

Los clientes conocen los servicios y deben saber cómo buscarlos, pero los servidores no necesitan conocer a los clientes.



Los que inician la comunicación son los clientes.

Los servidores no conocen la identidad de los clientes hasta que se han contactado.

Cientes y servidores son procesos lógicos diferentes.

El diseño de sistemas cliente-servidor debería reflejar la estructura lógica de la aplicación que se está desarrollando.

Los servidores se pueden replicar, esto es útil cuando la carga de un sistema es variable.

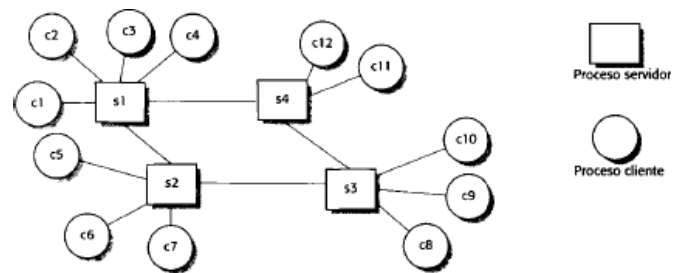


Figura 12.2 Un sistema cliente-servidor.

Si bien es una arquitectura distribuida, puede implementarse también en una sola computadora.

Es parte de la vista de ejecución (runtime). El uso más común es para explotar la capacidad gráfica de la PC y al mismo tiempo proteger los datos en un host central.

Los sistemas clientes-servidor dependen de que exista una clara separación entre la presentación de la información y los cálculos que crea y procesa esa información. En consecuencia, se debe diseñar la arquitectura de los sistemas distribuidos cliente-servidor para que se estructuren en varias capas lógicas, con interfaces claras entre dichas capas. Esto permite que cada capa se distribuya en diferentes computadoras. (Capa de presentación, capa lógica y capa de datos)

Variantes:

- Los conectores pueden ser sincrónicos o asincrónicos.
- Puede haber límites en el número de clientes o servidores.
- Las conexiones pueden ser con o sin estado (sesiones).
- Las topologías del sistema pueden ser estática o dinámica.
- Una variante permite al servidor, luego de que el cliente establezca contacto por primera vez, enviarle actualizaciones posteriores.
- Otra variante es el estilo N-Capas (N-Tier) que utiliza dos o más instancias del estilo Cliente-Servidor para formar una serie de capas.
 - Los pedidos fluyen en una única dirección.
 - Las capas tienen responsabilidades funcionales exclusivas
 - Este estilo es un híbrido entre la vista de ejecución y la de distribución, dado que las capas suelen (no siempre) asociarse a componentes de hardware distintos.

Calidad Resultante:

- Mejora la mantenibilidad: Soporta cambios en las reglas de negocio, cambiando el servidor, en lugar

de hacerlo en múltiples clientes.

- Control central ayuda a la evolución del sistema.
- Puede enmascarar sistemas legados y tratarlo como servidor.
- Este estilo es similar al de repositorio, solo que el de repositorio tiene la restricción de que los componentes vista y controlador no interactúan.
- En la práctica los clientes rara vez interactúan entre sí, pero el estilo no lo prohíbe.

Arquitectura Cliente Servidor y su distribución en el Hardware

Si a una arquitectura C/S de SW la quiero meter en un HW tengo que tener una red y tengo que ver cómo hago la distribución de los componentes de las distintas capas del software en los niveles de hardware. (Vista de despliegue) Recordando... Se le llama por convención "Capa" para el software y "Nivel" para el hardware.

para el hardware.

Hay varias formas de hacer el despliegue del SW en los niveles del HW según cómo fue evolucionando:

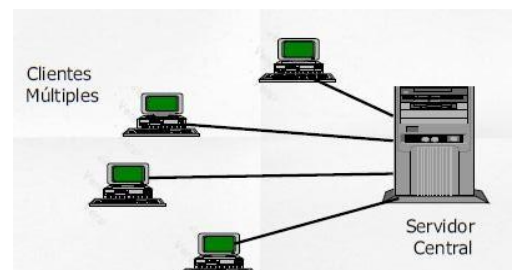
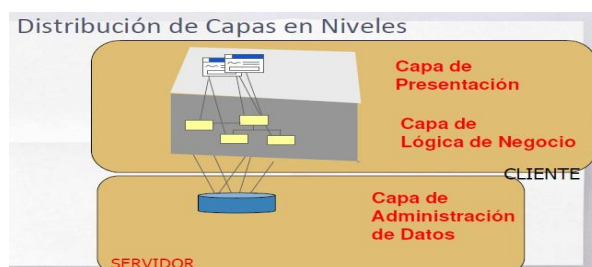
• **1) Arquitectura cliente-servidor de dos niveles**

Es la forma más simple de la arquitectura cliente servidor. El sistema se implementa en un solo servidor más un número indefinido de clientes que usan dicho servidor. Es decir, 1 sólo servidor central (servidor de base de datos con la capa de datos) y clientes múltiples (donde se implementa la capa de presentación y la capa de lógica de negocio). La motivación de esta primera forma de despliegue es aprovechar los recursos de las pc de los clientes.

- Tenemos dos niveles de HW:

1 nivel de cliente

1 nivel de servidor



Desventaja: las tres capas lógicas—presentación, procesamiento de la aplicación y gestión de los datos— deben asociarse con dos computadoras el cliente y el servidor. Aquí puede haber problemas con la escalabilidad y rendimiento. Esto se resuelve implementando una arquitectura C/S de 3 niveles.

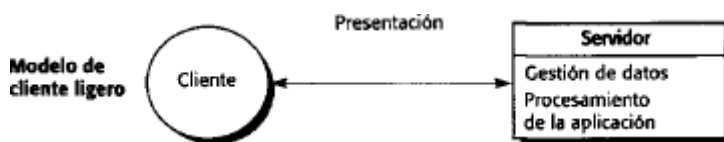
Cuando se distribuyen las capas de SW en los niveles de HW, tengo que decidir cómo se va a hacer esa distribución. Dependiendo de cuanta carga de procesamiento le asigne a los clientes puedo tener arquitecturas con cliente liviano o arquitecturas con cliente pesado.

Las arquitecturas cliente servidor de dos capas pueden ser de dos tipos:

Modelo cliente liviano/ ligero: todo el procesamiento de las aplicaciones y la gestión de los datos se lleva a cabo en el servidor. El cliente simplemente es responsable de la capa de presentación del software. Puede ser un programa de escritorio, pero es más común usar un navegador web. Utilizado cuando sistemas heredados centralizados son migrados a arquitecturas cliente servidor. El sistema heredado actúa como un servidor en sí mismo con una interfaz de usuario que se migra al cliente. También puede implementarse cuando los clientes son dispositivos de red sencillos en lugar de PCs. El dispositivo de red ejecuta un navegador de Internet y la interfaz de usuario es implementada a través de ese sistema.

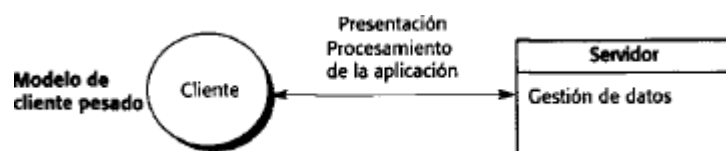
La ventaja del cliente ligero consiste en que es simple de manejar por los clientes. Puede ser costoso instalar y difícil instalar un software en cada uno de ellos si existe un número considerable de clientes. La solución es utilizar un navegador web, haciendo que no sea necesario instalar el software cliente. Además, implica que no haya exigencias en el HW del cliente. Esto facilita el control y despliegue.

Una gran desventaja del modelo de cliente ligero es que ubica una elevada carga de procesamiento tanto en el servidor como en la red. El servidor es responsable de todos los cálculos, y esto puede implicar la generación de un tráfico significativo en la red entre el cliente y el servidor.



Modelo cliente pesado: En este modelo, el servidor solamente es responsable de la gestión de los datos. El software del cliente implementa la lógica de la aplicación y las interacciones con el usuario del sistema. Se delega más procesamiento al cliente dado que el procesamiento de la aplicación es ejecutado localmente. Más adecuado para sistemas de C/S nuevos donde las capacidades del cliente son conocidas de ante mano. Utiliza el poder de procesamiento disponible en la máquina cliente. El servidor es esencialmente un servidor de transacciones que gestiona todas las transacciones de la base de datos. La ventaja es que se distribuye el procesamiento, aliviando la carga de procesamiento del servidor y de la red. La desventaja es que requiere gestión de sistema adicional para implementar y mantener el software en la computadora del cliente. Una nueva versión debe ser instalada en todos los clientes.

Un ejemplo de este tipo de arquitectura es la de los sistemas bancarios ATM, en donde cada ATM es un cliente y el servidor es un **mainframe** que procesa la cuenta del cliente en la base de datos. El hardware de los cajeros automáticos realiza una gran cantidad de procesamiento relacionado con el cliente y asociado a la transacción.



2) Arquitectura cliente-servidor de tres niveles

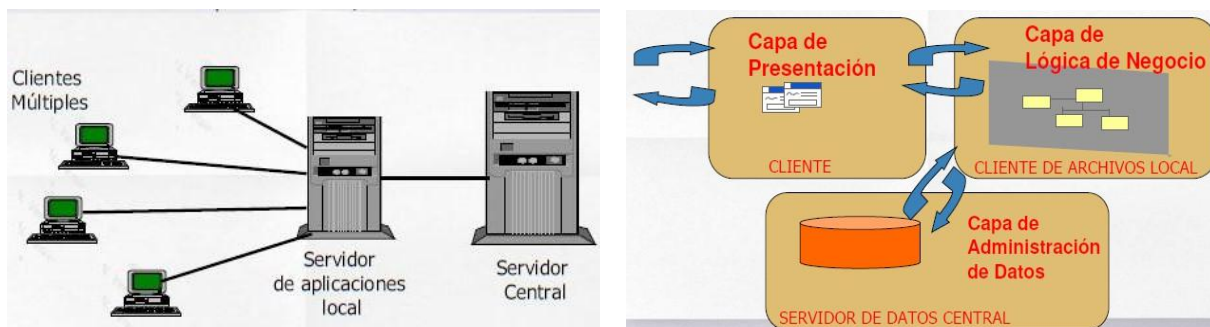
- Evoluciona porque crece la demanda de peticiones. En esta arquitectura, la presentación, la lógica de negocio y la gestión de los datos son procesos lógicamente separados que se ejecutan sobre procesadores diferentes.

Tenemos 3 niveles de HW:

1 nivel cliente (clientes múltiples) – en donde se ubica la capa de presentación

1 nivel de servidor de aplicaciones local – en donde se ubica la capa de lógica de negocio

1 nivel servidor central – es el servidor de BD, donde se ubica la capa de administración de datos.

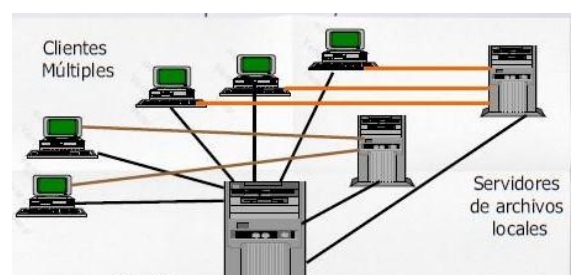


Un sistema bancario por Internet es un ejemplo de una arquitectura cliente / servidor de tres capas. La base de datos de clientes del banco (usualmente ubicada sobre una computadora mainframe) proporciona servicios de gestión de datos; un servidor web proporciona los servicios de aplicación tales como facilidades para transferir efectivo, generar estados de cuenta, pagar facturas, y así sucesivamente; y la propia computadora del usuario con un navegador de Internet es el cliente. El sistema es escalable debido a que es relativamente fácil añadir nuevos servidores web a medida que el número de clientes crece. El uso de una arquitectura de tres capas en este caso permite optimizar la transferencia de información entre el servidor web y el servidor de la base de datos. Las comunicaciones entre estos sistemas pueden usar protocolos de comunicación de bajo nivel muy rápidos.

3. Arquitectura Cliente Servidor de n niveles (o multinivel)

Los sistemas multinivel pueden usarse cuando las aplicaciones necesitan acceder y usar datos de diferentes bases de datos. En este caso, un servidor de integración se ubica entre el servidor de aplicaciones y los servidores de la base de datos. El servidor de integración recoge los datos distribuidos y los presenta a la aplicación como si se obtuviesen desde una única base de datos.

Los sistemas cliente-servidor multinivel que distribuyen el procesamiento de aplicación a través de varios servidores son inherentemente más escalables que las arquitecturas de dos niveles.



Arquitectura cliente-servidor N-Tier

Propiedades clave:

- Separación de intereses: capas diferentes y claramente divididas para presentación, negocios y manejo de datos.
- Comunicaciones sincrónicas: la comunicación entre niveles es pedido-respuesta sincrónica. Las peticiones emanan en una única dirección: del nivel de cliente a los niveles web y de la lógica de negocio al nivel EIS. Cada nivel espera la respuesta del otro nivel antes de proseguir.
- Distribución flexible: no hay restricciones para la distribución multi capas de la aplicación. En aplicaciones web, el cliente usualmente corre en un browser de una PC, comunicándose remotamente con internet con componentes a nivel web.

Evolución de las Arquitecturas Cliente/Servidor de Software

Por razones de seguridad e interoperabilidad, la computación distribuida ha sido principalmente implementada a nivel organizacional. Sin embargo, actualmente están disponibles modelos más recientes de computación distribuida que permiten computación distribuida interorganizacional en lugar de intraorganizacional. A continuación comentamos una de ellas.

Arquitectura Peer-to-Peer (P2P)

Los sistemas entre pares (peer-to-peer, p2p) son sistemas descentralizados en los que los cálculos pueden realizarse en cualquier nodo de la red. No se hacen distinciones entre clientes y servidores. En este tipo de sistemas, el sistema global está diseñado para sacar ventaja del poder computacional y de almacenamiento disponible en una red de computadoras potencialmente enorme. Aquí cada nodo, tiene la habilidad (no la obligación) de comportarse como cliente o como servidor. El resultado es un conjunto de nodos operando como pares donde cada nodo puede pedir proveer servicios a cualquier otro nodo de la red. Es parte de la vista de ejecución (runtime)

Calidad resultante:

- Disponibilidad promovida por el acceso a un archivo o parte de él desde cualquier nodo.
- Es altamente escalable y extensible.
- Aprovechan capacidad de procesamiento no utilizada de computadoras locales.

Desventajas:

- Muchos nodos pueden procesar la misma búsqueda.
- Excesiva carga de trabajo al replicar comunicaciones entre pares.
- Preocupaciones por la seguridad y la confianza.

Cuando se usa:

- Donde el sistema es de cómputo intensivo y es posible separar el procesamiento requerido en un gran número de cálculos independientes.
- Donde el sistema implica el intercambio de información entre computadoras individuales en una red y no hay necesidad de que esta información se almacene o gestione de manera centralizada.

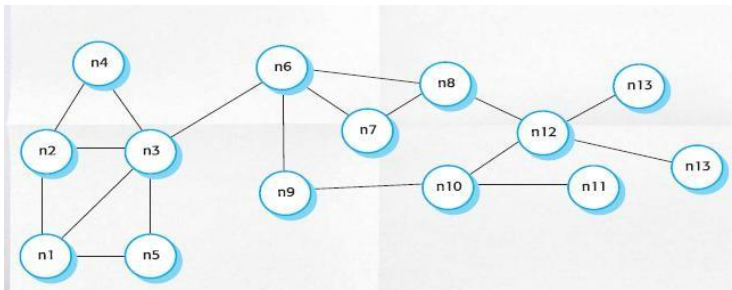
Usted puede ver la arquitectura de las aplicaciones p2p desde dos puntos de vista. La arquitectura lógica de la red es la distribución de la arquitectura del sistema, mientras que la arquitectura de la aplicación es la organización genérica de los componentes en cada tipo de aplicación. A continuación, vemos los dos tipos principales de arquitecturas lógicas de red que se pueden usar: arquitecturas descentralizadas y arquitecturas semicentralizadas.

Arquitectura Peer-To-Peer Descentralizada

- Los nodos en la red no son simplemente elementos funcionales, sino que también son interruptores de comunicaciones que pueden encaminar los datos y señales de control de un nodo a otro.
- Todos los componentes están en el mismo nivel jerárquico y ninguno controla a otro.
- Cada componente de software tiene un rol dual, puede actuar como cliente o como servidor.
- Surgen para atender muchas peticiones a la vez.

-Ventaja: es altamente redundante, y por lo tanto es tolerante a defectos y tolerante a nodos desconectados de la red. Un cliente hace una petición y, el primero que esté disponible o que la tome, atiende esa petición

-Desventaja: existen sobrecargas obvias en el sistema ya que la misma búsqueda puede ser procesada por muchos nodos diferentes y hay una sobrecarga significativa en comunicaciones entre iguales replicadas. Se disminuye la performance.

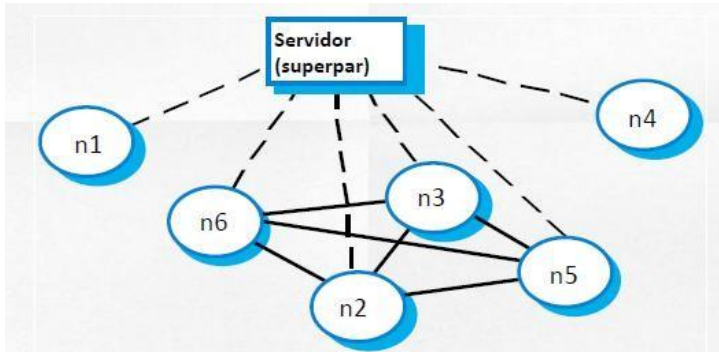


Arquitectura Peer-To-Peer Semi – centralizada

- Hay un nodo superpar (Servidor) y el papel que cumple es ayudar a establecer contacto entre iguales en la red o para coordinar los resultados de un cálculo, es decir, asigna las peticiones para evitar el procesamiento redundante.
- El superpar debe conocer a todos los otros nodos.
- Los nodos de la red se comunican con el servidor (indicado por líneas discontinuas), para encontrar

qué otros nodos están disponibles. Una vez que éstos son encontrados, se pueden establecer comunicaciones directas y la conexión con el servidor es innecesaria.

- **Desventaja:** el superpar puede saturarse y puede haber cuellos de botella en el mismo.



Arquitecturas de la vista de distribución – Arquitecturas de HW

1. Arquitectura Espejada (Mirrored).
2. Arquitectura Rack.
3. Arquitectura Granja de Servidores (Farm).

Arquitectura Espejada - Mirrored

- En este estilo se duplica hardware idéntico, que corre en paralelo.
- Generalmente opera en "lock-step", lo que duplica el esfuerzo, pero si uno falla el otro puede continuar por su cuenta. Realiza la misma operación varias veces en paralelo.
- El software puede actualizarse en forma separada uno de otro.
- La disponibilidad aumenta porque la probabilidad de que ambas fallen simultáneamente es baja.

Arquitectura Rack

- En este estilo los servidores se acomodan en pilas, para utilizar menos espacio del piso.
- Todas las computadoras de la pila se conectan con la misma red.
- La red en cambio puede tener varias conexiones a internet.
- La conexión a red para el rack es a menudo más rápida o la restricción de ancho de banda es menor, por lo que la comunicación entre computadoras de mismo rack es mucho más rápida.

Arquitectura granja de servidores (Farm)

- En este estilo muchas computadoras (usualmente idénticas) son ubicadas en la misma habitación (granja).
- La interconexión de las computadoras puede variar y la granja puede componerse de varios racks.
- Se las piensa como un recurso masivo que puede alojar cualquier aplicación.
- Las aplicaciones tienen restricciones para poder ejecutarse en una granja, como no tener estado.
- Es fácilmente escalable agregando más hardware del mismo tipo.

- Usos comunes para las granjas: capa de interfaz de usuario y capa intermedia de un sistema de tres capas donde cada granja aloja una capa diferente.

UNIDAD 4

- Propósito, estructura e importancia, rol en el ciclo de vida iterativo e incremental del PUD de los siguientes flujos de trabajo:
 - Flujo de Trabajo de Implementación
 - Flujo de Trabajo de Prueba
 - Flujo de Trabajo de Despliegue
 - ❖ Problemáticas y Técnicas del despliegue de software en función de las tecnologías existentes

Workflow de implementación

El workflow de diseño nos da los resultados que usaremos como entrada en el workflow de implementación, donde nos ocuparemos de implementar en código lo que diseñamos en la etapa anterior.

Los propósitos de este flujo de trabajo son:

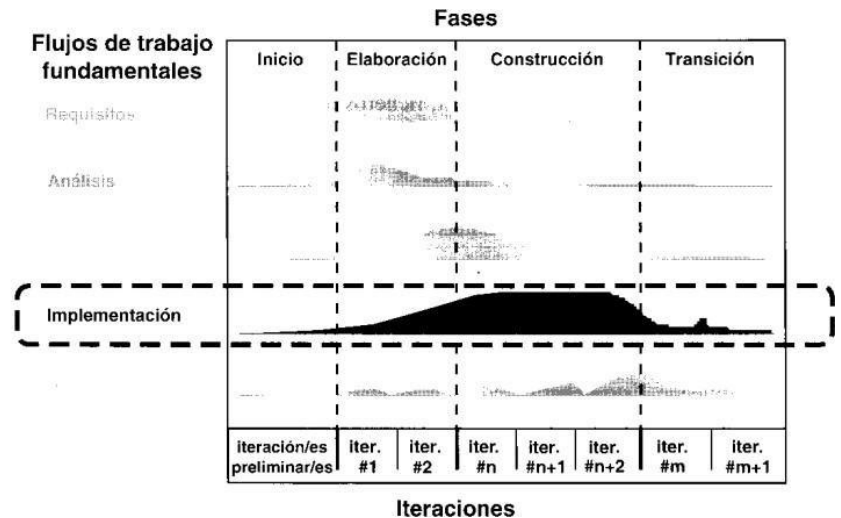
1. Planificar las integraciones de sistema necesarias en cada iteración, siguiendo un enfoque incremental
2. Distribuir el sistema asignando componentes a nodos en el diagrama de despliegue, basándonos sobre todo en las clases encontradas en el workflow anterior
3. Implementar clases y subsistemas encontrados en el workflow anterior
4. Probar los componentes de forma individual, luego compilarlos e integrarlos, enlazándolos en ejecutables que luego serán enviados a pruebas

La implementación es el centro en las iteraciones de la fase de construcción, aunque también podemos ver que se aplica al final de la fase de elaboración (sentando bases de la arquitectura) y al inicio de la fase de transición (para corregir errores que pueden surgir luego).

El resultado principal de la implementación es el modelo de implementación.

El papel de la implementación en el ciclo de vida del software

La implementación es el centro durante las iteraciones de construcción, aunque también se lleva a cabo trabajo de implementación durante la fase de elaboración, para crear la línea base ejecutable de la arquitectura, y durante la fase de transición, para tratar defectos tardíos.

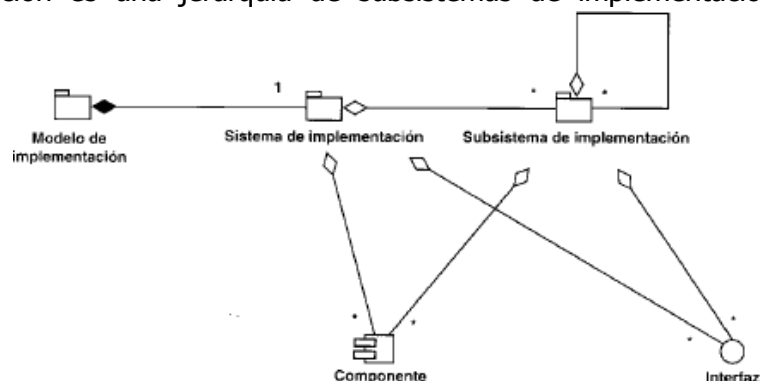


Artefactos (6)

1. Modelo de implementación:

Describe cómo los elementos del modelo de diseño se implementan en términos de componentes, y cómo estos se organizan de acuerdo a las tecnologías usadas y sus relaciones. como ficheros de código fuente, ejecutables, etc. Describe también cómo se organizan los componentes de acuerdo con los mecanismos de estructuración y modularización disponibles en el entorno de implementación y en el lenguaje o lenguajes de programación utilizados, y cómo dependen los componentes unos de otros. El modelo de implementación se representa con un sistema de implementación que denota el subsistema de nivel superior del modelo.

El modelo de implementación es una jerarquía de subsistemas de implementación que contiene componentes e interfaces.



2. Componente:

Es el Empaquetamiento físico de los elementos de un modelo, por ejemplo, las clases en el modelo de diseño, tienen trazabilidad con los elementos de modelo que implementan y también deben implementar la misma interfaz que estos, y pueden implementar más de un elemento. Puede haber dependencias de compilación entre componentes.

Algunos estereotipos estándar son:

- <<executable>> es un programa que puede ser ejecutado en un nodo.

- <<file>> es un fichero que contiene código fuente o datos.
- <<library>> es una librería estática o dinámica.
- <<table>> es una tabla de base de datos.
- <<document>> es un documento.

Durante la creación de los componentes en un entorno de implementación particular, los estereotipos pueden cambiar para reflejar el significado real de estos componentes.

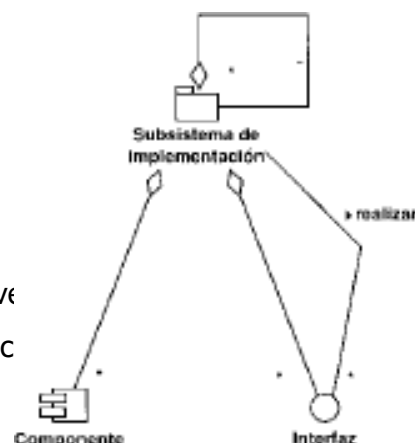
Características de los componentes:

- Tienen relaciones de traza con los elementos del modelo que implementan.
- Es normal que un componente implemente varios elementos, por ejemplo, varias clases. La traza dependerá de cómo van a ser estructurados y modularizados los ficheros de código fuente, dado el lenguaje de programación que se esté usando.
- Los componentes proporcionan las mismas interfaces que los elementos del modelo que implementan
- Puede haber dependencias de compilación entre componentes.

3. Subsistema de implementación:

Como los subsistemas en el modelo de diseño (con los cuales deberían mantener una traza reconocible), estos proporcionan una forma de estructurar los artefactos del workflow en piezas más manejables. Puede estar formado por componentes, otros subsistemas e interfaces. El mecanismo de empaquetamiento que “guarda” estos subsistemas dependen de la tecnología que se está usando. Los subsistemas de implementación están muy relacionados con los subsistemas de diseño en el modelo de diseño. Los subsistemas de implementación deberían seguir la traza uno a uno de sus subsistemas de diseño correspondientes:

- Definir dependencias análogas a las dependencias del subsistema de diseño con el que tiene traza
- Proporcionar las mismas interfaces que este subsistema de diseño
- Definir componentes y otros subsistemas contenidos, los cuales deben tener traza con las clases de diseño que representan.



Un subsistema de implementación se manifiesta a través de un entorno de implementación determinado, como:

- Un paquete en Java.
- Un proyecto en Visual Basic.

“empaquetamiento”

- Un directorio de ficheros en un proyecto de C++.
- Un paquete en una herramienta de modelado como Rational Rose.
- Un subsistema en un entorno de desarrollo integrado como Rational Apex.

La semántica de la noción de subsistema de implementación se refinará cuando se manifieste en un entorno de implementación determinado.

4. Interfaz:

Un componente puede implementar una interfaz, las cuales pueden ser usadas en este modelo para especificar las operaciones implementadas por esos componentes y otros subsistemas. El componente que implemente la interfaz (y por tanto proporciona) ha de implementar correctamente todas las operaciones definidas por la interfaz, y un subsistema que la implemente debe contener componentes que la implementen.

5. Descripción de la arquitectura (vista del modelo de implementación): Contiene una **visión del modelo de implementación**, el cual representa sus artefactos significativos arquitectónicamente. Muestra los artefactos clave del modelo de implementación, los cuales son generalmente los componentes clave (los que tengan traza directa con el diseño, sean ejecutables o muy generales) y la descomposición del modelo, interfaces y dependencias.

Artefactos considerados significativos para la arquitectura en el modelo de implementación:

- La descomposición del modelo en subsistemas, sus interfaces y las dependencias entre ellos.
- Componentes clave, como los componentes que siguen la traza de las clases de diseño significativas arquitectónicamente, ejecutables y los componentes que son generales, centrales, que implementan mecanismos de diseño genéricos de los que dependen muchos otros componentes.

6. Plan de integración de construcciones:

Al construir el software, nos sirve hacerlo de a pasos, "construcciones" que son piezas ejecutables del sistema, más pequeñas y manejables. Siguiendo un enfoque incremental, podemos realizar pruebas de integración sobre esas construcciones antes de que se cree otra construcción.

Beneficios de este enfoque:

- ❖ Podemos crear una versión ejecutable del sistema muy pronto que sirva como demostración
- ❖ Es más fácil encontrar defectos durante estas pruebas ya que el defecto estará en la nueva pequeña construcción agregada

- ❖ Suelen ser pruebas minuciosas al ser piezas pequeñas y manejables

La integración incremental se centra en un incremento pequeño y manejable de la funcionalidad. Cada iteración resultará en, al menos, una construcción ("al menos" porque la funcionalidad completa suele ser compleja como para que implique una construcción). Un plan de integración de construcciones describe la secuencia de construcciones necesarias en una iteración.

Un **plan de integración de construcciones** describe la secuencia de construcciones necesarias en una iteración.

Describe lo siguiente para cada construcción:

- La funcionalidad que se espera que sea implementada en dicha construcción. Consiste en una lista de casos de uso o escenarios o parte de ellos.
- Las partes del modelo de implementación que están afectadas por la construcción. Es una lista de los subsistemas y componentes necesarios para implementar la funcionalidad supuesta por la construcción.

Trabajadores (3)

1. Arquitecto

Responsable de la integridad del modelo de implementación y de que el mismo es un todo correcto, complejo y legible; esto es, que el modelo implementa la funcionalidad descrita en el modelo de diseño (y requisitos adicionales) y sólo esta funcionalidad. También es responsable de la arquitectura del modelo de implementación, y de la asignación de componentes ejecutables a nodos.

2. Ingeniero de componentes

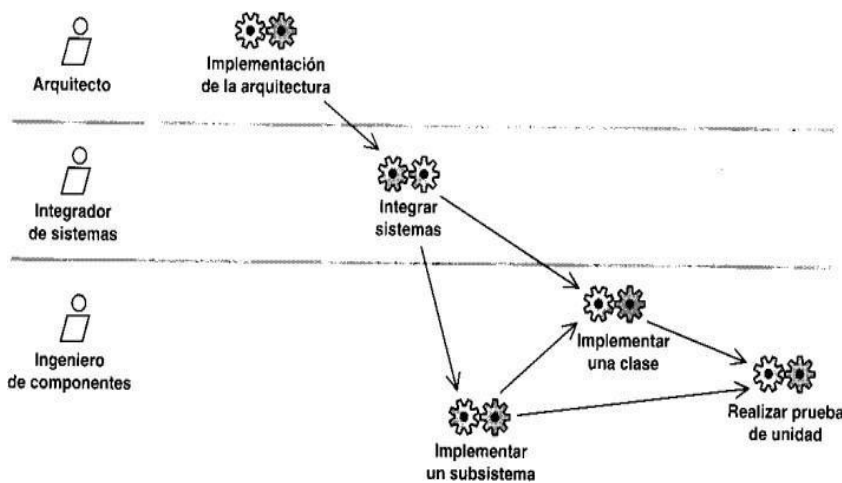
Define y mantiene el código fuente de los componentes, garantizando que cumplen con la funcionalidad correcta. Puede mantener la integridad de uno o varios subsistemas de implementación. Ya que los subsistemas de implementación siguen la traza uno a uno a los subsistemas de diseño necesita garantizar que los contenidos de los subsistemas de implementación son correctos, que sus dependencias con otros subsistemas o interfaces son correctas y que implementan correctamente las interfaces que proporciona.

3. Integrador de sistemas

Se encarga de planificar la secuencia de construcciones necesarias en cada iteración y la integración de cada construcción al terminar de implementarlas.

Flujo de trabajo

El objetivo principal de este workflow es implementar el sistema. Esto lo inicia el arquitecto, esbozando los componentes clave en el modelo, y luego el integrador de sistemas planea las integraciones necesarias en la iteración como una secuencia de construcciones. Él describe la funcionalidad a implementar y las partes del modelo que se verán afectadas. Los requisitos sobre el subsistema y componentes a construir son implementados por los ingenieros de componentes, y luego estos componentes son probados y enviados al integrador de sistemas para su integración. El mismo integra los componentes, y los pasan a los ingenieros de pruebas de integración. Luego, los desarrolladores inician la implementación de la siguiente construcción.

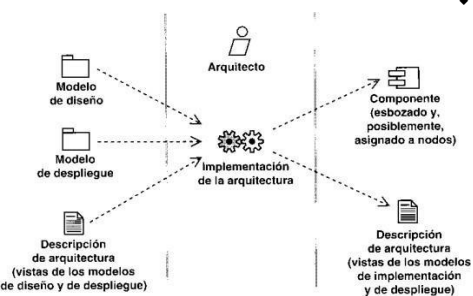


Actividades

1. Implementación de la arquitectura (Arq)

El propósito es esbozar el modelo de implementación y su arquitectura mediante las siguientes subactividades:

- ❖ Identificación de componentes significativos a la arquitectura
 - Hay varios componentes, como los de ficheros, que son triviales de crear porque son una forma de empaquetar por esto, solo basta con realizar un esbozo inicial de los componentes arquitectónicos dado que identificar muchos componentes y ahondar en detalles puede significar rehacer gran parte del trabajo cuando se implementen las clases.
- ❖ La asignación de componentes ejecutables a los nodos en las configuraciones de redes relevantes:



La identificación de los componentes ejecutables se consideran las clases activas encontradas durante el diseño y asignamos un componente ejecutable por cada clase activa. Esto debería representarse en la vista de la arquitectura del modelo de despliegue.

Usaremos subsistemas de implementación que siguen la traza con los subsistemas de diseño y que proporcionan las mismas interfaces. El reto está en crear, dentro de los subsistemas de implementación,

componentes que implementen los subsistemas de diseño correspondientes.

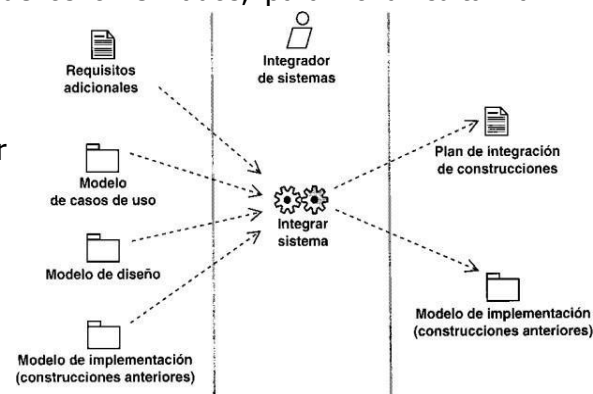
2. Integrar el sistema (Intg de Sist)

1. **Creación de un plan** de integración de construcciones que describa las construcciones necesarias en una iteración y los requisitos de cada una.

Planificación de una construcción

Como planificar los contenidos de una construcción, independientemente de que partamos de una construcción previa o de no partamos de ninguna. Suponemos que tenemos un número de casos de uso o escenarios y otros requerimientos que han de ser implementados en la iteración actual.

- Una construcción debería añadir funcionalidad implementando casos de uso, y las pruebas de integración estarán basadas en estos casos de uso
- No deberían incluir demasiados componentes nuevos o refinados, para no dificultar la integración
- Deberían basarse en la construcción anterior y expandirse hacia arriba (primero debemos sentar una buena base y luego expandirnos)



Pasos a seguir:

- a. Tomar la realización de caso de uso de diseño a implementar
- b. Identificar subsistemas y clases de diseño involucradas
- c. Identificar subsistemas y componentes de implementación a tener en cuenta
- d. Considerar el impacto de la implementación sobre la construcción actual y la anterior.

Los resultados deben estar descritos en el plan de integración de la construcción.

2. Integración de cada construcción antes de someterlas a pruebas de integración

Si la construcción fue planificada cuidadosamente, debería ser fácil integrar la construcción.

Esto se hace recopilando las versiones correctas de los subsistemas de implementación y de los componentes compilándolos y enlazándolos para generar una construcción.

3. Implementar un subsistema (Ing Comp)

Realizaremos esta implementación para asegurarnos de que el subsistema y sus componentes implementa correctamente la funcionalidad o requisito implementado en la construcción

Mantenimiento de los contenidos de los subsistemas

Si el contenido de un subsistema es esbozado por los arquitectos, de igual forma puede necesitar ser refinado por el ingeniero de componentes conforme evoluciona el modelo de implementación.

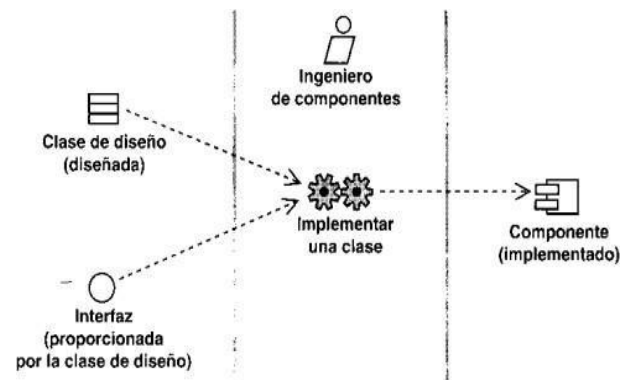
Dicho refinamiento puede incluir:

- Cada clase en el correspondiente subsistema de diseño que es necesaria en la construcción actual debería ser implementada mediante componentes en el subsistema de implementación-
- Cada interfaz proporcionada por el subsistema de diseño correspondiente requerido en la construcción actual también debería ser proporcionada por el subsistema de implementación.

4. Implementar una clase (Ing Comp)

Buscamos implementar una clase de diseño en un componente fichero

- Esbozo de un componente fichero que contendrá el código fuente
 - El código fuente que implementa una clase de diseño reside en un componente fichero
 - Los componentes ficheros elegidos deberían facilitar la compilación, instalación y mantenimiento del sistema.
- Generación de código a partir de una clase de diseño
 - Sólo se puede generar una parte del código fuente que implementa una clase a partir de los detalles que tiene esa clase de diseño. Pero, esto solo se cumple para las operaciones y atributos de la clase así como para las relaciones en las que la clase participa. Sin embargo, sólo se genera la signatura de las operaciones (el nombre del método), las operaciones en sí deben ser implementadas.
- Implementación de operaciones (métodos)
 - Cada operación definida por la clase de diseño tiene que ser implementada, a no ser que sea virtual (o abstracta) y ésta sea implementada por descendientes (como subtipos) de la clase
- Implementación de las interfaces adecuadas (las mismas que la clase de diseño que implementa)
 - El componente resultante debería proporcionar las mismas interfaces que las clases de diseño que éste implementa.



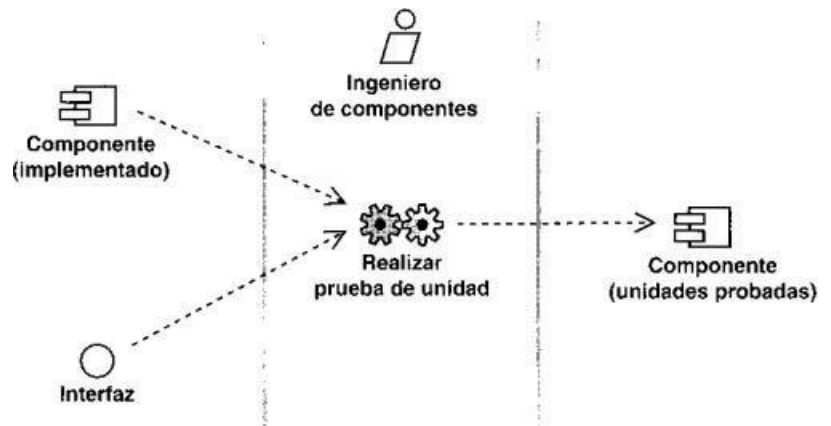
5. Realizar prueba de unidad (Ing Comp)

Busca probar los componentes implementados como unidades individuales, mediante:

- **Pruebas de caja negra** (especificación, prueba su comportamiento sin mirar su estructura (Sin tener en cuenta el cómo lo hace), entra concepto de clases de equivalencia, toene4 en cuenta la salida que devuelve el componente ante una determinada entrada)
- **Pruebas de caja blanca** (verifica la implementación interna de la unidad. Se fija en el código. Se

realizan para verificar que un componente funciona internamente como se quería., entra concepto de camino crítico)

Además, puede ser necesario llevar a cabo otras pruebas tales como pruebas de rendimiento, memoria, carga, capacidad como así también es necesario realizar pruebas de integración y de sistema.



RESUMEN DE LA IMPLEMENTACIÓN

El resultado principal de la implementación es el modelo de implementación, el cual incluye los siguientes elementos:

- Subsistemas de implementación y sus dependencias, interfaces y contenidos.
- Componentes, incluyendo componentes fichero y ejecutables, y las dependencias entre ellos. Los componentes son sometidos a pruebas de unidad.
- La vista de arquitectura del modelo de implementación, incluyendo sus elementos arquitectónicamente significativos.

La implementación también produce como resultado un refinamiento de la vista de arquitectura del modelo de despliegue, donde los componentes ejecutables son asignados a nodos.

El modelo de implementación es la entrada principal de la etapa de prueba que le sigue a la implementación.

Workflow de prueba

En este flujo de trabajo verificamos el resultado de la implementación probando cada construcción, incluyendo tanto construcciones internas como intermedias, así como las versiones finales del sistema a ser entregadas a los clientes.

Propósito

- Planificar las pruebas necesarias en cada iteración, incluyendo pruebas de integración (en cada construcción) y de sistema (al final de la iteración)
- Diseñar e implementar las pruebas creando los casos de prueba (Especifican que probar, como hacer las pruebas y creando componentes de prueba ejecutables para automatizar las mismas)
- Realizar pruebas y manejar los resultados, esto es, aprobar o reenviar a otro workflow aquel artefacto con defectos como diseño o implementación para que los defectos importantes puedan ser corregidos.

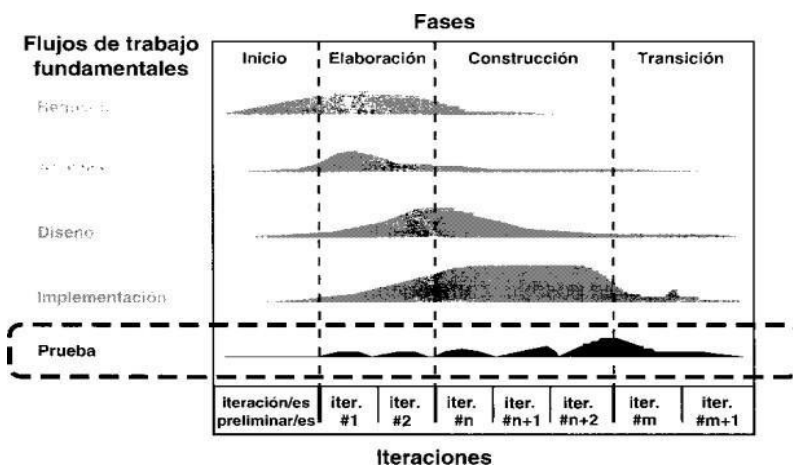
Podemos encontrar parte de la planificación inicial de las pruebas en la fase de inicio, pero las pruebas se llevan a cabo finalmente en la fase de elaboración (cuando se prueba la línea base ejecutable de la arquitectura) y de construcción (cuando el grueso del sistema está implementado), y en la fase de transición (cuando se corrigen defectos y se hacen pruebas de regresión). Es buena idea mantener el modelo de pruebas a lo largo de todo el desarrollo, ya que las pruebas necesarias no suelen cambiar tanto, excepto debido a eliminación de casos de prueba obsoletos, refinación o creación de los mismos.

El papel de la prueba en el ciclo de vida del software

Durante la fase de inicio, se incluye la planificación inicial de las pruebas al definir el alcance del sistema. La realización de pruebas se centra en las fases de elaboración, donde se prueba la línea base ejecutable de la arquitectura, y de construcción, cuando se implementa la mayor parte del sistema. En la fase de transición, el enfoque se desplaza hacia la corrección de defectos durante los primeros usos y las pruebas de regresión.

Debido a la naturaleza iterativa del desarrollo, algunos casos de prueba de las primeras construcciones pueden utilizarse como pruebas de regresión para las construcciones siguientes. El número de pruebas de regresión aumenta de manera constante a lo largo de las iteraciones, lo que implica un esfuerzo significativo en las últimas iteraciones.

El modelo de pruebas se mantiene a lo largo de todo el ciclo de vida del software, adaptándose mediante la eliminación de casos de prueba obsoletos, el refinamiento de algunos casos para pruebas de regresión y la creación de nuevos casos de uso para cada nueva construcción.



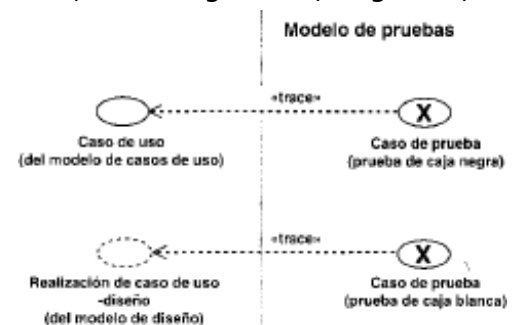
Describe cómo se prueban los componentes ejecutables en el modelo de implementación con pruebas de integración y de sistema, y otras pruebas específicas. Puede describir también como han de ser probados aspectos específicos del sistema. Es una colección de casos de prueba, procedimientos y componentes de prueba. Si el modelo es muy grande, podríamos subdividirlo en paquetes.

2. Caso de prueba

- Especifica una forma de probar el sistema, incluyendo la entrada y condiciones bajo las cuales probar. Normalmente, un caso de prueba puede especificar cómo probar un caso

de uso o un escenario del mismo, verificando resultados esperados, teniendo en cuenta pre y post condiciones y la secuencia de pasos establecidos (**prueba caja negra**), prueba el comportamiento observable.

- b. Un caso de prueba puede especificar cómo probar una realización de caso de uso de diseño o un escenario de la realización, Se prueba interacción interna entre los componentes (prueba caja blanca).
- c. El otro grupo de casos de prueba que no entran en los dos anteriores, los casos de prueba específicos, pueden ser pruebas de instalación, de configuración, negativas, de estrés.



Se pueden especificar otros casos de prueba para probar el sistema como un todo. Por ejemplo:

- **Pruebas de instalación:** verifican que el sistema puede ser instalado en la plataforma del cliente y que funcionará correctamente cuando sea instalado.
- **Pruebas de configuración:** verifican que el sistema funciona correctamente en diferentes configuraciones, ej: configuraciones de red.
- **Pruebas negativas:** intentan provocar que el sistema falle para poder así revelar sus debilidades. Consiste en utilizar el sistema en formas para los que no sido diseñado ("usar mal el sistema a propósito").
- **Pruebas de tensión o de estrés:** identifican problemas con el sistema cuando hay recursos insuficientes o cuando hay competencia por recursos.

3. Procedimiento de prueba

Especifica cómo realizar casos de prueba. Se puede reutilizar **un** procedimiento de prueba **para varios casos de prueba** y reutilizar **varios procedimientos** de pruebas **para un caso** de prueba

Por ejemplo, un procedimiento de prueba puede ser una instrucción para una persona sobre cómo realizar un caso de prueba manualmente o puede ser una especificación de cómo interaccionar con una herramienta de automatización de pruebas para crear componentes ejecutables de prueba.

4. Componente de prueba

Automatiza procedimientos de prueba, se usan para probar los componentes del modelo de implementación proporcionando todo lo necesario para que la prueba se ejecute e informando los resultados. Pueden ser automáticos.

5. Plan de prueba

Describe estrategias, recursos y planificación de la prueba. La estrategia incluye la definición del tipo de pruebas a realizar en cada iteración, sus objetivos, el nivel de cobertura de prueba y de código necesario y el porcentaje de pruebas que deberían ejecutarse con un resultado específico.

6. Defecto

Anomalía del sistema que necesitamos resolver, asumimos que existe. Puede ser utilizado para localizar cualquier cosa que los desarrolladores necesitan registrar como síntoma de un problema en el sistema que ellos necesitan controlar y resolver.

7. Evaluación de prueba

Se evalúan los resultados de los esfuerzos de prueba, tales como la cobertura del caso de prueba, la cobertura del código y el estado de los defectos.

Trabajadores (4)

1. Diseñador de pruebas.
2. Ingeniero de componentes.
3. Ingeniero de pruebas de integración.
4. Ingeniero de pruebas de sistema.

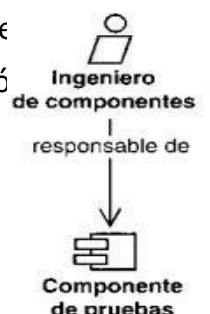


1. Diseñador de pruebas

Responsable de la integridad del modelo, asegurando que el mismo cumple su propósito. También planean las pruebas, seleccionan y describen los casos de prueba y sus procedimientos correspondientes, y son responsables de la evaluación de las pruebas de integración. Sólo se dedican a la preparación y evaluación de las pruebas, no las llevan a cabo.

2. Ingeniero de componentes

Responsables de los componentes de prueba que automatizan algunos procedimientos de prueba, es sólo algunos porque no todos los procedimientos de prueba pueden automatizarse.



3. Ingeniero de pruebas de integración

Responsables de realizar las pruebas de integración, las cuales se realizan para verificar que los componentes integrados en una construcción funcionan bien juntos. También se encargan de documentar los defectos encontrados. Prueba los resultados, es decir, una construcción creada por el integrador de sistemas en el workflow de implementación. Por eso, en muchos proyectos estos trabajadores suelen ser la misma persona o grupo.

4. Ingeniero de pruebas de sistema

Responsable de realizar las pruebas de sistema, que se realizan

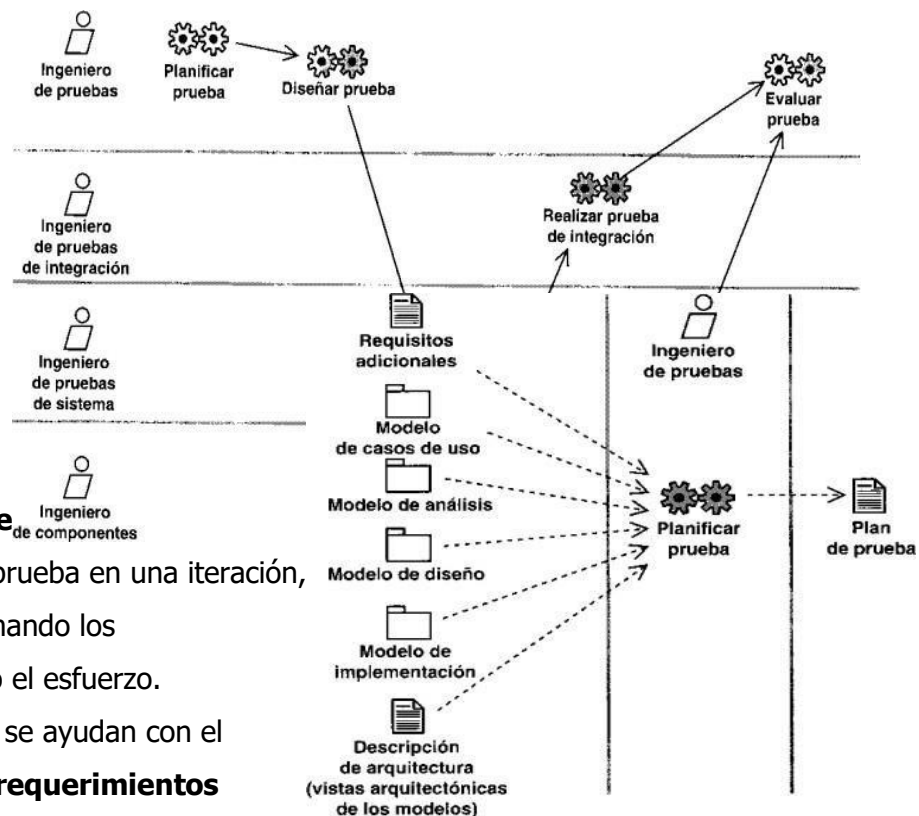


para verificar interacciones entre actores y sistema. También se encargan de documentar los defectos encontrados. Deben tener conocimiento del funcionamiento interno del sistema ya que prueban de forma general una construcción.

Flujo de trabajo

El principal objetivo de la prueba es realizar y evaluar las pruebas como se describe en el modelo de pruebas.

- Los ingenieros de pruebas inician planificando el esfuerzo de prueba en cada iteración, describiendo los casos de prueba y procedimientos necesarios.
- Los ingenieros de componentes crean los componentes de prueba para automatizar (si es posible)
- Los ingenieros de pruebas de integración y de sistema prueban cada construcción y detectan defectos. Los defectos sirven como retroalimentación tanto para otros flujos de trabajo, como el diseño y el de implementación



Actividades

1. Planificar prueba (Ing. de Prue

Busca planificar los esfuerzos de prueba en una iteración, describiendo una estrategia, estimando los recursos necesarios y planificando el esfuerzo.

Cuando arman el plan de prueba, se ayudan con el

modelo de casos de uso y los requerimientos

adicionales porque permiten decidir por un tipo adecuado de pruebas y a estimar el esfuerzo necesario. Los diseñadores de pruebas desarrollan una estrategia de prueba para la iteración: deciden **qué pruebas ejecutar, cuándo y cómo ejecutarlas y cómo determinar si el esfuerzo de prueba tiene éxito.** El principio general se basa en desarrollar casos y procedimientos de prueba con solapamiento mínimo para probar los casos de uso más importantes y probar los requerimientos que están asociados a los riesgos más altos.

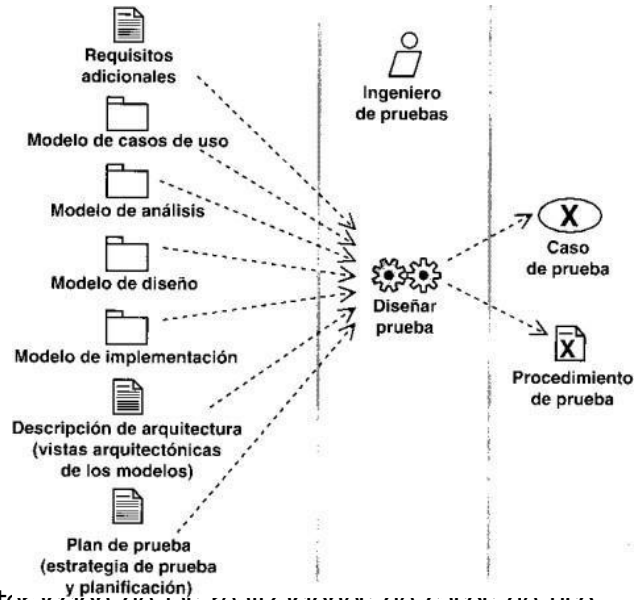
2. Diseñar una prueba (Ing. de Pruebas)

- **Identificar y describir los casos de prueba para cada construcción**, para verificar que los componentes interactúan de la forma correcta luego de haber sido integrados y que funcionan correctamente como un todo.
- **Identificar y estructurar los procedimientos de prueba**

Diseño de los casos de prueba de integración

Los casos de prueba de integración permiten verificar la interacción correcta de los componentes. La mayoría de los casos de prueba de integración pueden ser derivados de las realizaciones de casos de uso de diseño, que describen cómo interaccionan las clases y los objetos, y por tanto cómo interaccionan los componentes. Se crean un conjunto de casos de prueba que hicieran posible alcanzar los objetivos establecidos en el plan de prueba con esfuerzo

mínimo. Consideran como entrada los diagramas de interacción de las realizaciones de casos de uso. Buscan combinaciones de entrada, salida y estado inicial que den lugar a escenarios interesantes que empleen las clases y componentes que participan en los diagramas. Al realizar la prueba de integración, se compara la interacción actual con la del diagrama de interacciones, las cuales deberían ser iguales, en otro caso se trata de un defecto.



Diseño de los casos de prueba de sistema

Busca asegurar el correcto funcionamiento global del sistema, probando combinaciones de casos de uso en diversas condiciones (diferentes configuraciones de hardware, diferentes cargas del sistema, diferentes números de actores y diferentes tamaños de la base de datos). Se priorizan combinaciones que puedan **ejecutarse en paralelo, posiblemente influenciarse entre sí al ejecutarse en paralelo, involucrar múltiples procesadores y utilizar recursos del sistema.**

Diseño de los casos de prueba de regresión

Se centra en la reutilización de casos de prueba anteriores para pruebas en construcciones subsiguientes. Para ser adecuados y contribuir a la calidad del sistema, los casos de prueba deben ser flexibles a cambios en el software que se está probando. Esta flexibilidad puede requerir esfuerzo extra de desarrollo por lo tanto tenemos que convertir en casos de prueba de regresión sólo aquellos que valgan la pena.

Identificación y estructuración de los procedimientos de prueba

Se intenta reutilizar procedimientos de prueba existentes tanto como sea posible, lo que significa que

se puede llegar a necesitar modificarlos para especificar cómo realizar un caso de prueba nuevo o cambiado.

Los diseñadores de pruebas también intentan crear procedimientos de prueba que puedan ser reutilizados en varios casos de prueba.

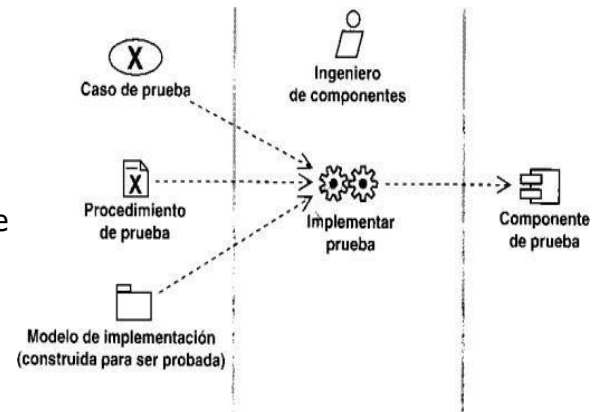
3. Implementar una prueba (Ing comp)

Busca automatizar los procedimientos de prueba creando componentes de prueba siempre que sea posible. Estos se crean usando los procedimientos de prueba como entrada

- Cuando usamos una herramienta de automatización de pruebas especificamos

las acciones como se describe en el procedimiento de prueba. Esto se guarda como un componente de prueba, por ejemplo, un script.

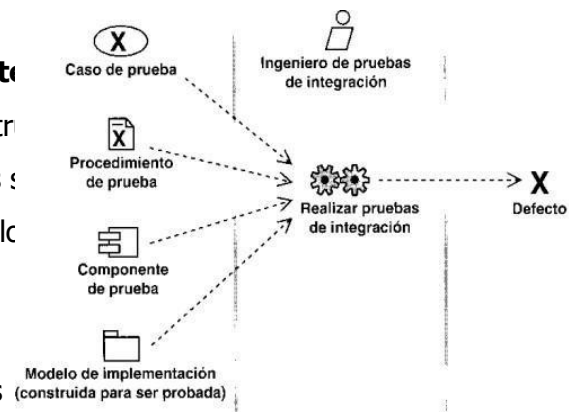
- Cuando programamos los componentes de prueba efectivamente se usan los procedimientos de prueba como especificaciones iniciales. Esto puede requerir personal de programación.



4. Realizar pruebas de integración (Ing. de pruebas de integración)

Se realizan las pruebas de integración necesarias para cada construcción. Los resultados. Las pruebas de integración se llevan a cabo en los siguientes pasos:

1. Realizar las pruebas de integración relevantes realizando lo necesario o ejecutando un componente de prueba que automatice esos procedimientos.
2. Comparar los resultados de las pruebas con los resultados esperados e investigar los que no coinciden.
3. Informar de los defectos a los ingenieros de componentes responsables de los componentes que tienen fallos.
4. Informar de los defectos a los diseñadores de pruebas, quienes usarán los defectos para evaluar los resultados del esfuerzo de prueba.



5. Realizar pruebas de sistema (Ing. de pruebas de sistema)

Se realizan las pruebas de sistema necesarias en cada iteración y se recopilan los resultados. Se inician cuando las pruebas de integración terminan e indican un nivel de satisfacción suficiente. Los pasos a seguir son los mismos que en las pruebas de integración

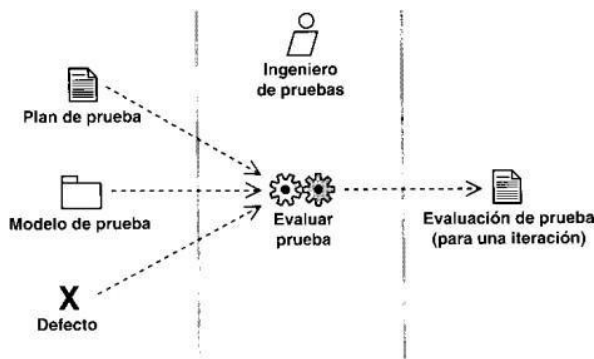
6. Evaluar una prueba (Ing. de pruebas)

Busca evaluar los esfuerzos de prueba de una iteración. Se evalúan comparando los resultados obtenidos con los objetivos esbozados en el plan, lo que nos da métricas para evaluar la calidad del

software. Las métricas son la compleción o aprobación de la prueba y la fiabilidad. Frente al análisis de fiabilidad, si vemos una tendencia alta de defectos, el diseñador puede elegir realizar pruebas adicionales, relajar el criterio de pruebas o aislar y entregar las partes del sistema con calidad aceptable.

En concreto, el ingeniero de pruebas observa dos métricas:

- **Compleción de la prueba:** obtenida a partir de la cobertura de casos de prueba y de la cobertura de los componentes probados. Indica el porcentaje de casos de prueba que han sido ejecutados y el porcentaje de código que ha sido probado.
- **Fiabilidad:** la cual se basa en el análisis de las tendencias en los defectos detectados y en las tendencias en las pruebas que se ejecutan como resultado esperado.



Workflow de despliegue

El propósito de este flujo de trabajo es volcar el producto finalizado a sus usuarios, presentar el software:

- Probar el software en el entorno donde va a operar
- Empaquetar, distribuir e instalar el software
- Capacitar usuarios
- Migrar el software existente o convertir bases de datos.

EL ROL DEL DESPLIEGUE EN EL CICLO DE VIDA ITERATIVO E INCREMENTAL DE RUP

El centro de atención del despliegue se centra en la fase de transición. Durante la fase de transición, el usuario final puede probar el sistema para saber cómo trabaja en un ambiente real de trabajo. El objetivo es lograr una transición sin problemas de un usuario autosuficiente al nuevo software. El éxito de un proyecto de desarrollo es la voluntad del cliente de usar el producto terminado más que una compilación limpia.

Artefactos (1)

Release

Consiste en software ejecutable, artefactos de instalación (físicos y lógicos), notas de la release, material de soporte y de capacitación. Podemos entregar otros artefactos al cliente, no necesarios, tales como

resultados de pruebas, de retroalimentación, etc.

Trabajadores (7)

1. Gerente de despliegue

Planifica y organiza el despliegue, es el responsable de la retroalimentación de las pruebas beta, y de asegurar que el producto es empaquetado apropiadamente para su envío

2. Gerente de proyecto

Intermediario con los clientes, responsable de aprobar el despliegue según la retroalimentación y las evaluaciones de las pruebas y la aceptación del cliente

3. Escritor técnico

Planea y produce material de soporte para usuario final

4. Desarrollador de cursos

Planea y produce material de capacitación

5. Artista gráfico

Responsable del material artístico del producto

6. Tester

Ejecuta pruebas de aceptación, es responsable de asegurar que el producto se probó adecuadamente

7. Implementador

Crea los scripts de instalación y lo necesario para la instalación

Flujo de trabajo (actividades)

1. Plan de despliegue.
2. Desarrollar material de soporte.
3. Probar el producto en el lugar de desarrollo.
4. Crear la release (producto final) – Crear una versión.
5. Lanzamiento de prueba beta.
6. Probar el producto en el lugar de instalación.
7. Proveer acceso a sitio de descarga (opc.)

Actividades

Plan de despliegue

El planeamiento del despliegue debe tener en cuenta cómo y cuándo entregar el software, y debe asegurar que el usuario final tiene todo lo que necesita para recibir y comenzar a usar el software. Para esto, podemos incluir pruebas beta para asesorar al usuario desde temprano. Necesitamos un alto grado de colaboración del usuario. La conclusión exitosa de un proyecto de software puede verse gravemente afectada por factores fuera del alcance del desarrollo de software, como el edificio, la falta de infraestructura de hardware y los usuarios que no están preparados para la transición al nuevo sistema.

Desarrollar material de soporte

Implica cubrir toda la información que requiere el usuario para instalar y usar el sistema operación, uso y mantenimiento del sistema entregado. También incluye material de formación para todos los puestos que serán necesarios para utilizar el nuevo sistema de forma eficaz.

Probar el producto en el lugar de desarrollo

Determina si el producto tiene la suficiente madurez para ser entregado al cliente como producto final o beta. La prueba beta de un producto enlatado se realiza encuestando a una amplia gama de usuarios finales para obtener retroalimentación. En el caso de un sistema hecho a medida, una prueba beta podría ser una instalación piloto en el sitio de destino. Para reforzar la idea de que las actividades y plataformas de prueba en el sitio de desarrollo deben aproximarse mucho al sitio de destino previsto, las actividades de prueba del producto son idénticas para las pruebas en el sitio de desarrollo y el sitio de destino, con una excepción. La excepción es que la prueba en el sitio de desarrollo debe usar simuladores. Las pruebas en el sitio de destino utilizarán el hardware de destino real.

Crear la reléase

Asegurar de que el producto está preparado para ser entregado al cliente. Una versión consiste en todo lo que el usuario final necesitara instalar y correr en el software ejecutable.

Lanzamiento de prueba beta

Nos ayuda a asegurarnos de que el producto cumple las expectativas del cliente y la información provista por el usuario como respuesta sirva de retroalimentación para la próxima iteración del desarrollo

Probar el producto en el lugar de instalación

Después de suficientes pruebas internas y pruebas beta, el producto debe ser instalado en el sitio y probado por el cliente. Basándonos en las pruebas de iteración anteriores y los compromisos del cliente, las pruebas finales en el sitio de instalación no deberían ser una sorpresa, ipero deberían ser solo una formalidad para que el cliente "acepte" el sistema.

Proveer acceso a sitio de descarga(opc.)

El atractivo del comercio web e Internet como canal de distribución de software es innegable. Aunque el producto es accesible a través de navegadores y sitios web, el desafío para el proveedor es garantizar su disponibilidad las 24/7 en un mercado global, superando posibles obstáculos de hardware y ancho de banda. Configurar la infraestructura de hardware para la presencia web corporativa es un proceso fuera del alcance del desarrollo de software, pero el director de despliegue debe conocer cómo añadir el producto a la lista web y comunicar su disponibilidad. La Web también sirve como mecanismo para obtener retroalimentación, anunciar actualizaciones y brindar acceso a otros servicios corporativos

mediante contraseñas si es necesario.

PRODUCTOS ENLATADOS.

Este conjunto de actividades opcionales describe lo que debe ocurrir para producir productos de "software enlatado". En este caso, la versión se guarda como original para la producción en masa y luego se empaqueta en cajas de productos junto con la lista de materiales para su envío al cliente.

RESUMEN DEL DESPLIEGUE

El flujo de trabajo de despliegue describe las actividades asociadas con las pruebas beta y la entrega de software instalable. La instalación puede ser realizada por el proveedor (en el caso de sistemas distribuidos complejos) o por el usuario (en el caso de software entregado como un producto enlatado o por Internet). Este flujo de trabajo depende en gran medida tanto del tipo de producto que se está desarrollando como del contexto empresarial, y debe ser especializado por la organización que está adoptando el Proceso Unificado de Rational.

UNIDAD 5

El software evolucionará con el tiempo dado que vivimos en un mundo que cambia rápidamente. El cambio impulsa este proceso. Para el software, el cambio ocurre cuando se corrigen los errores, cuando el sistema se adapta a un nuevo entorno, cuando el cliente solicita nuevas características o funciones y cuando la aplicación se somete a reingeniería para ofrecer beneficios en un contexto moderno.

El enfoque y los conceptos a lo largo del tiempo

Existe una distinción bastante marcada entre MANTENIMIENTO y EVOLUCIÓN del software

En los años 70 comenzó la problemática del mantenimiento del software en las empresas. Desde sus comienzos, se lo veía como una etapa más, pero sólo destinado a la corrección de errores cuando el software ya estaba terminado y se ponía en producción. Se consideraba una actividad aparte de lo hecho anteriormente. En esos momentos, se implementaba un ciclo de vida en cascada, secuencial con lo cual se tardaba mucho el proceso desde que comenzaba hasta ponerlo en producción. Esta actividad siempre fue y sigue siendo muy compleja para las empresas que conviven y usan el software dado que tienen que hacer que el software siga sirviendo para su negocio. Además, también se da el problema de que las personas que crearon dicho software probablemente ya no se encuentren trabajando en la misma empresa y no quedan personas que tengan conocimiento sobre el sistema heredado.

Por otro lado, se consideraba como evolución del software a los módulos o funcionalidades nuevas o el cambio a características ya existentes.

Con el PUD, se incorpora al mantenimiento como una actividad normal al proceso mientras dura el ciclo de vida iterativo e incremental.

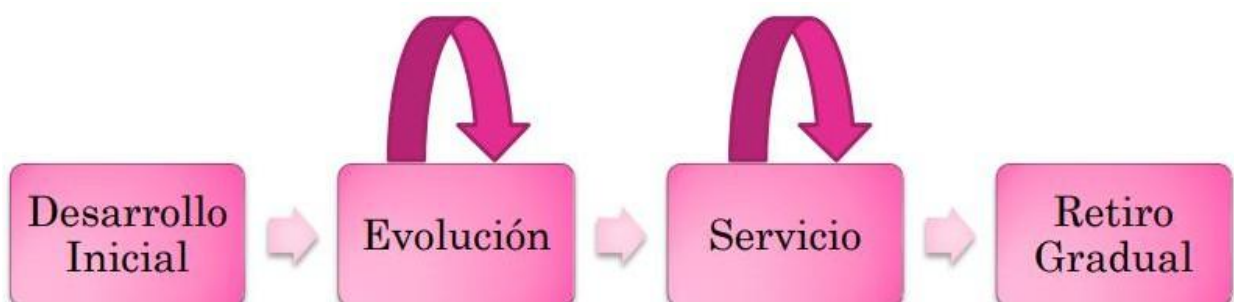
Evolución del software

El mantenimiento del software se caracteriza con un iceberg; “esperábamos que solo fuera lo que se veía, pero sabíamos que una enorme masa de problemas y costos yacía bajo la superficie”. La evolución del software es importante porque las organizaciones invierten grandes cantidades de dinero en él y en la actualidad son completamente dependientes de sus sistemas. Los cambios en el entorno conducen a cambios en el sistema, lo que puede provocar complejidad al momento de evolucionar. La ingeniería de software se debe considerar como un proceso en espiral, con requerimientos, diseño, implementación y pruebas continuas a lo largo de la vida del sistema.

Podemos diferenciar la evolución del servicio; **evolución es la fase donde es posible hacer cambios significativos a la arquitectura y funcionalidad del software**. Durante el servicio, los cambios son relativamente pequeños. Necesitamos el **mantenimiento y la evolución para no tener sistemas obsoletos y difíciles de modificar en un futuro, además, siempre hay necesidades nuevas que incluir**. Con el tiempo su estructura puede degradarse y esto implica que hacer un cambio sea cada vez más costoso.

¿Porque se necesita tanto mantenimiento?

- Arquitecturas muy mal diseñadas
- Aun cuando hayan sido construidos empleando las mejoras técnicas de análisis y diseño (y para la mayoría no fue así) se crearon cuando el tamaño de los programas y el espacio de almacenamiento eran las principales preocupaciones.
- Mala codificación
- Lógica inadecuada
- Escasa o inexistente documentación
- Software antiguo
- Se incorporan nuevas necesidades
- Se deben hacer traslados a nuevas plataformas



Procesos de evolución

Los procesos varían dependiendo del tipo de software que se mantiene, de los procesos de desarrollo usados y de las habilidades de las personas que intervienen. Las propuestas de cambio al sistema son el motor que motiva la evolución. Estas pueden venir de requerimientos que no se implementaron, que no se encontraron o que no existían; de reportes de errores y de nuevas ideas. Estamos en constante búsqueda de cambios. Las propuestas deben vincularse con los componentes a modificar, así podremos valorar los costos e impactos de los cambios que realicemos. Esto se valora para saber qué tanto resultará afectado el sistema por el cambio, y cuánto costaría implementarlo. Si los cambios se aceptan, se planea una nueva versión del sistema, se implementa, se valida y se libera. Podemos considerar la implementación del cambio como una iteración del proceso de desarrollo, donde deberíamos investigar y comprender el sistema original para evitar fallas al momento de introducir funcionalidades nuevas. Esta etapa debe modificar la especificación, diseño e implementación del sistema para reflejar los cambios. Se proponen, analizan y validan los nuevos requerimientos, los componentes se rediseñan e implementan, y el sistema vuelve a probarse. Hasta podríamos elaborar prototipos con los cambios propuestos. A veces las peticiones de cambio se relacionan con problemas que deben resolverse, que surgen por 3 razones principales:

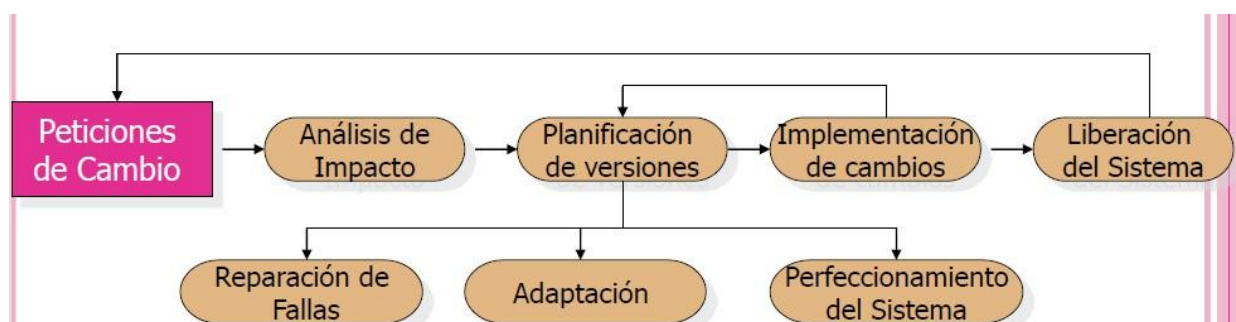
Ocurre una falla seria del sistema que debe repararse para poder continuar usándolo normalmente.

Cambios en el entorno afectan la operación normal.

Cambios no anticipados en la empresa.

En caso de que el cambio sea urgente, podemos evitar modificar requerimientos y diseño y hacer una reparación de emergencia, pero esto podría generar inconsistencia entre diseño y código. En cualquier caso, estas reparaciones deben completarse lo más rápido posible, buscando y aplicando una solución rápida y viable y no la mejor solución. Esto puede degenerar el software, y aún más si el equipo que desarrolló el sistema no es el mismo que el que lo reparó.

El Proceso de Evolución del Software



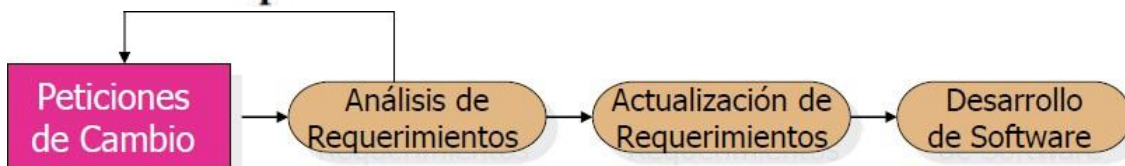
El proceso de evolución incluye las actividades fundamentales de:

- Análisis de impacto: significa ver que implica ese cambio a realizar, cuánto esfuerzo lleva, cuántos componentes va a afectar y hay que cambiar. Se realiza antes de decir que ese cambio se va a llevar a cabo.
- Planificación de versiones: consiste en tres subetapas de Reparación de Fallas, Adaptación (es ambiental, si hay un SO nuevo, una BD nueva, si se adapta a tecnología mobile, etc) y Perfeccionamiento del sistema.
- Implementación de cambios: se debe llevar adelante los workflows de análisis, diseño, implementación y prueba del PUD. Se define cuándo se va a realizar en función de las prioridades.
- Liberación del sistema a los clientes: se lanza una nueva versión del sistema.

El costo y el impacto de dichos cambios se valoran para saber que tanto resultará afectado el sistema por el cambio y cuánto costaría implementarlo. Si los cambios propuestos se aceptan, se planea una nueva versión del sistema. Durante la planeación de la versión se consideran todos los cambios propuestos (reparación de fallas, adaptación y nueva funcionalidad). Entonces se toma una decisión acerca de cuáles cambios implementar en la siguiente versión del sistema. Después de implementarse, se valida y libera una nueva versión del sistema. Luego, el proceso se repite con un conjunto nuevo de cambios propuestos para la siguiente liberación.

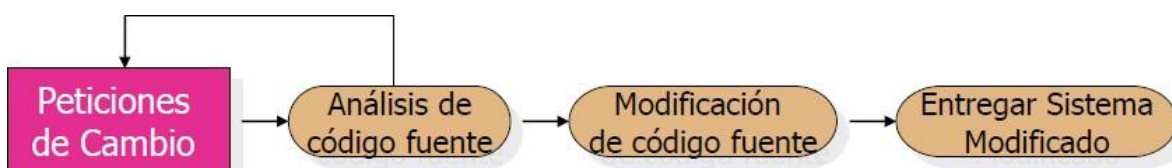
Procesos de Cambio

Proceso de Implementación de cambios



La actualización de requerimientos es lo que se debe hacer antes del desarrollo. Es la manera en que debemos mantener la arquitectura y el resto de los modelos actualizados y consistentes con el SW en producción.

Proceso de reparación de fallas de emergencia



Si la reparación es urgente entonces, directamente se entra al código, se lo arregla, se lo entrega y luego de eso se debería ir pasos hacia atrás (ingeniería inversa) para realizar la actualización de los modelos que están antes del código.

El problema es si no se realizan esas actualizaciones, esto generará inconsistencias y el esfuerzo de haber realizado los modelos se echa a perder y es lo que la mayoría de las veces ocurre, por eso realizar mantenimiento es costoso.

Dinámica de evolución del software

Es el estudio de cambio del sistema. Se propusieron las Leyes de Lheman, quien afirma que estas leyes suelen ser verdaderas para todos los tipos de sistemas de software organizacional grandes. Se trata de sistemas en los cuales los requerimientos se modifican para reflejar las necesidades cambiantes de la empresa. Las nuevas versiones del sistema son esenciales para que éste proporcione valor al negocio.

Ley	Descripción
Cambio continuo	Un programa usado en un entorno real debe cambiar; de otro modo, en dicho entorno se volvería progresivamente inútil.
Complejidad creciente	A medida que cambia un programa en evolución, su estructura tiende a volverse más compleja. Deben dedicarse recursos adicionales para conservar y simplificar su estructura.
Evolución de programa grande	La evolución del programa es un proceso autorregulador. Los atributos del sistema, como tamaño, tiempo entre versiones y número de errores reportados, son casi invariantes para cada versión del sistema.
Estabilidad organizacional	Durante la vida de un programa, su tasa de desarrollo es aproximadamente constante e independiente de los recursos dedicados al desarrollo del sistema.
Conservación de familiaridad	A lo largo de la existencia de un sistema, el cambio incremental en cada liberación es casi constante.
Crecimiento continuo	La funcionalidad ofrecida por los sistemas tiene que aumentar continuamente para mantener la satisfacción del usuario.
Declive de calidad	La calidad de los sistemas declinará, a menos que se modifiquen para reflejar los cambios en su entorno operacional.
Sistema de retroalimentación	Los procesos de evolución incorporan sistemas de retroalimentación multiagente y multiciclo. Además, deben tratarse como sistemas de retroalimentación para lograr una mejora significativa del producto.

La primera ley afirma que el mantenimiento del sistema es un proceso inevitable. A medida que cambia el entorno del sistema, surgen nuevos requerimientos y el sistema debe modificarse. Cuando el sistema modificado se reintroduce al entorno, promueve más cambios ambientales, de manera que el proceso de evolución comienza de nuevo.

La segunda ley afirma que, conforme cambia un sistema, su estructura se degrada. La única manera de evitar que esto ocurra es invertir en mantenimiento preventivo. Se invierte tiempo mejorando la estructura del software sin agregar nada a su funcionalidad.

La tercera ley sugiere que los sistemas grandes tienen una dinámica propia que se establece en una etapa temprana del proceso de desarrollo. Esto determina las grandes tendencias del proceso de mantenimiento del sistema y limita el número de cambios posibles al sistema. Esta ley es consecuencia de factores estructurales que influyen en el cambio al sistema y lo restringen, así como de factores organizacionales que afectan el proceso de evolución.

Los factores estructurales que afectan la tercera ley provienen de la complejidad de los sistemas grandes. Conforme cambia y se extiende un programa, su estructura tiende a degradarse. Ocurre porque una estructura con un propósito específico se adapta para uno diferente. Esta degradación, si no se controla, hace cada vez más difícil realizar cambios ulteriores al programa. Hacer pequeños cambios reduce el alcance de la degradación estructural y, por consiguiente, aminora los riesgos de causar serios problemas a la confiabilidad del sistema. Si se realizan grandes cambios, hay una alta probabilidad de

que se introduzcan nuevas fallas, las cuales impedirían más cambios al programa.

Los factores organizacionales que afectan la tercera ley reflejan el hecho de que, por lo general, los sistemas grandes se producen en organizaciones grandes. Las empresas deben tomar decisiones sobre los riesgos y el valor de los cambios, así como sobre los costos inherentes. Tales decisiones toman tiempo y, en ocasiones, tardan más para decidir acerca de los cambios por realizar que para implementarlos. En consecuencia, la rapidez de los procesos de toma de decisiones de la organización controla la tasa de cambio del sistema.

La cuarta ley sugiere que la mayoría de los grandes proyectos de programación funcionan en un estado "saturado". Es decir, un cambio en los recursos o en el personal tiene efectos imperceptibles en la evolución a largo plazo del sistema. Esta ley confirma que los grandes equipos de desarrollo de software con frecuencia son improductivos, porque los gastos en comunicación dominan el trabajo del equipo.

La quinta ley se relaciona con los incrementos de cambio en cada versión del sistema. Agregar nueva funcionalidad a un sistema introduce inevitablemente nuevas fallas al mismo. Cuanta más funcionalidad se agregue en cada versión, más fallas habrá. Esta ley sugiere que no se deben presupuestar grandes incrementos de funcionalidad en cada versión, sin tomar en cuenta la necesidad de reparación de las fallas.

Las leyes sexta y séptima son similares y, en esencia, indican que los usuarios de software se volverán cada vez más infortunados con el sistema, a menos que se le mantenga y se le agregue nueva funcionalidad.

Estrategias de evolución del software

Mantenimiento del software

Es el proceso general de cambiar un sistema luego de que el mismo se entregó. Los cambios se implementan modificando componentes existentes del sistema y agregándole componentes donde sea necesario. Este proceso comienza casi de inmediato. El software se libera a los usuarios finales y en cuestión de días, los reportes de errores están de vuelta hacia la organización de ingeniería de software. Se debe enfrentar a una lista de corrección de errores, peticiones de adaptación y mejoras categóricas que deben planearse, calendarizarse y lograrse. Conforme pasa el tiempo, se emplea más tiempo y dinero en mantener los programas existentes, de hecho, se puede emplear entre el 60% y 70% de todos sus recursos en mantenimiento del software.

- Hacer correcciones de errores: esto no se puede cobrar y hay que evitarlo porque provoca un cliente insatisfecho. Hay que tratarlos con urgencia y compensar al cliente.
- Hacer mejoras:
 - Que modifican requerimientos existentes

- Que crean nuevos requerimientos
- Que modifican la instalación sin cambiar los requerimientos (adaptación ambiental con RNF)

Tipos de mantenimiento de software:

Reparaciones de fallas ("mantenimiento correctivo"): sea por error de codificación, de diseño o de requerimientos (en ese orden de costos). Relacionado al concepto de mantenimiento correctivo. Llega un error que no se contempló en los test. Es el costo de la no calidad. Los errores de codificación por lo general son relativamente baratos de corregir; los errores de diseño son más costosos, ya que pueden implicar la reescritura de muchos componentes. Los errores de requerimientos son los más costosos de reparar debido a que podría ser necesario un extenso rediseño del sistema.

Adaptación ambiental ("mantenimiento adaptativo"): este tipo de mantenimiento se requiere cuando algún aspecto del entorno del sistema, como el hardware, la plataforma operativa del sistema u otro soporte, cambia el software. Modifican RNF. Problema: tener que pasar todo el proceso de adaptación y actualizar el presupuesto. Relacionado al concepto de mantenimiento adaptativo.

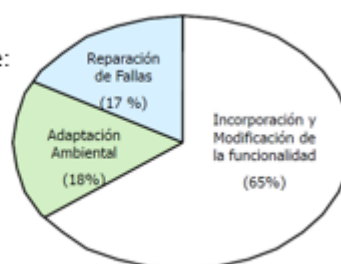
Adición de funcionalidad ("mantenimiento perfectivo"): este tipo de mantenimiento es necesario cuando se modifican o se crean requerimientos del sistema, en respuesta a un cambio organizacional o empresarial. es el que conlleva un mayor esfuerzo de mantenimiento, un mayor presupuesto, y es el más común y más frecuente. Cambian las políticas y reglas de negocio, disposiciones legales que impactan los procesos de negocio. Relacionado al concepto de mantenimiento perfectivo.

En resumen, los tres tipos de cambio tienen el mismo objetivo, que es mantener el sistema útil. Los mantenimientos adaptativo y perfectivo no son atribuibles a algo que hicimos mal, por lo que son cobrables. Cuando vienen peticiones de cambio (requerimientos), se debe tener rastreabilidad (trazabilidad). Lo que comparten estos tipos de mantenimientos es:

- 1) Análisis de impacto: qué (requerimiento), cuándo (calendario, fecha límite), cuánto (costo, es lo último que se calcula), esfuerzo
- 2) Comunicárselo al cliente.

Distribución del esfuerzo de mantenimiento

La tendencia en la industria actualmente es la siguiente:



Factores de costo de mantenimiento:

Se busca tener un buen diseño y una buena base en el software para evitar o reducir costos de mantenimiento. Suele ser costoso agregar funcionalidad en un sistema en operación por diferentes

motivos:

- **Estabilidad del equipo:** luego de entregar un sistema, el equipo de desarrollo suele separarse, y si un nuevo equipo queda a cargo del mantenimiento tendrá que comenzar de cero a entender el funcionamiento. Un buen diseño es un lenguaje común entre programadores, lo que evitaría que un nuevo equipo tenga que sentarse a entender qué se hizo
- **Práctica de desarrollo deficientes:** Muchas veces los códigos están escritos por programadores novatos los cuales tienen un conocimiento básico y no reciben capacitaciones adecuadas
- **Habilidades del personal**
- **Antigüedad y estructura del programa:** un sistema viejo suele estar degradado, se vuelve difícil de entender y de cambiar

Motivación del personal

- Es bueno organizar la gente que hace mantenimiento
- Embeber el trabajo de mantenimiento en nuevos proyectos o productos
- Hacer rotación de personal.
- Relacionar el desarrollo de software a los objetivos organizacionales-mantenimiento racional.
- Relacionar recompensas a la performance organizacional.
- Integrar mantenimiento con desarrollo.
- Crear un presupuesto discrecional para mantenimiento preventivo.
- Planificar el mantenimiento en las etapas tempranas del proceso de desarrollo.
- Planificar el gasto de esfuerzo en programas de mantenibilidad.
- Hay que incorporar el mantenimiento como una situación normal de evolución del SW en el tiempo. Esto facilita la responsabilidad de hacerse cargo de la situación.

Predicción de cambios

Se puede hacer una proyección de futuros cambios analizando los requerimientos y viendo cuáles son más volátiles que otros, cuáles son más susceptibles a cambiar con el tiempo analizando las decisiones que quedaron aferradas al ambiente o contexto del sistema. Si esto cambia, el sistema también deberá hacerlo (por ej. leyes contables, bases imponibles de impuesto a las ganancias, topes de aportes, etc).

Hay que analizar estos factores para estimar cuánto esfuerzo vamos a necesitar para mantenimiento. Tenemos que entender que cuando el SW se pone en producción, no termina nuestro trabajo sino que recién empieza.

Predecir el número de peticiones de cambio para un sistema requiere comprender las relaciones entre el sistema y su ambiente externo. Los sistemas fuertemente acoplados requieren cambios cada vez que cambia el ambiente. Para evaluar las relaciones entre un sistema y su ambiente, los factores que

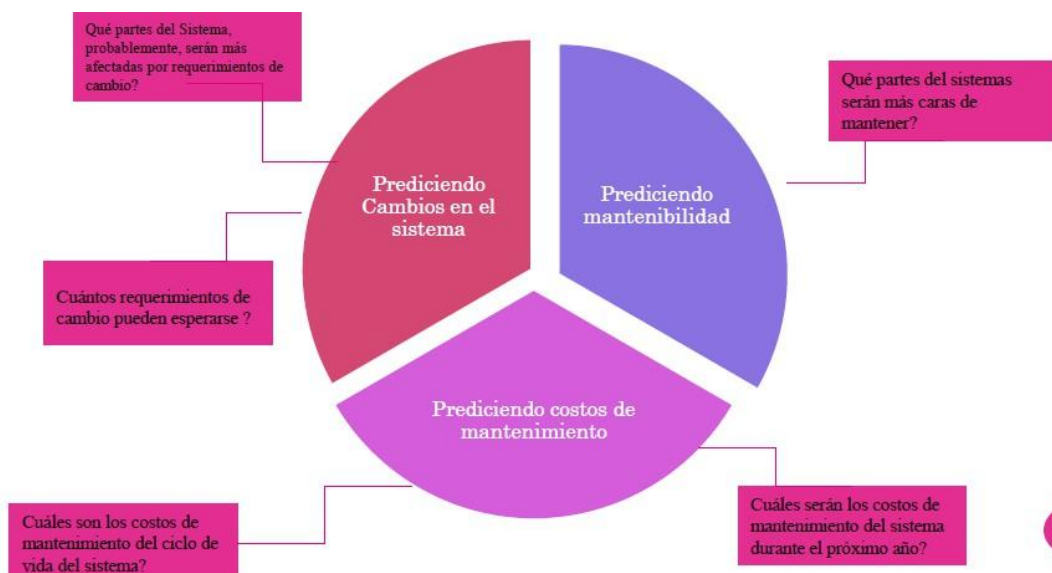
influyen son:

- 1. El número y la complejidad de las interfaces del sistema:** cuantas más interfaces y más complejas sean, más probable será que se requieran cambios de interfaz conforme se propongan nuevos requerimientos.
- 2. El número de requerimientos volátiles del sistema:** los requerimientos que reflejan políticas y procedimientos de la organización son más inestables que los requerimientos que se basan en características de un dominio estable.
- 3. Los procesos de negocio donde se usa el sistema:** a medida que evolucionan los procesos de negocio, generan peticiones de cambio del sistema.

Predicción de mantenimiento

La predicción del mantenimiento está relacionada con la evaluación de cuáles son las partes del sistema que pueden causar problemas y tener costos altos de mantenimiento:

- La aceptación del cambio depende de la mantenibilidad de los componentes afectados por el cambio.
- La implementación de los cambios degrada el sistema y reduce su mantenibilidad
- Los costos de mantenimiento dependen del número de cambios y los costos del cambio



La predicción de cambios será útil, aunque requiere un entendimiento de la relación entre el sistema y su entorno. Debemos bajar el acoplamiento entre ellos, así habrá menor cantidad de cambios en el sistema frente a cambios en el entorno. Para evaluar esta relación, debemos valorar:

El número y complejidad de interfaces del sistema

El número de requerimientos de sistema inherentemente inestables

Los procesos empresariales donde se usa el sistema

Mantenimiento preventivo mediante refactorización

La refactorización es un proceso de hacer mejoras a un programa para frenar la degradación por el cambio. Esto sirve para mejorar su estructura o reducir su complejidad. Es un proceso continuo de mejoramiento, que nos brinda una forma de reducir los costos de mantenimiento a largo plazo. Luego de haber mantenido un sistema durante cierto tiempo, podemos hacer reingeniería, usando herramientas automatizadas para procesar un sistema heredado. Podemos identificar cuándo un código es susceptible a mejorar al encontrarnos los "malos olores":

Código duplicado

Métodos largos

Enunciados switch case

Aglomeración de datos

Generalidad especulativa

Administración de sistemas heredados

Es común que encontremos en una organización un portafolio de sistemas heredados, muchas veces con necesidad de mantenimiento y actualización. Hay que decidir cómo invertir en estos sistemas para obtener el mejor resultado, y para esto tenemos cuatro opciones estratégicas:

1. Desechar completamente el sistema, lo que debemos hacer cuando el mismo ya no aporte nada efectivo
2. Dejar el sistema sin cambios, continuar con mantenimiento regular, lo que debemos hacer cuando el sistema todavía se necesite, pero esté lo suficientemente estable
3. Someter el sistema a reingeniería, que debe elegirse cuando la calidad del sistema esté degradada
4. Sustituir todo o parte del sistema con uno nuevo, lo que debemos hacer cuando ya no sea posible seguir utilizándolo, generalmente por cuestiones de incompatibilidad

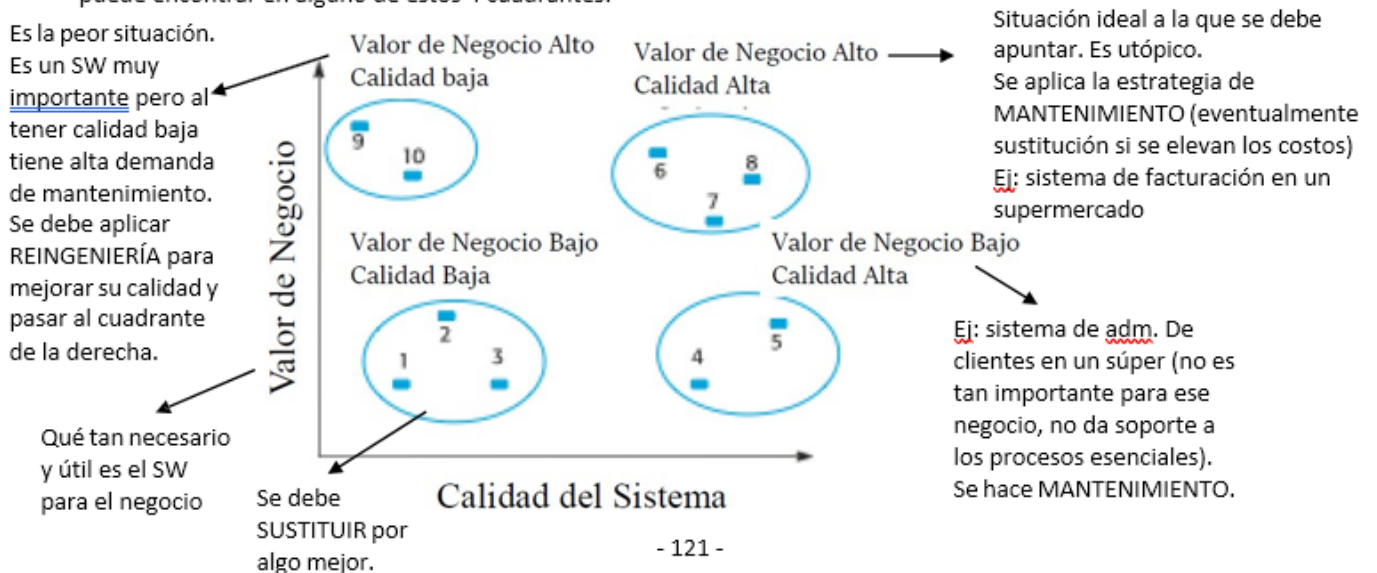
Al tomar esta decisión debemos considerar la perspectiva empresarial, decidiendo si la empresa necesita realmente el sistema, y la perspectiva técnica, valorando la calidad del software y el hardware que utiliza. Podemos armar una gráfica que tenga en cuenta estas dos valoraciones

- **Baja calidad, bajo valor:** mantener el sistema será costoso, tendrá tasa de retorno baja. Debemos descartarlo/sustituirlo
- **Baja calidad, alto valor:** pueden sustituirse o podemos aplicar reingeniería, ya que el sistema es útil
- **Alta calidad, bajo valor:** no aporta mucho a la empresa, pero su mantenimiento no sería costoso, podemos seguirlo manteniendo de forma regular y si surge un cambio costoso lo desechamos

- **Alta calidad, alto valor:** el sistema debe mantenerse en operación, debemos continuar con su mantenimiento regular

Ejemplo de valoración de un sistema heredado

Al SW que se encuentra funcionando y que tenemos que evolucionar se le llama sistema heredado. Un SW heredado se puede encontrar en alguno de estos 4 cuadrantes:



La perspectiva empresarial puede ser difícil de contemplar. Por eso, debemos basarnos en cuatro temas básicos, en los que debemos hacer hincapié al momento de entrevistar al personal:

- Uso del sistema
- Procesos empresariales actuales
- Confiabilidad del sistema
- Salidas del sistema

El entorno es todo lo que rodea al sistema, hardware y software. Para valorar el entorno, debemos tener en cuenta factores como:

- Estabilidad del proveedor
- Tasa de falla
- Edad/antigüedad
- Rendimiento
- Requerimientos de soporte
- Costos de mantenimiento
- Interoperabilidad

Para valorar la calidad técnica de un sistema, debemos tener en cuenta

- Entendimiento
- Documentación
- Datos
- Rendimiento
- Lenguaje de programación

- Administración de la configuración
- Datos de prueba
- Habilidades del personal

Reingeniería

Es el trabajo que se hace sobre el SW existente (sistemas heredados) que apunta a **mejorar la calidad** del software sin modificar la funcionalidad original para lograr que el mantenimiento sea menos costoso y más simple. Es una decisión de inversión sobre un proyecto de software, es una estrategia de evolución para mejorar su calidad interna y que quede en una posición más conveniente para hacer mantenimiento. El propósito de un proyecto de reingeniería es la necesidad de mantenimiento del software, pero con un costo menor.

Para hacer que los sistemas heredados sean más sencillos de mantener, se pueden someter a reingeniería para mejorar su estructura y entendimiento. La reingeniería puede implicar volver a documentar el sistema, refactorizar su arquitectura, traducir los programas a un lenguaje de programación moderno, y modificar y actualizar la estructura y los valores de los datos del sistema. La funcionalidad del software no cambia y, normalmente, conviene tratar de evitar grandes cambios en la arquitectura del sistema.

Cuando se aplica la reingeniería no necesariamente se requiere de todos las actividades.

Se debe tener en cuenta la trazabilidad existente entre los modelos para que sea menos costoso incluir cambios.

Actualmente la mayoría de las actividades del proceso de reingeniería pueden ser automatizadas.



Cuando ya se realizaron los modelos con la ingeniería inversa y se tiene el código se continúa con las actividades de ingeniería hacia adelante que implica realizar el mantenimiento con el PUD

Ingeniería Reversa: Partir desde el código para generar modelos

Ventajas de la reingeniería frente a la sustitución

1. **Reducción del riesgo:** pueden cometerse errores en la especificación del sistema o tal vez haya problemas de desarrollo, las demoras en la introducción del nuevo software podrían significar que la empresa está perdida y que se incurrirá en costos adicionales, hay un riesgo alto al realizar un software desde cero.
2. **Reducción de costos:** el costo de la reingeniería puede ser significativamente menor que el costo de desarrollar software nuevo

Actividades de la reingeniería

Son las actividades que se llevan a cabo en un proyecto de reingeniería de software

1. **Traducción del código fuente:** el programa se convierte de un lenguaje de programación

antiguo a una versión más moderna, o a un lenguaje diferente.

2. Ingeniería inversa: Es el proceso de analizar un programa para crear una representación del mismo en un nivel más alto de abstracción que el código. Por lo general es un proceso automatizado (ej: con el EA). Se realiza cuando los sistemas no cuentan con modelos y/o documentación. La ingeniería inversa permite crear modelos a partir del código. No se toca el producto si no que se está mejorando la calidad al generar los modelos que se deberían haber hecho antes para que sea más fácil de mantener. Es un proceso de recuperación del diseño.

3. Mejoramiento de la estructura del programa: la estructura de control del programa se analiza y modifica para facilitar su lectura y comprensión. Puede ser automatizado con alguna intervención manual.

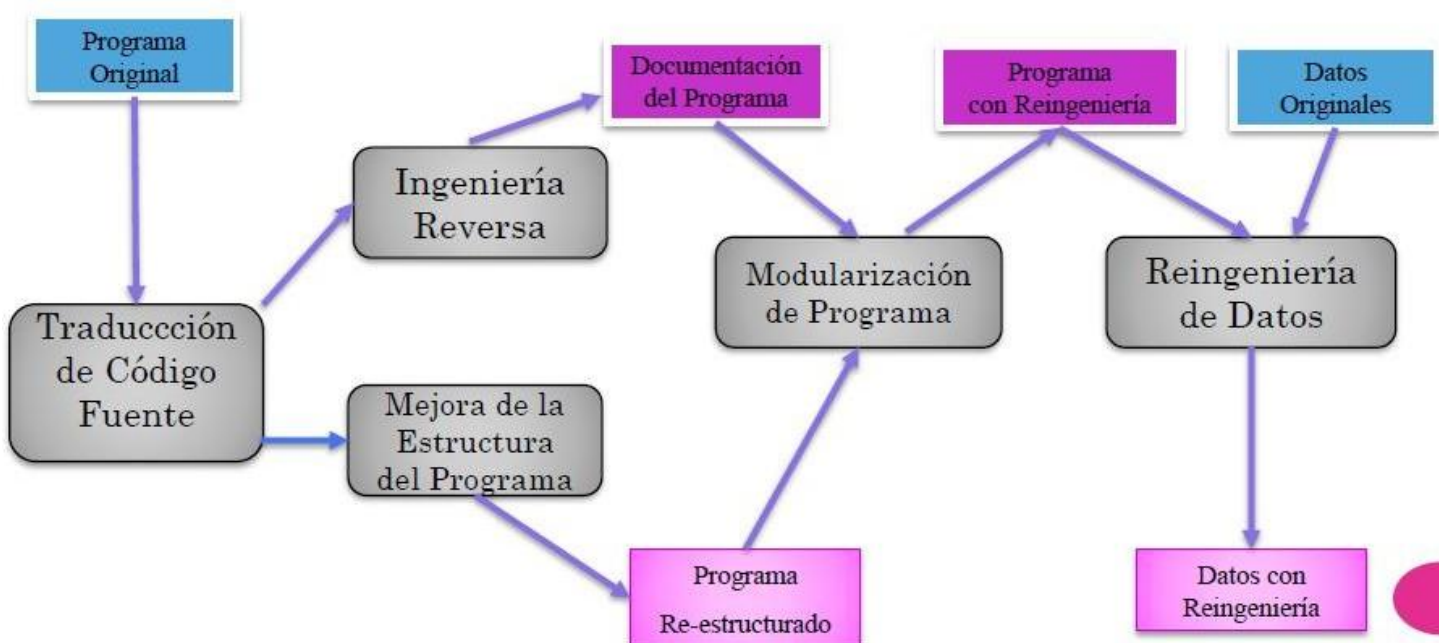
4. Modularización del programa: las partes relacionadas del programa se agrupan y, donde es adecuado, se elimina la redundancia. En algunos casos, implica refactorización arquitectónica. Es un proceso manual. Se debe tener en cuenta la alta cohesión y el bajo acoplamiento.

5. Reingeniería de datos: los datos procesados por el programa cambian para reflejar cambios al programa. Significa la redefinición de los esquemas de bases de datos (puede incluir cambios en el tamaño de los campos) y convertir las bases de datos existentes a la nueva estructura. Es una actividad de reingeniería gran escala.

Desventajas

- No es posible convertir un sistema funcional a un sistema orientado a objetos.
- Grandes cambios no pueden hacerse de forma automatizada.
- Los sistemas con reingeniería probablemente no serán tan mantenibles como los sistemas nuevos desarrollados con técnicas modernas.

El proceso de reingeniería



Factores de costo de la reingeniería

- La calidad de software al que se le va a aplicar re-ingeniería.
- La herramienta de soporte disponible.
- La extensión de la conversión de datos que se requiere.
- La disponibilidad de personal experto para realizar la re-ingeniería.

Factores de complejidad

- Complejidad de Control: puede medirse examinándose las sentencias condicionales en el programa.
- Complejidad de Datos: La complejidad de las estructuras de datos y los componentes de interfaces.
- Tamaño de los módulos
- Comentarios del Programa: Tal vez más comentarios significan mantenimiento más fácil.
- Extensión de los nombres identificadores: Los nombres más largos implican más legibilidad.

Refactorización

La refactorización es el proceso de hacer mejoras a un programa para frenar la degradación mediante el cambio. Estos significan modificar un programa para mejorar su estructura, reducir su complejidad o hacerlo más fácil de entender. Es una técnica para mejorar el código por dentro, pero sin cambiar su estructura por fuera. Se parte de un componente de código que funciona, pero queremos mejorarlo para que sea más eficiente o para que sea más entendible.

Mientras se refactorice un programa, no se debe agregar funcionalidad, sino que hay que concentrarse en la mejora del programa.

Aunque la reingeniería y la refactorización tienen la intención de hacer el software más fácil de entender y cambiar, no son lo mismo.

Refactoring

Es una estrategia de desarrollo de software, la cual esta incorporada como parte del TDD(Desarrollo guiado por pruebas), plantea mejorar la calidad interna del código introduciendo cambios sobre un código que funciona, mejorando la estructura interna y disminuyendo la complejidad e incrementando la capacidad de mantenimiento

Es una transformación del software que:

- Preserva la estructura externa
- Mejora la estructura interna

Son cambios que se realizan en el sistema que:

- No cambien el comportamiento observable (todas las pruebas aún pasan)
- Eliminan la duplicación o la complejidad innecesaria
- Mejoran la calidad del software
- Hacen que el código sea más simple y más fácil de entender

- Flexibiliza el código
- Hace que el código sea más fácil de cambiar

Para hacer refactoring se aplican los principios de diseño y los patrones de diseño.

Refactorización vs reingeniería

La reingeniería se lleva a cabo después de haber mantenido un sistema durante cierto tiempo. Se crea un nuevo sistema a partir de un sistema heredado utilizando herramientas automatizadas.

La refactorización es un proceso continuo de mejoramiento debido al proceso de desarrollo y evolución. Tiene la intención de evitar la degradación de la estructura y el código que aumentan los costos y las dificultades por mantener un sistema.

Mejoras de la refactorización

La refactorización mejora:

- 1. Código duplicado:** el mismo código se incluye en diferentes partes del programa. Éste se descarta o se implementa como un solo método.
- 2. Métodos largos:** si hay métodos largos, deben rediseñarse en varios métodos más pequeños.
- 3. Sentencias case (switch):** en un lenguaje orientado a objetos se pueden reemplazar mediante polimorfismo.
- 4. Aglomeración de datos:** las aglomeraciones de datos ocurren cuando el mismo grupo de objetos de datos vuelven a ocurrir en muchos lugares en un programa. Pueden sustituirse con un objeto que encapsule todos los datos.
- 5. Generalidad especulativa:** esto ocurre cuando los desarrolladores incluyen generalidad en un programa, en caso de que se requiera en el futuro. Por lo general, eso simplemente puede eliminarse.

¿Por qué es necesario hacer refactoring?

- Evitar el "deterioro del diseño"
- Limpiar desorden en el código
- Simplificar el código
- Incrementar la legibilidad y la comprensibilidad
- Encontrar errores
- Reducir el tiempo de depuración
- Incorporar el aprendizaje que hacemos sobre la aplicación
- Rehacer las cosas es fundamental en todo proceso creativo

El proceso de hacer Refactoring



Refactoring en el proceso de Desarrollo y Evolución del Software

¿Cuándo usamos esta estrategia?

Cuando estamos construyendo el código. Se puede aplicar de diversas formas:

- O **Extreme programming**: es una metodología ágil. Es refactoring embebido en desarrollo conducido por testing (TDD)
- O **Refactoring y TDD**: es una estrategia de desarrollo conducido por testing, se hace cuando se construye software por primera vez. Cuando se pasaron las pruebas, se entra al código y se mejora la calidad interna pero haciendo que las pruebas sigan pasando. Luego se vuelve a probar.
- O **Refactoring como mantenimiento preventivo**: sirve para mejorar la calidad y evitar la reingeniería.